

```
=====
FILE: data\output\canonical_model.json
ENCODING: utf-8
=====
```

```
{
  "records": [
    {
      "name": "COVERAGE",
      "primary_key": null,
      "primary_keys": [],
      "record_id": null,
      "location_mode": null,
      "fields": [
        {
          "name": "SELECTION_DATE_0400",
          "datatype": "DATE",
          "length": null,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "TERMINATION_DATE_0400",
          "datatype": "DATE",
          "length": null,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "TYPE_0400",
          "datatype": "CHAR",
          "length": 1,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "INS_PLAN_CODE_0400",
          "datatype": "CHAR",
          "length": 3,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        }
      ]
    },
    {
      "name": "DENTAL_CLAIM",
      "primary_key": null,
      "primary_keys": [],
      "record_id": null,
      "location_mode": null,
      "fields": [
        {
          "name": "CLAIM_DATE_0405",
          "datatype": "DATE",
          "length": null,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "PATIENT_BIRTH_DATE_0405",
          "datatype": "DATE",
          "length": null,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "SERVICE_DATE_0405",
          "datatype": "DATE",
          "length": null,
          "scale": null,

```

```

    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PATIENT_FIRST_NAME_0405",
    "datatype": "CHAR",
    "length": 10,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PATIENT_LAST_NAME_0405",
    "datatype": "CHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PATIENT_SEX_0405",
    "datatype": "CHAR",
    "length": 1,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "RELATION_TO_EMPLOYEE_0405",
    "datatype": "CHAR",
    "length": 10,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DENTIST_FIRST_NAME_0405",
    "datatype": "CHAR",
    "length": 10,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DENTIST_LAST_NAME_0405",
    "datatype": "CHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DENTIST_STREET_0405",
    "datatype": "CHAR",
    "length": 20,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DENTIST_CITY_0405",
    "datatype": "CHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DENTIST_STATE_0405",
    "datatype": "CHAR",
    "length": 2,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {

```

```

    "name": "DENTIST_ZIP_FIRST_FIVE_0405",
    "datatype": "CHAR",
    "length": 5,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DENTIST_ZIP_LAST_FOUR_0405",
    "datatype": "CHAR",
    "length": 4,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DENTIST_LICENSE_NUMBER_0405",
    "datatype": "DECIMAL",
    "length": 6,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "NUMBER_OF_PROCEDURES_0405",
    "datatype": "DECIMAL",
    "length": 2,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "TOOTH_NUMBER_0405",
    "datatype": "DECIMAL",
    "length": 2,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PROCEDURE_CODE_0405",
    "datatype": "DECIMAL",
    "length": 4,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DESCRIPTION_OF_SERVICE_0405",
    "datatype": "CHAR",
    "length": 60,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "FEE_0405",
    "datatype": "DECIMAL",
    "length": 5,
    "scale": 2,
    "occurs": false,
    "occurs_max": null
  }
]
},
{
  "name": "DEPARTMENT",
  "primary_key": "DEPT_ID_0410",
  "primary_keys": [
    "DEPT_ID_0410"
  ],
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "DEPT_ID_0410",

```

```

        "datatype": "DECIMAL",
        "length": 4,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DEPT_NAME_0410",
        "datatype": "CHAR",
        "length": 45,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DEPT_HEAD_ID_0410",
        "datatype": "DECIMAL",
        "length": 4,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    }
]
},
{
    "name": "EMPLOYEE",
    "primary_key": "EMP_ID_0415",
    "primary_keys": [
        "EMP_ID_0415"
    ],
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "START_DATE_0415",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "TERMINATION_DATE_0415",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "BIRTH_DATE_0415",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "EMP_ID_0415",
            "datatype": "DECIMAL",
            "length": 4,
            "scale": 0,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "EMP_FIRST_NAME_0415",
            "datatype": "CHAR",
            "length": 10,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "EMP_LAST_NAME_0415",
            "datatype": "CHAR",

```

```

        "length": 15,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_STREET_0415",
        "datatype": "CHAR",
        "length": 20,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_CITY_0415",
        "datatype": "CHAR",
        "length": 15,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_STATE_0415",
        "datatype": "CHAR",
        "length": 2,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_ZIP_FIRST_FIVE_0415",
        "datatype": "CHAR",
        "length": 5,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_ZIP_LAST_FOUR_0415",
        "datatype": "CHAR",
        "length": 4,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_PHONE_0415",
        "datatype": "DECIMAL",
        "length": 10,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "STATUS_0415",
        "datatype": "CHAR",
        "length": 2,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "SS_NUMBER_0415",
        "datatype": "DECIMAL",
        "length": 9,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    }
]
},
{
    "name": "EMPOSITION",
    "primary_key": null,
    "primary_keys": [],
    "record_id": null,

```

```

"location_mode": null,
"fields": [
  {
    "name": "START_DATE_0420",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "FINISH_DATE_0420",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "SALARY_GRADE_0420",
    "datatype": "DECIMAL",
    "length": 2,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "SALARY_AMOUNT_0420",
    "datatype": "DECIMAL",
    "length": 5,
    "scale": 2,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "BONUS_PERCENT_0420",
    "datatype": "DECIMAL",
    "length": 2,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "COMMISSION_PERCENT_0420",
    "datatype": "DECIMAL",
    "length": 2,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "OVERTIME_RATE_0420",
    "datatype": "DECIMAL",
    "length": 2,
    "scale": 2,
    "occurs": false,
    "occurs_max": null
  }
]
},
{
  "name": "EXPERTISE",
  "primary_key": null,
  "primary_keys": [],
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "EXPERTISE_DATE_0425",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ],
  {

```

```

        "name": "SKILL_LEVEL_0425",
        "datatype": "CHAR",
        "length": 2,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    }
]
},
{
    "name": "HOSPITAL_CLAIM",
    "primary_key": null,
    "primary_keys": [],
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "CLAIM_DATE_0430",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "PATIENT_BIRTH_DATE_0430",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "ADMIT_DATE_0430",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "DISCHARGE_DATE_0430",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "PATIENT_FIRST_NAME_0430",
            "datatype": "CHAR",
            "length": 10,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "PATIENT_LAST_NAME_0430",
            "datatype": "CHAR",
            "length": 15,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "PATIENT_SEX_0430",
            "datatype": "CHAR",
            "length": 1,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "RELATION_TO_EMPLOYEE_0430",
            "datatype": "CHAR",
            "length": 10,

```

```

    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "HOSPITAL_NAME_0430",
    "datatype": "CHAR",
    "length": 25,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "HOSP_STREET_0430",
    "datatype": "CHAR",
    "length": 20,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "HOSP_CITY_0430",
    "datatype": "CHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "HOSP_STATE_0430",
    "datatype": "CHAR",
    "length": 2,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "HOSP_ZIP_FIRST_FIVE_0430",
    "datatype": "CHAR",
    "length": 5,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "HOSP_ZIP_LAST_FOUR_0430",
    "datatype": "CHAR",
    "length": 4,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DIAGNOSIS_0430",
    "datatype": null,
    "length": 120,
    "scale": null,
    "occurs": true,
    "occurs_max": 2
  },
  {
    "name": "WARD_DAYS_0430",
    "datatype": "DECIMAL",
    "length": 3,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "WARD_RATE_0430",
    "datatype": "DECIMAL",
    "length": 5,
    "scale": 2,
    "occurs": false,
    "occurs_max": null
  },
  },

```



```

{
  "name": "WARD_TOTAL_0430",
  "datatype": "DECIMAL",
  "length": 5,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "SEMI_DAYS_0430",
  "datatype": "DECIMAL",
  "length": 3,
  "scale": 0,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "SEMI_RATE_0430",
  "datatype": "DECIMAL",
  "length": 5,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "SEMI_TOTAL_0430",
  "datatype": "DECIMAL",
  "length": 5,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "DELIVERY_COST_0430",
  "datatype": "DECIMAL",
  "length": 5,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "ANESTHESIA_COST_0430",
  "datatype": "DECIMAL",
  "length": 5,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "LAB_COST_0430",
  "datatype": "DECIMAL",
  "length": 5,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
}
],
{
  "name": "INSURANCE_PLAN",
  "primary_key": "INS_PLAN_CODE_0435",
  "primary_keys": [
    "INS_PLAN_CODE_0435"
  ],
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "INS_PLAN_CODE_0435",
      "datatype": "CHAR",
      "length": 3,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ],
  {

```

```

    "name": "INS_CO_NAME_0435",
    "datatype": "CHAR",
    "length": 45,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "INS_CO_STREET_0435",
    "datatype": "CHAR",
    "length": 20,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "INS_CO_CITY_0435",
    "datatype": "CHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "INS_CO_STATE_0435",
    "datatype": "CHAR",
    "length": 2,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "INS_CO_ZIP_FIRST_FIVE_0435",
    "datatype": "CHAR",
    "length": 5,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "INS_CO_ZIP_LAST_FOUR_0435",
    "datatype": "CHAR",
    "length": 4,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "INS_CO_PHONE_0435",
    "datatype": "DECIMAL",
    "length": 10,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "GROUP_NUMBER_0435",
    "datatype": "DECIMAL",
    "length": 6,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DEDUCT_0435",
    "datatype": "DECIMAL",
    "length": 5,
    "scale": 2,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "MAXIMUM_LIFE_COST_0435",
    "datatype": "DECIMAL",
    "length": 5,
    "scale": 2,

```

```

        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "FAMILY_COST_0435",
        "datatype": "DECIMAL",
        "length": 5,
        "scale": 2,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DEP_COST_0435",
        "datatype": "DECIMAL",
        "length": 5,
        "scale": 2,
        "occurs": false,
        "occurs_max": null
    }
]
},
{
    "name": "JOB",
    "primary_key": "JOB_ID_0440",
    "primary_keys": [
        "JOB_ID_0440"
    ],
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "JOB_ID_0440",
            "datatype": "DECIMAL",
            "length": 4,
            "scale": 0,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "TITLE_0440",
            "datatype": "CHAR",
            "length": 20,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "DESCRIPTION_LINE_0440",
            "datatype": null,
            "length": 120,
            "scale": null,
            "occurs": true,
            "occurs_max": 2
        },
        {
            "name": "REQUIREMENT_LINE_0440",
            "datatype": null,
            "length": 120,
            "scale": null,
            "occurs": true,
            "occurs_max": 2
        },
        {
            "name": "MINIMUM_SALARY_0440",
            "datatype": "DECIMAL",
            "length": 8,
            "scale": 2,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "MAXIMUM_SALARY_0440",
            "datatype": "DECIMAL",
            "length": 8,
            "scale": 2,
            "occurs": false,

```

```

    "occurs_max": null
  },
  {
    "name": "SALARY_GRADES_0440",
    "datatype": null,
    "length": 8,
    "scale": null,
    "occurs": true,
    "occurs_max": 4
  },
  {
    "name": "NUMBER_OF_POSITIONS_0440",
    "datatype": "DECIMAL",
    "length": 3,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "NUMBER_OPEN_0440",
    "datatype": "DECIMAL",
    "length": 3,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  }
]
},
{
  "name": "NON_HOSP_CLAIM",
  "primary_key": null,
  "primary_keys": [],
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "CLAIM_DATE_0445",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "PATIENT_BIRTH_DATE_0445",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "SERVICE_DATE_0445",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "PATIENT_FIRST_NAME_0445",
      "datatype": "CHAR",
      "length": 10,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "PATIENT_LAST_NAME_0445",
      "datatype": "CHAR",
      "length": 15,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ],
  {

```

```

    "name": "PATIENT_SEX_0445",
    "datatype": "CHAR",
    "length": 1,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "RELATION_TO_EMPLOYEE_0445",
    "datatype": "CHAR",
    "length": 10,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_FIRST_NAME_0445",
    "datatype": "CHAR",
    "length": 10,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_LAST_NAME_0445",
    "datatype": "CHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_STREET_0445",
    "datatype": "CHAR",
    "length": 20,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_CITY_0445",
    "datatype": "CHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_STATE_0445",
    "datatype": "CHAR",
    "length": 2,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_ZIP_FIRST_FIVE_0445",
    "datatype": "CHAR",
    "length": 5,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_ZIP_LAST_FOUR_0445",
    "datatype": "CHAR",
    "length": 4,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_ID_0445",
    "datatype": "DECIMAL",
    "length": 6,
    "scale": 0,

```

```

        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DIAGNOSIS_0445",
        "datatype": null,
        "length": 120,
        "scale": null,
        "occurs": true,
        "occurs_max": 2
    },
    {
        "name": "NUMBER_OF_PROCEDURES_0445",
        "datatype": "DECIMAL",
        "length": 2,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "PROCEDURE_CODE_0445",
        "datatype": "DECIMAL",
        "length": 4,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DESCRIPTION_OF_SERVICE_0445",
        "datatype": "CHAR",
        "length": 60,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "FEE_0445",
        "datatype": "DECIMAL",
        "length": 5,
        "scale": 2,
        "occurs": false,
        "occurs_max": null
    }
}
],
{
    "name": "OFFICE",
    "primary_key": "OFFICE_CODE_0450",
    "primary_keys": [
        "OFFICE_CODE_0450"
    ],
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "OFFICE_CODE_0450",
            "datatype": "CHAR",
            "length": 3,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "OFFICE_STREET_0450",
            "datatype": "CHAR",
            "length": 20,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "OFFICE_CITY_0450",
            "datatype": "CHAR",
            "length": 15,
            "scale": null,
            "occurs": false,

```

```

    "occurs_max": null
  },
  {
    "name": "OFFICE_STATE_0450",
    "datatype": "CHAR",
    "length": 2,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "OFFICE_ZIP_FIRST_FIVE_0450",
    "datatype": "CHAR",
    "length": 5,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "OFFICE_ZIP_LAST_FOUR_0450",
    "datatype": "CHAR",
    "length": 4,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "OFFICE_PHONE_0450",
    "datatype": null,
    "length": 21,
    "scale": null,
    "occurs": true,
    "occurs_max": 3
  },
  {
    "name": "OFFICE_AREA_CODE_0450",
    "datatype": "CHAR",
    "length": 3,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "SPEED_DIAL_0450",
    "datatype": "CHAR",
    "length": 3,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  }
]
},
{
  "name": "SKILL",
  "primary_key": "SKILL_ID_0455",
  "primary_keys": [
    "SKILL_ID_0455"
  ],
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "SKILL_ID_0455",
      "datatype": "DECIMAL",
      "length": 4,
      "scale": 0,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "SKILL_NAME_0455",
      "datatype": "CHAR",
      "length": 12,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ]
}

```

```

    },
    {
      "name": "SKILL_DESCRIPTION_0455",
      "datatype": "CHAR",
      "length": 60,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ]
},
{
  "name": "STRUCTURE",
  "primary_key": null,
  "primary_keys": [],
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "STRUCTURE_DATE_0460",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "STRUCTURE_CODE_0460",
      "datatype": "CHAR",
      "length": 2,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ]
}
],
"sets": [
  {
    "name": "COVERAGE_CLAIMS",
    "owner_record": "COVERAGE",
    "member_record": "DENTAL_CLAIM",
    "order_mode": null
  },
  {
    "name": "COVERAGE_CLAIMS",
    "owner_record": "COVERAGE",
    "member_record": "HOSPITAL_CLAIM",
    "order_mode": null
  },
  {
    "name": "COVERAGE_CLAIMS",
    "owner_record": "COVERAGE",
    "member_record": "NON_HOSP_CLAIM",
    "order_mode": null
  },
  {
    "name": "DEPT_EMPLOYEE",
    "owner_record": "DEPARTMENT",
    "member_record": "EMPLOYEE",
    "order_mode": null
  },
  {
    "name": "EMP_COVERAGE",
    "owner_record": "EMPLOYEE",
    "member_record": "COVERAGE",
    "order_mode": null
  },
  {
    "name": "EMP_EMPOSITION",
    "owner_record": "EMPLOYEE",
    "member_record": "EMPOSITION",
    "order_mode": null
  },
  {
    "name": "EMP_EXPERTISE",

```



```

    "owner_record": "EMPLOYEE",
    "member_record": "EXPERTISE",
    "order_mode": null
  },
  {
    "name": "JOB_EMPOSITION",
    "owner_record": "JOB",
    "member_record": "EMPOSITION",
    "order_mode": null
  },
  {
    "name": "MANAGES",
    "owner_record": "EMPLOYEE",
    "member_record": "STRUCTURE",
    "order_mode": null
  },
  {
    "name": "OFFICE_EMPLOYEE",
    "owner_record": "OFFICE",
    "member_record": "EMPLOYEE",
    "order_mode": null
  },
  {
    "name": "REPORTS_TO",
    "owner_record": "EMPLOYEE",
    "member_record": "STRUCTURE",
    "order_mode": null
  },
  {
    "name": "SKILL_EXPERTISE",
    "owner_record": "SKILL",
    "member_record": "EXPERTISE",
    "order_mode": null
  }
],
"relationships": [
  {
    "set_name": "COVERAGE_CLAIMS",
    "parent_record": "COVERAGE",
    "child_record": "DENTAL_CLAIM",
    "cardinality": "1:N"
  },
  {
    "set_name": "COVERAGE_CLAIMS",
    "parent_record": "COVERAGE",
    "child_record": "HOSPITAL_CLAIM",
    "cardinality": "1:N"
  },
  {
    "set_name": "COVERAGE_CLAIMS",
    "parent_record": "COVERAGE",
    "child_record": "NON_HOSP_CLAIM",
    "cardinality": "1:N"
  },
  {
    "set_name": "DEPT_EMPLOYEE",
    "parent_record": "DEPARTMENT",
    "child_record": "EMPLOYEE",
    "cardinality": "1:N"
  },
  {
    "set_name": "EMP_COVERAGE",
    "parent_record": "EMPLOYEE",
    "child_record": "COVERAGE",
    "cardinality": "1:N"
  },
  {
    "set_name": "EMP_EMPOSITION",
    "parent_record": "EMPLOYEE",
    "child_record": "EMPOSITION",
    "cardinality": "1:N"
  },
  {
    "set_name": "EMP_EXPERTISE",
    "parent_record": "EMPLOYEE",
    "child_record": "EXPERTISE",

```

```

    "cardinality": "1:N"
  },
  {
    "set_name": "JOB_EMPOSITION",
    "parent_record": "JOB",
    "child_record": "EMPOSITION",
    "cardinality": "1:N"
  },
  {
    "set_name": "MANAGES",
    "parent_record": "EMPLOYEE",
    "child_record": "STRUCTURE",
    "cardinality": "1:N"
  },
  {
    "set_name": "OFFICE_EMPLOYEE",
    "parent_record": "OFFICE",
    "child_record": "EMPLOYEE",
    "cardinality": "1:N"
  },
  {
    "set_name": "REPORTS_TO",
    "parent_record": "EMPLOYEE",
    "child_record": "STRUCTURE",
    "cardinality": "1:N"
  },
  {
    "set_name": "SKILL_EXPERTISE",
    "parent_record": "SKILL",
    "child_record": "EXPERTISE",
    "cardinality": "1:N"
  }
}
]
}

```

```

=====
FILE: data\output\converted_db2_cobol.cb1
ENCODING: utf-8
=====

```

```

*RETRIEVAL
*DMLIST
*NO-ACTIVITY-LOG
*SCHEMA-COMMENTS
IDENTIFICATION DIVISION.
PROGRAM-ID. DEPTRPT.
AUTHOR. CA.
DATE-WRITTEN. APRIL 1995.
REMARKS. THIS PROGRAM DEMONSTRATES
CA IDMS DATABASE ACCESS USING
COBOL DML STATEMENTS. IT READS
DEPARTMENT ID NUMBERS AND RETRIEVES
RELATED RECORD OCCURRENCES,
PRINTING A REPORT THAT INCLUDES
DEPARTMENT, EMPLOYEE, JOB, AND
OFFICE INFORMATION.
*****
ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
SELECT DEPT-FILE-IN ASSIGN TO INFILE.
SELECT DEPT-FILE-OUT ASSIGN TO OUTFILE.
SELECT ERR-FILE-OUT ASSIGN TO ERRFILE.
*****

DATA DIVISION.

FILE SECTION.
FD DEPT-FILE-IN
RECORD CONTAINS 80
BLOCK CONTAINS 80 CHARACTERS
RECORDING MODE IS F
LABEL RECORDS ARE OMITTED.
01 DEPT-REC-IN.
02 DEPT-ID-IN PIC 9(4).

```

```

02 DEPT-IN-FILLER PIC X(76).
FD DEPT-FILE-OUT
RECORD CONTAINS 133
BLOCK CONTAINS 133 CHARACTERS
RECORDING MODE IS F
LABEL RECORDS ARE OMITTED.
01 DEPT-REC-OUT.
02 CC PIC X.
02 PRINT-LINE PIC X(132).
FD ERR-FILE-OUT
RECORD CONTAINS 133
BLOCK CONTAINS 133 CHARACTERS
RECORDING MODE IS F
LABEL RECORDS ARE OMITTED.
01 ERR-REC-OUT.
02 ERR-CC PIC X.
02 ERR-LINE PIC X(132).
*****
WORKING-STORAGE SECTION.
01 EOF-SW PIC X VALUE 'N'.
88 END-OF-FILE VALUE 'Y'.
01 LINE-COUNT PIC 99 VALUE 0.
01 ERR-LINE-COUNT PIC 99 VALUE 0.
01 LINE-MAX PIC 99 VALUE 50.
*****
01 DEPT-HEADER.
05 FILLER PIC X(30) VALUE SPACES.
05 FILLER PIC X(13) VALUE 'DEPARTMENT ID'.
05 FILLER PIC X(10) VALUE SPACES.
05 FILLER PIC X(9) VALUE 'DEPT NAME'.
05 FILLER PIC X(70) VALUE SPACES.
01 DEPT-DETAIL-LINE.
05 FILLER PIC X(33) VALUE SPACES.
05 DEPT-ID-OUT PIC X(4).
05 FILLER PIC X(16) VALUE SPACES.
05 DEPT-NAME-OUT PIC X(45).
05 FILLER PIC X(34) VALUE SPACES.
01 EMP-HEADER.
05 FILLER PIC X(5) VALUE SPACES.
05 FILLER PIC X(6) VALUE 'EMP ID'.
05 FILLER PIC X(2) VALUE SPACES.
05 FILLER PIC X(9) VALUE 'LAST NAME'.
05 FILLER PIC X(8) VALUE SPACES.
05 FILLER PIC X(10) VALUE 'FIRST NAME'.
05 FILLER PIC X(3) VALUE SPACES.
05 FILLER PIC X(10) VALUE 'START DATE'.
05 FILLER PIC X(2) VALUE SPACES.
05 FILLER PIC X(9) VALUE 'JOB TITLE'.
05 FILLER PIC X(13) VALUE SPACES.
05 FILLER PIC X(14) VALUE 'OFFICE ADDRESS'.
05 FILLER PIC X(42) VALUE SPACES.
01 EMP-DETAIL-LINE.
05 FILLER PIC X(5) VALUE SPACES.
05 ID-OUT PIC X(4).
05 FILLER PIC X(4) VALUE SPACES.
05 LAST-OUT PIC X(15).
05 FILLER PIC X(2) VALUE SPACES.
05 FIRST-OUT PIC X(10).
05 FILLER PIC X(3) VALUE SPACES.
05 SD-OUT.
10 SD-MM PIC XX.
10 FILLER PIC X VALUE '/'.
10 SD-DD PIC XX.
10 FILLER PIC X VALUE '/'.
10 SD-YY PIC XX.
05 FILLER PIC X(4) VALUE SPACES.
05 TITLE-OUT PIC X(20).
05 FILLER PIC X(2) VALUE SPACES.
05 OFF-ADDRESS-OUT.
10 STREET-OUT PIC X(20).
10 FILLER PIC XX VALUE SPACES.
10 CITY-OUT PIC X(15).
10 FILLER PIC XX VALUE SPACES.
10 STATE-OUT PIC XX.
10 FILLER PIC XX VALUE SPACES.
10 ZIP-OUT PIC X(5).

```

```

05 FILLER PIC X(8) VALUE SPACES.
01 ERR-HEADER-1.
05 FILLER PIC X(40) VALUE SPACES.
05 FILLER PIC X(12) VALUE 'ERROR REPORT'.
05 FILLER PIC X(80) VALUE SPACES.
01 ERR-HEADER-2.
05 FILLER PIC X(10) VALUE SPACES.
05 FILLER PIC X(4) VALUE '*** '.
05 FILLER PIC X(51)
VALUE 'THIS REPORT LISTS EMPTY AND NONEXISTENT DEPARTMENTS'.
05 FILLER PIC X(4) VALUE ' ***'.
05 FILLER PIC X(63) VALUE SPACES.
01 ERR-HEADER-3.
05 FILLER PIC X(20) VALUE SPACES.
05 FILLER PIC X(7) VALUE 'DEPT ID'.
05 FILLER PIC X(9) VALUE SPACES.
05 FILLER PIC X(7) VALUE 'MESSAGE'.
05 FILLER PIC X(89) VALUE SPACES.
01 ERR-DETAIL-LINE.
05 FILLER PIC X(20) VALUE SPACES.
05 ERR-ID-OUT PIC X(4).
05 FILLER PIC X(12) VALUE SPACES.
05 ERR-MESS-OUT PIC X(15).
05 FILLER PIC X(79) VALUE SPACES.
*****
01 MESSAGES.
05 NO-JOB-MESSAGE.
10 FILLER PIC X(20)
VALUE 'NO JOB ASSIGNED'.
05 NO-OFFICE-MESSAGE.
10 FILLER PIC X(20)
VALUE 'NO OFFICE ASSIGNED'.
05 NO-DEPT-MESSAGE.
10 FILLER PIC X(15)
VALUE 'DOES NOT EXIST'.
05 NO-EMP-MESSAGE.
10 FILLER PIC X(15)
VALUE 'IS EMPTY'.
05 NO-INPUT-MESSAGE.
10 FILLER PIC XX VALUE SPACES.
10 FILLER PIC X(11)
VALUE '=====> '.
10 FILLER PIC X(8)
VALUE 'NO INPUT'.
10 FILLER PIC X(11)
VALUE ' <====='.
10 FILLER PIC X(100)
VALUE SPACES.

EJECT
*****
* DB2 SQLCA, HOST VARIABLES, AND CURSORS
*****

EXEC SQL
    INCLUDE SQLCA
END-EXEC.
EXEC SQL
    BEGIN DECLARE SECTION
END-EXEC.

01 HV-DEPARTMENT.
    05 HV-DEPARTMENT-DEPT-ID-479MENT          PIC S9(4) COMP-3.
    05 HV-DEPARTMENT-DEPT-NAME-479MENT        PIC X(45).
    05 HV-DEPARTMENT-DEPT-HEADID-479MENT      PIC S9(4) COMP-3.

01 HV-EMPLOYEE.
    05 HV-EMPLOYEE-DA-STARTDATE-479OYEE      PIC X(10).
    05 HV-EMPLOYEE-DA-TERMINATIONDATE-479OYEE PIC X(10).
    05 HV-EMPLOYEE-DA-BIRTHDATE-479OYEE      PIC X(10).
    05 HV-EMPLOYEE-EMP-ID-479OYEE            PIC S9(4) COMP-3.
    05 HV-EMPLOYEE-EMP-FIRSTNAME-479OYEE     PIC X(10).
    05 HV-EMPLOYEE-EMP-LASTNAME-479OYEE      PIC X(15).
    05 HV-EMPLOYEE-EMP-STREET-479OYEE        PIC X(20).
    05 HV-EMPLOYEE-EMP-CITY-479OYEE          PIC X(15).
    05 HV-EMPLOYEE-EMP-STATE-479OYEE         PIC X(2).

```

05 HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE	PIC X(5).
05 HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE	PIC X(4).
05 HV-EMPLOYEE-EMP-PHONE-4790YEE	PIC S9(10) COMP-3.
05 HV-EMPLOYEE-STATUS-4790YEE	PIC X(2).
05 HV-EMPLOYEE-SS-NUMBER-4790YEE	PIC S9(9) COMP-3.
05 HV-EMPLOYEE-DEPT-ID-479MENT	PIC S9(4) COMP-3.
05 HV-EMPLOYEE-OFFICE-CODE-479FICE	PIC X(3).
01 HV-EMPOSITION.	
05 HV-EMPOSITION-ID-RECORD-TION	PIC X(20).
05 HV-EMPOSITION-DA-STARTDATE-479TION	PIC X(10).
05 HV-EMPOSITION-DA-FINISHDATE-479TION	PIC X(10).
05 HV-EMPOSITION-SALARY-GRADE-479TION	PIC S9(2) COMP-3.
05 HV-EMPOSITION-SALARY-AMOUNT-479TION	PIC S9(3)V9(2) COMP-3.
05 HV-EMPOSITION-BONUS-PERCENT-479TION	PIC S9(2) COMP-3.
05 HV-EMPOSITION-COMMISSION-PERCENT-479TION	PIC S9(2) COMP-3.
05 HV-EMPOSITION-OVERTIME-RATE-479TION	PIC S9(1)V9(2) COMP-3.
05 HV-EMPOSITION-EMP-ID-4790YEE	PIC S9(4) COMP-3.
05 HV-EMPOSITION-JOB-ID-479JOB	PIC S9(4) COMP-3.
01 HV-JOB.	
05 HV-JOB-JOB-ID-479JOB	PIC S9(4) COMP-3.
05 HV-JOB-TITLE-479JOB	PIC X(20).
05 HV-JOB-DESCRIPTION-LINE-479JOB	PIC X(120).
05 HV-JOB-REQUIREMENT-LINE-479JOB	PIC X(120).
05 HV-JOB-MINIMUM-SALARY-479JOB	PIC S9(6)V9(2) COMP-3.
05 HV-JOB-MAXIMUM-SALARY-479JOB	PIC S9(6)V9(2) COMP-3.
05 HV-JOB-SALARY-GRADES-479JOB	PIC X(8).
05 HV-JOB-NUMBER-OFPOSITIONS-479JOB	PIC S9(3) COMP-3.
05 HV-JOB-NUMBER-OPEN-479JOB	PIC S9(3) COMP-3.
01 HV-OFFICE.	
05 HV-OFFICE-OFFICE-CODE-479FICE	PIC X(3).
05 HV-OFFICE-OFFICE-STREET-479FICE	PIC X(20).
05 HV-OFFICE-OFFICE-CITY-479FICE	PIC X(15).
05 HV-OFFICE-OFFICE-STATE-479FICE	PIC X(2).
05 HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE	PIC X(5).
05 HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE	PIC X(4).
05 HV-OFFICE-OFFICE-PHONE-479FICE	PIC X(21).
05 HV-OFFICE-OFFICE-AREACODE-479FICE	PIC X(3).
05 HV-OFFICE-SPEED-DIAL-479FICE	PIC X(3).
01 NI-DEPARTMENT-DEPT-NAME-479MENT	PIC S9(4) COMP.
01 NI-DEPARTMENT-DEPT-HEADID-479MENT	PIC S9(4) COMP.
01 NI-EMPLOYEE-DA-STARTDATE-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-DA-TERMINATIONDATE-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-DA-BIRTHDATE-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-FIRSTNAME-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-LASTNAME-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-STREET-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-CITY-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-STATE-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-PHONE-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-STATUS-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-SS-NUMBER-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-DEPT-ID-479MENT	PIC S9(4) COMP.
01 NI-EMPLOYEE-OFFICE-CODE-479FICE	PIC S9(4) COMP.
01 NI-EMPOSITION-DA-STARTDATE-479TION	PIC S9(4) COMP.
01 NI-EMPOSITION-DA-FINISHDATE-479TION	PIC S9(4) COMP.
01 NI-EMPOSITION-SALARY-GRADE-479TION	PIC S9(4) COMP.
01 NI-EMPOSITION-SALARY-AMOUNT-479TION	PIC S9(4) COMP.
01 NI-EMPOSITION-BONUS-PERCENT-479TION	PIC S9(4) COMP.
01 NI-EMPOSITION-COMMISSION-PERCENT-479TION	PIC S9(4) COMP.
01 NI-EMPOSITION-OVERTIME-RATE-479TION	PIC S9(4) COMP.
01 NI-EMPOSITION-EMP-ID-4790YEE	PIC S9(4) COMP.
01 NI-EMPOSITION-JOB-ID-479JOB	PIC S9(4) COMP.
01 NI-JOB-TITLE-479JOB	PIC S9(4) COMP.
01 NI-JOB-DESCRIPTION-LINE-479JOB	PIC S9(4) COMP.
01 NI-JOB-REQUIREMENT-LINE-479JOB	PIC S9(4) COMP.
01 NI-JOB-MINIMUM-SALARY-479JOB	PIC S9(4) COMP.
01 NI-JOB-MAXIMUM-SALARY-479JOB	PIC S9(4) COMP.
01 NI-JOB-SALARY-GRADES-479JOB	PIC S9(4) COMP.
01 NI-JOB-NUMBER-OFPOSITIONS-479JOB	PIC S9(4) COMP.
01 NI-JOB-NUMBER-OPEN-479JOB	PIC S9(4) COMP.

```

01 NI-OFFICE-OFFICE-STREET-479FICE          PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-CITY-479FICE            PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-STATE-479FICE          PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE    PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-ZIPLASTFOUR-479FICE     PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-PHONE-479FICE          PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-AREACODE-479FICE        PIC S9(4) COMP.
01 NI-OFFICE-SPEED-DIAL-479FICE            PIC S9(4) COMP.

```

EXEC SQL

END DECLARE SECTION

END-EXEC.

EXEC SQL

DECLARE C-DEPT-EMPLOYEE CURSOR FOR

```

SELECT DA_STARTDATE_479OYEE,
       DA_TERMINATIONDATE_479OYEE,
       DA_BIRTHDATE_479OYEE,
       EMP_ID_479OYEE,
       EMP_FIRSTNAME_479OYEE,
       EMP_LASTNAME_479OYEE,
       EMP_STREET_479OYEE,
       EMP_CITY_479OYEE,
       EMP_STATE_479OYEE,
       EMP_ZIPFIRSTFIVE_479OYEE,
       EMP_ZIPLASTFOUR_479OYEE,
       EMP_PHONE_479OYEE,
       STATUS_479OYEE,
       SS_NUMBER_479OYEE,
       DEPT_ID_479MENT,
       OFFICE_CODE_479FICE

```

FROM EMPLOYEE

WHERE DEPT_ID_479MENT = :HV-DEPARTMENT-DEPT-ID-479MENT

ORDER BY DEPT_ID_479MENT

END-EXEC.

PROCEDURE DIVISION.

* PROCEDURE DIVISION GENERAL STRATEGY: *

* *

* 1) READ DEPT-ID-IN, WHICH CONTAINS THE *

* DEPT-ID NUMBER *

* *

* 2) ACCESS THE DATABASE USING THE DEPT-ID NUMBER *

* WITH AN OBTAIN CALC ON THE DEPARTMENT RECORD *

* *

* 3) ACCESS ALL EMPLOYEES IN THE DEPT-EMPLOYEE SET *

* AND RETRIEVE RELATED JOB AND OFFICE DATA *

* *

* 4) PRINT A REPORT FOR EACH DEPARTMENT *

* *

* 5) PRINT AN ERROR REPORT FOR EMPTY DEPARTMENTS *

* AND NONEXISTENT DEPARTMENTS (NO MATCHING *

* DEPT-ID) *

FETCH-DEPT-EMPLOYEE.

EXEC SQL

FETCH C-DEPT-EMPLOYEE

```

INTO :HV-EMPLOYEE-DA-STARTDATE-479OYEE :NI-EMPLOYEE-DA-STARTDATE-479OYEE,
     :HV-EMPLOYEE-DA-TERMINATIONDATE-479OYEE :NI-EMPLOYEE-DA-TERMINATIONDATE-479OYEE,
     :HV-EMPLOYEE-DA-BIRTHDATE-479OYEE :NI-EMPLOYEE-DA-BIRTHDATE-479OYEE,
     :HV-EMPLOYEE-EMP-ID-479OYEE,
     :HV-EMPLOYEE-EMP-FIRSTNAME-479OYEE :NI-EMPLOYEE-EMP-FIRSTNAME-479OYEE,
     :HV-EMPLOYEE-EMP-LASTNAME-479OYEE :NI-EMPLOYEE-EMP-LASTNAME-479OYEE,
     :HV-EMPLOYEE-EMP-STREET-479OYEE :NI-EMPLOYEE-EMP-STREET-479OYEE,
     :HV-EMPLOYEE-EMP-CITY-479OYEE :NI-EMPLOYEE-EMP-CITY-479OYEE,
     :HV-EMPLOYEE-EMP-STATE-479OYEE :NI-EMPLOYEE-EMP-STATE-479OYEE,
     :HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-479OYEE :NI-EMPLOYEE-EMP-ZIPFIRSTFIVE-479OYEE,
     :HV-EMPLOYEE-EMP-ZIPLASTFOUR-479OYEE :NI-EMPLOYEE-EMP-ZIPLASTFOUR-479OYEE,
     :HV-EMPLOYEE-EMP-PHONE-479OYEE :NI-EMPLOYEE-EMP-PHONE-479OYEE,
     :HV-EMPLOYEE-STATUS-479OYEE :NI-EMPLOYEE-STATUS-479OYEE,
     :HV-EMPLOYEE-SS-NUMBER-479OYEE :NI-EMPLOYEE-SS-NUMBER-479OYEE,
     :HV-EMPLOYEE-DEPT-ID-479MENT :NI-EMPLOYEE-DEPT-ID-479MENT,
     :HV-EMPLOYEE-OFFICE-CODE-479FICE :NI-EMPLOYEE-OFFICE-CODE-479FICE

```

END-EXEC.

IF SQLCODE NOT = 0 AND SQLCODE NOT = 100

PERFORM SQL-ERROR

END-IF.

```

MAIN-LINE.
    PERFORM INIT-FILES.
    IF END-OF-FILE
        PERFORM EMPTY-INPUT-PROCESSING
    ELSE
        PERFORM INIT-DB2-SETUP
        PERFORM U220-ERR-HEADER
        PERFORM DEPT-PROCESSING THRU DEPT-PROCESSING-EXIT
        UNTIL END-OF-FILE.
        PERFORM END-PROCESSING.
        GOBACK.
INIT-DB2-SETUP.
    CONTINUE.
INIT-FILES.
    OPEN INPUT DEPT-FILE-IN.
    OPEN OUTPUT DEPT-FILE-OUT.
    OPEN OUTPUT ERR-FILE-OUT.
    MOVE SPACES TO PRINT-LINE.
    MOVE SPACES TO ERR-LINE.
    READ DEPT-FILE-IN
        AT END MOVE 'Y' TO EOF-SW.
EMPTY-INPUT-PROCESSING.
    MOVE NO-INPUT-MESSAGE TO PRINT-LINE.
    MOVE '1' TO CC.
    PERFORM U000-WRITE-LINE.
*****
* THIS PARAGRAPH ACCESSES THE DATABASE USING THE DEPT-ID-0415 *
* CALCKEY VALUE. *
*****
DEPT-PROCESSING.
    MOVE DEPT-ID-IN TO HV-DEPARTMENT-DEPT-ID-479MENT.
    EXEC SQL
        SELECT DEPT_ID_479MENT,
            DEPT_NAME_479MENT,
            DEPT_HEADID_479MENT
        INTO :HV-DEPARTMENT-DEPT-ID-479MENT,
            :HV-DEPARTMENT-DEPT-NAME-479MENT :NI-DEPARTMENT-DEPT-NAME-479MENT,
            :HV-DEPARTMENT-DEPT-HEADID-479MENT :NI-DEPARTMENT-DEPT-HEADID-479MENT
        FROM DEPARTMENT
        WHERE DEPT_ID_479MENT = :HV-DEPARTMENT-DEPT-ID-479MENT
    END-EXEC.
    IF SQLCODE = 100
        PERFORM NO-DEPT-PROCESSING
    ELSE IF SQLCODE NOT = 0
        PERFORM SQL-ERROR
    ELSE
        EXEC SQL
            OPEN C-DEPT-EMPLOYEE
        END-EXEC.
        IF SQLCODE NOT = 0
            PERFORM SQL-ERROR
        ELSE
            PERFORM FETCH-DEPT-EMPLOYEE

    IF SQLCODE = 100
        PERFORM EMPTY-SET
    ELSE
        PERFORM U020-VALID-HEADER
        MOVE HV-DEPARTMENT-DEPT-ID-479MENT TO DEPT-ID-OUT
        MOVE HV-DEPARTMENT-DEPT-NAME-479MENT TO DEPT-NAME-OUT
        MOVE DEPT-DETAIL-LINE TO PRINT-LINE
        PERFORM U000-WRITE-LINE
        PERFORM U030-EMP-HEADERS

    PERFORM UNTIL SQLCODE = 100
        PERFORM SET-WALK THRU SET-WALK-EXIT
        PERFORM FETCH-DEPT-EMPLOYEE
    END-PERFORM.
    END-IF.
    EXEC SQL
        CLOSE C-DEPT-EMPLOYEE
    END-EXEC.
    END-IF.
    END-IF.
    END-IF.
    READ DEPT-FILE-IN

```

AT END MOVE 'Y' TO EOF-SW.
DEPT-PROCESSING-EXIT.
EXIT.

* THIS PARAGRAPH RETRIEVES EMPLOYEE, JOB, AND OFFICE DATA *
* FOR EACH EMPLOYEE IN THE DEPT-EMPLOYEE SET. *

SET-WALK.

CONTINUE.

MOVE HV-EMPLOYEE-EMP-ID-4790YEE TO ID-OUT.
MOVE HV-EMPLOYEE-EMP-LASTNAME-4790YEE TO LAST-OUT.
MOVE HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE TO FIRST-OUT.
MOVE HV-EMPLOYEE-DA-STARTDATE-4790YEE(3:2) TO SD-YY.
MOVE HV-EMPLOYEE-DA-STARTDATE-4790YEE(6:2) TO SD-MM.
MOVE HV-EMPLOYEE-DA-STARTDATE-4790YEE(9:2) TO SD-DD.

EXEC SQL

SELECT ID_RECORD_TION,
DA_STARTDATE_479TION,
DA_FINISHDATE_479TION,
SALARY_GRADE_479TION,
SALARY_AMOUNT_479TION,
BONUS_PERCENT_479TION,
COMMISSION_PERCENT_479TION,
OVERTIME_RATE_479TION,
EMP_ID_4790YEE,
JOB_ID_479JOB
INTO :HV-EMPOSITION-ID-RECORD-TION,
:HV-EMPOSITION-DA-STARTDATE-479TION :NI-EMPOSITION-DA-STARTDATE-479TION,
:HV-EMPOSITION-DA-FINISHDATE-479TION :NI-EMPOSITION-DA-FINISHDATE-479TION,
:HV-EMPOSITION-SALARY-GRADE-479TION :NI-EMPOSITION-SALARY-GRADE-479TION,
:HV-EMPOSITION-SALARY-AMOUNT-479TION :NI-EMPOSITION-SALARY-AMOUNT-479TION,
:HV-EMPOSITION-BONUS-PERCENT-479TION :NI-EMPOSITION-BONUS-PERCENT-479TION,
:HV-EMPOSITION-COMMISSION-PERCENT-479TION :NI-EMPOSITION-COMMISSION-PERCENT-479TION,
:HV-EMPOSITION-OVERTIME-RATE-479TION :NI-EMPOSITION-OVERTIME-RATE-479TION,
:HV-EMPOSITION-EMP-ID-4790YEE :NI-EMPOSITION-EMP-ID-4790YEE,
:HV-EMPOSITION-JOB-ID-479JOB :NI-EMPOSITION-JOB-ID-479JOB
FROM EMPOSITION
WHERE EMP_ID_4790YEE = :HV-EMPLOYEE-EMP-ID-4790YEE
ORDER BY EMP_ID_4790YEE
FETCH FIRST 1 ROW ONLY

END-EXEC.

IF SQLCODE = 100

MOVE NO-JOB-MESSAGE TO TITLE-OUT

ELSE IF SQLCODE NOT = 0

PERFORM SQL-ERROR

ELSE

EXEC SQL

SELECT JOB_ID_479JOB,
TITLE_479JOB,
DESCRIPTION_LINE_479JOB,
REQUIREMENT_LINE_479JOB,
MINIMUM_SALARY_479JOB,
MAXIMUM_SALARY_479JOB,
SALARY_GRADES_479JOB,
NUMBER_OFPOSITIONS_479JOB,
NUMBER_OPEN_479JOB
INTO :HV-JOB-JOB-ID-479JOB,
:HV-JOB-TITLE-479JOB :NI-JOB-TITLE-479JOB,
:HV-JOB-DESCRIPTION-LINE-479JOB :NI-JOB-DESCRIPTION-LINE-479JOB,
:HV-JOB-REQUIREMENT-LINE-479JOB :NI-JOB-REQUIREMENT-LINE-479JOB,
:HV-JOB-MINIMUM-SALARY-479JOB :NI-JOB-MINIMUM-SALARY-479JOB,
:HV-JOB-MAXIMUM-SALARY-479JOB :NI-JOB-MAXIMUM-SALARY-479JOB,
:HV-JOB-SALARY-GRADES-479JOB :NI-JOB-SALARY-GRADES-479JOB,
:HV-JOB-NUMBER-OFPOSITIONS-479JOB :NI-JOB-NUMBER-OFPOSITIONS-479JOB,
:HV-JOB-NUMBER-OPEN-479JOB :NI-JOB-NUMBER-OPEN-479JOB
FROM JOB
WHERE JOB_ID_479JOB = :HV-EMPOSITION-JOB-ID-479JOB

END-EXEC.

IF SQLCODE = 100

MOVE NO-JOB-MESSAGE TO TITLE-OUT

ELSE IF SQLCODE NOT = 0

PERFORM SQL-ERROR

ELSE

CONTINUE.

END-IF.

END-IF.


```

END-IF.
END-IF.
MOVE HV-JOB-TITLE-479JOB TO TITLE-OUT.
IF NI-EMPLOYEE-OFFICE-CODE-479FICE < 0
    MOVE NO-OFFICE-MESSAGE TO STREET-OUT
MOVE SPACES TO CITY-OUT
MOVE SPACES TO STATE-OUT
MOVE SPACES TO ZIP-OUT
ELSE
EXEC SQL
    SELECT OFFICE_CODE_479FICE,
           OFFICE_STREET_479FICE,
           OFFICE_CITY_479FICE,
           OFFICE_STATE_479FICE,
           OFFICE_ZIPFIRSTFIVE_479FICE,
           OFFICE_ZIPLASTFOUR_479FICE,
           OFFICE_PHONE_479FICE,
           OFFICE_AREACODE_479FICE,
           SPEED_DIAL_479FICE
    INTO :HV-OFFICE-OFFICE-CODE-479FICE,
         :HV-OFFICE-OFFICE-STREET-479FICE :NI-OFFICE-OFFICE-STREET-479FICE,
         :HV-OFFICE-OFFICE-CITY-479FICE :NI-OFFICE-OFFICE-CITY-479FICE,
         :HV-OFFICE-OFFICE-STATE-479FICE :NI-OFFICE-OFFICE-STATE-479FICE,
         :HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE :NI-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE,
         :HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE :NI-OFFICE-OFFICE-ZIPLASTFOUR-479FICE,
         :HV-OFFICE-OFFICE-PHONE-479FICE :NI-OFFICE-OFFICE-PHONE-479FICE,
         :HV-OFFICE-OFFICE-AREACODE-479FICE :NI-OFFICE-OFFICE-AREACODE-479FICE,
         :HV-OFFICE-SPEED-DIAL-479FICE :NI-OFFICE-SPEED-DIAL-479FICE
    FROM OFFICE
    WHERE OFFICE_CODE_479FICE = :HV-EMPLOYEE-OFFICE-CODE-479FICE
END-EXEC.
IF SQLCODE = 0
    CONTINUE.
ELSE
    MOVE NO-OFFICE-MESSAGE TO STREET-OUT
MOVE SPACES TO CITY-OUT
MOVE SPACES TO STATE-OUT
MOVE SPACES TO ZIP-OUT
END-IF.
END-IF.
MOVE HV-OFFICE-OFFICE-STREET-479FICE TO STREET-OUT
MOVE HV-OFFICE-OFFICE-CITY-479FICE TO CITY-OUT
MOVE HV-OFFICE-OFFICE-STATE-479FICE TO STATE-OUT
MOVE HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE TO ZIP-OUT
MOVE EMP-DETAIL-LINE TO PRINT-LINE.
PERFORM U000-WRITE-LINE.
SET-WALK-EXIT.
EXIT.
END-PROCESSING.
CLOSE DEPT-FILE-OUT.
CLOSE ERR-FILE-OUT.
CLOSE DEPT-FILE-IN.
EMPTY-SET.
MOVE SPACES TO ERR-LINE.
MOVE HV-DEPARTMENT-DEPT-ID-479MENT TO ERR-ID-OUT.
MOVE NO-EMP-MESSAGE TO ERR-MESS-OUT.
MOVE ERR-DETAIL-LINE TO ERR-LINE.
PERFORM U200-WRITE-ERR-LINE.
NO-DEPT-PROCESSING.
MOVE DEPT-ID-IN TO ERR-ID-OUT.
MOVE NO-DEPT-MESSAGE TO ERR-MESS-OUT.
MOVE ERR-DETAIL-LINE TO ERR-LINE.
PERFORM U200-WRITE-ERR-LINE.
U000-WRITE-LINE.
WRITE DEPT-REC-OUT AFTER POSITIONING CC.
IF CC = '1'
    THEN MOVE 0 TO LINE-COUNT
ELSE IF CC = ' '
    THEN ADD 1 TO LINE-COUNT
ELSE IF CC = '0'
    THEN ADD 2 TO LINE-COUNT.
IF LINE-COUNT > LINE-MAX
    THEN PERFORM U010-NEW-PAGE-ROUTINE.
U010-NEW-PAGE-ROUTINE.
PERFORM U020-VALID-HEADER.
MOVE DEPT-DETAIL-LINE TO PRINT-LINE.

```

```

        PERFORM U000-WRITE-LINE.
        PERFORM U030-EMP-HEADERS.
U020-VALID-HEADER.
        MOVE DEPT-HEADER TO PRINT-LINE.
        MOVE '1' TO CC.
        PERFORM U000-WRITE-LINE.
        MOVE ' ' TO CC.
U030-EMP-HEADERS.
        MOVE '0' TO CC.
        MOVE EMP-HEADER TO PRINT-LINE.
        PERFORM U000-WRITE-LINE.
        MOVE SPACES TO PRINT-LINE.
        MOVE ' ' TO CC.
        PERFORM U000-WRITE-LINE.
U200-WRITE-ERR-LINE.
        WRITE ERR-REC-OUT AFTER POSITIONING ERR-CC.
        IF ERR-CC = '1'
            THEN MOVE 0 TO ERR-LINE-COUNT
        ELSE IF ERR-CC = ' '
            THEN ADD 1 TO ERR-LINE-COUNT
        ELSE IF ERR-CC = '0'
            THEN ADD 2 TO ERR-LINE-COUNT.
        IF ERR-LINE-COUNT > LINE-MAX
            THEN PERFORM U220-ERR-HEADER.
U220-ERR-HEADER.
        MOVE ERR-HEADER-1 TO ERR-LINE.
        MOVE '1' TO ERR-CC.
        PERFORM U200-WRITE-ERR-LINE.
        MOVE '0' TO ERR-CC.
        MOVE ERR-HEADER-2 TO ERR-LINE.
        PERFORM U200-WRITE-ERR-LINE.
        MOVE ERR-HEADER-3 TO ERR-LINE.
        PERFORM U200-WRITE-ERR-LINE.
        MOVE SPACES TO ERR-LINE.
        MOVE ' ' TO ERR-CC.
        PERFORM U200-WRITE-ERR-LINE.
SQL-ERROR.
        MOVE SPACES TO ERR-LINE.
        MOVE 'SQL ERROR' TO ERR-MESS-OUT.
        MOVE ERR-DETAIL-LINE TO ERR-LINE.
        PERFORM U200-WRITE-ERR-LINE.
SQL-ERROR-EXIT.
        EXIT.

```

```

=====
FILE: data\output\db2_dd1.sql
ENCODING: utf-8
=====

```

```

CREATE TABLE COVERAGE
(
    ID_RECORD_RAGE CHAR(20) NOT NULL,
    DA_SELECTIONDATE_479RAGE DATE,
    DA_TERMINATIONDATE_479RAGE DATE,
    TYPE_479RAGE CHAR(1),
    INS_PLancode_479RAGE CHAR(3),
    EMP_ID_479OYEE DECIMAL(4),
    PRIMARY KEY (ID_RECORD_RAGE)
);

CREATE TABLE DENTAL_CLAIM
(
    ID_RECORD_LAIM CHAR(20) NOT NULL,
    DA_CLAIMDATE_479LAIM DATE,
    DA_PATIENTBIRTHDATE_479LAIM DATE,
    DA_SERVICEDATE_479LAIM DATE,
    PATIENT_FIRSTNAME_479LAIM CHAR(10),
    PATIENT_LASTNAME_479LAIM CHAR(15),
    PATIENT_SEX_479LAIM CHAR(1),
    RELATION_TOEMPLOYEE_479LAIM CHAR(10),
    DENTIST_FIRSTNAME_479LAIM CHAR(10),
    DENTIST_LASTNAME_479LAIM CHAR(15),
    DENTIST_STREET_479LAIM CHAR(20),
    DENTIST_CITY_479LAIM CHAR(15),
    DENTIST_STATE_479LAIM CHAR(2),

```

```

DENTIST_ZIPFIRSTFIVE_479LAIM CHAR(5),
DENTIST_ZIPLASTFOUR_479LAIM CHAR(4),
DENTIST_LICENSENUMBER_479LAIM DECIMAL(6),
NUMBER_OFPROCEDURES_479LAIM DECIMAL(2),
TOOTH_NUMBER_479LAIM DECIMAL(2),
PROCEDURE_CODE_479LAIM DECIMAL(4),
DESCRIPTION_OFSERVICE_479LAIM CHAR(60),
FEE_479LAIM DECIMAL(5,2),
ID_RECORD_RAGE CHAR(20),
PRIMARY KEY (ID_RECORD_LAIM)
);

```

```

CREATE TABLE DEPARTMENT
(
    DEPT_ID_479MENT DECIMAL(4) NOT NULL,
    DEPT_NAME_479MENT CHAR(45),
    DEPT_HEADID_479MENT DECIMAL(4),
    PRIMARY KEY (DEPT_ID_479MENT)
);

```

```

CREATE TABLE EMPLOYEE
(
    DA_STARTDATE_479OYEE DATE,
    DA_TERMINATIONDATE_479OYEE DATE,
    DA_BIRTHDATE_479OYEE DATE,
    EMP_ID_479OYEE DECIMAL(4) NOT NULL,
    EMP_FIRSTNAME_479OYEE CHAR(10),
    EMP_LASTNAME_479OYEE CHAR(15),
    EMP_STREET_479OYEE CHAR(20),
    EMP_CITY_479OYEE CHAR(15),
    EMP_STATE_479OYEE CHAR(2),
    EMP_ZIPFIRSTFIVE_479OYEE CHAR(5),
    EMP_ZIPLASTFOUR_479OYEE CHAR(4),
    EMP_PHONE_479OYEE DECIMAL(10),
    STATUS_479OYEE CHAR(2),
    SS_NUMBER_479OYEE DECIMAL(9),
    DEPT_ID_479MENT DECIMAL(4),
    OFFICE_CODE_479FICE CHAR(3),
    PRIMARY KEY (EMP_ID_479OYEE)
);

```

```

CREATE TABLE EMPOSITION
(
    ID_RECORD_TION CHAR(20) NOT NULL,
    DA_STARTDATE_479TION DATE,
    DA_FINISHDATE_479TION DATE,
    SALARY_GRADE_479TION DECIMAL(2),
    SALARY_AMOUNT_479TION DECIMAL(5,2),
    BONUS_PERCENT_479TION DECIMAL(2),
    COMMISSION_PERCENT_479TION DECIMAL(2),
    OVERTIME_RATE_479TION DECIMAL(2,2),
    EMP_ID_479OYEE DECIMAL(4),
    JOB_ID_479JOB DECIMAL(4),
    PRIMARY KEY (ID_RECORD_TION)
);

```

```

CREATE TABLE EXPERTISE
(
    ID_RECORD_TISE CHAR(20) NOT NULL,
    DA_EXPERTISEDATE_479TISE DATE,
    SKILL_LEVEL_479TISE CHAR(2),
    EMP_ID_479OYEE DECIMAL(4),
    SKILL_ID_479KILL DECIMAL(4),
    PRIMARY KEY (ID_RECORD_TISE)
);

```

```

CREATE TABLE HOSPITAL_CLAIM
(
    ID_RECORD_LAIM CHAR(20) NOT NULL,
    DA_CLAIMDATE_479LAIM DATE,
    DA_PATIENTBIRTHDATE_479LAIM DATE,
    DA_ADMITDATE_479LAIM DATE,
    DA_DISCHARGEDATE_479LAIM DATE,
    PATIENT_FIRSTNAME_479LAIM CHAR(10),
    PATIENT_LASTNAME_479LAIM CHAR(15),
    PATIENT_SEX_479LAIM CHAR(1),

```

```

RELATION_TOEMPLOYEE_479LAIM CHAR(10),
HOSPITAL_NAME_479LAIM CHAR(25),
HOSP_STREET_479LAIM CHAR(20),
HOSP_CITY_479LAIM CHAR(15),
HOSP_STATE_479LAIM CHAR(2),
HOSP_ZIPFIRSTFIVE_479LAIM CHAR(5),
HOSP_ZIPLASTFOUR_479LAIM CHAR(4),
DIAGNOSIS_479LAIM VARCHAR(120),
WARD_DAYS_479LAIM DECIMAL(3),
WARD_RATE_479LAIM DECIMAL(5,2),
WARD_TOTAL_479LAIM DECIMAL(5,2),
SEMI_DAYS_479LAIM DECIMAL(3),
SEMI_RATE_479LAIM DECIMAL(5,2),
SEMI_TOTAL_479LAIM DECIMAL(5,2),
DELIVERY_COST_479LAIM DECIMAL(5,2),
ANESTHESIA_COST_479LAIM DECIMAL(5,2),
LAB_COST_479LAIM DECIMAL(5,2),
ID_RECORD_RAGE CHAR(20),
PRIMARY KEY (ID_RECORD_LAIM)
);

```

```

CREATE TABLE INSURANCE_PLAN
(
    INS_PLANCODE_479PLAN CHAR(3) NOT NULL,
    INS_CONAME_479PLAN CHAR(45),
    INS_COSTREET_479PLAN CHAR(20),
    INS_COCITY_479PLAN CHAR(15),
    INS_COSTATE_479PLAN CHAR(2),
    INS_COZIPFIRSTFIVE_479PLAN CHAR(5),
    INS_COZIPLASTFOUR_479PLAN CHAR(4),
    INS_COPHONE_479PLAN DECIMAL(10),
    GROUP_NUMBER_479PLAN DECIMAL(6),
    DEDUCT_479PLAN DECIMAL(5,2),
    MAXIMUM_LIFECOST_479PLAN DECIMAL(5,2),
    FAMILY_COST_479PLAN DECIMAL(5,2),
    DEP_COST_479PLAN DECIMAL(5,2),
    PRIMARY KEY (INS_PLANCODE_479PLAN)
);

```

```

CREATE TABLE JOB
(
    JOB_ID_479JOB DECIMAL(4) NOT NULL,
    TITLE_479JOB CHAR(20),
    DESCRIPTION_LINE_479JOB VARCHAR(120),
    REQUIREMENT_LINE_479JOB VARCHAR(120),
    MINIMUM_SALARY_479JOB DECIMAL(8,2),
    MAXIMUM_SALARY_479JOB DECIMAL(8,2),
    SALARY_GRADES_479JOB VARCHAR(8),
    NUMBER_OFPOSITIONS_479JOB DECIMAL(3),
    NUMBER_OPEN_479JOB DECIMAL(3),
    PRIMARY KEY (JOB_ID_479JOB)
);

```

```

CREATE TABLE NON_HOSP_CLAIM
(
    ID_RECORD_LAIM CHAR(20) NOT NULL,
    DA_CLAIMDATE_479LAIM DATE,
    DA_PATIENTBIRTHDATE_479LAIM DATE,
    DA_SERVICEDATE_479LAIM DATE,
    PATIENT_FIRSTNAME_479LAIM CHAR(10),
    PATIENT_LASTNAME_479LAIM CHAR(15),
    PATIENT_SEX_479LAIM CHAR(1),
    RELATION_TOEMPLOYEE_479LAIM CHAR(10),
    PHYSICIAN_FIRSTNAME_479LAIM CHAR(10),
    PHYSICIAN_LASTNAME_479LAIM CHAR(15),
    PHYSICIAN_STREET_479LAIM CHAR(20),
    PHYSICIAN_CITY_479LAIM CHAR(15),
    PHYSICIAN_STATE_479LAIM CHAR(2),
    PHYSICIAN_ZIPFIRSTFIVE_479LAIM CHAR(5),
    PHYSICIAN_ZIPLASTFOUR_479LAIM CHAR(4),
    PHYSICIAN_ID_479LAIM DECIMAL(6),
    DIAGNOSIS_479LAIM VARCHAR(120),
    NUMBER_OFPROCEDURES_479LAIM DECIMAL(2),
    PROCEDURE_CODE_479LAIM DECIMAL(4),
    DESCRIPTION_OFSERVICE_479LAIM CHAR(60),
    FEE_479LAIM DECIMAL(5,2),

```

```

        ID_RECORD_RAGE CHAR(20),
        PRIMARY KEY (ID_RECORD_LAIM)
    );

CREATE TABLE OFFICE
(
    OFFICE_CODE_479FICE CHAR(3) NOT NULL,
    OFFICE_STREET_479FICE CHAR(20),
    OFFICE_CITY_479FICE CHAR(15),
    OFFICE_STATE_479FICE CHAR(2),
    OFFICE_ZIPFIRSTFIVE_479FICE CHAR(5),
    OFFICE_ZIPLASTFOUR_479FICE CHAR(4),
    OFFICE_PHONE_479FICE VARCHAR(21),
    OFFICE_AREACODE_479FICE CHAR(3),
    SPEED_DIAL_479FICE CHAR(3),
    PRIMARY KEY (OFFICE_CODE_479FICE)
);

CREATE TABLE SKILL
(
    SKILL_ID_479KILL DECIMAL(4) NOT NULL,
    SKILL_NAME_479KILL CHAR(12),
    SKILL_DESCRIPTION_479KILL CHAR(60),
    PRIMARY KEY (SKILL_ID_479KILL)
);

CREATE TABLE STRUCTURE
(
    ID_RECORD_TURE CHAR(20) NOT NULL,
    DA_STRUCTUREDATE_479TURE DATE,
    STRUCTURE_CODE_479TURE CHAR(2),
    EMP_ID_479OYEE DECIMAL(4),
    PRIMARY KEY (ID_RECORD_TURE)
);

ALTER TABLE COVERAGE ADD CONSTRAINT FK_COVERAGE_EMP_ID_479OYEE FOREIGN KEY (EMP_ID_479OYEE) REFERENCES EMPLOYEE (EMP_ID_479OYEE);
ALTER TABLE DENTAL_CLAIM ADD CONSTRAINT FK_DENTAL_CLAIM_ID_RECORD_RAGE FOREIGN KEY (ID_RECORD_RAGE) REFERENCES COVERAGE (ID_RECORD_RAGE);
ALTER TABLE EMPLOYEE ADD CONSTRAINT FK_EMPLOYEE_DEPT_ID_479MENT FOREIGN KEY (DEPT_ID_479MENT) REFERENCES DEPARTMENT (DEPT_ID_479MENT);
ALTER TABLE EMPLOYEE ADD CONSTRAINT FK_EMPLOYEE_OFFICE_CODE_479FICE FOREIGN KEY (OFFICE_CODE_479FICE) REFERENCES OFFICE (OFFICE_CODE_479FICE);
ALTER TABLE EMPOSITION ADD CONSTRAINT FK_EMPOSITION_EMP_ID_479OYEE FOREIGN KEY (EMP_ID_479OYEE) REFERENCES EMPLOYEE (EMP_ID_479OYEE);
ALTER TABLE EMPOSITION ADD CONSTRAINT FK_EMPOSITION_JOB_ID_479JOB FOREIGN KEY (JOB_ID_479JOB) REFERENCES JOB (JOB_ID_479JOB);
ALTER TABLE EXPERTISE ADD CONSTRAINT FK_EXPERTISE_EMP_ID_479OYEE FOREIGN KEY (EMP_ID_479OYEE) REFERENCES EMPLOYEE (EMP_ID_479OYEE);
ALTER TABLE EXPERTISE ADD CONSTRAINT FK_EXPERTISE_SKILL_ID_479KILL FOREIGN KEY (SKILL_ID_479KILL) REFERENCES SKILL (SKILL_ID_479KILL);
ALTER TABLE HOSPITAL_CLAIM ADD CONSTRAINT FK_HOSPITAL_CLAIM_ID_RECORD_RAGE FOREIGN KEY (ID_RECORD_RAGE) REFERENCES COVERAGE (ID_RECORD_RAGE);
ALTER TABLE NON_HOSP_CLAIM ADD CONSTRAINT FK_NON_HOSP_CLAIM_ID_RECORD_RAGE FOREIGN KEY (ID_RECORD_RAGE) REFERENCES COVERAGE (ID_RECORD_RAGE);
ALTER TABLE STRUCTURE ADD CONSTRAINT FK_STRUCTURE_EMP_ID_479OYEE FOREIGN KEY (EMP_ID_479OYEE) REFERENCES EMPLOYEE (EMP_ID_479OYEE);

```

```

=====
FILE: data\output\phase2_metadata.json
ENCODING: utf-8
=====

```

```

{
  "version": "1.0",
  "generated_at": "2026-08-07T10:23:41.967586Z",
  "records": [
    {
      "name": "COVERAGE",
      "table": "COVERAGE",
      "primary_key": "ID_RECORD_RAGE",
      "primary_keys": [
        "ID_RECORD_RAGE"
      ],
    },
  ],
}

```

```

"fields": [
  {
    "name": "ID_RECORD_RAGE",
    "column": "ID_RECORD_RAGE",
    "datatype": "CHAR(20)",
    "length": null,
    "scale": null,
    "nullable": false,
    "primary_key": true,
    "generated": true,
    "source_kind": "GENERATED PK"
  },
  {
    "name": "DA_SELECTIONDATE_479RAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_TERMINATIONDATE_479RAGE",
    "column": "DA_TERMINATIONDATE_479RAGE",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "TYPE_479RAGE",
    "column": "TYPE_479RAGE",
    "datatype": "CHAR(1)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "INS_PLANCODE_479RAGE",
    "column": "INS_PLANCODE_479RAGE",
    "datatype": "CHAR(3)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_ID_479OYEE",
    "column": "EMP_ID_479OYEE",
    "datatype": "DECIMAL(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": true,
    "source_kind": "SET_FK"
  }
],
{
  "name": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "primary_key": "ID_RECORD_LAIM",
  "primary_keys": [
    "ID_RECORD_LAIM"
  ],

```

```

"fields": [
  {
    "name": "ID_RECORD_LAIM",
    "column": "ID_RECORD_LAIM",
    "datatype": "CHAR(20)",
    "length": null,
    "scale": null,
    "nullable": false,
    "primary_key": true,
    "generated": true,
    "source_kind": "GENERATED PK"
  },
  {
    "name": "DA_CLAIMDATE_479LAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_PATIENTBIRTHDATE_479LAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_SERVICEDATE_479LAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PATIENT_FIRSTNAME_479LAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "datatype": "CHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PATIENT_LASTNAME_479LAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PATIENT_SEX_479LAIM",
    "column": "PATIENT_SEX_479LAIM",
    "datatype": "CHAR(1)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
  }
]

```

```

    "source_kind": "COBOL"
  },
  {
    "name": "RELATION_TOEMPLOYEE_479LAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "datatype": "CHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_FIRSTNAME_479LAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "datatype": "CHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_LASTNAME_479LAIM",
    "column": "DENTIST_LASTNAME_479LAIM",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_STREET_479LAIM",
    "column": "DENTIST_STREET_479LAIM",
    "datatype": "CHAR(20)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_CITY_479LAIM",
    "column": "DENTIST_CITY_479LAIM",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_STATE_479LAIM",
    "column": "DENTIST_STATE_479LAIM",
    "datatype": "CHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "datatype": "CHAR(5)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,

```



```

    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_ZIPLASTFOUR_479LAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "datatype": "CHAR(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_LICENSENUMBER_479LAIM",
    "column": "DENTIST_LICENSENUMBER_479LAIM",
    "datatype": "DECIMAL(6)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "NUMBER_OFPROCEDURES_479LAIM",
    "column": "NUMBER_OFPROCEDURES_479LAIM",
    "datatype": "DECIMAL(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "TOOTH_NUMBER_479LAIM",
    "column": "TOOTH_NUMBER_479LAIM",
    "datatype": "DECIMAL(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PROCEDURE_CODE_479LAIM",
    "column": "PROCEDURE_CODE_479LAIM",
    "datatype": "DECIMAL(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DESCRIPTION_OFSERVICE_479LAIM",
    "column": "DESCRIPTION_OFSERVICE_479LAIM",
    "datatype": "CHAR(60)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "FEE_479LAIM",
    "column": "FEE_479LAIM",
    "datatype": "DECIMAL(5,2)",
    "length": null,
    "scale": null,
    "nullable": true,

```

```

        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "ID_RECORD_RAGE",
        "column": "ID_RECORD_RAGE",
        "datatype": "CHAR(20)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    }
]
},
{
    "name": "DEPARTMENT",
    "table": "DEPARTMENT",
    "primary_key": "DEPT_ID_479MENT",
    "primary_keys": [
        "DEPT_ID_479MENT"
    ],
    "fields": [
        {
            "name": "DEPT_ID_479MENT",
            "column": "DEPT_ID_479MENT",
            "datatype": "DECIMAL(4)",
            "length": null,
            "scale": null,
            "nullable": false,
            "primary_key": true,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "DEPT_NAME_479MENT",
            "column": "DEPT_NAME_479MENT",
            "datatype": "CHAR(45)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "DEPT_HEADID_479MENT",
            "column": "DEPT_HEADID_479MENT",
            "datatype": "DECIMAL(4)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,
            "generated": false,
            "source_kind": "COBOL"
        }
    ]
},
{
    "name": "EMPLOYEE",
    "table": "EMPLOYEE",
    "primary_key": "EMP_ID_4790YEE",
    "primary_keys": [
        "EMP_ID_4790YEE"
    ],
    "fields": [
        {
            "name": "DA_STARTDATE_4790YEE",
            "column": "DA_STARTDATE_4790YEE",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,

```

```

    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_TERMINATIONDATE_4790YEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_BIRTHDATE_4790YEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_ID_4790YEE",
    "column": "EMP_ID_4790YEE",
    "datatype": "DECIMAL(4)",
    "length": null,
    "scale": null,
    "nullable": false,
    "primary_key": true,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_FIRSTNAME_4790YEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "datatype": "CHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_LASTNAME_4790YEE",
    "column": "EMP_LASTNAME_4790YEE",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_STREET_4790YEE",
    "column": "EMP_STREET_4790YEE",
    "datatype": "CHAR(20)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_CITY_4790YEE",
    "column": "EMP_CITY_4790YEE",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,

```

```

    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_STATE_4790YEE",
    "column": "EMP_STATE_4790YEE",
    "datatype": "CHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_ZIPFIRSTFIVE_4790YEE",
    "column": "EMP_ZIPFIRSTFIVE_4790YEE",
    "datatype": "CHAR(5)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_ZIPLASTFOUR_4790YEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "datatype": "CHAR(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_PHONE_4790YEE",
    "column": "EMP_PHONE_4790YEE",
    "datatype": "DECIMAL(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "STATUS_4790YEE",
    "column": "STATUS_4790YEE",
    "datatype": "CHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "SS_NUMBER_4790YEE",
    "column": "SS_NUMBER_4790YEE",
    "datatype": "DECIMAL(9)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DEPT_ID_479MENT",
    "column": "DEPT_ID_479MENT",
    "datatype": "DECIMAL(4)",
    "length": null,
    "scale": null,

```

```

        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    },
    {
        "name": "OFFICE_CODE_479FICE",
        "column": "OFFICE_CODE_479FICE",
        "datatype": "CHAR(3)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    }
]
},
{
    "name": "EMPOSITION",
    "table": "EMPOSITION",
    "primary_key": "ID_RECORD_TION",
    "primary_keys": [
        "ID_RECORD_TION"
    ],
    "fields": [
        {
            "name": "ID_RECORD_TION",
            "column": "ID_RECORD_TION",
            "datatype": "CHAR(20)",
            "length": null,
            "scale": null,
            "nullable": false,
            "primary_key": true,
            "generated": true,
            "source_kind": "GENERATED PK"
        },
        {
            "name": "DA_STARTDATE_479TION",
            "column": "DA_STARTDATE_479TION",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "DA_FINISHDATE_479TION",
            "column": "DA_FINISHDATE_479TION",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "SALARY_GRADE_479TION",
            "column": "SALARY_GRADE_479TION",
            "datatype": "DECIMAL(2)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "SALARY_AMOUNT_479TION",
            "column": "SALARY_AMOUNT_479TION",
            "datatype": "DECIMAL(5,2)",
            "length": null,
            "scale": null,

```

```

        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "BONUS_PERCENT_479TION",
        "column": "BONUS_PERCENT_479TION",
        "datatype": "DECIMAL(2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "COMMISSION_PERCENT_479TION",
        "column": "COMMISSION_PERCENT_479TION",
        "datatype": "DECIMAL(2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "OVERTIME_RATE_479TION",
        "column": "OVERTIME_RATE_479TION",
        "datatype": "DECIMAL(2,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "EMP_ID_479OYEE",
        "column": "EMP_ID_479OYEE",
        "datatype": "DECIMAL(4)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    },
    {
        "name": "JOB_ID_479JOB",
        "column": "JOB_ID_479JOB",
        "datatype": "DECIMAL(4)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    }
]
},
{
    "name": "EXPERTISE",
    "table": "EXPERTISE",
    "primary_key": "ID_RECORD_TISE",
    "primary_keys": [
        "ID_RECORD_TISE"
    ],
    "fields": [
        {
            "name": "ID_RECORD_TISE",
            "column": "ID_RECORD_TISE",
            "datatype": "CHAR(20)",
            "length": null,
            "scale": null,

```

```

        "nullable": false,
        "primary_key": true,
        "generated": true,
        "source_kind": "GENERATED PK"
    },
    {
        "name": "DA_EXPERTISEDATE_479TISE",
        "column": "DA_EXPERTISEDATE_479TISE",
        "datatype": "DATE",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "SKILL_LEVEL_479TISE",
        "column": "SKILL_LEVEL_479TISE",
        "datatype": "CHAR(2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "EMP_ID_479OYEE",
        "column": "EMP_ID_479OYEE",
        "datatype": "DECIMAL(4)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    },
    {
        "name": "SKILL_ID_479KILL",
        "column": "SKILL_ID_479KILL",
        "datatype": "DECIMAL(4)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    }
]
},
{
    "name": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "primary_key": "ID_RECORD_LAIM",
    "primary_keys": [
        "ID_RECORD_LAIM"
    ],
    "fields": [
        {
            "name": "ID_RECORD_LAIM",
            "column": "ID_RECORD_LAIM",
            "datatype": "CHAR(20)",
            "length": null,
            "scale": null,
            "nullable": false,
            "primary_key": true,
            "generated": true,
            "source_kind": "GENERATED PK"
        },
        {
            "name": "DA_CLAIMDATE_479LAIM",
            "column": "DA_CLAIMDATE_479LAIM",
            "datatype": "DATE",
            "length": null,
            "scale": null,

```

```

    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_PATIENTBIRTHDATE_479LAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_ADMITDATE_479LAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_DISCHARGEDATE_479LAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PATIENT_FIRSTNAME_479LAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "datatype": "CHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PATIENT_LASTNAME_479LAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PATIENT_SEX_479LAIM",
    "column": "PATIENT_SEX_479LAIM",
    "datatype": "CHAR(1)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "RELATION_TOEMPLOYEE_479LAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "datatype": "CHAR(10)",
    "length": null,

```



```

    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "HOSPITAL_NAME_479LAIM",
    "column": "HOSPITAL_NAME_479LAIM",
    "datatype": "CHAR(25)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "HOSP_STREET_479LAIM",
    "column": "HOSP_STREET_479LAIM",
    "datatype": "CHAR(20)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "HOSP_CITY_479LAIM",
    "column": "HOSP_CITY_479LAIM",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "HOSP_STATE_479LAIM",
    "column": "HOSP_STATE_479LAIM",
    "datatype": "CHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "datatype": "CHAR(5)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "HOSP_ZIPLASTFOUR_479LAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "datatype": "CHAR(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DIAGNOSIS_479LAIM",
    "column": "DIAGNOSIS_479LAIM",
    "datatype": "VARCHAR(120)",

```

```

    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "WARD_DAYS_479LAIM",
    "column": "WARD_DAYS_479LAIM",
    "datatype": "DECIMAL(3)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "WARD_RATE_479LAIM",
    "column": "WARD_RATE_479LAIM",
    "datatype": "DECIMAL(5,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "WARD_TOTAL_479LAIM",
    "column": "WARD_TOTAL_479LAIM",
    "datatype": "DECIMAL(5,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "SEMI_DAYS_479LAIM",
    "column": "SEMI_DAYS_479LAIM",
    "datatype": "DECIMAL(3)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "SEMI_RATE_479LAIM",
    "column": "SEMI_RATE_479LAIM",
    "datatype": "DECIMAL(5,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "SEMI_TOTAL_479LAIM",
    "column": "SEMI_TOTAL_479LAIM",
    "datatype": "DECIMAL(5,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DELIVERY_COST_479LAIM",
    "column": "DELIVERY_COST_479LAIM",

```

```

        "datatype": "DECIMAL(5,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "ANESTHESIA_COST_479LAIM",
        "column": "ANESTHESIA_COST_479LAIM",
        "datatype": "DECIMAL(5,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "LAB_COST_479LAIM",
        "column": "LAB_COST_479LAIM",
        "datatype": "DECIMAL(5,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "ID_RECORD_RAGE",
        "column": "ID_RECORD_RAGE",
        "datatype": "CHAR(20)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    }
]
},
{
    "name": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "primary_key": "INS_PLANCODE_479PLAN",
    "primary_keys": [
        "INS_PLANCODE_479PLAN"
    ],
    "fields": [
        {
            "name": "INS_PLANCODE_479PLAN",
            "column": "INS_PLANCODE_479PLAN",
            "datatype": "CHAR(3)",
            "length": null,
            "scale": null,
            "nullable": false,
            "primary_key": true,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "INS_CONAME_479PLAN",
            "column": "INS_CONAME_479PLAN",
            "datatype": "CHAR(45)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "INS_COSTREET_479PLAN",
            "column": "INS_COSTREET_479PLAN",

```

```

    "datatype": "CHAR(20)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "INS_COCITY_479PLAN",
    "column": "INS_COCITY_479PLAN",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "INS_COSTATE_479PLAN",
    "column": "INS_COSTATE_479PLAN",
    "datatype": "CHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "INS_COZIPFIRSTFIVE_479PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "datatype": "CHAR(5)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "INS_COZIPLASTFOUR_479PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "datatype": "CHAR(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "INS_COPHONE_479PLAN",
    "column": "INS_COPHONE_479PLAN",
    "datatype": "DECIMAL(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "GROUP_NUMBER_479PLAN",
    "column": "GROUP_NUMBER_479PLAN",
    "datatype": "DECIMAL(6)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DEDUCT_479PLAN",

```

```

        "column": "DEDUCT_479PLAN",
        "datatype": "DECIMAL(5,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "MAXIMUM_LIFECOST_479PLAN",
        "column": "MAXIMUM_LIFECOST_479PLAN",
        "datatype": "DECIMAL(5,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "FAMILY_COST_479PLAN",
        "column": "FAMILY_COST_479PLAN",
        "datatype": "DECIMAL(5,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "DEP_COST_479PLAN",
        "column": "DEP_COST_479PLAN",
        "datatype": "DECIMAL(5,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    }
]
},
{
    "name": "JOB",
    "table": "JOB",
    "primary_key": "JOB_ID_479JOB",
    "primary_keys": [
        "JOB_ID_479JOB"
    ],
    "fields": [
        {
            "name": "JOB_ID_479JOB",
            "column": "JOB_ID_479JOB",
            "datatype": "DECIMAL(4)",
            "length": null,
            "scale": null,
            "nullable": false,
            "primary_key": true,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "TITLE_479JOB",
            "column": "TITLE_479JOB",
            "datatype": "CHAR(20)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "DESCRIPTION_LINE_479JOB",

```

```

    "column": "DESCRIPTION_LINE_479JOB",
    "datatype": "VARCHAR(120)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "REQUIREMENT_LINE_479JOB",
    "column": "REQUIREMENT_LINE_479JOB",
    "datatype": "VARCHAR(120)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "MINIMUM_SALARY_479JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "datatype": "DECIMAL(8,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "MAXIMUM_SALARY_479JOB",
    "column": "MAXIMUM_SALARY_479JOB",
    "datatype": "DECIMAL(8,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "SALARY_GRADES_479JOB",
    "column": "SALARY_GRADES_479JOB",
    "datatype": "VARCHAR(8)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "NUMBER_OFPOSITIONS_479JOB",
    "column": "NUMBER_OFPOSITIONS_479JOB",
    "datatype": "DECIMAL(3)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "NUMBER_OPEN_479JOB",
    "column": "NUMBER_OPEN_479JOB",
    "datatype": "DECIMAL(3)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  }
]

```

```

},
{
  "name": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "primary_key": "ID_RECORD_LAIM",
  "primary_keys": [
    "ID_RECORD_LAIM"
  ],
  "fields": [
    {
      "name": "ID_RECORD_LAIM",
      "column": "ID_RECORD_LAIM",
      "datatype": "CHAR(20)",
      "length": null,
      "scale": null,
      "nullable": false,
      "primary_key": true,
      "generated": true,
      "source_kind": "GENERATED PK"
    },
    {
      "name": "DA_CLAIMDATE_479LAIM",
      "column": "DA_CLAIMDATE_479LAIM",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "DA_PATIENTBIRTHDATE_479LAIM",
      "column": "DA_PATIENTBIRTHDATE_479LAIM",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "DA_SERVICEDATE_479LAIM",
      "column": "DA_SERVICEDATE_479LAIM",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "PATIENT_FIRSTNAME_479LAIM",
      "column": "PATIENT_FIRSTNAME_479LAIM",
      "datatype": "CHAR(10)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "PATIENT_LASTNAME_479LAIM",
      "column": "PATIENT_LASTNAME_479LAIM",
      "datatype": "CHAR(15)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    }
  ],
{

```

```

    "name": "PATIENT_SEX_479LAIM",
    "column": "PATIENT_SEX_479LAIM",
    "datatype": "CHAR(1)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "RELATION_TOEMPLOYEE_479LAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "datatype": "CHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PHYSICIAN_FIRSTNAME_479LAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "datatype": "CHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PHYSICIAN_LASTNAME_479LAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PHYSICIAN_STREET_479LAIM",
    "column": "PHYSICIAN_STREET_479LAIM",
    "datatype": "CHAR(20)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PHYSICIAN_CITY_479LAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PHYSICIAN_STATE_479LAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "datatype": "CHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },

```



```

{
  "name": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
  "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
  "datatype": "CHAR(5)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false,
  "generated": false,
  "source_kind": "COBOL"
},
{
  "name": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
  "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
  "datatype": "CHAR(4)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false,
  "generated": false,
  "source_kind": "COBOL"
},
{
  "name": "PHYSICIAN_ID_479LAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "datatype": "DECIMAL(6)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false,
  "generated": false,
  "source_kind": "COBOL"
},
{
  "name": "DIAGNOSIS_479LAIM",
  "column": "DIAGNOSIS_479LAIM",
  "datatype": "VARCHAR(120)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false,
  "generated": false,
  "source_kind": "COBOL"
},
{
  "name": "NUMBER_OFPROCEDURES_479LAIM",
  "column": "NUMBER_OFPROCEDURES_479LAIM",
  "datatype": "DECIMAL(2)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false,
  "generated": false,
  "source_kind": "COBOL"
},
{
  "name": "PROCEDURE_CODE_479LAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "datatype": "DECIMAL(4)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false,
  "generated": false,
  "source_kind": "COBOL"
},
{
  "name": "DESCRIPTION_OFSERVICE_479LAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "datatype": "CHAR(60)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false,
  "generated": false,
  "source_kind": "COBOL"
}

```

```

    },
    {
      "name": "FEE_479LAIM",
      "column": "FEE_479LAIM",
      "datatype": "DECIMAL(5,2)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "ID_RECORD_RAGE",
      "column": "ID_RECORD_RAGE",
      "datatype": "CHAR(20)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": true,
      "source_kind": "SET_FK"
    }
  ]
},
{
  "name": "OFFICE",
  "table": "OFFICE",
  "primary_key": "OFFICE_CODE_479FICE",
  "primary_keys": [
    "OFFICE_CODE_479FICE"
  ],
  "fields": [
    {
      "name": "OFFICE_CODE_479FICE",
      "column": "OFFICE_CODE_479FICE",
      "datatype": "CHAR(3)",
      "length": null,
      "scale": null,
      "nullable": false,
      "primary_key": true,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "OFFICE_STREET_479FICE",
      "column": "OFFICE_STREET_479FICE",
      "datatype": "CHAR(20)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "OFFICE_CITY_479FICE",
      "column": "OFFICE_CITY_479FICE",
      "datatype": "CHAR(15)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "OFFICE_STATE_479FICE",
      "column": "OFFICE_STATE_479FICE",
      "datatype": "CHAR(2)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    }
  ]
}

```

```

    },
    {
      "name": "OFFICE_ZIPFIRSTFIVE_479FICE",
      "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
      "datatype": "CHAR(5)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "OFFICE_ZIPLASTFOUR_479FICE",
      "column": "OFFICE_ZIPLASTFOUR_479FICE",
      "datatype": "CHAR(4)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "OFFICE_PHONE_479FICE",
      "column": "OFFICE_PHONE_479FICE",
      "datatype": "VARCHAR(21)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "OFFICE_AREACODE_479FICE",
      "column": "OFFICE_AREACODE_479FICE",
      "datatype": "CHAR(3)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "SPEED_DIAL_479FICE",
      "column": "SPEED_DIAL_479FICE",
      "datatype": "CHAR(3)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    }
  ],
  {
    "name": "SKILL",
    "table": "SKILL",
    "primary_key": "SKILL_ID_479KILL",
    "primary_keys": [
      "SKILL_ID_479KILL"
    ],
    "fields": [
      {
        "name": "SKILL_ID_479KILL",
        "column": "SKILL_ID_479KILL",
        "datatype": "DECIMAL(4)",
        "length": null,
        "scale": null,
        "nullable": false,
        "primary_key": true,
        "generated": false,
        "source_kind": "COBOL"
      }
    ]
  }
}

```

```

    },
    {
      "name": "SKILL_NAME_479KILL",
      "column": "SKILL_NAME_479KILL",
      "datatype": "CHAR(12)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "SKILL_DESCRIPTION_479KILL",
      "column": "SKILL_DESCRIPTION_479KILL",
      "datatype": "CHAR(60)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    }
  ]
},
{
  "name": "STRUCTURE",
  "table": "STRUCTURE",
  "primary_key": "ID_RECORD_TURE",
  "primary_keys": [
    "ID_RECORD_TURE"
  ],
  "fields": [
    {
      "name": "ID_RECORD_TURE",
      "column": "ID_RECORD_TURE",
      "datatype": "CHAR(20)",
      "length": null,
      "scale": null,
      "nullable": false,
      "primary_key": true,
      "generated": true,
      "source_kind": "GENERATED PK"
    },
    {
      "name": "DA_STRUCTUREDATE_479TURE",
      "column": "DA_STRUCTUREDATE_479TURE",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "STRUCTURE_CODE_479TURE",
      "column": "STRUCTURE_CODE_479TURE",
      "datatype": "CHAR(2)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "EMP_ID_479OYEE",
      "column": "EMP_ID_479OYEE",
      "datatype": "DECIMAL(4)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": true,
      "source_kind": "SET_FK"
    }
  ]
}

```

```

    }
  ]
}
],
"record_table_map": {
  "COVERAGE": "COVERAGE",
  "DENTAL_CLAIM": "DENTAL_CLAIM",
  "DEPARTMENT": "DEPARTMENT",
  "EMPLOYEE": "EMPLOYEE",
  "EMPOSITION": "EMPOSITION",
  "EXPERTISE": "EXPERTISE",
  "HOSPITAL_CLAIM": "HOSPITAL_CLAIM",
  "INSURANCE_PLAN": "INSURANCE_PLAN",
  "JOB": "JOB",
  "NON_HOSP_CLAIM": "NON_HOSP_CLAIM",
  "OFFICE": "OFFICE",
  "SKILL": "SKILL",
  "STRUCTURE": "STRUCTURE"
},
"field_map": {
  "SELECTION_DATE_0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
  "SELECTION-DATE-0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
  "SELECTION_DATE": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
  "SELECTIONDATE0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
  "DA_SELECTIONDATE_479RAGE": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
  "DA-SELECTIONDATE-479RAGE": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
  "DA_SELECTIONDATE": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
  "DASELECTIONDATE": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
},

```

```

"TERMINATION_DATE_0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "legacy_field": "TERMINATION_DATE_0400"
},
"TERMINATION-DATE-0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "legacy_field": "TERMINATION_DATE_0400"
},
"TERMINATION_DATE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "DA_TERMINATIONDATE_479OYEE",
  "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-479OYEE",
  "legacy_field": "TERMINATION_DATE_0415"
},
"TERMINATIONDATE0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "legacy_field": "TERMINATION_DATE_0400"
},
"DA_TERMINATIONDATE_479RAGE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "legacy_field": "TERMINATION_DATE_0400"
},
"DA-TERMINATIONDATE-479RAGE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "legacy_field": "TERMINATION_DATE_0400"
},
"DA_TERMINATIONDATE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "legacy_field": "TERMINATION_DATE_0400"
},
"DATERMINATIONDATE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "legacy_field": "TERMINATION_DATE_0400"
},
"TYPE_0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "TYPE_479RAGE",
  "host": "HV-COVERAGE-TYPE-479RAGE",
  "legacy_field": "TYPE_0400"
},
"TYPE-0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "TYPE_479RAGE",
  "host": "HV-COVERAGE-TYPE-479RAGE",
  "legacy_field": "TYPE_0400"
},
"TYPE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "TYPE_479RAGE",
  "host": "HV-COVERAGE-TYPE-479RAGE",
  "legacy_field": "TYPE_0400"
}

```

```

},
"TYPE0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "TYPE_479RAGE",
  "host": "HV-COVERAGE-TYPE-479RAGE",
  "legacy_field": "TYPE_0400"
},
"TYPE_479RAGE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "TYPE_479RAGE",
  "host": "HV-COVERAGE-TYPE-479RAGE",
  "legacy_field": "TYPE_0400"
},
"TYPE-479RAGE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "TYPE_479RAGE",
  "host": "HV-COVERAGE-TYPE-479RAGE",
  "legacy_field": "TYPE_0400"
},
"INS_PLAN_CODE_0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "INS_PLANCODE_479RAGE",
  "host": "HV-COVERAGE-INS-PLANCODE-479RAGE",
  "legacy_field": "INS_PLAN_CODE_0400"
},
"INS-PLAN-CODE-0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "INS_PLANCODE_479RAGE",
  "host": "HV-COVERAGE-INS-PLANCODE-479RAGE",
  "legacy_field": "INS_PLAN_CODE_0400"
},
"INS_PLAN_CODE": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_PLANCODE_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-PLANCODE-479PLAN",
  "legacy_field": "INS_PLAN_CODE_0435"
},
"INSPLANCODE0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "INS_PLANCODE_479RAGE",
  "host": "HV-COVERAGE-INS-PLANCODE-479RAGE",
  "legacy_field": "INS_PLAN_CODE_0400"
},
"INS_PLANCODE_479RAGE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "INS_PLANCODE_479RAGE",
  "host": "HV-COVERAGE-INS-PLANCODE-479RAGE",
  "legacy_field": "INS_PLAN_CODE_0400"
},
"INS-PLANCODE-479RAGE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "INS_PLANCODE_479RAGE",
  "host": "HV-COVERAGE-INS-PLANCODE-479RAGE",
  "legacy_field": "INS_PLAN_CODE_0400"
},
"INS_PLANCODE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "INS_PLANCODE_479RAGE",
  "host": "HV-COVERAGE-INS-PLANCODE-479RAGE",
  "legacy_field": "INS_PLAN_CODE_0400"
},
"INSPLANCODE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "INS_PLANCODE_479RAGE",
  "host": "HV-COVERAGE-INS-PLANCODE-479RAGE",

```

```

    "legacy_field": "INS_PLAN_CODE_0400"
  },
  "CLAIM_DATE_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0405"
  },
  "CLAIM-DATE-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0405"
  },
  "CLAIM_DATE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0445"
  },
  "CLAIMDATE0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0405"
  },
  "DA_CLAIMDATE_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0405"
  },
  "DA-CLAIMDATE-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0405"
  },
  "DA_CLAIMDATE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0405"
  },
  "DACLAIMDATE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0405"
  },
  "PATIENT_FIRST_NAME_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0405"
  },
  "PATIENT-FIRST-NAME-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0405"
  },
  "PATIENT_FIRST_NAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",

```



```

    "host": "HV-NON-HOSP-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0445"
  },
  "PATIENTFIRSTNAME0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0405"
  },
  "PATIENT_FIRSTNAME_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0405"
  },
  "PATIENT-FIRSTNAME-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0405"
  },
  "PATIENT_FIRSTNAME": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0405"
  },
  "PATIENTFIRSTNAME": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0405"
  },
  "PATIENT_LAST_NAME_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-LASTNAME-479LAIM",
    "legacy_field": "PATIENT_LAST_NAME_0405"
  },
  "PATIENT-LAST-NAME-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-LASTNAME-479LAIM",
    "legacy_field": "PATIENT_LAST_NAME_0405"
  },
  "PATIENT_LAST_NAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PATIENT-LASTNAME-479LAIM",
    "legacy_field": "PATIENT_LAST_NAME_0445"
  },
  "PATIENTLASTNAME0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-LASTNAME-479LAIM",
    "legacy_field": "PATIENT_LAST_NAME_0405"
  },
  "PATIENT_LASTNAME_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-LASTNAME-479LAIM",
    "legacy_field": "PATIENT_LAST_NAME_0405"
  },
  "PATIENT-LASTNAME-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",

```

```

    "column": "PATIENT_LASTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-LASTNAME-479LAIM",
    "legacy_field": "PATIENT_LAST_NAME_0405"
  },
  "PATIENT_LASTNAME": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-LASTNAME-479LAIM",
    "legacy_field": "PATIENT_LAST_NAME_0405"
  },
  "PATIENTLASTNAME": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-LASTNAME-479LAIM",
    "legacy_field": "PATIENT_LAST_NAME_0405"
  },
  "PATIENT_BIRTH_DATE_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "legacy_field": "PATIENT_BIRTH_DATE_0405"
  },
  "PATIENT-BIRTH-DATE-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "legacy_field": "PATIENT_BIRTH_DATE_0405"
  },
  "PATIENT_BIRTH_DATE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "legacy_field": "PATIENT_BIRTH_DATE_0445"
  },
  "PATIENTBIRTHDATE0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "legacy_field": "PATIENT_BIRTH_DATE_0405"
  },
  "DA_PATIENTBIRTHDATE_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "legacy_field": "PATIENT_BIRTH_DATE_0405"
  },
  "DA-PATIENTBIRTHDATE-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "legacy_field": "PATIENT_BIRTH_DATE_0405"
  },
  "DA_PATIENTBIRTHDATE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "legacy_field": "PATIENT_BIRTH_DATE_0405"
  },
  "DAPATIENTBIRTHDATE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "legacy_field": "PATIENT_BIRTH_DATE_0405"
  },
  "PATIENT_SEX_0405": {
    "record": "DENTAL_CLAIM",

```

```

    "table": "DENTAL_CLAIM",
    "column": "PATIENT_SEX_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-SEX-479LAIM",
    "legacy_field": "PATIENT_SEX_0405"
  },
  "PATIENT-SEX-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_SEX_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-SEX-479LAIM",
    "legacy_field": "PATIENT_SEX_0405"
  },
  "PATIENT_SEX": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PATIENT_SEX_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PATIENT-SEX-479LAIM",
    "legacy_field": "PATIENT_SEX_0445"
  },
  "PATIENTSEX0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_SEX_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-SEX-479LAIM",
    "legacy_field": "PATIENT_SEX_0405"
  },
  "PATIENT_SEX_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_SEX_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-SEX-479LAIM",
    "legacy_field": "PATIENT_SEX_0405"
  },
  "PATIENT-SEX-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_SEX_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-SEX-479LAIM",
    "legacy_field": "PATIENT_SEX_0405"
  },
  "PATIENTSEX": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_SEX_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-SEX-479LAIM",
    "legacy_field": "PATIENT_SEX_0405"
  },
  "RELATION_TO_EMPLOYEE_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "RELATION-TO-EMPLOYEE-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "RELATION_TO_EMPLOYEE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0445"
  },
  "RELATIONTOEMPLOYEE0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "RELATION_TOEMPLOYEE_479LAIM": {

```

```

    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "RELATION-TOEMPLOYEE-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "RELATION_TOEMPLOYEE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "RELATIONTOEMPLOYEE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "DENTIST_FIRST_NAME_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },
  "DENTIST-FIRST-NAME-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },
  "DENTIST_FIRST_NAME": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },
  "DENTISTFIRSTNAME0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },
  "DENTIST_FIRSTNAME_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },
  "DENTIST-FIRSTNAME-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },
  "DENTIST_FIRSTNAME": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },

```

```

"DENTISTFIRSTNAME": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_FIRSTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
  "legacy_field": "DENTIST_FIRST_NAME_0405"
},
"DENTIST_LAST_NAME_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTIST-LAST-NAME-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTIST_LAST_NAME": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTISTLASTNAME0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTIST_LASTNAME_479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTIST-LASTNAME-479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTIST_LASTNAME": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTISTLASTNAME": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTIST_STREET_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_STREET_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-STREET-479LAIM",
  "legacy_field": "DENTIST_STREET_0405"
},
"DENTIST-STREET-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_STREET_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-STREET-479LAIM",
  "legacy_field": "DENTIST_STREET_0405"
}

```

```

},
"DENTIST_STREET": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_STREET_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-STREET-479LAIM",
  "legacy_field": "DENTIST_STREET_0405"
},
"DENTISTSTREET0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_STREET_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-STREET-479LAIM",
  "legacy_field": "DENTIST_STREET_0405"
},
"DENTIST_STREET_479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_STREET_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-STREET-479LAIM",
  "legacy_field": "DENTIST_STREET_0405"
},
"DENTIST-STREET-479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_STREET_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-STREET-479LAIM",
  "legacy_field": "DENTIST_STREET_0405"
},
"DENTISTSTREET": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_STREET_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-STREET-479LAIM",
  "legacy_field": "DENTIST_STREET_0405"
},
"DENTIST_CITY_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_CITY_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-CITY-479LAIM",
  "legacy_field": "DENTIST_CITY_0405"
},
"DENTIST-CITY-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_CITY_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-CITY-479LAIM",
  "legacy_field": "DENTIST_CITY_0405"
},
"DENTIST_CITY": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_CITY_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-CITY-479LAIM",
  "legacy_field": "DENTIST_CITY_0405"
},
"DENTISTCITY0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_CITY_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-CITY-479LAIM",
  "legacy_field": "DENTIST_CITY_0405"
},
"DENTIST_CITY_479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_CITY_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-CITY-479LAIM",
  "legacy_field": "DENTIST_CITY_0405"
},
"DENTIST-CITY-479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_CITY_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-CITY-479LAIM",

```

```

    "legacy_field": "DENTIST_CITY_0405"
  },
  "DENTISTCITY": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_CITY_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-CITY-479LAIM",
    "legacy_field": "DENTIST_CITY_0405"
  },
  "DENTIST_STATE_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE-479LAIM",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTIST-STATE-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE-479LAIM",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTIST_STATE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE-479LAIM",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTISTSTATE0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE-479LAIM",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTIST_STATE_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE-479LAIM",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTIST-STATE-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE-479LAIM",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTISTSTATE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE-479LAIM",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTIST_ZIP_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTIST-ZIP-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTIST_ZIP": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",

```

```

    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTISTZIP0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTIST_ZIPFIRSTFIVE_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTIST-ZIPFIRSTFIVE-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTIST_ZIPFIRSTFIVE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTISTZIPFIRSTFIVE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTIST_ZIP_FIRST_FIVE_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_FIRST_FIVE_0405"
  },
  "DENTIST-ZIP-FIRST-FIVE-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_FIRST_FIVE_0405"
  },
  "DENTIST_ZIP_FIRST_FIVE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_FIRST_FIVE_0405"
  },
  "DENTISTZIPFIRSTFIVE0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_FIRST_FIVE_0405"
  },
  "DENTIST_ZIP_LAST_FOUR_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTIST-ZIP-LAST-FOUR-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",

```



```

    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTIST_ZIP_LAST_FOUR": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTISTZIPLASTFOUR0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTIST_ZIPLASTFOUR_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTIST-ZIPLASTFOUR-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTIST_ZIPLASTFOUR": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTISTZIPLASTFOUR": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTIST_LICENSE_NUMBER_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_LICENSENUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
    "legacy_field": "DENTIST_LICENSE_NUMBER_0405"
  },
  "DENTIST-LICENSE-NUMBER-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_LICENSENUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
    "legacy_field": "DENTIST_LICENSE_NUMBER_0405"
  },
  "DENTIST_LICENSE_NUMBER": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_LICENSENUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
    "legacy_field": "DENTIST_LICENSE_NUMBER_0405"
  },
  "DENTISTLICENSENUMBER0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_LICENSENUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
    "legacy_field": "DENTIST_LICENSE_NUMBER_0405"
  },
  "DENTIST_LICENSENUMBER_479LAIM": {
    "record": "DENTAL_CLAIM",

```

```

"table": "DENTAL_CLAIM",
"column": "DENTIST_LICENSENUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
"legacy_field": "DENTIST_LICENSE_NUMBER_0405"
},
"DENTIST-LICENSENUMBER-479LAIM": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "DENTIST_LICENSENUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
"legacy_field": "DENTIST_LICENSE_NUMBER_0405"
},
"DENTIST_LICENSENUMBER": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "DENTIST_LICENSENUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
"legacy_field": "DENTIST_LICENSE_NUMBER_0405"
},
"DENTISTLICENSENUMBER": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "DENTIST_LICENSENUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
"legacy_field": "DENTIST_LICENSE_NUMBER_0405"
},
"NUMBER_OF_PROCEDURES_0405": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-DENTAL-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEDURES_0405"
},
"NUMBER-OF-PROCEDURES-0405": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-DENTAL-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEDURES_0405"
},
"NUMBER_OF_PROCEDURES": {
"record": "NON_HOSP_CLAIM",
"table": "NON_HOSP_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-NON-HOSP-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEDURES_0445"
},
"NUMBEROFPROCEDURES0405": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-DENTAL-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEDURES_0405"
},
"NUMBER_OFPROCEDURES_479LAIM": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-DENTAL-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEDURES_0405"
},
"NUMBER-OFPROCEDURES-479LAIM": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-DENTAL-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEDURES_0405"
},
"NUMBER_OFPROCEDURES": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-DENTAL-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEDURES_0405"
},
"NUMBEROFPROCEDURES": {

```

```

"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-DENTAL-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEEDURES_0405"
},
"TOOTH_NUMBER_0405": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "TOOTH_NUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-TOOTH-NUMBER-479LAIM",
"legacy_field": "TOOTH_NUMBER_0405"
},
"TOOTH-NUMBER-0405": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "TOOTH_NUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-TOOTH-NUMBER-479LAIM",
"legacy_field": "TOOTH_NUMBER_0405"
},
"TOOTH_NUMBER": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "TOOTH_NUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-TOOTH-NUMBER-479LAIM",
"legacy_field": "TOOTH_NUMBER_0405"
},
"TOOTHNUMBER0405": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "TOOTH_NUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-TOOTH-NUMBER-479LAIM",
"legacy_field": "TOOTH_NUMBER_0405"
},
"TOOTH_NUMBER_479LAIM": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "TOOTH_NUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-TOOTH-NUMBER-479LAIM",
"legacy_field": "TOOTH_NUMBER_0405"
},
"TOOTH-NUMBER-479LAIM": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "TOOTH_NUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-TOOTH-NUMBER-479LAIM",
"legacy_field": "TOOTH_NUMBER_0405"
},
"TOOTHNUMBER": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "TOOTH_NUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-TOOTH-NUMBER-479LAIM",
"legacy_field": "TOOTH_NUMBER_0405"
},
"SERVICE_DATE_0405": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "DA_SERVICEDATE_479LAIM",
"host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
"legacy_field": "SERVICE_DATE_0405"
},
"SERVICE-DATE-0405": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "DA_SERVICEDATE_479LAIM",
"host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
"legacy_field": "SERVICE_DATE_0405"
},
"SERVICE_DATE": {
"record": "NON_HOSP_CLAIM",
"table": "NON_HOSP_CLAIM",
"column": "DA_SERVICEDATE_479LAIM",
"host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
"legacy_field": "SERVICE_DATE_0445"
},

```

```

"SERVICE_DATE_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_SERVICE_DATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-SERVICE_DATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0405"
},
"DA_SERVICE_DATE_479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_SERVICE_DATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-SERVICE_DATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0405"
},
"DA-SERVICE_DATE-479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_SERVICE_DATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-SERVICE_DATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0405"
},
"DA_SERVICE_DATE": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_SERVICE_DATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-SERVICE_DATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0405"
},
"DASERVICE_DATE": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_SERVICE_DATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-SERVICE_DATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0405"
},
"PROCEDURE_CODE_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-DENTAL-CLAIM-PROCEDURE-CODE-479LAIM",
  "legacy_field": "PROCEDURE_CODE_0405"
},
"PROCEDURE-CODE-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-DENTAL-CLAIM-PROCEDURE-CODE-479LAIM",
  "legacy_field": "PROCEDURE_CODE_0405"
},
"PROCEDURE_CODE": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PROCEDURE-CODE-479LAIM",
  "legacy_field": "PROCEDURE_CODE_0445"
},
"PROCEDURECODE0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-DENTAL-CLAIM-PROCEDURE-CODE-479LAIM",
  "legacy_field": "PROCEDURE_CODE_0405"
},
"PROCEDURE_CODE_479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-DENTAL-CLAIM-PROCEDURE-CODE-479LAIM",
  "legacy_field": "PROCEDURE_CODE_0405"
},
"PROCEDURE-CODE-479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-DENTAL-CLAIM-PROCEDURE-CODE-479LAIM",
  "legacy_field": "PROCEDURE_CODE_0405"
}

```

```

},
"PROCEDURECODE": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-DENTAL-CLAIM-PROCEDURE-CODE-479LAIM",
  "legacy_field": "PROCEDURE_CODE_0405"
},
"DESCRIPTION_OF_SERVICE_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
},
"DESCRIPTION-OF-SERVICE-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
},
"DESCRIPTION_OF_SERVICE": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0445"
},
"DESCRIPTIONOFSERVICE0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
},
"DESCRIPTION_OFSERVICE_479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
},
"DESCRIPTION-OFSERVICE-479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
},
"DESCRIPTION_OFSERVICE": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
},
"DESCRIPTIONOFSERVICE": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
},
"FEE_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "FEE_479LAIM",
  "host": "HV-DENTAL-CLAIM-FEE-479LAIM",
  "legacy_field": "FEE_0405"
},
"FEE-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "FEE_479LAIM",
  "host": "HV-DENTAL-CLAIM-FEE-479LAIM",

```

```

    "legacy_field": "FEE_0405"
  },
  "FEE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "FEE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-FEE-479LAIM",
    "legacy_field": "FEE_0445"
  },
  "FEE0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "FEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-FEE-479LAIM",
    "legacy_field": "FEE_0405"
  },
  "FEE_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "FEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-FEE-479LAIM",
    "legacy_field": "FEE_0405"
  },
  "FEE-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "FEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-FEE-479LAIM",
    "legacy_field": "FEE_0405"
  },
  "DEPT_ID_0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-ID-479MENT",
    "legacy_field": "DEPT_ID_0410"
  },
  "DEPT-ID-0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-ID-479MENT",
    "legacy_field": "DEPT_ID_0410"
  },
  "DEPT_ID": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-ID-479MENT",
    "legacy_field": "DEPT_ID_0410"
  },
  "DEPTID0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-ID-479MENT",
    "legacy_field": "DEPT_ID_0410"
  },
  "DEPT_ID_479MENT": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-ID-479MENT",
    "legacy_field": "DEPT_ID_0410"
  },
  "DEPT-ID-479MENT": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-ID-479MENT",
    "legacy_field": "DEPT_ID_0410"
  },
  "DEPTID": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_479MENT",

```

```

    "host": "HV-DEPARTMENT-DEPT-ID-479MENT",
    "legacy_field": "DEPT_ID_0410"
  },
  "DEPT_NAME_0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_NAME_479MENT",
    "host": "HV-DEPARTMENT-DEPT-NAME-479MENT",
    "legacy_field": "DEPT_NAME_0410"
  },
  "DEPT-NAME-0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_NAME_479MENT",
    "host": "HV-DEPARTMENT-DEPT-NAME-479MENT",
    "legacy_field": "DEPT_NAME_0410"
  },
  "DEPT_NAME": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_NAME_479MENT",
    "host": "HV-DEPARTMENT-DEPT-NAME-479MENT",
    "legacy_field": "DEPT_NAME_0410"
  },
  "DEPTNAME0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_NAME_479MENT",
    "host": "HV-DEPARTMENT-DEPT-NAME-479MENT",
    "legacy_field": "DEPT_NAME_0410"
  },
  "DEPT_NAME_479MENT": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_NAME_479MENT",
    "host": "HV-DEPARTMENT-DEPT-NAME-479MENT",
    "legacy_field": "DEPT_NAME_0410"
  },
  "DEPT-NAME-479MENT": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_NAME_479MENT",
    "host": "HV-DEPARTMENT-DEPT-NAME-479MENT",
    "legacy_field": "DEPT_NAME_0410"
  },
  "DEPTNAME": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_NAME_479MENT",
    "host": "HV-DEPARTMENT-DEPT-NAME-479MENT",
    "legacy_field": "DEPT_NAME_0410"
  },
  "DEPT_HEAD_ID_0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "DEPT-HEAD-ID-0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "DEPT_HEAD_ID": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "DEPTHEADID0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",

```

```

    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "DEPT_HEADID_479MENT": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "DEPT-HEADID-479MENT": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "DEPT_HEADID": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "DEPTHEADID": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "EMP_ID_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE",
    "legacy_field": "EMP_ID_0415"
  },
  "EMP-ID-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE",
    "legacy_field": "EMP_ID_0415"
  },
  "EMP_ID": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE",
    "legacy_field": "EMP_ID_0415"
  },
  "EMPID0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE",
    "legacy_field": "EMP_ID_0415"
  },
  "EMP_ID_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE",
    "legacy_field": "EMP_ID_0415"
  },
  "EMP-ID-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE",
    "legacy_field": "EMP_ID_0415"
  },
  "EMPID": {
    "record": "EMPLOYEE",

```



```

    "table": "EMPLOYEE",
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE",
    "legacy_field": "EMP_ID_0415"
  },
  "EMP_FIRST_NAME_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMP-FIRST-NAME-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMP_FIRST_NAME": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMPFIRSTNAME0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMP_FIRSTNAME_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMP-FIRSTNAME-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMP_FIRSTNAME": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMPFIRSTNAME": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMP_LAST_NAME_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMP-LAST-NAME-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMP_LAST_NAME": {

```

```

    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMPLASTNAME0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMP_LASTNAME_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMP-LASTNAME-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMP_LASTNAME": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMPLASTNAME": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMP_STREET_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_STREET_4790YEE",
    "host": "HV-EMPLOYEE-EMP-STREET-4790YEE",
    "legacy_field": "EMP_STREET_0415"
  },
  "EMP-STREET-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_STREET_4790YEE",
    "host": "HV-EMPLOYEE-EMP-STREET-4790YEE",
    "legacy_field": "EMP_STREET_0415"
  },
  "EMP_STREET": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_STREET_4790YEE",
    "host": "HV-EMPLOYEE-EMP-STREET-4790YEE",
    "legacy_field": "EMP_STREET_0415"
  },
  "EMPSTREET0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_STREET_4790YEE",
    "host": "HV-EMPLOYEE-EMP-STREET-4790YEE",
    "legacy_field": "EMP_STREET_0415"
  },
  "EMP_STREET_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_STREET_4790YEE",
    "host": "HV-EMPLOYEE-EMP-STREET-4790YEE",
    "legacy_field": "EMP_STREET_0415"
  },

```

```

"EMP-STREET-4790YEE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STREET_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STREET-4790YEE",
  "legacy_field": "EMP_STREET_0415"
},
"EMPSTREET": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STREET_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STREET-4790YEE",
  "legacy_field": "EMP_STREET_0415"
},
"EMP_CITY_0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_4790YEE",
  "host": "HV-EMPLOYEE-EMP-CITY-4790YEE",
  "legacy_field": "EMP_CITY_0415"
},
"EMP-CITY-0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_4790YEE",
  "host": "HV-EMPLOYEE-EMP-CITY-4790YEE",
  "legacy_field": "EMP_CITY_0415"
},
"EMP_CITY": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_4790YEE",
  "host": "HV-EMPLOYEE-EMP-CITY-4790YEE",
  "legacy_field": "EMP_CITY_0415"
},
"EMPCITY0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_4790YEE",
  "host": "HV-EMPLOYEE-EMP-CITY-4790YEE",
  "legacy_field": "EMP_CITY_0415"
},
"EMP_CITY_4790YEE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_4790YEE",
  "host": "HV-EMPLOYEE-EMP-CITY-4790YEE",
  "legacy_field": "EMP_CITY_0415"
},
"EMP-CITY-4790YEE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_4790YEE",
  "host": "HV-EMPLOYEE-EMP-CITY-4790YEE",
  "legacy_field": "EMP_CITY_0415"
},
"EMPCITY": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_4790YEE",
  "host": "HV-EMPLOYEE-EMP-CITY-4790YEE",
  "legacy_field": "EMP_CITY_0415"
},
"EMP_STATE_0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STATE-4790YEE",
  "legacy_field": "EMP_STATE_0415"
},
"EMP-STATE-0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STATE-4790YEE",
  "legacy_field": "EMP_STATE_0415"
}

```

```

},
"EMP_STATE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STATE-4790YEE",
  "legacy_field": "EMP_STATE_0415"
},
"EMPSTATE0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STATE-4790YEE",
  "legacy_field": "EMP_STATE_0415"
},
"EMP_STATE_4790YEE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STATE-4790YEE",
  "legacy_field": "EMP_STATE_0415"
},
"EMP-STATE-4790YEE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STATE-4790YEE",
  "legacy_field": "EMP_STATE_0415"
},
"EMPSTATE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STATE-4790YEE",
  "legacy_field": "EMP_STATE_0415"
},
"EMP_ZIP_0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIPFIRSTFIVE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
  "legacy_field": "EMP_ZIP_0415"
},
"EMP-ZIP-0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIPFIRSTFIVE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
  "legacy_field": "EMP_ZIP_0415"
},
"EMP_ZIP": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIPFIRSTFIVE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
  "legacy_field": "EMP_ZIP_0415"
},
"EMPZIP0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIPFIRSTFIVE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
  "legacy_field": "EMP_ZIP_0415"
},
"EMP_ZIPFIRSTFIVE_4790YEE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIPFIRSTFIVE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
  "legacy_field": "EMP_ZIP_0415"
},
"EMP-ZIPFIRSTFIVE-4790YEE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIPFIRSTFIVE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",

```

```

    "legacy_field": "EMP_ZIP_0415"
  },
  "EMP_ZIPFIRSTFIVE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPFIRSTFIVE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
    "legacy_field": "EMP_ZIP_0415"
  },
  "EMPZIPFIRSTFIVE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPFIRSTFIVE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
    "legacy_field": "EMP_ZIP_0415"
  },
  "EMP_ZIP_FIRST_FIVE_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPFIRSTFIVE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
    "legacy_field": "EMP_ZIP_FIRST_FIVE_0415"
  },
  "EMP-ZIP-FIRST-FIVE-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPFIRSTFIVE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
    "legacy_field": "EMP_ZIP_FIRST_FIVE_0415"
  },
  "EMP_ZIP_FIRST_FIVE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPFIRSTFIVE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
    "legacy_field": "EMP_ZIP_FIRST_FIVE_0415"
  },
  "EMPZIPFIRSTFIVE0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPFIRSTFIVE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
    "legacy_field": "EMP_ZIP_FIRST_FIVE_0415"
  },
  "EMP_ZIP_LAST_FOUR_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMP-ZIP-LAST-FOUR-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMP_ZIP_LAST_FOUR": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMPZIPLASTFOUR0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMP_ZIPLASTFOUR_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",

```

```

    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMP-ZIPLASTFOUR-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMP_ZIPLASTFOUR": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMPZIPLASTFOUR": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMP_PHONE_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-PHONE-4790YEE",
    "legacy_field": "EMP_PHONE_0415"
  },
  "EMP-PHONE-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-PHONE-4790YEE",
    "legacy_field": "EMP_PHONE_0415"
  },
  "EMP_PHONE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-PHONE-4790YEE",
    "legacy_field": "EMP_PHONE_0415"
  },
  "EMPPHONE0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-PHONE-4790YEE",
    "legacy_field": "EMP_PHONE_0415"
  },
  "EMP_PHONE_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-PHONE-4790YEE",
    "legacy_field": "EMP_PHONE_0415"
  },
  "EMP-PHONE-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-PHONE-4790YEE",
    "legacy_field": "EMP_PHONE_0415"
  },
  "EMPPHONE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-PHONE-4790YEE",
    "legacy_field": "EMP_PHONE_0415"
  },
  "STATUS_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",

```

```

    "column": "STATUS_4790YEE",
    "host": "HV-EMPLOYEE-STATUS-4790YEE",
    "legacy_field": "STATUS_0415"
  },
  "STATUS-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "STATUS_4790YEE",
    "host": "HV-EMPLOYEE-STATUS-4790YEE",
    "legacy_field": "STATUS_0415"
  },
  "STATUS": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "STATUS_4790YEE",
    "host": "HV-EMPLOYEE-STATUS-4790YEE",
    "legacy_field": "STATUS_0415"
  },
  "STATUS0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "STATUS_4790YEE",
    "host": "HV-EMPLOYEE-STATUS-4790YEE",
    "legacy_field": "STATUS_0415"
  },
  "STATUS_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "STATUS_4790YEE",
    "host": "HV-EMPLOYEE-STATUS-4790YEE",
    "legacy_field": "STATUS_0415"
  },
  "STATUS-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "STATUS_4790YEE",
    "host": "HV-EMPLOYEE-STATUS-4790YEE",
    "legacy_field": "STATUS_0415"
  },
  "SS_NUMBER_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "SS_NUMBER_4790YEE",
    "host": "HV-EMPLOYEE-SS-NUMBER-4790YEE",
    "legacy_field": "SS_NUMBER_0415"
  },
  "SS-NUMBER-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "SS_NUMBER_4790YEE",
    "host": "HV-EMPLOYEE-SS-NUMBER-4790YEE",
    "legacy_field": "SS_NUMBER_0415"
  },
  "SS_NUMBER": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "SS_NUMBER_4790YEE",
    "host": "HV-EMPLOYEE-SS-NUMBER-4790YEE",
    "legacy_field": "SS_NUMBER_0415"
  },
  "SSNUMBER0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "SS_NUMBER_4790YEE",
    "host": "HV-EMPLOYEE-SS-NUMBER-4790YEE",
    "legacy_field": "SS_NUMBER_0415"
  },
  "SS_NUMBER_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "SS_NUMBER_4790YEE",
    "host": "HV-EMPLOYEE-SS-NUMBER-4790YEE",
    "legacy_field": "SS_NUMBER_0415"
  },
  "SS-NUMBER-4790YEE": {
    "record": "EMPLOYEE",

```

```

    "table": "EMPLOYEE",
    "column": "SS_NUMBER_4790YEE",
    "host": "HV-EMPLOYEE-SS-NUMBER-4790YEE",
    "legacy_field": "SS_NUMBER_0415"
  },
  "SSNUMBER": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "SS_NUMBER_4790YEE",
    "host": "HV-EMPLOYEE-SS-NUMBER-4790YEE",
    "legacy_field": "SS_NUMBER_0415"
  },
  "START_DATE_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "legacy_field": "START_DATE_0415"
  },
  "START-DATE-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "legacy_field": "START_DATE_0415"
  },
  "START_DATE": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_STARTDATE_479TION",
    "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
    "legacy_field": "START_DATE_0420"
  },
  "STARTDATE0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "legacy_field": "START_DATE_0415"
  },
  "DA_STARTDATE_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "legacy_field": "START_DATE_0415"
  },
  "DA-STARTDATE-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "legacy_field": "START_DATE_0415"
  },
  "DA_STARTDATE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "legacy_field": "START_DATE_0415"
  },
  "DASTARTDATE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "legacy_field": "START_DATE_0415"
  },
  "TERMINATION_DATE_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "legacy_field": "TERMINATION_DATE_0415"
  },
  "TERMINATION-DATE-0415": {

```



```

    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "legacy_field": "TERMINATION_DATE_0415"
  },
  "TERMINATIONDATE0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "legacy_field": "TERMINATION_DATE_0415"
  },
  "DA_TERMINATIONDATE_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "legacy_field": "TERMINATION_DATE_0415"
  },
  "DA-TERMINATIONDATE-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "legacy_field": "TERMINATION_DATE_0415"
  },
  "BIRTH_DATE_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "legacy_field": "BIRTH_DATE_0415"
  },
  "BIRTH-DATE-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "legacy_field": "BIRTH_DATE_0415"
  },
  "BIRTH_DATE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "legacy_field": "BIRTH_DATE_0415"
  },
  "BIRTHDATE0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "legacy_field": "BIRTH_DATE_0415"
  },
  "DA_BIRTHDATE_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "legacy_field": "BIRTH_DATE_0415"
  },
  "DA-BIRTHDATE-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "legacy_field": "BIRTH_DATE_0415"
  },
  "DA_BIRTHDATE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "legacy_field": "BIRTH_DATE_0415"
  },

```

```

"DABIRTHDATE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "DA_BIRTHDATE_4790YEE",
  "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
  "legacy_field": "BIRTH_DATE_0415"
},
"START_DATE_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "legacy_field": "START_DATE_0420"
},
"START-DATE-0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "legacy_field": "START_DATE_0420"
},
"STARTDATE0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "legacy_field": "START_DATE_0420"
},
"DA_STARTDATE_479TION": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "legacy_field": "START_DATE_0420"
},
"DA-STARTDATE-479TION": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "legacy_field": "START_DATE_0420"
},
"FINISH_DATE_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
},
"FINISH-DATE-0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
},
"FINISH_DATE": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
},
"FINISHDATE0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
},
"DA_FINISHDATE_479TION": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
}

```

```

},
"DA-FINISHDATE-479TION": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
},
"DA_FINISHDATE": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
},
"DAFINISHDATE": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
},
"SALARY_GRADE_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_GRADE_479TION",
  "host": "HV-EMPOSITION-SALARY-GRADE-479TION",
  "legacy_field": "SALARY_GRADE_0420"
},
"SALARY-GRADE-0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_GRADE_479TION",
  "host": "HV-EMPOSITION-SALARY-GRADE-479TION",
  "legacy_field": "SALARY_GRADE_0420"
},
"SALARY_GRADE": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_GRADE_479TION",
  "host": "HV-EMPOSITION-SALARY-GRADE-479TION",
  "legacy_field": "SALARY_GRADE_0420"
},
"SALARYGRADE0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_GRADE_479TION",
  "host": "HV-EMPOSITION-SALARY-GRADE-479TION",
  "legacy_field": "SALARY_GRADE_0420"
},
"SALARY_GRADE_479TION": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_GRADE_479TION",
  "host": "HV-EMPOSITION-SALARY-GRADE-479TION",
  "legacy_field": "SALARY_GRADE_0420"
},
"SALARY-GRADE-479TION": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_GRADE_479TION",
  "host": "HV-EMPOSITION-SALARY-GRADE-479TION",
  "legacy_field": "SALARY_GRADE_0420"
},
"SALARYGRADE": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_GRADE_479TION",
  "host": "HV-EMPOSITION-SALARY-GRADE-479TION",
  "legacy_field": "SALARY_GRADE_0420"
},
"SALARY_AMOUNT_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_AMOUNT_479TION",
  "host": "HV-EMPOSITION-SALARY-AMOUNT-479TION",

```

```

    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "SALARY-AMOUNT-0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_AMOUNT_479TION",
    "host": "HV-EMPOSITION-SALARY-AMOUNT-479TION",
    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "SALARY_AMOUNT": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_AMOUNT_479TION",
    "host": "HV-EMPOSITION-SALARY-AMOUNT-479TION",
    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "SALARYAMOUNT0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_AMOUNT_479TION",
    "host": "HV-EMPOSITION-SALARY-AMOUNT-479TION",
    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "SALARY_AMOUNT_479TION": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_AMOUNT_479TION",
    "host": "HV-EMPOSITION-SALARY-AMOUNT-479TION",
    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "SALARY-AMOUNT-479TION": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_AMOUNT_479TION",
    "host": "HV-EMPOSITION-SALARY-AMOUNT-479TION",
    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "SALARYAMOUNT": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_AMOUNT_479TION",
    "host": "HV-EMPOSITION-SALARY-AMOUNT-479TION",
    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "BONUS_PERCENT_0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_479TION",
    "host": "HV-EMPOSITION-BONUS-PERCENT-479TION",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "BONUS-PERCENT-0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_479TION",
    "host": "HV-EMPOSITION-BONUS-PERCENT-479TION",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "BONUS_PERCENT": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_479TION",
    "host": "HV-EMPOSITION-BONUS-PERCENT-479TION",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "BONUSPERCENT0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_479TION",
    "host": "HV-EMPOSITION-BONUS-PERCENT-479TION",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "BONUS_PERCENT_479TION": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_479TION",

```

```

    "host": "HV-EMPOSITION-BONUS-PERCENT-479TION",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "BONUS-PERCENT-479TION": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_479TION",
    "host": "HV-EMPOSITION-BONUS-PERCENT-479TION",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "BONUSPERCENT": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_479TION",
    "host": "HV-EMPOSITION-BONUS-PERCENT-479TION",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "COMMISSION_PERCENT_0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_479TION",
    "host": "HV-EMPOSITION-COMMISSION-PERCENT-479TION",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "COMMISSION-PERCENT-0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_479TION",
    "host": "HV-EMPOSITION-COMMISSION-PERCENT-479TION",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "COMMISSION_PERCENT": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_479TION",
    "host": "HV-EMPOSITION-COMMISSION-PERCENT-479TION",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "COMMISSIONPERCENT0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_479TION",
    "host": "HV-EMPOSITION-COMMISSION-PERCENT-479TION",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "COMMISSION_PERCENT_479TION": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_479TION",
    "host": "HV-EMPOSITION-COMMISSION-PERCENT-479TION",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "COMMISSION-PERCENT-479TION": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_479TION",
    "host": "HV-EMPOSITION-COMMISSION-PERCENT-479TION",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "COMMISSIONPERCENT": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_479TION",
    "host": "HV-EMPOSITION-COMMISSION-PERCENT-479TION",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "OVERTIME_RATE_0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "OVERTIME_RATE_479TION",
    "host": "HV-EMPOSITION-OVERTIME-RATE-479TION",
    "legacy_field": "OVERTIME_RATE_0420"
  },
  "OVERTIME-RATE-0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",

```

```

"column": "OVERTIME_RATE_479TION",
"host": "HV-EMPOSITION-OVERTIME-RATE-479TION",
"legacy_field": "OVERTIME_RATE_0420"
},
"OVERTIME_RATE": {
"record": "EMPOSITION",
"table": "EMPOSITION",
"column": "OVERTIME_RATE_479TION",
"host": "HV-EMPOSITION-OVERTIME-RATE-479TION",
"legacy_field": "OVERTIME_RATE_0420"
},
"OVERTIMERATE0420": {
"record": "EMPOSITION",
"table": "EMPOSITION",
"column": "OVERTIME_RATE_479TION",
"host": "HV-EMPOSITION-OVERTIME-RATE-479TION",
"legacy_field": "OVERTIME_RATE_0420"
},
"OVERTIME_RATE_479TION": {
"record": "EMPOSITION",
"table": "EMPOSITION",
"column": "OVERTIME_RATE_479TION",
"host": "HV-EMPOSITION-OVERTIME-RATE-479TION",
"legacy_field": "OVERTIME_RATE_0420"
},
"OVERTIME-RATE-479TION": {
"record": "EMPOSITION",
"table": "EMPOSITION",
"column": "OVERTIME_RATE_479TION",
"host": "HV-EMPOSITION-OVERTIME-RATE-479TION",
"legacy_field": "OVERTIME_RATE_0420"
},
"OVERTIMERATE": {
"record": "EMPOSITION",
"table": "EMPOSITION",
"column": "OVERTIME_RATE_479TION",
"host": "HV-EMPOSITION-OVERTIME-RATE-479TION",
"legacy_field": "OVERTIME_RATE_0420"
},
"SKILL_LEVEL_0425": {
"record": "EXPERTISE",
"table": "EXPERTISE",
"column": "SKILL_LEVEL_479TISE",
"host": "HV-EXPERTISE-SKILL-LEVEL-479TISE",
"legacy_field": "SKILL_LEVEL_0425"
},
"SKILL-LEVEL-0425": {
"record": "EXPERTISE",
"table": "EXPERTISE",
"column": "SKILL_LEVEL_479TISE",
"host": "HV-EXPERTISE-SKILL-LEVEL-479TISE",
"legacy_field": "SKILL_LEVEL_0425"
},
"SKILL_LEVEL": {
"record": "EXPERTISE",
"table": "EXPERTISE",
"column": "SKILL_LEVEL_479TISE",
"host": "HV-EXPERTISE-SKILL-LEVEL-479TISE",
"legacy_field": "SKILL_LEVEL_0425"
},
"SKILLLEVEL0425": {
"record": "EXPERTISE",
"table": "EXPERTISE",
"column": "SKILL_LEVEL_479TISE",
"host": "HV-EXPERTISE-SKILL-LEVEL-479TISE",
"legacy_field": "SKILL_LEVEL_0425"
},
"SKILL_LEVEL_479TISE": {
"record": "EXPERTISE",
"table": "EXPERTISE",
"column": "SKILL_LEVEL_479TISE",
"host": "HV-EXPERTISE-SKILL-LEVEL-479TISE",
"legacy_field": "SKILL_LEVEL_0425"
},
"SKILL-LEVEL-479TISE": {
"record": "EXPERTISE",

```

```

    "table": "EXPERTISE",
    "column": "SKILL_LEVEL_479TISE",
    "host": "HV-EXPERTISE-SKILL-LEVEL-479TISE",
    "legacy_field": "SKILL_LEVEL_0425"
  },
  "SKILLLEVEL": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "SKILL_LEVEL_479TISE",
    "host": "HV-EXPERTISE-SKILL-LEVEL-479TISE",
    "legacy_field": "SKILL_LEVEL_0425"
  },
  "EXPERTISE_DATE_0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "EXPERTISE-DATE-0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "EXPERTISE_DATE": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "EXPERTISEDATE0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "DA_EXPERTISEDATE_479TISE": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "DA-EXPERTISEDATE-479TISE": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "DA_EXPERTISEDATE": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "DAEXPERTISEDATE": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "CLAIM_DATE_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0430"
  },
  "CLAIM-DATE-0430": {

```

```

"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_CLAIMDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
"legacy_field": "CLAIM_DATE_0430"
},
"CLAIMDATE0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_CLAIMDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
"legacy_field": "CLAIM_DATE_0430"
},
"PATIENT_FIRST_NAME_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "PATIENT_FIRSTNAME_479LAIM",
"host": "HV-HOSPITAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
"legacy_field": "PATIENT_FIRST_NAME_0430"
},
"PATIENT-FIRST-NAME-0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "PATIENT_FIRSTNAME_479LAIM",
"host": "HV-HOSPITAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
"legacy_field": "PATIENT_FIRST_NAME_0430"
},
"PATIENTFIRSTNAME0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "PATIENT_FIRSTNAME_479LAIM",
"host": "HV-HOSPITAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
"legacy_field": "PATIENT_FIRST_NAME_0430"
},
"PATIENT_LAST_NAME_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "PATIENT_LASTNAME_479LAIM",
"host": "HV-HOSPITAL-CLAIM-PATIENT-LASTNAME-479LAIM",
"legacy_field": "PATIENT_LAST_NAME_0430"
},
"PATIENT-LAST-NAME-0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "PATIENT_LASTNAME_479LAIM",
"host": "HV-HOSPITAL-CLAIM-PATIENT-LASTNAME-479LAIM",
"legacy_field": "PATIENT_LAST_NAME_0430"
},
"PATIENTLASTNAME0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "PATIENT_LASTNAME_479LAIM",
"host": "HV-HOSPITAL-CLAIM-PATIENT-LASTNAME-479LAIM",
"legacy_field": "PATIENT_LAST_NAME_0430"
},
"PATIENT_BIRTH_DATE_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_PATIENTBIRTHDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
"legacy_field": "PATIENT_BIRTH_DATE_0430"
},
"PATIENT-BIRTH-DATE-0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_PATIENTBIRTHDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
"legacy_field": "PATIENT_BIRTH_DATE_0430"
},
"PATIENTBIRTHDATE0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_PATIENTBIRTHDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
"legacy_field": "PATIENT_BIRTH_DATE_0430"
},

```



```

"PATIENT_SEX_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "PATIENT_SEX_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-PATIENT-SEX-479LAIM",
  "legacy_field": "PATIENT_SEX_0430"
},
"PATIENT-SEX-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "PATIENT_SEX_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-PATIENT-SEX-479LAIM",
  "legacy_field": "PATIENT_SEX_0430"
},
"PATIENTSEX0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "PATIENT_SEX_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-PATIENT-SEX-479LAIM",
  "legacy_field": "PATIENT_SEX_0430"
},
"RELATION_TO_EMPLOYEE_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "RELATION_TOEMPLOYEE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
  "legacy_field": "RELATION_TO_EMPLOYEE_0430"
},
"RELATION-TO-EMPLOYEE-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "RELATION_TOEMPLOYEE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
  "legacy_field": "RELATION_TO_EMPLOYEE_0430"
},
"RELATIONTOEMPLOYEE0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "RELATION_TOEMPLOYEE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
  "legacy_field": "RELATION_TO_EMPLOYEE_0430"
},
"HOSPITAL_NAME_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSPITAL_NAME_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME-479LAIM",
  "legacy_field": "HOSPITAL_NAME_0430"
},
"HOSPITAL-NAME-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSPITAL_NAME_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME-479LAIM",
  "legacy_field": "HOSPITAL_NAME_0430"
},
"HOSPITAL_NAME": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSPITAL_NAME_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME-479LAIM",
  "legacy_field": "HOSPITAL_NAME_0430"
},
"HOSPITALNAME0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSPITAL_NAME_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME-479LAIM",
  "legacy_field": "HOSPITAL_NAME_0430"
},
"HOSPITAL_NAME_479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSPITAL_NAME_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME-479LAIM",
  "legacy_field": "HOSPITAL_NAME_0430"
}

```

```

},
"HOSPITAL-NAME-479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSPITAL_NAME_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME-479LAIM",
  "legacy_field": "HOSPITAL_NAME_0430"
},
"HOSPITALNAME": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSPITAL_NAME_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME-479LAIM",
  "legacy_field": "HOSPITAL_NAME_0430"
},
"HOSP_STREET_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_STREET_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-STREET-479LAIM",
  "legacy_field": "HOSP_STREET_0430"
},
"HOSP-STREET-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_STREET_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-STREET-479LAIM",
  "legacy_field": "HOSP_STREET_0430"
},
"HOSP_STREET": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_STREET_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-STREET-479LAIM",
  "legacy_field": "HOSP_STREET_0430"
},
"HOSPSTREET0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_STREET_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-STREET-479LAIM",
  "legacy_field": "HOSP_STREET_0430"
},
"HOSP_STREET_479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_STREET_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-STREET-479LAIM",
  "legacy_field": "HOSP_STREET_0430"
},
"HOSP-STREET-479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_STREET_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-STREET-479LAIM",
  "legacy_field": "HOSP_STREET_0430"
},
"HOSPSTREET": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_STREET_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-STREET-479LAIM",
  "legacy_field": "HOSP_STREET_0430"
},
"HOSP_CITY_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_CITY_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-CITY-479LAIM",
  "legacy_field": "HOSP_CITY_0430"
},
"HOSP-CITY-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_CITY_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-CITY-479LAIM",

```

```

    "legacy_field": "HOSP_CITY_0430"
  },
  "HOSP_CITY": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_CITY_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-CITY-479LAIM",
    "legacy_field": "HOSP_CITY_0430"
  },
  "HOSPCITY0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_CITY_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-CITY-479LAIM",
    "legacy_field": "HOSP_CITY_0430"
  },
  "HOSP_CITY_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_CITY_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-CITY-479LAIM",
    "legacy_field": "HOSP_CITY_0430"
  },
  "HOSP-CITY-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_CITY_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-CITY-479LAIM",
    "legacy_field": "HOSP_CITY_0430"
  },
  "HOSPCITY": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_CITY_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-CITY-479LAIM",
    "legacy_field": "HOSP_CITY_0430"
  },
  "HOSP_STATE_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE-479LAIM",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSP-STATE-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE-479LAIM",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSP_STATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE-479LAIM",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSPSTATE0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE-479LAIM",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSP_STATE_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE-479LAIM",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSP-STATE-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_479LAIM",

```

```

    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE-479LAIM",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSPSTATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE-479LAIM",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSP_ZIP_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSP-ZIP-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSP_ZIP": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSPZIP0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSP_ZIPFIRSTFIVE_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSP-ZIPFIRSTFIVE-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSP_ZIPFIRSTFIVE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSPZIPFIRSTFIVE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSP_ZIP_FIRST_FIVE_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_FIRST_FIVE_0430"
  },
  "HOSP-ZIP-FIRST-FIVE-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",

```

```

    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_FIRST_FIVE_0430"
  },
  "HOSP_ZIP_FIRST_FIVE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_FIRST_FIVE_0430"
  },
  "HOSPZIPFIRSTFIVE0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_FIRST_FIVE_0430"
  },
  "HOSP_ZIP_LAST_FOUR_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSP-ZIP-LAST-FOUR-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSP_ZIP_LAST_FOUR": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSPZIPLASTFOUR0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSP_ZIPLASTFOUR_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSP-ZIPLASTFOUR-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSP_ZIPLASTFOUR": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSPZIPLASTFOUR": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "ADMIT_DATE_0430": {
    "record": "HOSPITAL_CLAIM",

```

```

    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "legacy_field": "ADMIT_DATE_0430"
  },
  "ADMIT-DATE-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "legacy_field": "ADMIT_DATE_0430"
  },
  "ADMIT_DATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "legacy_field": "ADMIT_DATE_0430"
  },
  "ADMITDATE0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "legacy_field": "ADMIT_DATE_0430"
  },
  "DA_ADMITDATE_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "legacy_field": "ADMIT_DATE_0430"
  },
  "DA-ADMITDATE-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "legacy_field": "ADMIT_DATE_0430"
  },
  "DA_ADMITDATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "legacy_field": "ADMIT_DATE_0430"
  },
  "DAADMITDATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "legacy_field": "ADMIT_DATE_0430"
  },
  "DISCHARGE_DATE_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "legacy_field": "DISCHARGE_DATE_0430"
  },
  "DISCHARGE-DATE-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "legacy_field": "DISCHARGE_DATE_0430"
  },
  "DISCHARGE_DATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "legacy_field": "DISCHARGE_DATE_0430"
  },
  "DISCHARGEDATE0430": {

```

```

    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "legacy_field": "DISCHARGE_DATE_0430"
  },
  "DA_DISCHARGEDATE_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "legacy_field": "DISCHARGE_DATE_0430"
  },
  "DA-DISCHARGEDATE-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "legacy_field": "DISCHARGE_DATE_0430"
  },
  "DA_DISCHARGEDATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "legacy_field": "DISCHARGE_DATE_0430"
  },
  "DADISCHARGEDATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "legacy_field": "DISCHARGE_DATE_0430"
  },
  "DIAGNOSIS_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DIAGNOSIS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DIAGNOSIS-479LAIM",
    "legacy_field": "DIAGNOSIS_0430"
  },
  "DIAGNOSIS-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DIAGNOSIS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DIAGNOSIS-479LAIM",
    "legacy_field": "DIAGNOSIS_0430"
  },
  "DIAGNOSIS": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DIAGNOSIS_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DIAGNOSIS-479LAIM",
    "legacy_field": "DIAGNOSIS_0445"
  },
  "DIAGNOSIS0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DIAGNOSIS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DIAGNOSIS-479LAIM",
    "legacy_field": "DIAGNOSIS_0430"
  },
  "DIAGNOSIS_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DIAGNOSIS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DIAGNOSIS-479LAIM",
    "legacy_field": "DIAGNOSIS_0430"
  },
  "DIAGNOSIS-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DIAGNOSIS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DIAGNOSIS-479LAIM",
    "legacy_field": "DIAGNOSIS_0430"
  },

```

```

"WARD_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_0430"
},
"WARD-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_0430"
},
"WARD": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_0430"
},
"WARD0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_0430"
},
"WARD_DAYS_479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_0430"
},
"WARD-DAYS-479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_0430"
},
"WARD_DAYS": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_DAYS_0430"
},
"WARD_DAYS": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_DAYS_0430"
},
"WARD_DAYS_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_DAYS_0430"
},
"WARD-DAYS-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_DAYS_0430"
},
"WARD_DAYS0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_DAYS_0430"
}

```



```

},
"WARD_RATE_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_RATE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-RATE-479LAIM",
  "legacy_field": "WARD_RATE_0430"
},
"WARD-RATE-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_RATE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-RATE-479LAIM",
  "legacy_field": "WARD_RATE_0430"
},
"WARD_RATE": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_RATE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-RATE-479LAIM",
  "legacy_field": "WARD_RATE_0430"
},
"WARDRATE0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_RATE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-RATE-479LAIM",
  "legacy_field": "WARD_RATE_0430"
},
"WARD_RATE_479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_RATE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-RATE-479LAIM",
  "legacy_field": "WARD_RATE_0430"
},
"WARD-RATE-479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_RATE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-RATE-479LAIM",
  "legacy_field": "WARD_RATE_0430"
},
"WARDRATE": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_RATE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-RATE-479LAIM",
  "legacy_field": "WARD_RATE_0430"
},
"WARD_TOTAL_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_TOTAL_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL-479LAIM",
  "legacy_field": "WARD_TOTAL_0430"
},
"WARD-TOTAL-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_TOTAL_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL-479LAIM",
  "legacy_field": "WARD_TOTAL_0430"
},
"WARD_TOTAL": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_TOTAL_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL-479LAIM",
  "legacy_field": "WARD_TOTAL_0430"
},
"WARDTOTAL0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_TOTAL_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL-479LAIM",

```

```

    "legacy_field": "WARD_TOTAL_0430"
  },
  "WARD_TOTAL_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "WARD_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL-479LAIM",
    "legacy_field": "WARD_TOTAL_0430"
  },
  "WARD-TOTAL-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "WARD_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL-479LAIM",
    "legacy_field": "WARD_TOTAL_0430"
  },
  "WARDTOTAL": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "WARD_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL-479LAIM",
    "legacy_field": "WARD_TOTAL_0430"
  },
  "SEMI_DAYS_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS-479LAIM",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMI-DAYS-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS-479LAIM",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMI_DAYS": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS-479LAIM",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMIDAYS0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS-479LAIM",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMI_DAYS_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS-479LAIM",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMI-DAYS-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS-479LAIM",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMIDAYS": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS-479LAIM",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMI_RATE_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_479LAIM",

```

```

    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE-479LAIM",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI-RATE-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE-479LAIM",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI_RATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE-479LAIM",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI-RATE-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE-479LAIM",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI_RATE_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE-479LAIM",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI-RATE-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE-479LAIM",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI-RATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE-479LAIM",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI_TOTAL_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL-479LAIM",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "SEMI-TOTAL-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL-479LAIM",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "SEMI_TOTAL": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL-479LAIM",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "SEMITOTAL0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL-479LAIM",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "SEMI_TOTAL_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",

```

```

    "column": "SEMI_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL-479LAIM",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "SEMI-TOTAL-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL-479LAIM",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "SEMITOTAL": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL-479LAIM",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "DELIVERY_COST_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST-479LAIM",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "DELIVERY-COST-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST-479LAIM",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "DELIVERY_COST": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST-479LAIM",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "DELIVERYCOST0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST-479LAIM",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "DELIVERY_COST_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST-479LAIM",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "DELIVERY-COST-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST-479LAIM",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "DELIVERYCOST": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST-479LAIM",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "ANESTHESIA_COST_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST-479LAIM",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "ANESTHESIA-COST-0430": {
    "record": "HOSPITAL_CLAIM",

```

```

    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST-479LAIM",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "ANESTHESIA_COST": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST-479LAIM",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "ANESTHESIACOST0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST-479LAIM",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "ANESTHESIA_COST_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST-479LAIM",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "ANESTHESIA-COST-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST-479LAIM",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "ANESTHESIACOST": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST-479LAIM",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "LAB_COST_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-LAB-COST-479LAIM",
    "legacy_field": "LAB_COST_0430"
  },
  "LAB-COST-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-LAB-COST-479LAIM",
    "legacy_field": "LAB_COST_0430"
  },
  "LAB_COST": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-LAB-COST-479LAIM",
    "legacy_field": "LAB_COST_0430"
  },
  "LABCOST0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-LAB-COST-479LAIM",
    "legacy_field": "LAB_COST_0430"
  },
  "LAB_COST_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-LAB-COST-479LAIM",
    "legacy_field": "LAB_COST_0430"
  },
  "LAB-COST-479LAIM": {

```

```

    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-LAB-COST-479LAIM",
    "legacy_field": "LAB_COST_0430"
  },
  "LABCOST": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-LAB-COST-479LAIM",
    "legacy_field": "LAB_COST_0430"
  },
  "INS_PLAN_CODE_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_PLANCODE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-PLANCODE-479PLAN",
    "legacy_field": "INS_PLAN_CODE_0435"
  },
  "INS-PLAN-CODE-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_PLANCODE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-PLANCODE-479PLAN",
    "legacy_field": "INS_PLAN_CODE_0435"
  },
  "INSPLANCODE0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_PLANCODE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-PLANCODE-479PLAN",
    "legacy_field": "INS_PLAN_CODE_0435"
  },
  "INS_PLANCODE_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_PLANCODE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-PLANCODE-479PLAN",
    "legacy_field": "INS_PLAN_CODE_0435"
  },
  "INS-PLANCODE-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_PLANCODE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-PLANCODE-479PLAN",
    "legacy_field": "INS_PLAN_CODE_0435"
  },
  "INS_CO_NAME_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_CONAME_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
    "legacy_field": "INS_CO_NAME_0435"
  },
  "INS-CO-NAME-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_CONAME_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
    "legacy_field": "INS_CO_NAME_0435"
  },
  "INS_CO_NAME": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_CONAME_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
    "legacy_field": "INS_CO_NAME_0435"
  },
  "INSCONAME0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_CONAME_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
    "legacy_field": "INS_CO_NAME_0435"
  },
  },

```

```

"INS_CONAME_479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CONAME_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
  "legacy_field": "INS_CO_NAME_0435"
},
"INS-CONAME-479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CONAME_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
  "legacy_field": "INS_CO_NAME_0435"
},
"INS_CONAME": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CONAME_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
  "legacy_field": "INS_CO_NAME_0435"
},
"INSCONAME": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CONAME_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
  "legacy_field": "INS_CO_NAME_0435"
},
"INS_CO_STREET_0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
},
"INS-CO-STREET-0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
},
"INS_CO_STREET": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
},
"INSCOSTREET0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
},
"INS_COSTREET_479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
},
"INS-COSTREET-479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
},
"INS_COSTREET": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
}

```

```

},
"INSCOSTREET": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
},
"INS_CO_CITY_0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INS-CO-CITY-0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INS_CO_CITY": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INSCOCITY0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INS_COCITY_479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INS-COCITY-479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INS_COCITY": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INSCOCITY": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INS_CO_STATE_0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTATE_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",
  "legacy_field": "INS_CO_STATE_0435"
},
"INS-CO-STATE-0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTATE_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",

```



```

    "legacy_field": "INS_CO_STATE_0435"
  },
  "INS_CO_STATE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COSTATE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",
    "legacy_field": "INS_CO_STATE_0435"
  },
  "INSCOSTATE0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COSTATE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",
    "legacy_field": "INS_CO_STATE_0435"
  },
  "INS_COSTATE_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COSTATE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",
    "legacy_field": "INS_CO_STATE_0435"
  },
  "INS-COSTATE-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COSTATE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",
    "legacy_field": "INS_CO_STATE_0435"
  },
  "INS_COSTATE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COSTATE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",
    "legacy_field": "INS_CO_STATE_0435"
  },
  "INSCOSTATE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COSTATE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",
    "legacy_field": "INS_CO_STATE_0435"
  },
  "INS_CO_ZIP_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INS-CO-ZIP-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INS_CO_ZIP": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INSCOZIP0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INS_COZIPFIRSTFIVE_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",

```

```

    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INS-COZIPFIRSTFIVE-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INS_COZIPFIRSTFIVE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INSCOZIPFIRSTFIVE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INS_CO_ZIP_FIRST_FIVE_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_FIRST_FIVE_0435"
  },
  "INS-CO-ZIP-FIRST-FIVE-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_FIRST_FIVE_0435"
  },
  "INS_CO_ZIP_FIRST_FIVE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_FIRST_FIVE_0435"
  },
  "INSCOZIPFIRSTFIVE0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_FIRST_FIVE_0435"
  },
  "INS_CO_ZIP_LAST_FOUR_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INS-CO-ZIP-LAST-FOUR-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INS_CO_ZIP_LAST_FOUR": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INSCOZIPLASTFOUR0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",

```

```

    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INS_COZIPLASTFOUR_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INS-COZIPLASTFOUR-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INS_COZIPLASTFOUR": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INSCOZIPLASTFOUR": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INS_CO_PHONE_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COPHONE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
    "legacy_field": "INS_CO_PHONE_0435"
  },
  "INS-CO-PHONE-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COPHONE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
    "legacy_field": "INS_CO_PHONE_0435"
  },
  "INS_CO_PHONE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COPHONE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
    "legacy_field": "INS_CO_PHONE_0435"
  },
  "INSCOPHONE0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COPHONE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
    "legacy_field": "INS_CO_PHONE_0435"
  },
  "INS_COPHONE_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COPHONE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
    "legacy_field": "INS_CO_PHONE_0435"
  },
  "INS-COPHONE-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COPHONE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
    "legacy_field": "INS_CO_PHONE_0435"
  },
  "INS_COPHONE": {
    "record": "INSURANCE_PLAN",

```

```

    "table": "INSURANCE_PLAN",
    "column": "INS_COPHONE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
    "legacy_field": "INS_CO_PHONE_0435"
  },
  "INSCOPHONE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COPHONE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
    "legacy_field": "INS_CO_PHONE_0435"
  },
  "GROUP_NUMBER_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "GROUP_NUMBER_479PLAN",
    "host": "HV-INSURANCE-PLAN-GROUP-NUMBER-479PLAN",
    "legacy_field": "GROUP_NUMBER_0435"
  },
  "GROUP-NUMBER-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "GROUP_NUMBER_479PLAN",
    "host": "HV-INSURANCE-PLAN-GROUP-NUMBER-479PLAN",
    "legacy_field": "GROUP_NUMBER_0435"
  },
  "GROUP_NUMBER": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "GROUP_NUMBER_479PLAN",
    "host": "HV-INSURANCE-PLAN-GROUP-NUMBER-479PLAN",
    "legacy_field": "GROUP_NUMBER_0435"
  },
  "GROUPNUMBER0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "GROUP_NUMBER_479PLAN",
    "host": "HV-INSURANCE-PLAN-GROUP-NUMBER-479PLAN",
    "legacy_field": "GROUP_NUMBER_0435"
  },
  "GROUP_NUMBER_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "GROUP_NUMBER_479PLAN",
    "host": "HV-INSURANCE-PLAN-GROUP-NUMBER-479PLAN",
    "legacy_field": "GROUP_NUMBER_0435"
  },
  "GROUP-NUMBER-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "GROUP_NUMBER_479PLAN",
    "host": "HV-INSURANCE-PLAN-GROUP-NUMBER-479PLAN",
    "legacy_field": "GROUP_NUMBER_0435"
  },
  "GROUPNUMBER": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "GROUP_NUMBER_479PLAN",
    "host": "HV-INSURANCE-PLAN-GROUP-NUMBER-479PLAN",
    "legacy_field": "GROUP_NUMBER_0435"
  },
  "DEDUCT_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "DEDUCT_479PLAN",
    "host": "HV-INSURANCE-PLAN-DEDUCT-479PLAN",
    "legacy_field": "DEDUCT_0435"
  },
  "DEDUCT-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "DEDUCT_479PLAN",
    "host": "HV-INSURANCE-PLAN-DEDUCT-479PLAN",
    "legacy_field": "DEDUCT_0435"
  },
  "DEDUCT": {

```

```

    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "DEDUCT_479PLAN",
    "host": "HV-INSURANCE-PLAN-DEDUCT-479PLAN",
    "legacy_field": "DEDUCT_0435"
  },
  "DEDUCT0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "DEDUCT_479PLAN",
    "host": "HV-INSURANCE-PLAN-DEDUCT-479PLAN",
    "legacy_field": "DEDUCT_0435"
  },
  "DEDUCT_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "DEDUCT_479PLAN",
    "host": "HV-INSURANCE-PLAN-DEDUCT-479PLAN",
    "legacy_field": "DEDUCT_0435"
  },
  "DEDUCT-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "DEDUCT_479PLAN",
    "host": "HV-INSURANCE-PLAN-DEDUCT-479PLAN",
    "legacy_field": "DEDUCT_0435"
  },
  "MAXIMUM_LIFE_COST_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "MAXIMUM_LIFECOST_479PLAN",
    "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
    "legacy_field": "MAXIMUM_LIFE_COST_0435"
  },
  "MAXIMUM-LIFE-COST-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "MAXIMUM_LIFECOST_479PLAN",
    "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
    "legacy_field": "MAXIMUM_LIFE_COST_0435"
  },
  "MAXIMUM_LIFE_COST": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "MAXIMUM_LIFECOST_479PLAN",
    "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
    "legacy_field": "MAXIMUM_LIFE_COST_0435"
  },
  "MAXIMUMLIFECOST0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "MAXIMUM_LIFECOST_479PLAN",
    "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
    "legacy_field": "MAXIMUM_LIFE_COST_0435"
  },
  "MAXIMUM_LIFECOST_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "MAXIMUM_LIFECOST_479PLAN",
    "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
    "legacy_field": "MAXIMUM_LIFE_COST_0435"
  },
  "MAXIMUM-LIFECOST-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "MAXIMUM_LIFECOST_479PLAN",
    "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
    "legacy_field": "MAXIMUM_LIFE_COST_0435"
  },
  "MAXIMUM_LIFECOST": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "MAXIMUM_LIFECOST_479PLAN",
    "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
    "legacy_field": "MAXIMUM_LIFE_COST_0435"
  },

```

```

"MAXIMUMLIFECOST": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "MAXIMUM_LIFECOST_479PLAN",
  "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
  "legacy_field": "MAXIMUM_LIFE_COST_0435"
},
"FAMILY_COST_0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-FAMILY-COST-479PLAN",
  "legacy_field": "FAMILY_COST_0435"
},
"FAMILY-COST-0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-FAMILY-COST-479PLAN",
  "legacy_field": "FAMILY_COST_0435"
},
"FAMILY_COST": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-FAMILY-COST-479PLAN",
  "legacy_field": "FAMILY_COST_0435"
},
"FAMILYCOST0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-FAMILY-COST-479PLAN",
  "legacy_field": "FAMILY_COST_0435"
},
"FAMILY_COST_479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-FAMILY-COST-479PLAN",
  "legacy_field": "FAMILY_COST_0435"
},
"FAMILY-COST-479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-FAMILY-COST-479PLAN",
  "legacy_field": "FAMILY_COST_0435"
},
"FAMILYCOST": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-FAMILY-COST-479PLAN",
  "legacy_field": "FAMILY_COST_0435"
},
"DEP_COST_0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEP_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-DEP-COST-479PLAN",
  "legacy_field": "DEP_COST_0435"
},
"DEP-COST-0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEP_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-DEP-COST-479PLAN",
  "legacy_field": "DEP_COST_0435"
},
"DEP_COST": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEP_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-DEP-COST-479PLAN",
  "legacy_field": "DEP_COST_0435"
}

```

```

},
"DEPCOST0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEP_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-DEP-COST-479PLAN",
  "legacy_field": "DEP_COST_0435"
},
"DEP_COST_479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEP_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-DEP-COST-479PLAN",
  "legacy_field": "DEP_COST_0435"
},
"DEP-COST-479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEP_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-DEP-COST-479PLAN",
  "legacy_field": "DEP_COST_0435"
},
"DEPCOST": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEP_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-DEP-COST-479PLAN",
  "legacy_field": "DEP_COST_0435"
},
"JOB_ID_0440": {
  "record": "JOB",
  "table": "JOB",
  "column": "JOB_ID_479JOB",
  "host": "HV-JOB-JOB-ID-479JOB",
  "legacy_field": "JOB_ID_0440"
},
"JOB-ID-0440": {
  "record": "JOB",
  "table": "JOB",
  "column": "JOB_ID_479JOB",
  "host": "HV-JOB-JOB-ID-479JOB",
  "legacy_field": "JOB_ID_0440"
},
"JOB_ID": {
  "record": "JOB",
  "table": "JOB",
  "column": "JOB_ID_479JOB",
  "host": "HV-JOB-JOB-ID-479JOB",
  "legacy_field": "JOB_ID_0440"
},
"JOBID0440": {
  "record": "JOB",
  "table": "JOB",
  "column": "JOB_ID_479JOB",
  "host": "HV-JOB-JOB-ID-479JOB",
  "legacy_field": "JOB_ID_0440"
},
"JOB_ID_479JOB": {
  "record": "JOB",
  "table": "JOB",
  "column": "JOB_ID_479JOB",
  "host": "HV-JOB-JOB-ID-479JOB",
  "legacy_field": "JOB_ID_0440"
},
"JOB-ID-479JOB": {
  "record": "JOB",
  "table": "JOB",
  "column": "JOB_ID_479JOB",
  "host": "HV-JOB-JOB-ID-479JOB",
  "legacy_field": "JOB_ID_0440"
},
"JOBID": {
  "record": "JOB",
  "table": "JOB",
  "column": "JOB_ID_479JOB",
  "host": "HV-JOB-JOB-ID-479JOB",

```

```

    "legacy_field": "JOB_ID_0440"
  },
  "TITLE_0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "TITLE_479JOB",
    "host": "HV-JOB-TITLE-479JOB",
    "legacy_field": "TITLE_0440"
  },
  "TITLE-0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "TITLE_479JOB",
    "host": "HV-JOB-TITLE-479JOB",
    "legacy_field": "TITLE_0440"
  },
  "TITLE": {
    "record": "JOB",
    "table": "JOB",
    "column": "TITLE_479JOB",
    "host": "HV-JOB-TITLE-479JOB",
    "legacy_field": "TITLE_0440"
  },
  "TITLE0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "TITLE_479JOB",
    "host": "HV-JOB-TITLE-479JOB",
    "legacy_field": "TITLE_0440"
  },
  "TITLE_479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "TITLE_479JOB",
    "host": "HV-JOB-TITLE-479JOB",
    "legacy_field": "TITLE_0440"
  },
  "TITLE-479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "TITLE_479JOB",
    "host": "HV-JOB-TITLE-479JOB",
    "legacy_field": "TITLE_0440"
  },
  "DESCRIPTION_0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_0440"
  },
  "DESCRIPTION-0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_0440"
  },
  "DESCRIPTION": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_0440"
  },
  "DESCRIPTION0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_0440"
  },
  "DESCRIPTION_LINE_479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",

```



```

    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_0440"
  },
  "DESCRIPTION-LINE-479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_0440"
  },
  "DESCRIPTION_LINE": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_LINE_0440"
  },
  "DESCRIPTIONLINE": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_0440"
  },
  "DESCRIPTION_LINE_0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_LINE_0440"
  },
  "DESCRIPTION-LINE-0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_LINE_0440"
  },
  "DESCRIPTIONLINE0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_LINE_0440"
  },
  "REQUIREMENT_LINE_0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "REQUIREMENT_LINE_479JOB",
    "host": "HV-JOB-REQUIREMENT-LINE-479JOB",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "REQUIREMENT-LINE-0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "REQUIREMENT_LINE_479JOB",
    "host": "HV-JOB-REQUIREMENT-LINE-479JOB",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "REQUIREMENT_LINE": {
    "record": "JOB",
    "table": "JOB",
    "column": "REQUIREMENT_LINE_479JOB",
    "host": "HV-JOB-REQUIREMENT-LINE-479JOB",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "REQUIREMENTLINE0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "REQUIREMENT_LINE_479JOB",
    "host": "HV-JOB-REQUIREMENT-LINE-479JOB",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "REQUIREMENT_LINE_479JOB": {
    "record": "JOB",
    "table": "JOB",

```

```

    "column": "REQUIREMENT_LINE_479JOB",
    "host": "HV-JOB-REQUIREMENT-LINE-479JOB",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "REQUIREMENT-LINE-479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "REQUIREMENT_LINE_479JOB",
    "host": "HV-JOB-REQUIREMENT-LINE-479JOB",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "REQUIREMENTLINE": {
    "record": "JOB",
    "table": "JOB",
    "column": "REQUIREMENT_LINE_479JOB",
    "host": "HV-JOB-REQUIREMENT-LINE-479JOB",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "MINIMUM_SALARY_0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "host": "HV-JOB-MINIMUM-SALARY-479JOB",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MINIMUM-SALARY-0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "host": "HV-JOB-MINIMUM-SALARY-479JOB",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MINIMUM_SALARY": {
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "host": "HV-JOB-MINIMUM-SALARY-479JOB",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MINIMUMSALARY0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "host": "HV-JOB-MINIMUM-SALARY-479JOB",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MINIMUM_SALARY_479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "host": "HV-JOB-MINIMUM-SALARY-479JOB",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MINIMUM-SALARY-479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "host": "HV-JOB-MINIMUM-SALARY-479JOB",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MINIMUMSALARY": {
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "host": "HV-JOB-MINIMUM-SALARY-479JOB",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MAXIMUM_SALARY_0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "MAXIMUM_SALARY_479JOB",
    "host": "HV-JOB-MAXIMUM-SALARY-479JOB",
    "legacy_field": "MAXIMUM_SALARY_0440"
  },
  "MAXIMUM-SALARY-0440": {
    "record": "JOB",

```

```

"table": "JOB",
"column": "MAXIMUM_SALARY_479JOB",
"host": "HV-JOB-MAXIMUM-SALARY-479JOB",
"legacy_field": "MAXIMUM_SALARY_0440"
},
"MAXIMUM_SALARY": {
"record": "JOB",
"table": "JOB",
"column": "MAXIMUM_SALARY_479JOB",
"host": "HV-JOB-MAXIMUM-SALARY-479JOB",
"legacy_field": "MAXIMUM_SALARY_0440"
},
"MAXIMUMSALARY0440": {
"record": "JOB",
"table": "JOB",
"column": "MAXIMUM_SALARY_479JOB",
"host": "HV-JOB-MAXIMUM-SALARY-479JOB",
"legacy_field": "MAXIMUM_SALARY_0440"
},
"MAXIMUM_SALARY_479JOB": {
"record": "JOB",
"table": "JOB",
"column": "MAXIMUM_SALARY_479JOB",
"host": "HV-JOB-MAXIMUM-SALARY-479JOB",
"legacy_field": "MAXIMUM_SALARY_0440"
},
"MAXIMUM-SALARY-479JOB": {
"record": "JOB",
"table": "JOB",
"column": "MAXIMUM_SALARY_479JOB",
"host": "HV-JOB-MAXIMUM-SALARY-479JOB",
"legacy_field": "MAXIMUM_SALARY_0440"
},
"MAXIMUMSALARY": {
"record": "JOB",
"table": "JOB",
"column": "MAXIMUM_SALARY_479JOB",
"host": "HV-JOB-MAXIMUM-SALARY-479JOB",
"legacy_field": "MAXIMUM_SALARY_0440"
},
"SALARY_GRADES_0440": {
"record": "JOB",
"table": "JOB",
"column": "SALARY_GRADES_479JOB",
"host": "HV-JOB-SALARY-GRADES-479JOB",
"legacy_field": "SALARY_GRADES_0440"
},
"SALARY-GRADES-0440": {
"record": "JOB",
"table": "JOB",
"column": "SALARY_GRADES_479JOB",
"host": "HV-JOB-SALARY-GRADES-479JOB",
"legacy_field": "SALARY_GRADES_0440"
},
"SALARY_GRADES": {
"record": "JOB",
"table": "JOB",
"column": "SALARY_GRADES_479JOB",
"host": "HV-JOB-SALARY-GRADES-479JOB",
"legacy_field": "SALARY_GRADES_0440"
},
"SALARYGRADES0440": {
"record": "JOB",
"table": "JOB",
"column": "SALARY_GRADES_479JOB",
"host": "HV-JOB-SALARY-GRADES-479JOB",
"legacy_field": "SALARY_GRADES_0440"
},
"SALARY_GRADES_479JOB": {
"record": "JOB",
"table": "JOB",
"column": "SALARY_GRADES_479JOB",
"host": "HV-JOB-SALARY-GRADES-479JOB",
"legacy_field": "SALARY_GRADES_0440"
},
"SALARY-GRADES-479JOB": {

```

```

    "record": "JOB",
    "table": "JOB",
    "column": "SALARY_GRADES_479JOB",
    "host": "HV-JOB-SALARY-GRADES-479JOB",
    "legacy_field": "SALARY_GRADES_0440"
  },
  "SALARYGRADES": {
    "record": "JOB",
    "table": "JOB",
    "column": "SALARY_GRADES_479JOB",
    "host": "HV-JOB-SALARY-GRADES-479JOB",
    "legacy_field": "SALARY_GRADES_0440"
  },
  "NUMBER_OF_POSITIONS_0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "NUMBER_OFPOSITIONS_479JOB",
    "host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
    "legacy_field": "NUMBER_OF_POSITIONS_0440"
  },
  "NUMBER-OF-POSITIONS-0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "NUMBER_OFPOSITIONS_479JOB",
    "host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
    "legacy_field": "NUMBER_OF_POSITIONS_0440"
  },
  "NUMBER_OF_POSITIONS": {
    "record": "JOB",
    "table": "JOB",
    "column": "NUMBER_OFPOSITIONS_479JOB",
    "host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
    "legacy_field": "NUMBER_OF_POSITIONS_0440"
  },
  "NUMBEROFPOSITIONS0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "NUMBER_OFPOSITIONS_479JOB",
    "host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
    "legacy_field": "NUMBER_OF_POSITIONS_0440"
  },
  "NUMBER_OFPOSITIONS_479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "NUMBER_OFPOSITIONS_479JOB",
    "host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
    "legacy_field": "NUMBER_OF_POSITIONS_0440"
  },
  "NUMBER-OFPOSITIONS-479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "NUMBER_OFPOSITIONS_479JOB",
    "host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
    "legacy_field": "NUMBER_OF_POSITIONS_0440"
  },
  "NUMBER_OFPOSITIONS": {
    "record": "JOB",
    "table": "JOB",
    "column": "NUMBER_OFPOSITIONS_479JOB",
    "host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
    "legacy_field": "NUMBER_OF_POSITIONS_0440"
  },
  "NUMBEROFPOSITIONS": {
    "record": "JOB",
    "table": "JOB",
    "column": "NUMBER_OFPOSITIONS_479JOB",
    "host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
    "legacy_field": "NUMBER_OF_POSITIONS_0440"
  },
  "NUMBER_OPEN_0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "NUMBER_OPEN_479JOB",
    "host": "HV-JOB-NUMBER-OPEN-479JOB",
    "legacy_field": "NUMBER_OPEN_0440"
  },

```

```

"NUMBER-OPEN-0440": {
  "record": "JOB",
  "table": "JOB",
  "column": "NUMBER_OPEN_479JOB",
  "host": "HV-JOB-NUMBER-OPEN-479JOB",
  "legacy_field": "NUMBER_OPEN_0440"
},
"NUMBER_OPEN": {
  "record": "JOB",
  "table": "JOB",
  "column": "NUMBER_OPEN_479JOB",
  "host": "HV-JOB-NUMBER-OPEN-479JOB",
  "legacy_field": "NUMBER_OPEN_0440"
},
"NUMBEROPEN0440": {
  "record": "JOB",
  "table": "JOB",
  "column": "NUMBER_OPEN_479JOB",
  "host": "HV-JOB-NUMBER-OPEN-479JOB",
  "legacy_field": "NUMBER_OPEN_0440"
},
"NUMBER_OPEN_479JOB": {
  "record": "JOB",
  "table": "JOB",
  "column": "NUMBER_OPEN_479JOB",
  "host": "HV-JOB-NUMBER-OPEN-479JOB",
  "legacy_field": "NUMBER_OPEN_0440"
},
"NUMBER-OPEN-479JOB": {
  "record": "JOB",
  "table": "JOB",
  "column": "NUMBER_OPEN_479JOB",
  "host": "HV-JOB-NUMBER-OPEN-479JOB",
  "legacy_field": "NUMBER_OPEN_0440"
},
"NUMBEROPEN": {
  "record": "JOB",
  "table": "JOB",
  "column": "NUMBER_OPEN_479JOB",
  "host": "HV-JOB-NUMBER-OPEN-479JOB",
  "legacy_field": "NUMBER_OPEN_0440"
},
"CLAIM_DATE_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "legacy_field": "CLAIM_DATE_0445"
},
"CLAIM-DATE-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "legacy_field": "CLAIM_DATE_0445"
},
"CLAIMDATE0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "legacy_field": "CLAIM_DATE_0445"
},
"PATIENT_FIRST_NAME_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_FIRSTNAME_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-FIRSTNAME-479LAIM",
  "legacy_field": "PATIENT_FIRST_NAME_0445"
},
"PATIENT-FIRST-NAME-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_FIRSTNAME_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-FIRSTNAME-479LAIM",
  "legacy_field": "PATIENT_FIRST_NAME_0445"
}

```

```

},
"PATIENTFIRSTNAME0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_FIRSTNAME_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-FIRSTNAME-479LAIM",
  "legacy_field": "PATIENT_FIRST_NAME_0445"
},
"PATIENT_LAST_NAME_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_LASTNAME_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-LASTNAME-479LAIM",
  "legacy_field": "PATIENT_LAST_NAME_0445"
},
"PATIENT-LAST-NAME-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_LASTNAME_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-LASTNAME-479LAIM",
  "legacy_field": "PATIENT_LAST_NAME_0445"
},
"PATIENTLASTNAME0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_LASTNAME_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-LASTNAME-479LAIM",
  "legacy_field": "PATIENT_LAST_NAME_0445"
},
"PATIENT_BIRTH_DATE_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "legacy_field": "PATIENT_BIRTH_DATE_0445"
},
"PATIENT-BIRTH-DATE-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "legacy_field": "PATIENT_BIRTH_DATE_0445"
},
"PATIENTBIRTHDATE0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "legacy_field": "PATIENT_BIRTH_DATE_0445"
},
"PATIENT_SEX_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_SEX_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-SEX-479LAIM",
  "legacy_field": "PATIENT_SEX_0445"
},
"PATIENT-SEX-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_SEX_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-SEX-479LAIM",
  "legacy_field": "PATIENT_SEX_0445"
},
"PATIENTSEX0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_SEX_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-SEX-479LAIM",
  "legacy_field": "PATIENT_SEX_0445"
},
"RELATION_TO_EMPLOYEE_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "RELATION_TOEMPLOYEE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-RELATION-TOEMPLOYEE-479LAIM",

```

```

    "legacy_field": "RELATION_TO_EMPLOYEE_0445"
  },
  "RELATION-TO-EMPLOYEE-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0445"
  },
  "RELATIONTOEMPLOYEE0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0445"
  },
  "PHYSICIAN_FIRST_NAME_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIAN-FIRST-NAME-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIAN_FIRST_NAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIANFIRSTNAME0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIAN_FIRSTNAME_479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIAN-FIRSTNAME-479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIAN_FIRSTNAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIANFIRSTNAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIAN_LAST_NAME_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",

```

```

    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIAN-LAST-NAME-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIAN_LAST_NAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIANLASTNAME0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIAN_LASTNAME_479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIAN-LASTNAME-479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIAN_LASTNAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIANLASTNAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIAN_STREET_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STREET_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET-479LAIM",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIAN-STREET-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STREET_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET-479LAIM",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIAN_STREET": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STREET_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET-479LAIM",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIANSTREET0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",

```



```

    "column": "PHYSICIAN_STREET_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET-479LAIM",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIAN_STREET_479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STREET_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET-479LAIM",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIAN-STREET-479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STREET_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET-479LAIM",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIANSTREET": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STREET_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET-479LAIM",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIAN_CITY_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY-479LAIM",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIAN-CITY-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY-479LAIM",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIAN_CITY": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY-479LAIM",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIANCITY0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY-479LAIM",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIAN_CITY_479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY-479LAIM",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIAN-CITY-479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY-479LAIM",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIANCITY": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY-479LAIM",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIAN_STATE_0445": {
    "record": "NON_HOSP_CLAIM",

```

```

    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE-479LAIM",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIAN-STATE-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE-479LAIM",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIAN_STATE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE-479LAIM",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIANSTATE0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE-479LAIM",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIAN_STATE_479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE-479LAIM",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIAN-STATE-479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE-479LAIM",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIANSTATE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE-479LAIM",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIAN_ZIP_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_0445"
  },
  "PHYSICIAN-ZIP-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_0445"
  },
  "PHYSICIAN_ZIP": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_0445"
  },
  "PHYSICIANZIP0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_0445"
  },
  "PHYSICIAN_ZIPFIRSTFIVE_479LAIM": {

```

```

    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_0445"
  },
  "PHYSICIAN-ZIPFIRSTFIVE-479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_0445"
  },
  "PHYSICIAN_ZIPFIRSTFIVE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_0445"
  },
  "PHYSICIANZIPFIRSTFIVE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_0445"
  },
  "PHYSICIAN_ZIP_FIRST_FIVE_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_FIRST_FIVE_0445"
  },
  "PHYSICIAN-ZIP-FIRST-FIVE-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_FIRST_FIVE_0445"
  },
  "PHYSICIAN_ZIP_FIRST_FIVE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_FIRST_FIVE_0445"
  },
  "PHYSICIANZIPFIRSTFIVE0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_FIRST_FIVE_0445"
  },
  "PHYSICIAN_ZIP_LAST_FOUR_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
  },
  "PHYSICIAN-ZIP-LAST-FOUR-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
  },
  "PHYSICIAN_ZIP_LAST_FOUR": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
  },

```

```

"PHYSICIANZIPLASTFOUR0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
  "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
},
"PHYSICIAN_ZIPLASTFOUR_479LAIM": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
  "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
},
"PHYSICIAN-ZIPLASTFOUR-479LAIM": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
  "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
},
"PHYSICIAN_ZIPLASTFOUR": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
  "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
},
"PHYSICIANZIPLASTFOUR": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
  "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
},
"PHYSICIAN_ID_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID-479LAIM",
  "legacy_field": "PHYSICIAN_ID_0445"
},
"PHYSICIAN-ID-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID-479LAIM",
  "legacy_field": "PHYSICIAN_ID_0445"
},
"PHYSICIAN_ID": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID-479LAIM",
  "legacy_field": "PHYSICIAN_ID_0445"
},
"PHYSICIANID0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID-479LAIM",
  "legacy_field": "PHYSICIAN_ID_0445"
},
"PHYSICIAN_ID_479LAIM": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID-479LAIM",
  "legacy_field": "PHYSICIAN_ID_0445"
},
"PHYSICIAN-ID-479LAIM": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID-479LAIM",
  "legacy_field": "PHYSICIAN_ID_0445"
}

```

```

},
"PHYSICIANID": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID-479LAIM",
  "legacy_field": "PHYSICIAN_ID_0445"
},
"DIAGNOSIS_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DIAGNOSIS_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DIAGNOSIS-479LAIM",
  "legacy_field": "DIAGNOSIS_0445"
},
"DIAGNOSIS-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DIAGNOSIS_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DIAGNOSIS-479LAIM",
  "legacy_field": "DIAGNOSIS_0445"
},
"DIAGNOSIS0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DIAGNOSIS_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DIAGNOSIS-479LAIM",
  "legacy_field": "DIAGNOSIS_0445"
},
"NUMBER_OF_PROCEDURES_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "NUMBER_OFPROCEDURES_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
  "legacy_field": "NUMBER_OF_PROCEDURES_0445"
},
"NUMBER-OF-PROCEDURES-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "NUMBER_OFPROCEDURES_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
  "legacy_field": "NUMBER_OF_PROCEDURES_0445"
},
"NUMBEROFPROCEDURES0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "NUMBER_OFPROCEDURES_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
  "legacy_field": "NUMBER_OF_PROCEDURES_0445"
},
"SERVICE_DATE_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_SERVICEDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0445"
},
"SERVICE-DATE-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_SERVICEDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0445"
},
"SERVICEDATE0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_SERVICEDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0445"
},
"PROCEDURE_CODE_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PROCEDURE-CODE-479LAIM",

```

```

    "legacy_field": "PROCEDURE_CODE_0445"
  },
  "PROCEDURE-CODE-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PROCEDURE_CODE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PROCEDURE-CODE-479LAIM",
    "legacy_field": "PROCEDURE_CODE_0445"
  },
  "PROCEDURECODE0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PROCEDURE_CODE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PROCEDURE-CODE-479LAIM",
    "legacy_field": "PROCEDURE_CODE_0445"
  },
  "DESCRIPTION_OF_SERVICE_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DESCRIPTION_OFSERVICE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
    "legacy_field": "DESCRIPTION_OF_SERVICE_0445"
  },
  "DESCRIPTION-OF-SERVICE-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DESCRIPTION_OFSERVICE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
    "legacy_field": "DESCRIPTION_OF_SERVICE_0445"
  },
  "DESCRIPTIONOFSERVICE0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DESCRIPTION_OFSERVICE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
    "legacy_field": "DESCRIPTION_OF_SERVICE_0445"
  },
  "FEE_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "FEE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-FEE-479LAIM",
    "legacy_field": "FEE_0445"
  },
  "FEE-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "FEE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-FEE-479LAIM",
    "legacy_field": "FEE_0445"
  },
  "FEE0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "FEE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-FEE-479LAIM",
    "legacy_field": "FEE_0445"
  },
  "OFFICE_CODE_0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_479FICE",
    "host": "HV-OFFICE-OFFICE-CODE-479FICE",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICE-CODE-0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_479FICE",
    "host": "HV-OFFICE-OFFICE-CODE-479FICE",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICE_CODE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_479FICE",

```

```

    "host": "HV-OFFICE-OFFICE-CODE-479FICE",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICECODE0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_479FICE",
    "host": "HV-OFFICE-OFFICE-CODE-479FICE",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICE_CODE_479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_479FICE",
    "host": "HV-OFFICE-OFFICE-CODE-479FICE",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICE-CODE-479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_479FICE",
    "host": "HV-OFFICE-OFFICE-CODE-479FICE",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICECODE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_479FICE",
    "host": "HV-OFFICE-OFFICE-CODE-479FICE",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICE_STREET_0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STREET_479FICE",
    "host": "HV-OFFICE-OFFICE-STREET-479FICE",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICE-STREET-0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STREET_479FICE",
    "host": "HV-OFFICE-OFFICE-STREET-479FICE",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICE_STREET": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STREET_479FICE",
    "host": "HV-OFFICE-OFFICE-STREET-479FICE",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICESTREET0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STREET_479FICE",
    "host": "HV-OFFICE-OFFICE-STREET-479FICE",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICE_STREET_479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STREET_479FICE",
    "host": "HV-OFFICE-OFFICE-STREET-479FICE",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICE-STREET-479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STREET_479FICE",
    "host": "HV-OFFICE-OFFICE-STREET-479FICE",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICESTREET": {
    "record": "OFFICE",
    "table": "OFFICE",

```

```

    "column": "OFFICE_STREET_479FICE",
    "host": "HV-OFFICE-OFFICE-STREET-479FICE",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICE_CITY_0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_479FICE",
    "host": "HV-OFFICE-OFFICE-CITY-479FICE",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICE-CITY-0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_479FICE",
    "host": "HV-OFFICE-OFFICE-CITY-479FICE",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICE_CITY": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_479FICE",
    "host": "HV-OFFICE-OFFICE-CITY-479FICE",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICECITY0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_479FICE",
    "host": "HV-OFFICE-OFFICE-CITY-479FICE",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICE_CITY_479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_479FICE",
    "host": "HV-OFFICE-OFFICE-CITY-479FICE",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICE-CITY-479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_479FICE",
    "host": "HV-OFFICE-OFFICE-CITY-479FICE",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICECITY": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_479FICE",
    "host": "HV-OFFICE-OFFICE-CITY-479FICE",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICE_STATE_0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STATE_479FICE",
    "host": "HV-OFFICE-OFFICE-STATE-479FICE",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICE-STATE-0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STATE_479FICE",
    "host": "HV-OFFICE-OFFICE-STATE-479FICE",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICE_STATE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STATE_479FICE",
    "host": "HV-OFFICE-OFFICE-STATE-479FICE",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICESTATE0450": {
    "record": "OFFICE",

```



```

    "table": "OFFICE",
    "column": "OFFICE_STATE_479FICE",
    "host": "HV-OFFICE-OFFICE-STATE-479FICE",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICE_STATE_479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STATE_479FICE",
    "host": "HV-OFFICE-OFFICE-STATE-479FICE",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICE-STATE-479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STATE_479FICE",
    "host": "HV-OFFICE-OFFICE-STATE-479FICE",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICESTATE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STATE_479FICE",
    "host": "HV-OFFICE-OFFICE-STATE-479FICE",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICE_ZIP_0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICE-ZIP-0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICE_ZIP": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICEZIP0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICE_ZIPFIRSTFIVE_479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICE-ZIPFIRSTFIVE-479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICE_ZIPFIRSTFIVE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICEZIPFIRSTFIVE": {

```

```

    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICE_ZIP_FIRST_FIVE_0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_FIRST_FIVE_0450"
  },
  "OFFICE-ZIP-FIRST-FIVE-0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_FIRST_FIVE_0450"
  },
  "OFFICE_ZIP_FIRST_FIVE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_FIRST_FIVE_0450"
  },
  "OFFICEZIPFIRSTFIVE0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_FIRST_FIVE_0450"
  },
  "OFFICE_ZIP_LAST_FOUR_0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPLASTFOUR_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
    "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
  },
  "OFFICE-ZIP-LAST-FOUR-0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPLASTFOUR_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
    "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
  },
  "OFFICE_ZIP_LAST_FOUR": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPLASTFOUR_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
    "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
  },
  "OFFICEZIPLASTFOUR0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPLASTFOUR_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
    "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
  },
  "OFFICE_ZIPLASTFOUR_479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPLASTFOUR_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
    "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
  },
  "OFFICE-ZIPLASTFOUR-479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPLASTFOUR_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
    "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
  },

```

```

"OFFICE_ZIPLASTFOUR": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_ZIPLASTFOUR_479FICE",
  "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
  "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
},
"OFFICEZIPLASTFOUR": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_ZIPLASTFOUR_479FICE",
  "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
  "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
},
"OFFICE_PHONE_0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_479FICE",
  "host": "HV-OFFICE-OFFICE-PHONE-479FICE",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICE-PHONE-0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_479FICE",
  "host": "HV-OFFICE-OFFICE-PHONE-479FICE",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICE_PHONE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_479FICE",
  "host": "HV-OFFICE-OFFICE-PHONE-479FICE",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICEPHONE0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_479FICE",
  "host": "HV-OFFICE-OFFICE-PHONE-479FICE",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICE_PHONE_479FICE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_479FICE",
  "host": "HV-OFFICE-OFFICE-PHONE-479FICE",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICE-PHONE-479FICE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_479FICE",
  "host": "HV-OFFICE-OFFICE-PHONE-479FICE",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICEPHONE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_479FICE",
  "host": "HV-OFFICE-OFFICE-PHONE-479FICE",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICE_AREA_CODE_0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"OFFICE-AREA-CODE-0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
}

```

```

},
"OFFICE_AREA_CODE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"OFFICEAREACODE0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"OFFICE_AREACODE_479FICE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"OFFICE-AREACODE-479FICE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"OFFICE_AREACODE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"OFFICEAREACODE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"SPEED_DIAL_0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "SPEED_DIAL_479FICE",
  "host": "HV-OFFICE-SPEED-DIAL-479FICE",
  "legacy_field": "SPEED_DIAL_0450"
},
"SPEED-DIAL-0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "SPEED_DIAL_479FICE",
  "host": "HV-OFFICE-SPEED-DIAL-479FICE",
  "legacy_field": "SPEED_DIAL_0450"
},
"SPEED_DIAL": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "SPEED_DIAL_479FICE",
  "host": "HV-OFFICE-SPEED-DIAL-479FICE",
  "legacy_field": "SPEED_DIAL_0450"
},
"SPEEDDIAL0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "SPEED_DIAL_479FICE",
  "host": "HV-OFFICE-SPEED-DIAL-479FICE",
  "legacy_field": "SPEED_DIAL_0450"
},
"SPEED_DIAL_479FICE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "SPEED_DIAL_479FICE",
  "host": "HV-OFFICE-SPEED-DIAL-479FICE",

```

```

    "legacy_field": "SPEED_DIAL_0450"
  },
  "SPEED-DIAL-479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "SPEED_DIAL_479FICE",
    "host": "HV-OFFICE-SPEED-DIAL-479FICE",
    "legacy_field": "SPEED_DIAL_0450"
  },
  "SPEEDDIAL": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "SPEED_DIAL_479FICE",
    "host": "HV-OFFICE-SPEED-DIAL-479FICE",
    "legacy_field": "SPEED_DIAL_0450"
  },
  "SKILL_ID_0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL",
    "legacy_field": "SKILL_ID_0455"
  },
  "SKILL-ID-0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL",
    "legacy_field": "SKILL_ID_0455"
  },
  "SKILL_ID": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL",
    "legacy_field": "SKILL_ID_0455"
  },
  "SKILLID0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL",
    "legacy_field": "SKILL_ID_0455"
  },
  "SKILL_ID_479KILL": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL",
    "legacy_field": "SKILL_ID_0455"
  },
  "SKILL-ID-479KILL": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL",
    "legacy_field": "SKILL_ID_0455"
  },
  "SKILLID": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL",
    "legacy_field": "SKILL_ID_0455"
  },
  "SKILL_NAME_0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_NAME_479KILL",
    "host": "HV-SKILL-SKILL-NAME-479KILL",
    "legacy_field": "SKILL_NAME_0455"
  },
  "SKILL-NAME-0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_NAME_479KILL",

```

```

    "host": "HV-SKILL-SKILL-NAME-479KILL",
    "legacy_field": "SKILL_NAME_0455"
  },
  "SKILL_NAME": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_NAME_479KILL",
    "host": "HV-SKILL-SKILL-NAME-479KILL",
    "legacy_field": "SKILL_NAME_0455"
  },
  "SKILLNAME0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_NAME_479KILL",
    "host": "HV-SKILL-SKILL-NAME-479KILL",
    "legacy_field": "SKILL_NAME_0455"
  },
  "SKILL_NAME_479KILL": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_NAME_479KILL",
    "host": "HV-SKILL-SKILL-NAME-479KILL",
    "legacy_field": "SKILL_NAME_0455"
  },
  "SKILL-NAME-479KILL": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_NAME_479KILL",
    "host": "HV-SKILL-SKILL-NAME-479KILL",
    "legacy_field": "SKILL_NAME_0455"
  },
  "SKILLNAME": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_NAME_479KILL",
    "host": "HV-SKILL-SKILL-NAME-479KILL",
    "legacy_field": "SKILL_NAME_0455"
  },
  "SKILL_DESCRIPTION_0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_DESCRIPTION_479KILL",
    "host": "HV-SKILL-SKILL-DESCRIPTION-479KILL",
    "legacy_field": "SKILL_DESCRIPTION_0455"
  },
  "SKILL-DESCRIPTION-0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_DESCRIPTION_479KILL",
    "host": "HV-SKILL-SKILL-DESCRIPTION-479KILL",
    "legacy_field": "SKILL_DESCRIPTION_0455"
  },
  "SKILL_DESCRIPTION": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_DESCRIPTION_479KILL",
    "host": "HV-SKILL-SKILL-DESCRIPTION-479KILL",
    "legacy_field": "SKILL_DESCRIPTION_0455"
  },
  "SKILLDESCRIPTION0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_DESCRIPTION_479KILL",
    "host": "HV-SKILL-SKILL-DESCRIPTION-479KILL",
    "legacy_field": "SKILL_DESCRIPTION_0455"
  },
  "SKILL_DESCRIPTION_479KILL": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_DESCRIPTION_479KILL",
    "host": "HV-SKILL-SKILL-DESCRIPTION-479KILL",
    "legacy_field": "SKILL_DESCRIPTION_0455"
  },
  "SKILL-DESCRIPTION-479KILL": {
    "record": "SKILL",
    "table": "SKILL",

```

```

    "column": "SKILL_DESCRIPTION_479KILL",
    "host": "HV-SKILL-SKILL-DESCRIPTION-479KILL",
    "legacy_field": "SKILL_DESCRIPTION_0455"
  },
  "SKILLDESCRIPTION": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_DESCRIPTION_479KILL",
    "host": "HV-SKILL-SKILL-DESCRIPTION-479KILL",
    "legacy_field": "SKILL_DESCRIPTION_0455"
  },
  "STRUCTURE_CODE_0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_CODE_479TURE",
    "host": "HV-STRUCTURE-STRUCTURE-CODE-479TURE",
    "legacy_field": "STRUCTURE_CODE_0460"
  },
  "STRUCTURE-CODE-0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_CODE_479TURE",
    "host": "HV-STRUCTURE-STRUCTURE-CODE-479TURE",
    "legacy_field": "STRUCTURE_CODE_0460"
  },
  "STRUCTURE_CODE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_CODE_479TURE",
    "host": "HV-STRUCTURE-STRUCTURE-CODE-479TURE",
    "legacy_field": "STRUCTURE_CODE_0460"
  },
  "STRUCTURECODE0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_CODE_479TURE",
    "host": "HV-STRUCTURE-STRUCTURE-CODE-479TURE",
    "legacy_field": "STRUCTURE_CODE_0460"
  },
  "STRUCTURE_CODE_479TURE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_CODE_479TURE",
    "host": "HV-STRUCTURE-STRUCTURE-CODE-479TURE",
    "legacy_field": "STRUCTURE_CODE_0460"
  },
  "STRUCTURE-CODE-479TURE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_CODE_479TURE",
    "host": "HV-STRUCTURE-STRUCTURE-CODE-479TURE",
    "legacy_field": "STRUCTURE_CODE_0460"
  },
  "STRUCTURECODE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_CODE_479TURE",
    "host": "HV-STRUCTURE-STRUCTURE-CODE-479TURE",
    "legacy_field": "STRUCTURE_CODE_0460"
  },
  "STRUCTURE_DATE_0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  },
  "STRUCTURE-DATE-0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  },
  "STRUCTURE_DATE": {
    "record": "STRUCTURE",

```

```

    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  },
  "STRUCTUREDATE0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  },
  "DA_STRUCTUREDATE_479TURE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  },
  "DA-STRUCTUREDATE-479TURE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  },
  "DA_STRUCTUREDATE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  },
  "DASTRUCTUREDATE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  }
},
"calc_key_map": {
  "DEPARTMENT": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "key": "DEPT_ID_479MENT",
    "primary_key": "DEPT_ID_479MENT",
    "primary_keys": [
      "DEPT_ID_479MENT"
    ],
    "column": "DEPT_ID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-ID-479MENT"
  },
  "EMPLOYEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "key": "EMP_ID_4790YEE",
    "primary_key": "EMP_ID_4790YEE",
    "primary_keys": [
      "EMP_ID_4790YEE"
    ],
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE"
  },
  "INSURANCE_PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "key": "INS_PLANCODE_479PLAN",
    "primary_key": "INS_PLANCODE_479PLAN",
    "primary_keys": [
      "INS_PLANCODE_479PLAN"
    ],
    "column": "INS_PLANCODE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-PLANCODE-479PLAN"
  },
  "JOB": {

```



```

    "record": "JOB",
    "table": "JOB",
    "key": "JOB_ID_479JOB",
    "primary_key": "JOB_ID_479JOB",
    "primary_keys": [
        "JOB_ID_479JOB"
    ],
    "column": "JOB_ID_479JOB",
    "host": "HV-JOB-JOB-ID-479JOB"
},
"OFFICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "key": "OFFICE_CODE_479FICE",
    "primary_key": "OFFICE_CODE_479FICE",
    "primary_keys": [
        "OFFICE_CODE_479FICE"
    ],
    "column": "OFFICE_CODE_479FICE",
    "host": "HV-OFFICE-OFFICE-CODE-479FICE"
},
"SKILL": {
    "record": "SKILL",
    "table": "SKILL",
    "key": "SKILL_ID_479KILL",
    "primary_key": "SKILL_ID_479KILL",
    "primary_keys": [
        "SKILL_ID_479KILL"
    ],
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL"
}
},
"relationships": [
    {
        "set_name": "COVERAGE_CLAIMS",
        "parent_record": "COVERAGE",
        "child_record": "DENTAL_CLAIM",
        "owner_record": "COVERAGE",
        "member_record": "DENTAL_CLAIM",
        "parent_table": "COVERAGE",
        "child_table": "DENTAL_CLAIM",
        "parent_key": "ID_RECORD_RAGE",
        "parent_keys": [
            "ID_RECORD_RAGE"
        ],
        "child_fk": "ID_RECORD_RAGE",
        "child_fks": [
            "ID_RECORD_RAGE"
        ],
        "cardinality": "1:N",
        "order_by": [
            "ID_RECORD_RAGE"
        ]
    },
    {
        "set_name": "COVERAGE_CLAIMS",
        "parent_record": "COVERAGE",
        "child_record": "HOSPITAL_CLAIM",
        "owner_record": "COVERAGE",
        "member_record": "HOSPITAL_CLAIM",
        "parent_table": "COVERAGE",
        "child_table": "HOSPITAL_CLAIM",
        "parent_key": "ID_RECORD_RAGE",
        "parent_keys": [
            "ID_RECORD_RAGE"
        ],
        "child_fk": "ID_RECORD_RAGE",
        "child_fks": [
            "ID_RECORD_RAGE"
        ],
        "cardinality": "1:N",
        "order_by": [
            "ID_RECORD_RAGE"
        ]
    }
]
},

```

```

{
  "set_name": "COVERAGE_CLAIMS",
  "parent_record": "COVERAGE",
  "child_record": "NON_HOSP_CLAIM",
  "owner_record": "COVERAGE",
  "member_record": "NON_HOSP_CLAIM",
  "parent_table": "COVERAGE",
  "child_table": "NON_HOSP_CLAIM",
  "parent_key": "ID_RECORD_RAGE",
  "parent_keys": [
    "ID_RECORD_RAGE"
  ],
  "child_fk": "ID_RECORD_RAGE",
  "child_fks": [
    "ID_RECORD_RAGE"
  ],
  "cardinality": "1:N",
  "order_by": [
    "ID_RECORD_RAGE"
  ]
},
{
  "set_name": "DEPT_EMPLOYEE",
  "parent_record": "DEPARTMENT",
  "child_record": "EMPLOYEE",
  "owner_record": "DEPARTMENT",
  "member_record": "EMPLOYEE",
  "parent_table": "DEPARTMENT",
  "child_table": "EMPLOYEE",
  "parent_key": "DEPT_ID_479MENT",
  "parent_keys": [
    "DEPT_ID_479MENT"
  ],
  "child_fk": "DEPT_ID_479MENT",
  "child_fks": [
    "DEPT_ID_479MENT"
  ],
  "cardinality": "1:N",
  "order_by": [
    "DEPT_ID_479MENT"
  ]
},
{
  "set_name": "EMP_COVERAGE",
  "parent_record": "EMPLOYEE",
  "child_record": "COVERAGE",
  "owner_record": "EMPLOYEE",
  "member_record": "COVERAGE",
  "parent_table": "EMPLOYEE",
  "child_table": "COVERAGE",
  "parent_key": "EMP_ID_479OYEE",
  "parent_keys": [
    "EMP_ID_479OYEE"
  ],
  "child_fk": "EMP_ID_479OYEE",
  "child_fks": [
    "EMP_ID_479OYEE"
  ],
  "cardinality": "1:N",
  "order_by": [
    "EMP_ID_479OYEE"
  ]
},
{
  "set_name": "EMP_EMPOSITION",
  "parent_record": "EMPLOYEE",
  "child_record": "EMPOSITION",
  "owner_record": "EMPLOYEE",
  "member_record": "EMPOSITION",
  "parent_table": "EMPLOYEE",
  "child_table": "EMPOSITION",
  "parent_key": "EMP_ID_479OYEE",
  "parent_keys": [
    "EMP_ID_479OYEE"
  ],
  "child_fk": "EMP_ID_479OYEE",

```

```

"child_fks": [
  "EMP_ID_4790YEE"
],
"cardinality": "1:N",
"order_by": [
  "EMP_ID_4790YEE"
]
},
{
  "set_name": "EMP_EXPERTISE",
  "parent_record": "EMPLOYEE",
  "child_record": "EXPERTISE",
  "owner_record": "EMPLOYEE",
  "member_record": "EXPERTISE",
  "parent_table": "EMPLOYEE",
  "child_table": "EXPERTISE",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ],
  "child_fk": "EMP_ID_4790YEE",
  "child_fks": [
    "EMP_ID_4790YEE"
  ],
  "cardinality": "1:N",
  "order_by": [
    "EMP_ID_4790YEE"
  ]
},
{
  "set_name": "JOB_EMPOSITION",
  "parent_record": "JOB",
  "child_record": "EMPOSITION",
  "owner_record": "JOB",
  "member_record": "EMPOSITION",
  "parent_table": "JOB",
  "child_table": "EMPOSITION",
  "parent_key": "JOB_ID_479JOB",
  "parent_keys": [
    "JOB_ID_479JOB"
  ],
  "child_fk": "JOB_ID_479JOB",
  "child_fks": [
    "JOB_ID_479JOB"
  ],
  "cardinality": "1:N",
  "order_by": [
    "JOB_ID_479JOB"
  ]
},
{
  "set_name": "MANAGES",
  "parent_record": "EMPLOYEE",
  "child_record": "STRUCTURE",
  "owner_record": "EMPLOYEE",
  "member_record": "STRUCTURE",
  "parent_table": "EMPLOYEE",
  "child_table": "STRUCTURE",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ],
  "child_fk": "EMP_ID_4790YEE",
  "child_fks": [
    "EMP_ID_4790YEE"
  ],
  "cardinality": "1:N",
  "order_by": [
    "EMP_ID_4790YEE"
  ]
},
{
  "set_name": "OFFICE_EMPLOYEE",
  "parent_record": "OFFICE",
  "child_record": "EMPLOYEE",
  "owner_record": "OFFICE",

```

```

"member_record": "EMPLOYEE",
"parent_table": "OFFICE",
"child_table": "EMPLOYEE",
"parent_key": "OFFICE_CODE_479FICE",
"parent_keys": [
  "OFFICE_CODE_479FICE"
],
"child_fk": "OFFICE_CODE_479FICE",
"child_fks": [
  "OFFICE_CODE_479FICE"
],
"cardinality": "1:N",
"order_by": [
  "OFFICE_CODE_479FICE"
]
},
{
  "set_name": "REPORTS_TO",
  "parent_record": "EMPLOYEE",
  "child_record": "STRUCTURE",
  "owner_record": "EMPLOYEE",
  "member_record": "STRUCTURE",
  "parent_table": "EMPLOYEE",
  "child_table": "STRUCTURE",
  "parent_key": "EMP_ID_479OYEE",
  "parent_keys": [
    "EMP_ID_479OYEE"
  ],
  "child_fk": null,
  "child_fks": [],
  "cardinality": "1:N",
  "order_by": []
},
{
  "set_name": "SKILL_EXPERTISE",
  "parent_record": "SKILL",
  "child_record": "EXPERTISE",
  "owner_record": "SKILL",
  "member_record": "EXPERTISE",
  "parent_table": "SKILL",
  "child_table": "EXPERTISE",
  "parent_key": "SKILL_ID_479KILL",
  "parent_keys": [
    "SKILL_ID_479KILL"
  ],
  "child_fk": "SKILL_ID_479KILL",
  "child_fks": [
    "SKILL_ID_479KILL"
  ],
  "cardinality": "1:N",
  "order_by": [
    "SKILL_ID_479KILL"
  ]
}
],
"set_ordering_map": {
  "COVERAGE_CLAIMS": {
    "parent_record": "COVERAGE",
    "child_record": "NON_HOSP_CLAIM",
    "parent_key": "ID_RECORD_RAGE",
    "parent_keys": [
      "ID_RECORD_RAGE"
    ],
    "child_fk": "ID_RECORD_RAGE",
    "child_fks": [
      "ID_RECORD_RAGE"
    ],
    "order_by": [
      "ID_RECORD_RAGE"
    ]
  },
  "DEPT_EMPLOYEE": {
    "parent_record": "DEPARTMENT",
    "child_record": "EMPLOYEE",
    "parent_key": "DEPT_ID_479MENT",
    "parent_keys": [

```

```

    "DEPT_ID_479MENT"
  ],
  "child_fk": "DEPT_ID_479MENT",
  "child_fks": [
    "DEPT_ID_479MENT"
  ],
  "order_by": [
    "DEPT_ID_479MENT"
  ]
},
"EMP_COVERAGE": {
  "parent_record": "EMPLOYEE",
  "child_record": "COVERAGE",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ],
  "child_fk": "EMP_ID_4790YEE",
  "child_fks": [
    "EMP_ID_4790YEE"
  ],
  "order_by": [
    "EMP_ID_4790YEE"
  ]
},
"EMP_EMPOSITION": {
  "parent_record": "EMPLOYEE",
  "child_record": "EMPOSITION",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ],
  "child_fk": "EMP_ID_4790YEE",
  "child_fks": [
    "EMP_ID_4790YEE"
  ],
  "order_by": [
    "EMP_ID_4790YEE"
  ]
},
"EMP_EXPERTISE": {
  "parent_record": "EMPLOYEE",
  "child_record": "EXPERTISE",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ],
  "child_fk": "EMP_ID_4790YEE",
  "child_fks": [
    "EMP_ID_4790YEE"
  ],
  "order_by": [
    "EMP_ID_4790YEE"
  ]
},
"JOB_EMPOSITION": {
  "parent_record": "JOB",
  "child_record": "EMPOSITION",
  "parent_key": "JOB_ID_479JOB",
  "parent_keys": [
    "JOB_ID_479JOB"
  ],
  "child_fk": "JOB_ID_479JOB",
  "child_fks": [
    "JOB_ID_479JOB"
  ],
  "order_by": [
    "JOB_ID_479JOB"
  ]
},
"MANAGES": {
  "parent_record": "EMPLOYEE",
  "child_record": "STRUCTURE",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ]
}

```

```

    ],
    "child_fk": "EMP_ID_4790YEE",
    "child_fks": [
        "EMP_ID_4790YEE"
    ],
    "order_by": [
        "EMP_ID_4790YEE"
    ]
},
"OFFICE_EMPLOYEE": {
    "parent_record": "OFFICE",
    "child_record": "EMPLOYEE",
    "parent_key": "OFFICE_CODE_479FICE",
    "parent_keys": [
        "OFFICE_CODE_479FICE"
    ],
    "child_fk": "OFFICE_CODE_479FICE",
    "child_fks": [
        "OFFICE_CODE_479FICE"
    ],
    "order_by": [
        "OFFICE_CODE_479FICE"
    ]
},
"REPORTS_TO": {
    "parent_record": "EMPLOYEE",
    "child_record": "STRUCTURE",
    "parent_key": "EMP_ID_4790YEE",
    "parent_keys": [
        "EMP_ID_4790YEE"
    ],
    "child_fk": null,
    "child_fks": [],
    "order_by": []
},
"SKILL_EXPERTISE": {
    "parent_record": "SKILL",
    "child_record": "EXPERTISE",
    "parent_key": "SKILL_ID_479KILL",
    "parent_keys": [
        "SKILL_ID_479KILL"
    ],
    "child_fk": "SKILL_ID_479KILL",
    "child_fks": [
        "SKILL_ID_479KILL"
    ],
    "order_by": [
        "SKILL_ID_479KILL"
    ]
}
},
"nullable_fk_map": {
    "COVERAGE.EMP_ID_4790YEE": {
        "nullable": true,
        "set_name": "EMP_COVERAGE",
        "reference_table": "EMPLOYEE",
        "reference_column": "EMP_ID_4790YEE",
        "table": "COVERAGE",
        "column": "EMP_ID_4790YEE",
        "null_indicator": "NI-COVERAGE-EMP-ID-4790YEE"
    },
    "EMP_COVERAGE": {
        "nullable": true,
        "set_name": "EMP_COVERAGE",
        "reference_table": "EMPLOYEE",
        "reference_column": "EMP_ID_4790YEE",
        "table": "COVERAGE",
        "column": "EMP_ID_4790YEE",
        "null_indicator": "NI-COVERAGE-EMP-ID-4790YEE"
    },
    "DENTAL_CLAIM.ID_RECORD_RAGE": {
        "nullable": true,
        "set_name": "COVERAGE_CLAIMS",
        "reference_table": "COVERAGE",
        "reference_column": "ID_RECORD_RAGE",
        "table": "DENTAL_CLAIM",

```

```

    "column": "ID_RECORD_RAGE",
    "null_indicator": "NI-DENTAL-CLAIM-ID-RECORD-RAGE"
  },
  "COVERAGE_CLAIMS": {
    "nullable": true,
    "set_name": "COVERAGE_CLAIMS",
    "reference_table": "COVERAGE",
    "reference_column": "ID_RECORD_RAGE",
    "table": "NON_HOSP_CLAIM",
    "column": "ID_RECORD_RAGE",
    "null_indicator": "NI-NON-HOSP-CLAIM-ID-RECORD-RAGE"
  },
  "EMPLOYEE.DEPT_ID_479MENT": {
    "nullable": true,
    "set_name": "DEPT_EMPLOYEE",
    "reference_table": "DEPARTMENT",
    "reference_column": "DEPT_ID_479MENT",
    "table": "EMPLOYEE",
    "column": "DEPT_ID_479MENT",
    "null_indicator": "NI-EMPLOYEE-DEPT-ID-479MENT"
  },
  "DEPT_EMPLOYEE": {
    "nullable": true,
    "set_name": "DEPT_EMPLOYEE",
    "reference_table": "DEPARTMENT",
    "reference_column": "DEPT_ID_479MENT",
    "table": "EMPLOYEE",
    "column": "DEPT_ID_479MENT",
    "null_indicator": "NI-EMPLOYEE-DEPT-ID-479MENT"
  },
  "EMPLOYEE.OFFICE_CODE_479FICE": {
    "nullable": true,
    "set_name": "OFFICE_EMPLOYEE",
    "reference_table": "OFFICE",
    "reference_column": "OFFICE_CODE_479FICE",
    "table": "EMPLOYEE",
    "column": "OFFICE_CODE_479FICE",
    "null_indicator": "NI-EMPLOYEE-OFFICE-CODE-479FICE"
  },
  "OFFICE_EMPLOYEE": {
    "nullable": true,
    "set_name": "OFFICE_EMPLOYEE",
    "reference_table": "OFFICE",
    "reference_column": "OFFICE_CODE_479FICE",
    "table": "EMPLOYEE",
    "column": "OFFICE_CODE_479FICE",
    "null_indicator": "NI-EMPLOYEE-OFFICE-CODE-479FICE"
  },
  "EMPOSITION.EMP_ID_479OYEE": {
    "nullable": true,
    "set_name": "EMP_EMPOSITION",
    "reference_table": "EMPLOYEE",
    "reference_column": "EMP_ID_479OYEE",
    "table": "EMPOSITION",
    "column": "EMP_ID_479OYEE",
    "null_indicator": "NI-EMPOSITION-EMP-ID-479OYEE"
  },
  "EMP_EMPOSITION": {
    "nullable": true,
    "set_name": "EMP_EMPOSITION",
    "reference_table": "EMPLOYEE",
    "reference_column": "EMP_ID_479OYEE",
    "table": "EMPOSITION",
    "column": "EMP_ID_479OYEE",
    "null_indicator": "NI-EMPOSITION-EMP-ID-479OYEE"
  },
  "EMPOSITION.JOB_ID_479JOB": {
    "nullable": true,
    "set_name": "JOB_EMPOSITION",
    "reference_table": "JOB",
    "reference_column": "JOB_ID_479JOB",
    "table": "EMPOSITION",
    "column": "JOB_ID_479JOB",
    "null_indicator": "NI-EMPOSITION-JOB-ID-479JOB"
  },
  "JOB_EMPOSITION": {

```

```

    "nullable": true,
    "set_name": "JOB_EMPOSITION",
    "reference_table": "JOB",
    "reference_column": "JOB_ID_479JOB",
    "table": "EMPOSITION",
    "column": "JOB_ID_479JOB",
    "null_indicator": "NI-EMPOSITION-JOB-ID-479JOB"
  },
  "EXPERTISE.EMP_ID_479OYEE": {
    "nullable": true,
    "set_name": "EMP_EXPERTISE",
    "reference_table": "EMPLOYEE",
    "reference_column": "EMP_ID_479OYEE",
    "table": "EXPERTISE",
    "column": "EMP_ID_479OYEE",
    "null_indicator": "NI-EXPERTISE-EMP-ID-479OYEE"
  },
  "EMP_EXPERTISE": {
    "nullable": true,
    "set_name": "EMP_EXPERTISE",
    "reference_table": "EMPLOYEE",
    "reference_column": "EMP_ID_479OYEE",
    "table": "EXPERTISE",
    "column": "EMP_ID_479OYEE",
    "null_indicator": "NI-EXPERTISE-EMP-ID-479OYEE"
  },
  "EXPERTISE.SKILL_ID_479KILL": {
    "nullable": true,
    "set_name": "SKILL_EXPERTISE",
    "reference_table": "SKILL",
    "reference_column": "SKILL_ID_479KILL",
    "table": "EXPERTISE",
    "column": "SKILL_ID_479KILL",
    "null_indicator": "NI-EXPERTISE-SKILL-ID-479KILL"
  },
  "SKILL_EXPERTISE": {
    "nullable": true,
    "set_name": "SKILL_EXPERTISE",
    "reference_table": "SKILL",
    "reference_column": "SKILL_ID_479KILL",
    "table": "EXPERTISE",
    "column": "SKILL_ID_479KILL",
    "null_indicator": "NI-EXPERTISE-SKILL-ID-479KILL"
  },
  "HOSPITAL_CLAIM.ID_RECORD_RAGE": {
    "nullable": true,
    "set_name": "COVERAGE_CLAIMS",
    "reference_table": "COVERAGE",
    "reference_column": "ID_RECORD_RAGE",
    "table": "HOSPITAL_CLAIM",
    "column": "ID_RECORD_RAGE",
    "null_indicator": "NI-HOSPITAL-CLAIM-ID-RECORD-RAGE"
  },
  "NON_HOSP_CLAIM.ID_RECORD_RAGE": {
    "nullable": true,
    "set_name": "COVERAGE_CLAIMS",
    "reference_table": "COVERAGE",
    "reference_column": "ID_RECORD_RAGE",
    "table": "NON_HOSP_CLAIM",
    "column": "ID_RECORD_RAGE",
    "null_indicator": "NI-NON-HOSP-CLAIM-ID-RECORD-RAGE"
  },
  "STRUCTURE.EMP_ID_479OYEE": {
    "nullable": true,
    "set_name": "MANAGES",
    "reference_table": "EMPLOYEE",
    "reference_column": "EMP_ID_479OYEE",
    "table": "STRUCTURE",
    "column": "EMP_ID_479OYEE",
    "null_indicator": "NI-STRUCTURE-EMP-ID-479OYEE"
  },
  "MANAGES": {
    "nullable": true,
    "set_name": "MANAGES",
    "reference_table": "EMPLOYEE",
    "reference_column": "EMP_ID_479OYEE",

```



```

    "table": "STRUCTURE",
    "column": "EMP_ID_4790YEE",
    "null_indicator": "NI-STRUCTURE-EMP-ID-4790YEE"
  }
},
"table_key_map": {
  "COVERAGE": {
    "primary_keys": [
      "ID_RECORD_RAGE"
    ],
    "foreign_keys": [
      {
        "column_name": "EMP_ID_4790YEE",
        "reference_table": "EMPLOYEE",
        "reference_column": "EMP_ID_4790YEE",
        "set_name": "EMP_COVERAGE"
      }
    ]
  },
  "DENTAL_CLAIM": {
    "primary_keys": [
      "ID_RECORD_LAIM"
    ],
    "foreign_keys": [
      {
        "column_name": "ID_RECORD_RAGE",
        "reference_table": "COVERAGE",
        "reference_column": "ID_RECORD_RAGE",
        "set_name": "COVERAGE_CLAIMS"
      }
    ]
  },
  "DEPARTMENT": {
    "primary_keys": [
      "DEPT_ID_479MENT"
    ],
    "foreign_keys": []
  },
  "EMPLOYEE": {
    "primary_keys": [
      "EMP_ID_4790YEE"
    ],
    "foreign_keys": [
      {
        "column_name": "DEPT_ID_479MENT",
        "reference_table": "DEPARTMENT",
        "reference_column": "DEPT_ID_479MENT",
        "set_name": "DEPT_EMPLOYEE"
      },
      {
        "column_name": "OFFICE_CODE_479FICE",
        "reference_table": "OFFICE",
        "reference_column": "OFFICE_CODE_479FICE",
        "set_name": "OFFICE_EMPLOYEE"
      }
    ]
  },
  "EMPOSITION": {
    "primary_keys": [
      "ID_RECORD_TION"
    ],
    "foreign_keys": [
      {
        "column_name": "EMP_ID_4790YEE",
        "reference_table": "EMPLOYEE",
        "reference_column": "EMP_ID_4790YEE",
        "set_name": "EMP_EMPOSITION"
      },
      {
        "column_name": "JOB_ID_479JOB",
        "reference_table": "JOB",
        "reference_column": "JOB_ID_479JOB",
        "set_name": "JOB_EMPOSITION"
      }
    ]
  }
},

```

```

"EXPERTISE": {
  "primary_keys": [
    "ID_RECORD_TISE"
  ],
  "foreign_keys": [
    {
      "column_name": "EMP_ID_4790YEE",
      "reference_table": "EMPLOYEE",
      "reference_column": "EMP_ID_4790YEE",
      "set_name": "EMP_EXPERTISE"
    },
    {
      "column_name": "SKILL_ID_479KILL",
      "reference_table": "SKILL",
      "reference_column": "SKILL_ID_479KILL",
      "set_name": "SKILL_EXPERTISE"
    }
  ]
},
"HOSPITAL_CLAIM": {
  "primary_keys": [
    "ID_RECORD_LAIM"
  ],
  "foreign_keys": [
    {
      "column_name": "ID_RECORD_RAGE",
      "reference_table": "COVERAGE",
      "reference_column": "ID_RECORD_RAGE",
      "set_name": "COVERAGE_CLAIMS"
    }
  ]
},
"INSURANCE_PLAN": {
  "primary_keys": [
    "INS_PLANCODE_479PLAN"
  ],
  "foreign_keys": []
},
"JOB": {
  "primary_keys": [
    "JOB_ID_479JOB"
  ],
  "foreign_keys": []
},
"NON_HOSP_CLAIM": {
  "primary_keys": [
    "ID_RECORD_LAIM"
  ],
  "foreign_keys": [
    {
      "column_name": "ID_RECORD_RAGE",
      "reference_table": "COVERAGE",
      "reference_column": "ID_RECORD_RAGE",
      "set_name": "COVERAGE_CLAIMS"
    }
  ]
},
"OFFICE": {
  "primary_keys": [
    "OFFICE_CODE_479FICE"
  ],
  "foreign_keys": []
},
"SKILL": {
  "primary_keys": [
    "SKILL_ID_479KILL"
  ],
  "foreign_keys": []
},
"STRUCTURE": {
  "primary_keys": [
    "ID_RECORD_TURE"
  ],
  "foreign_keys": [
    {
      "column_name": "EMP_ID_4790YEE",

```

```

        "reference_table": "EMPLOYEE",
        "reference_column": "EMP_ID_4790YEE",
        "set_name": "MANAGES"
    }
}
}
},
"date_part_map": {
    "SELECTION_YEAR_0400": {
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "DA_SELECTIONDATE_479RAGE",
        "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
        "substring_start": 3,
        "substring_length": 2,
        "part": "YEAR",
        "date_part": "YEAR",
        "legacy_field": "SELECTION_YEAR_0400"
    },
    "SELECTION-YEAR-0400": {
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "DA_SELECTIONDATE_479RAGE",
        "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
        "substring_start": 3,
        "substring_length": 2,
        "part": "YEAR",
        "date_part": "YEAR",
        "legacy_field": "SELECTION_YEAR_0400"
    },
    "SELECTION_YEAR": {
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "DA_SELECTIONDATE_479RAGE",
        "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
        "substring_start": 3,
        "substring_length": 2,
        "part": "YEAR",
        "date_part": "YEAR",
        "legacy_field": "SELECTION_YEAR_0400"
    },
    "SELECTIONYEAR0400": {
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "DA_SELECTIONDATE_479RAGE",
        "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
        "substring_start": 3,
        "substring_length": 2,
        "part": "YEAR",
        "date_part": "YEAR",
        "legacy_field": "SELECTION_YEAR_0400"
    },
    "SELECTION_MONTH_0400": {
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "DA_SELECTIONDATE_479RAGE",
        "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
        "substring_start": 6,
        "substring_length": 2,
        "part": "MONTH",
        "date_part": "MONTH",
        "legacy_field": "SELECTION_MONTH_0400"
    },
    "SELECTION-MONTH-0400": {
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "DA_SELECTIONDATE_479RAGE",
        "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
        "substring_start": 6,
        "substring_length": 2,
        "part": "MONTH",
        "date_part": "MONTH",
        "legacy_field": "SELECTION_MONTH_0400"
    },
    "SELECTION_MONTH": {
        "record": "COVERAGE",

```

```

"table": "COVERAGE",
"column": "DA_SELECTIONDATE_479RAGE",
"host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "SELECTION_MONTH_0400"
},
"SELECTIONMONTH0400": {
"record": "COVERAGE",
"table": "COVERAGE",
"column": "DA_SELECTIONDATE_479RAGE",
"host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "SELECTION_MONTH_0400"
},
"SELECTION_DAY_0400": {
"record": "COVERAGE",
"table": "COVERAGE",
"column": "DA_SELECTIONDATE_479RAGE",
"host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "SELECTION_DAY_0400"
},
"SELECTION-DAY-0400": {
"record": "COVERAGE",
"table": "COVERAGE",
"column": "DA_SELECTIONDATE_479RAGE",
"host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "SELECTION_DAY_0400"
},
"SELECTION_DAY": {
"record": "COVERAGE",
"table": "COVERAGE",
"column": "DA_SELECTIONDATE_479RAGE",
"host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "SELECTION_DAY_0400"
},
"SELECTIONDAY0400": {
"record": "COVERAGE",
"table": "COVERAGE",
"column": "DA_SELECTIONDATE_479RAGE",
"host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "SELECTION_DAY_0400"
},
"TERMINATION_YEAR_0400": {
"record": "COVERAGE",
"table": "COVERAGE",
"column": "DA_TERMINATIONDATE_479RAGE",
"host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "TERMINATION_YEAR_0400"
},
"TERMINATION-YEAR-0400": {

```

```

"record": "COVERAGE",
"table": "COVERAGE",
"column": "DA_TERMINATIONDATE_479RAGE",
"host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "TERMINATION_YEAR_0400"
},
"TERMINATION_YEAR": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_TERMINATIONDATE_479OYEE",
"host": "HV-EMPLOYEE-DA-TERMINATIONDATE-479OYEE",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "TERMINATION_YEAR_0415"
},
"TERMINATIONYEAR0400": {
"record": "COVERAGE",
"table": "COVERAGE",
"column": "DA_TERMINATIONDATE_479RAGE",
"host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "TERMINATION_YEAR_0400"
},
"TERMINATION_MONTH_0400": {
"record": "COVERAGE",
"table": "COVERAGE",
"column": "DA_TERMINATIONDATE_479RAGE",
"host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "TERMINATION_MONTH_0400"
},
"TERMINATION-MONTH-0400": {
"record": "COVERAGE",
"table": "COVERAGE",
"column": "DA_TERMINATIONDATE_479RAGE",
"host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "TERMINATION_MONTH_0400"
},
"TERMINATION_MONTH": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_TERMINATIONDATE_479OYEE",
"host": "HV-EMPLOYEE-DA-TERMINATIONDATE-479OYEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "TERMINATION_MONTH_0415"
},
"TERMINATIONMONTH0400": {
"record": "COVERAGE",
"table": "COVERAGE",
"column": "DA_TERMINATIONDATE_479RAGE",
"host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "TERMINATION_MONTH_0400"
},

```

```

"TERMINATION_DAY_0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "TERMINATION_DAY_0400"
},
"TERMINATION-DAY-0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "TERMINATION_DAY_0400"
},
"TERMINATION_DAY": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "DA_TERMINATIONDATE_479OYEE",
  "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-479OYEE",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "TERMINATION_DAY_0415"
},
"TERMINATIONDAY0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "TERMINATION_DAY_0400"
},
"CLAIM_YEAR_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "CLAIM_YEAR_0405"
},
"CLAIM-YEAR-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "CLAIM_YEAR_0405"
},
"CLAIM_YEAR": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "CLAIM_YEAR_0445"
}

```

```

},
"CLAIMYEAR0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "CLAIM_YEAR_0405"
},
"CLAIM_MONTH_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "CLAIM_MONTH_0405"
},
"CLAIM-MONTH-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "CLAIM_MONTH_0405"
},
"CLAIM_MONTH": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "CLAIM_MONTH_0445"
},
"CLAIMMONTH0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "CLAIM_MONTH_0405"
},
"CLAIM_DAY_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "CLAIM_DAY_0405"
},
"CLAIM-DAY-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",

```

```

    "legacy_field": "CLAIM_DAY_0405"
  },
  "CLAIM_DAY": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "CLAIM_DAY_0445"
  },
  "CLAIMDAY0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "CLAIM_DAY_0405"
  },
  "PATIENT_BIRTH_YEAR_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "PATIENT_BIRTH_YEAR_0405"
  },
  "PATIENT-BIRTH-YEAR-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "PATIENT_BIRTH_YEAR_0405"
  },
  "PATIENT_BIRTH_YEAR": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "PATIENT_BIRTH_YEAR_0445"
  },
  "PATIENTBIRTHYEAR0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "PATIENT_BIRTH_YEAR_0405"
  },
  "PATIENT_BIRTH_MONTH_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",

```



```

    "date_part": "MONTH",
    "legacy_field": "PATIENT_BIRTH_MONTH_0405"
  },
  "PATIENT-BIRTH-MONTH-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "PATIENT_BIRTH_MONTH_0405"
  },
  "PATIENT_BIRTH_MONTH": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "PATIENT_BIRTH_MONTH_0445"
  },
  "PATIENTBIRTHMONTH0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "PATIENT_BIRTH_MONTH_0405"
  },
  "PATIENT_BIRTH_DAY_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "PATIENT_BIRTH_DAY_0405"
  },
  "PATIENT-BIRTH-DAY-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "PATIENT_BIRTH_DAY_0405"
  },
  "PATIENT_BIRTH_DAY": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "PATIENT_BIRTH_DAY_0445"
  },
  "PATIENTBIRTHDAY0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,

```

```

    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "PATIENT_BIRTH_DAY_0405"
  },
  "SERVICE_YEAR_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "SERVICE_YEAR_0405"
  },
  "SERVICE-YEAR-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "SERVICE_YEAR_0405"
  },
  "SERVICE_YEAR": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "SERVICE_YEAR_0445"
  },
  "SERVICEYEAR0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "SERVICE_YEAR_0405"
  },
  "SERVICE_MONTH_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SERVICE_MONTH_0405"
  },
  "SERVICE-MONTH-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SERVICE_MONTH_0405"
  },
  "SERVICE_MONTH": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 6,

```

```

    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SERVICE_MONTH_0445"
  },
  "SERVICEMONTH0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SERVICE_MONTH_0405"
  },
  "SERVICE_DAY_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SERVICE_DAY_0405"
  },
  "SERVICE-DAY-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SERVICE_DAY_0405"
  },
  "SERVICE_DAY": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SERVICE_DAY_0445"
  },
  "SERVICEDAY0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SERVICE_DAY_0405"
  },
  "START_YEAR_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "START_YEAR_0415"
  },
  "START-YEAR-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",

```

```

    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "START_YEAR_0415"
  },
  "START_YEAR": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_STARTDATE_479TION",
    "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "START_YEAR_0420"
  },
  "STARTYEAR0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_479OYEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-479OYEE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "START_YEAR_0415"
  },
  "START_MONTH_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_479OYEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-479OYEE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "START_MONTH_0415"
  },
  "START-MONTH-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_479OYEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-479OYEE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "START_MONTH_0415"
  },
  "START_MONTH": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_STARTDATE_479TION",
    "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "START_MONTH_0420"
  },
  "STARTMONTH0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_479OYEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-479OYEE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "START_MONTH_0415"
  },
  "START_DAY_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_479OYEE",

```

```

    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "START_DAY_0415"
  },
  "START-DAY-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "START_DAY_0415"
  },
  "START_DAY": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_STARTDATE_479TION",
    "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "START_DAY_0420"
  },
  "STARTDAY0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "START_DAY_0415"
  },
  "TERMINATION_YEAR_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "TERMINATION_YEAR_0415"
  },
  "TERMINATION-YEAR-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "TERMINATION_YEAR_0415"
  },
  "TERMINATIONYEAR0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "TERMINATION_YEAR_0415"
  },
  "TERMINATION_MONTH_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",

```

```

"column": "DA_TERMINATIONDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "TERMINATION_MONTH_0415"
},
"TERMINATION-MONTH-0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_TERMINATIONDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "TERMINATION_MONTH_0415"
},
"TERMINATIONMONTH0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_TERMINATIONDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "TERMINATION_MONTH_0415"
},
"TERMINATION_DAY_0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_TERMINATIONDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "TERMINATION_DAY_0415"
},
"TERMINATION-DAY-0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_TERMINATIONDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "TERMINATION_DAY_0415"
},
"TERMINATIONDAY0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_TERMINATIONDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "TERMINATION_DAY_0415"
},
"BIRTH_YEAR_0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "BIRTH_YEAR_0415"
},
"BIRTH-YEAR-0415": {
"record": "EMPLOYEE",

```

```

"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "BIRTH_YEAR_0415"
},
"BIRTH_YEAR": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "BIRTH_YEAR_0415"
},
"BIRTHYEAR0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "BIRTH_YEAR_0415"
},
"BIRTH_MONTH_0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "BIRTH_MONTH_0415"
},
"BIRTH-MONTH-0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "BIRTH_MONTH_0415"
},
"BIRTH_MONTH": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "BIRTH_MONTH_0415"
},
"BIRTHMONTH0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "BIRTH_MONTH_0415"
},
"BIRTH_DAY_0415": {

```

```

    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "BIRTH_DAY_0415"
  },
  "BIRTH-DAY-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "BIRTH_DAY_0415"
  },
  "BIRTH_DAY": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "BIRTH_DAY_0415"
  },
  "BIRTHDAY0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "BIRTH_DAY_0415"
  },
  "START_YEAR_0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_STARTDATE_479TION",
    "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "START_YEAR_0420"
  },
  "START-YEAR-0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_STARTDATE_479TION",
    "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "START_YEAR_0420"
  },
  "STARTYEAR0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_STARTDATE_479TION",
    "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "START_YEAR_0420"
  },

```



```

"START_MONTH_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "START_MONTH_0420"
},
"START-MONTH-0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "START_MONTH_0420"
},
"STARTMONTH0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "START_MONTH_0420"
},
"START_DAY_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "START_DAY_0420"
},
"START-DAY-0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "START_DAY_0420"
},
"STARTDAY0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "START_DAY_0420"
},
"FINISH_YEAR_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "FINISH_YEAR_0420"
}

```

```

},
"FINISH-YEAR-0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "FINISH_YEAR_0420"
},
"FINISH_YEAR": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "FINISH_YEAR_0420"
},
"FINISHYEAR0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "FINISH_YEAR_0420"
},
"FINISH_MONTH_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "FINISH_MONTH_0420"
},
"FINISH-MONTH-0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "FINISH_MONTH_0420"
},
"FINISH_MONTH": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "FINISH_MONTH_0420"
},
"FINISHMONTH0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",

```

```

    "legacy_field": "FINISH_MONTH_0420"
  },
  "FINISH_DAY_0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_FINISHDATE_479TION",
    "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "FINISH_DAY_0420"
  },
  "FINISH-DAY-0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_FINISHDATE_479TION",
    "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "FINISH_DAY_0420"
  },
  "FINISH_DAY": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_FINISHDATE_479TION",
    "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "FINISH_DAY_0420"
  },
  "FINISHDAY0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_FINISHDATE_479TION",
    "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "FINISH_DAY_0420"
  },
  "EXPERTISE_YEAR_0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "EXPERTISE_YEAR_0425"
  },
  "EXPERTISE-YEAR-0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "EXPERTISE_YEAR_0425"
  },
  "EXPERTISE_YEAR": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",

```

```

    "date_part": "YEAR",
    "legacy_field": "EXPERTISE_YEAR_0425"
  },
  "EXPERTISEYEAR0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "EXPERTISE_YEAR_0425"
  },
  "EXPERTISE_MONTH_0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "EXPERTISE_MONTH_0425"
  },
  "EXPERTISE-MONTH-0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "EXPERTISE_MONTH_0425"
  },
  "EXPERTISE_MONTH": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "EXPERTISE_MONTH_0425"
  },
  "EXPERTISEMONTH0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "EXPERTISE_MONTH_0425"
  },
  "EXPERTISE_DAY_0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "EXPERTISE_DAY_0425"
  },
  "EXPERTISE-DAY-0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 9,
    "substring_length": 2,

```

```

    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "EXPERTISE_DAY_0425"
  },
  "EXPERTISE_DAY": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "EXPERTISE_DAY_0425"
  },
  "EXPERTISEDAY0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "EXPERTISE_DAY_0425"
  },
  "CLAIM_YEAR_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "CLAIM_YEAR_0430"
  },
  "CLAIM-YEAR-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "CLAIM_YEAR_0430"
  },
  "CLAIMYEAR0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "CLAIM_YEAR_0430"
  },
  "CLAIM_MONTH_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "CLAIM_MONTH_0430"
  },
  "CLAIM-MONTH-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 6,

```

```

    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "CLAIM_MONTH_0430"
  },
  "CLAIMMONTH0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "CLAIM_MONTH_0430"
  },
  "CLAIM_DAY_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "CLAIM_DAY_0430"
  },
  "CLAIM-DAY-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "CLAIM_DAY_0430"
  },
  "CLAIMDAY0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "CLAIM_DAY_0430"
  },
  "PATIENT_BIRTH_YEAR_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "PATIENT_BIRTH_YEAR_0430"
  },
  "PATIENT-BIRTH-YEAR-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "PATIENT_BIRTH_YEAR_0430"
  },
  "PATIENTBIRTHYEAR0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",

```

```

    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "PATIENT_BIRTH_YEAR_0430"
  },
  "PATIENT_BIRTH_MONTH_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "PATIENT_BIRTH_MONTH_0430"
  },
  "PATIENT-BIRTH-MONTH-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "PATIENT_BIRTH_MONTH_0430"
  },
  "PATIENTBIRTHMONTH0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "PATIENT_BIRTH_MONTH_0430"
  },
  "PATIENT_BIRTH_DAY_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "PATIENT_BIRTH_DAY_0430"
  },
  "PATIENT-BIRTH-DAY-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "PATIENT_BIRTH_DAY_0430"
  },
  "PATIENTBIRTHDAY0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "PATIENT_BIRTH_DAY_0430"
  },
  "ADMIT_YEAR_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",

```

```

    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "ADMIT_YEAR_0430"
  },
  "ADMIT-YEAR-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "ADMIT_YEAR_0430"
  },
  "ADMIT_YEAR": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "ADMIT_YEAR_0430"
  },
  "ADMITYEAR0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "ADMIT_YEAR_0430"
  },
  "ADMIT_MONTH_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "ADMIT_MONTH_0430"
  },
  "ADMIT-MONTH-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "ADMIT_MONTH_0430"
  },
  "ADMIT_MONTH": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "ADMIT_MONTH_0430"
  },
  "ADMITMONTH0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",

```



```

"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "ADMIT_MONTH_0430"
},
"ADMIT_DAY_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "ADMIT_DAY_0430"
},
"ADMIT-DAY-0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "ADMIT_DAY_0430"
},
"ADMIT_DAY": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "ADMIT_DAY_0430"
},
"ADMITDAY0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "ADMIT_DAY_0430"
},
"DISCHARGE_YEAR_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "DISCHARGE_YEAR_0430"
},
"DISCHARGE-YEAR-0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "DISCHARGE_YEAR_0430"
},
"DISCHARGE_YEAR": {
"record": "HOSPITAL_CLAIM",

```

```

"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "DISCHARGE_YEAR_0430"
},
"DISCHARGEYEAR0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "DISCHARGE_YEAR_0430"
},
"DISCHARGE_MONTH_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "DISCHARGE_MONTH_0430"
},
"DISCHARGE-MONTH-0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "DISCHARGE_MONTH_0430"
},
"DISCHARGE_MONTH": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "DISCHARGE_MONTH_0430"
},
"DISCHARGEMONTH0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "DISCHARGE_MONTH_0430"
},
"DISCHARGE_DAY_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "DISCHARGE_DAY_0430"
},
"DISCHARGE-DAY-0430": {

```

```

    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "DISCHARGE_DAY_0430"
  },
  "DISCHARGE_DAY": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "DISCHARGE_DAY_0430"
  },
  "DISCHARGEDAY0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "DISCHARGE_DAY_0430"
  },
  "CLAIM_YEAR_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "CLAIM_YEAR_0445"
  },
  "CLAIM-YEAR-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "CLAIM_YEAR_0445"
  },
  "CLAIMYEAR0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "CLAIM_YEAR_0445"
  },
  "CLAIM_MONTH_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "CLAIM_MONTH_0445"
  },

```

```

"CLAIM-MONTH-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "CLAIM_MONTH_0445"
},
"CLAIMMONTH0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "CLAIM_MONTH_0445"
},
"CLAIM_DAY_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "CLAIM_DAY_0445"
},
"CLAIM-DAY-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "CLAIM_DAY_0445"
},
"CLAIMDAY0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "CLAIM_DAY_0445"
},
"PATIENT_BIRTH_YEAR_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "PATIENT_BIRTH_YEAR_0445"
},
"PATIENT-BIRTH-YEAR-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "PATIENT_BIRTH_YEAR_0445"
}

```

```

},
"PATIENTBIRTHYEAR0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "PATIENT_BIRTH_YEAR_0445"
},
"PATIENT_BIRTH_MONTH_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "PATIENT_BIRTH_MONTH_0445"
},
"PATIENT-BIRTH-MONTH-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "PATIENT_BIRTH_MONTH_0445"
},
"PATIENTBIRTHMONTH0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "PATIENT_BIRTH_MONTH_0445"
},
"PATIENT_BIRTH_DAY_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "PATIENT_BIRTH_DAY_0445"
},
"PATIENT-BIRTH-DAY-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "PATIENT_BIRTH_DAY_0445"
},
"PATIENTBIRTHDAY0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",

```

```

    "legacy_field": "PATIENT_BIRTH_DAY_0445"
  },
  "SERVICE_YEAR_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "SERVICE_YEAR_0445"
  },
  "SERVICE_YEAR-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "SERVICE_YEAR_0445"
  },
  "SERVICEYEAR0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "SERVICE_YEAR_0445"
  },
  "SERVICE_MONTH_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SERVICE_MONTH_0445"
  },
  "SERVICE-MONTH-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SERVICE_MONTH_0445"
  },
  "SERVICEMONTH0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SERVICE_MONTH_0445"
  },
  "SERVICE_DAY_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",

```

```

    "date_part": "DAY",
    "legacy_field": "SERVICE_DAY_0445"
  },
  "SERVICE-DAY-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SERVICE_DAY_0445"
  },
  "SERVICEDAY0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SERVICE_DAY_0445"
  },
  "STRUCTURE_YEAR_0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "STRUCTURE_YEAR_0460"
  },
  "STRUCTURE-YEAR-0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "STRUCTURE_YEAR_0460"
  },
  "STRUCTURE_YEAR": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "STRUCTURE_YEAR_0460"
  },
  "STRUCTUREYEAR0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "STRUCTURE_YEAR_0460"
  },
  "STRUCTURE_MONTH_0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 6,
    "substring_length": 2,

```

```

    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "STRUCTURE_MONTH_0460"
  },
  "STRUCTURE-MONTH-0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "STRUCTURE_MONTH_0460"
  },
  "STRUCTURE_MONTH": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "STRUCTURE_MONTH_0460"
  },
  "STRUCTUREMONTH0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "STRUCTURE_MONTH_0460"
  },
  "STRUCTURE_DAY_0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "STRUCTURE_DAY_0460"
  },
  "STRUCTURE-DAY-0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "STRUCTURE_DAY_0460"
  },
  "STRUCTURE_DAY": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "STRUCTURE_DAY_0460"
  },
  "STRUCTUREDAY0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 9,

```



```

        "substring_length": 2,
        "part": "DAY",
        "date_part": "DAY",
        "legacy_field": "STRUCTURE_DAY_0460"
    }
},
"navigation_intent": {
    "COVERAGE_CLAIMS": {
        "access_pattern": "SET_NAVIGATION",
        "parent_record": "COVERAGE",
        "child_record": "NON_HOSP_CLAIM",
        "parent_key": "ID_RECORD_RAGE",
        "parent_keys": [
            "ID_RECORD_RAGE"
        ],
        "child_fk": "ID_RECORD_RAGE",
        "child_fks": [
            "ID_RECORD_RAGE"
        ],
        "order_by": [
            "ID_RECORD_RAGE"
        ]
    },
    "DEPT_EMPLOYEE": {
        "access_pattern": "SET_NAVIGATION",
        "parent_record": "DEPARTMENT",
        "child_record": "EMPLOYEE",
        "parent_key": "DEPT_ID_479MENT",
        "parent_keys": [
            "DEPT_ID_479MENT"
        ],
        "child_fk": "DEPT_ID_479MENT",
        "child_fks": [
            "DEPT_ID_479MENT"
        ],
        "order_by": [
            "DEPT_ID_479MENT"
        ]
    },
    "EMP_COVERAGE": {
        "access_pattern": "SET_NAVIGATION",
        "parent_record": "EMPLOYEE",
        "child_record": "COVERAGE",
        "parent_key": "EMP_ID_4790YEE",
        "parent_keys": [
            "EMP_ID_4790YEE"
        ],
        "child_fk": "EMP_ID_4790YEE",
        "child_fks": [
            "EMP_ID_4790YEE"
        ],
        "order_by": [
            "EMP_ID_4790YEE"
        ]
    },
    "EMP_EMPOSITION": {
        "access_pattern": "SET_NAVIGATION",
        "parent_record": "EMPLOYEE",
        "child_record": "EMPOSITION",
        "parent_key": "EMP_ID_4790YEE",
        "parent_keys": [
            "EMP_ID_4790YEE"
        ],
        "child_fk": "EMP_ID_4790YEE",
        "child_fks": [
            "EMP_ID_4790YEE"
        ],
        "order_by": [
            "EMP_ID_4790YEE"
        ]
    },
    "EMP_EXPERTISE": {
        "access_pattern": "SET_NAVIGATION",
        "parent_record": "EMPLOYEE",
        "child_record": "EXPERTISE",
        "parent_key": "EMP_ID_4790YEE",

```

```

"parent_keys": [
  "EMP_ID_4790YEE"
],
"child_fk": "EMP_ID_4790YEE",
"child_fks": [
  "EMP_ID_4790YEE"
],
"order_by": [
  "EMP_ID_4790YEE"
]
},
"JOB_EMPOSITION": {
  "access_pattern": "SET_NAVIGATION",
  "parent_record": "JOB",
  "child_record": "EMPOSITION",
  "parent_key": "JOB_ID_479JOB",
  "parent_keys": [
    "JOB_ID_479JOB"
  ],
  "child_fk": "JOB_ID_479JOB",
  "child_fks": [
    "JOB_ID_479JOB"
  ],
  "order_by": [
    "JOB_ID_479JOB"
  ]
},
"MANAGES": {
  "access_pattern": "SET_NAVIGATION",
  "parent_record": "EMPLOYEE",
  "child_record": "STRUCTURE",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ],
  "child_fk": "EMP_ID_4790YEE",
  "child_fks": [
    "EMP_ID_4790YEE"
  ],
  "order_by": [
    "EMP_ID_4790YEE"
  ]
},
"OFFICE_EMPLOYEE": {
  "access_pattern": "SET_NAVIGATION",
  "parent_record": "OFFICE",
  "child_record": "EMPLOYEE",
  "parent_key": "OFFICE_CODE_479FICE",
  "parent_keys": [
    "OFFICE_CODE_479FICE"
  ],
  "child_fk": "OFFICE_CODE_479FICE",
  "child_fks": [
    "OFFICE_CODE_479FICE"
  ],
  "order_by": [
    "OFFICE_CODE_479FICE"
  ]
},
"REPORTS_TO": {
  "access_pattern": "SET_NAVIGATION",
  "parent_record": "EMPLOYEE",
  "child_record": "STRUCTURE",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ],
  "child_fk": null,
  "child_fks": [],
  "order_by": []
},
"SKILL_EXPERTISE": {
  "access_pattern": "SET_NAVIGATION",
  "parent_record": "SKILL",
  "child_record": "EXPERTISE",
  "parent_key": "SKILL_ID_479KILL",

```

```

    "parent_keys": [
        "SKILL_ID_479KILL"
    ],
    "child_fk": "SKILL_ID_479KILL",
    "child_fks": [
        "SKILL_ID_479KILL"
    ],
    "order_by": [
        "SKILL_ID_479KILL"
    ]
}
},
"output_semantics": {
    "generated_by": "idms-db2-modernizer-phase1",
    "usage": "IDMS retrieval COBOL to DB2 embedded SQL conversion",
    "physical_types_source": "DB2_DDL",
    "logical_mapping_source": "PHASE1_CANONICAL_METADATA",
    "db2_key_values": [
        "PK",
        "FK",
        "PK/FK"
    ],
    "generated_primary_key_rule": "ID_RECORD_<record_name>",
    "generated_primary_key_datatype": "CHAR(20)",
    "foreign_key_rule": "Use SET relationship and owner primary key columns only.",
    "character_rule": "CHAR by default; VARCHAR only when length equals 100.",
    "numeric_rule": "Numeric COBOL PIC maps to DECIMAL.",
    "date_rule": "Date part recognition remains generic for YEAR, YR, Y, YY, YYYY, DY, MONTH, MON, MO, M, MM, DM, DAY,
D, DD."
},
"validation_messages": [],
"canonical_record_lookup_count": 21
}

```

```

=====
FILE: installed.txt
ENCODING: utf-8
=====

```

```

aistudio-sdk==0.3.8
altair==6.2.1
annotated-doc==0.0.4
annotated-types==0.7.0
anyio==4.14.0
ast-serialize==0.5.0
astor==0.8.1
attrs==26.1.0
bce-python-sdk==0.9.72
blinker==1.9.0
cachetools==7.1.4
certifi==2026.6.17
cffi==2.0.0
cfgv==3.5.0
charset==7.4.3
charset-normalizer==3.4.7
click==8.4.1
colorama==0.4.6
colorlog==6.10.1
coverage==7.14.1
crc32c==2.8
cryptography==49.0.0
decorator==5.3.1
distlib==0.4.3
easyocr==1.7.2
et-xmlfile==2.0.0
filelock==3.29.4
fsspec==2026.6.0
future==1.0.0
gitdb==4.0.12
gitpython==3.1.50
graphviz==0.21
h11==0.16.0
hf-xet==1.5.1
httpcore==1.0.9
httpx==0.27.0
httpxtools==0.8.0

```

httpx==0.28.1
huggingface-hub==1.19.0
identify==2.6.19
idna==3.18
imageio==2.37.3
imagesize==2.0.0
iniconfig==2.3.0
itsdangerous==2.2.0
jinja2==3.1.6
jsonschema==4.26.0
jsonschema-specifications==2025.9.1
lazy-loader==0.5
librt==0.11.0
markdown-it-py==4.2.0
markupsafe==3.0.3
mdurl==0.1.2
modelscope==1.37.1
mpmath==1.3.0
mypy==2.1.0
mypy-extensions==1.1.0
narwhals==2.22.1
networkx==3.6.1
ninja==1.13.0
nodeenv==1.10.0
numpy==2.4.6
opencv-contrib-python==4.10.0.84
opencv-python==4.13.0.92
opencv-python-headless==4.13.0.92
openpyxl==3.1.5
opt-einsum==3.3.0
packaging==26.2
paddleocr==3.4.1
paddlepaddle==3.0.0
paddlex==3.4.3
pandas==3.0.3
pathspec==1.1.1
pdfminer-six==20260107
pdfplumber==0.11.10
pillow==12.2.0
platformdirs==4.10.0
pluggy==1.6.0
pre-commit==4.6.0
prettytable==3.17.0
protobuf==7.35.1
psutil==7.2.2
py-cpuinfo==9.0.0
pyarrow==24.0.0
pyclipper==1.4.0
pycparser==3.0
pycryptodome==3.23.0
pydantic==2.13.4
pydantic-core==2.46.4
pydantic-settings==2.14.1
pydeck==0.9.2
pygments==2.20.0
pymupdf==1.27.2.3
pypdfium2==5.10.1
pytest==9.1.0
pytest-cov==7.1.0
pytest-mock==3.15.1
python-bidi==0.6.10
python-dateutil==2.9.0.post0
python-discovery==1.4.2
python-dotenv==1.2.2
python-multipart==0.0.32
pyyaml==6.0.2
referencing==0.37.0
reportlab==4.5.1
requests==2.34.2
rich==15.0.0
rpds-py==2026.5.1
ruamel-yaml==0.19.1
ruff==0.15.17
safetensors==0.8.0
scikit-image==0.26.0
scipy==1.17.1

```
setuptools==81.0.0
shapely==2.1.2
shellingham==1.5.4
six==1.17.0
smmap==5.0.3
starlette==1.3.1
streamlit==1.58.0
structlog==26.1.0
sympy==1.14.0
tenacity==9.1.4
tiffiff==2026.6.1
toml==0.10.2
torch==2.12.1
torchvision==0.27.1
tqdm==4.68.3
typer==0.25.1
typing-extensions==4.15.0
typing-inspection==0.4.2
tzdata==2026.2
ujson==5.13.0
urllib3==2.7.0
uvicorn==0.49.0
virtualenv==21.5.1
watchdog==6.0.0
wcwidth==0.8.1
websockets==16.0
```

```
=====
FILE: main.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.services.metadata_service import (
    MetadataService
)

from idms_modernizer.services.validation_service import (
    ValidationService
)

from idms_modernizer.ui.metadata_page import (
    render_metadata
)

from idms_modernizer.ui.upload_page import (
    render_upload_page
)

metadata_service = MetadataService()

validator = ValidationService()

pdf_path = render_upload_page()

if pdf_path:
    metadata = (
        metadata_service.build_metadata(
            pdf_path
        )
    )

    validator.validate_metadata(
        metadata
    )

    render_metadata(
        metadata
    )
```

```
=====
FILE: pyproject.toml
ENCODING: utf-8
=====
```

```
[project]
name = "idms-db2-modernizer"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "graphviz>=0.21",
    "networkx>=3.6.1",
    "pdfplumber>=0.11.10",
    "pydantic>=2.13.4",
    "pydantic-settings>=2.14.1",
    "streamlit>=1.58.0",
]
```

```
[tool.setuptools]
package-dir = {"" = "src"}
```

```
[tool.setuptools.packages.find]
where = ["src"]
```

```
[dependency-groups]
dev = [
    "pytest>=9.1.0",
    "pdfminer.six>=20250506",
]
```

```
=====
FILE: readme
ENCODING: utf-8
=====
```

```
set PYTHONPATH=src
python -m streamlit run src\idms_modernizer\app.py --server.port 8502
```

```
.venv\Scripts\Activate
python -m streamlit run app\streamlit_app.py
```

```
python src\tools\audit_hardcoding.py
```

```
python -m streamlit run app\streamlit_app.py --server.port 8502
```

```
streamlit run src/idms_modernizer/ui/schema_listing_only_app.py
```

```
taskkill /F /IM python.exe
--
```

```
=====
FILE: README.md
ENCODING: utf-8
=====
```

```
[Empty file]
```

```
=====
FILE: src\idms_db2_converter\__init__.py
ENCODING: utf-8
=====
```

```
[Empty file]
```

```
=====
FILE: src\idms_db2_converter\exceptions.py
ENCODING: utf-8
=====
```

```
class ConversionError(Exception):
    """Raised when IDMS to DB2 conversion fails."""
```

```
=====
FILE: src\idms_db2_converter\generators\__init__.py
ENCODING: utf-8
=====
```

```
[Empty file]
```

```
=====
FILE: src\idms_db2_converter\generators\cobol_builder.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Protocol
```

```
class CobolNode(Protocol):
```

```
    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:
        ...
```

```
def pad(
    indent: int
) -> str:

    return " " * indent
```

```
def ensure_period(
    value: str
) -> str:

    value = value.rstrip()

    if value.endswith("."):
        return value

    return value + "."
```

```
@dataclass
class PreRenderedBlock:
```

```
    lines: list[str]

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        result = list(
            self.lines
        )

        if final and result:
            result[-1] = ensure_period(
                result[-1]
            )
```

```
    return result
```

```
@dataclass
```

```
class RawLines:
```

```
    lines: list[str]
```

```
    def render(  
        self,  
        indent: int = 7,  
        final: bool = False  
    ) -> list[str]:
```

```
        result: list[str] = []  
  
        for line in self.lines:  
            if line.strip():  
                result.append(  
                    pad(indent) + line.strip()  
                )  
            else:  
                result.append("")
```

```
        if final and result:  
            result[-1] = ensure_period(  
                result[-1]  
            )
```

```
        return result
```

```
@dataclass
```

```
class Move:
```

```
    source: str
```

```
    target: str
```

```
    def render(  
        self,  
        indent: int = 7,  
        final: bool = False  
    ) -> list[str]:
```

```
        return [  
            f"{pad(indent)}MOVE {self.source} TO {self.target}."  
        ]
```

```
@dataclass
```

```
class Perform:
```

```
    paragraph: str
```

```
    def render(  
        self,  
        indent: int = 7,  
        final: bool = False  
    ) -> list[str]:
```

```
        return [  
            f"{pad(indent)}PERFORM {self.paragraph}."  
        ]
```

```
@dataclass
```

```
class Continue:
```

```
    def render(  
        self,  
        indent: int = 7,  
        final: bool = False  
    ) -> list[str]:
```

```
        return [  
  
            ]
```



```

        f"{pad(indent)}CONTINUE."
    ]

```

```

@dataclass
class ReadAtEndMove:

    file_name: str
    source: str
    target: str

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        source = self.source

        if (
            len(source) == 1
            and not source.startswith('"')
            and not source.endswith('"')
        ):
            source = f"'{source}'"

        return [
            f"{pad(indent)}READ {self.file_name} AT END MOVE {source} TO {self.target}."
        ]

```

```

@dataclass
class Write:

    record: str
    source: str | None = None

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        if self.source:
            return [
                f"{pad(indent)}WRITE {self.record} FROM {self.source}."
            ]

        return [
            f"{pad(indent)}WRITE {self.record}."
        ]

```

```

@dataclass
class ExecSql:

    sql: str

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        normalized_lines = self._normalize_sql_lines(
            self.sql
        )

        result: list[str] = []

        for line in normalized_lines:
            if line.strip():
                result.append(
                    pad(indent) + line.rstrip()
                )
            else:

```

```

        result.append("")

    if final and result:
        result[-1] = ensure_period(
            result[-1]
        )

    return result

def _normalize_sql_lines(
    self,
    sql: str
) -> list[str]:

    raw_lines = sql.splitlines()

    stripped_lines = [
        line.rstrip()
        for line in raw_lines
        if line.strip()
    ]

    if not stripped_lines:
        return []

    min_indent = min(
        len(line) - len(line.lstrip(" "))
        for line in stripped_lines
    )

    normalized: list[str] = []

    for line in raw_lines:
        if line.strip():
            normalized.append(
                line[min_indent:].rstrip()
            )
        else:
            normalized.append("")

    return normalized

```

```

@dataclass
class IfBlock:

    condition: str
    then_body: list[CobolNode] = field(
        default_factory=list
    )
    else_body: list[CobolNode] = field(
        default_factory=list
    )

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        result: list[str] = [
            f"{pad(indent)}IF {self.condition}"
        ]

        result.extend(
            render_nodes(
                self.then_body,
                indent + 3
            )
        )

        if self.else_body:
            result.append(
                f"{pad(indent)}ELSE"
            )

```

```

        result.extend(
            render_nodes(
                self.else_body,
                indent + 3
            )
        )

    terminator = "END-IF."

    if not final:
        terminator = "END-IF"

    result.append(
        f"{pad(indent)}{terminator}"
    )

    return result

def with_prefix(
    self,
    prefix: CobolNode
) -> PrefixedBlock:

    return PrefixedBlock(
        prefix=prefix,
        block=self
    )

```

```

@dataclass
class PrefixedBlock:

    prefix: CobolNode
    block: CobolNode

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        result: list[str] = []

        result.extend(
            self.prefix.render(
                indent=indent
            )
        )

        result.append("")

        result.extend(
            self.block.render(
                indent=indent,
                final=final
            )
        )

        return result

```

```

@dataclass
class PerformUntil:

    condition: str
    body: list[CobolNode] = field(
        default_factory=list
    )

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        result: list[str] = [

```

```

        f"{pad(indent)}PERFORM UNTIL {self.condition}"
    ]

    result.extend(
        render_nodes(
            self.body,
            indent + 3
        )
    )

    terminator = "END-PERFORM."

    if not final:
        terminator = "END-PERFORM"

    result.append(
        f"{pad(indent)}{terminator}"
    )

    return result

```

```

@dataclass
class Paragraph:

    name: str
    body: list[CobolNode] = field(
        default_factory=list
    )

    def render(
        self,
        indent: int = 7,
        final: bool = False
    ) -> list[str]:

        result: list[str] = [
            f"{self.name}."
        ]

        if self.body:
            result.extend(
                render_nodes(
                    self.body,
                    indent,
                    terminate_last=True
                )
            )

        return result

```

```

def render_nodes(
    nodes: list[CobolNode],
    indent: int = 7,
    terminate_last: bool = False
) -> list[str]:

    result: list[str] = []

    for index, node in enumerate(nodes):
        is_last = (
            terminate_last
            and index == len(nodes) - 1
        )

        result.extend(
            node.render(
                indent=indent,
                final=is_last
            )
        )

    return result

```

```
def render_paragraph(
    name: str,
    nodes: list[CobolNode]
) -> str:
```

```
    paragraph = Paragraph(
        name=name,
        body=nodes
    )

    return "\n".join(
        paragraph.render()
    )
```

```
=====
FILE: src\idms_db2_converter\generators\cursors.py
ENCODING: utf-8
=====
```

```
from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.generators.formatting import comma_block
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel
```

```
class CursorGenerator:
    """
```

```
    Generates DECLARE CURSOR statements for IDMS set navigation.
```

```
    Composite-key support:
```

- Uses relationship.parent_keys / relationship.child_fks when available.
 - Falls back to relationship.parent_key / relationship.child_fk.
 - WHERE clause emits all FK-to-parent-host conditions.
- ```
 """
```

```
def generate(
 self,
 schema: SchemaModel,
 used_sets: list[str],
) -> str:
 print("USING CURSOR_GENERATOR VERSION COMPOSITE-KEYS-2026-08-05")
```

```
 blocks: list[str] = []
 seen: set[str] = set()
```

```
 for set_name in used_sets:
 set_name = set_name.upper()
```

```
 if set_name in seen:
 continue
```

```
 seen.add(set_name)
```

```
 relationship = schema.relationships.get(
 set_name,
)
```

```
 if not relationship:
 raise ConversionError(
 f"Set {set_name} missing from schema relationships."
)
```

```
 parent_keys = self.effective_parent_keys(
 relationship=relationship,
)
```

```
 child_fks = self.effective_child_fks(
 relationship=relationship,
)
```

```
 self.validate_key_pairing(
 set_name=set_name,
 parent_keys=parent_keys,
 child_fks=child_fks,
)
```

```

 child_record = schema.records.get(
 self._physical_record_name(
 schema=schema,
 logical_record_name=relationship.child_record,
)
)

 if not child_record:
 raise ConversionError(
 f"Child record {relationship.child_record} missing."
)

 columns = list(child_record.fields.keys())

 where_items = []

 for parent_key, child_fk in zip(parent_keys, child_fks):
 parent_host = Naming.hv(
 relationship.parent_record,
 parent_key,
)

 where_items.append(
 f"{child_fk} = :{parent_host}"
)

 order_by = relationship.order_by or child_fks

 lines = [
 "EXEC SQL",
 f"DECLARE {Naming.cursor(set_name)} CURSOR FOR",
]

 lines.extend(
 comma_block(
 items=columns,
 first_prefix="SELECT ",
 next_prefix=" ",
)
)

 lines.append(
 f"FROM {self._physical_record_name(schema, relationship.child_record)}"
)

 lines.extend(
 self.where_lines(
 where_items=where_items,
)
)

 if order_by:
 lines.append(
 f"ORDER BY {' ', ' '.join(order_by)}"
)

 lines.append(
 "END-EXEC."
)

 blocks.append(
 "\n".join(lines)
)

 return "\n\n".join(blocks)

def effective_parent_keys(
 self,
 relationship,
) -> list[str]:
 if hasattr(relationship, "effective_parent_keys"):
 keys = relationship.effective_parent_keys()
 else:
 keys = list(getattr(relationship, "parent_keys", []) or [])

```

```

 if getattr(relationship, "parent_key", None):
 if relationship.parent_key not in keys:
 keys.append(relationship.parent_key)

 return [
 key
 for key in keys
 if key
]

def effective_child_fks(
 self,
 relationship,
) -> list[str]:
 if hasattr(relationship, "effective_child_fks"):
 keys = relationship.effective_child_fks()
 else:
 keys = list(getattr(relationship, "child_fks", []) or [])

 if getattr(relationship, "child_fk", None):
 if relationship.child_fk not in keys:
 keys.append(relationship.child_fk)

 return [
 key
 for key in keys
 if key
]

def validate_key_pairing(
 self,
 set_name: str,
 parent_keys: list[str],
 child_fks: list[str],
) -> None:
 if not parent_keys:
 raise ConversionError(
 f"Set {set_name} missing parent_key or parent_keys."
)

 if not child_fks:
 raise ConversionError(
 f"Set {set_name} missing child_fk or child_fks."
)

 if len(parent_keys) != len(child_fks):
 raise ConversionError(
 f"Set {set_name} has mismatched composite keys: "
 f"{len(parent_keys)} parent key(s), {len(child_fks)} child FK(s)."
)

def where_lines(
 self,
 where_items: list[str],
) -> list[str]:
 lines = []

 for index, item in enumerate(where_items):
 if index == 0:
 lines.append(
 f"WHERE {item}"
)
 else:
 lines.append(
 f" AND {item}"
)

 return lines

def _physical_record_name(
 self,
 schema: SchemaModel,
 logical_record_name: str,
) -> str:
 logical_record_name = logical_record_name.upper()

```

```

mapped = schema.record_table_map.get(
 logical_record_name,
)

if mapped and mapped in schema.records:
 return mapped

normalized = logical_record_name.replace(
 "-",
 "_",
)

mapped = schema.record_table_map.get(
 normalized,
)

if mapped and mapped in schema.records:
 return mapped

if logical_record_name in schema.records:
 return logical_record_name

if normalized in schema.records:
 return normalized

return logical_record_name

```

```

=====
FILE: src\idms_db2_converter\generators\fetch_paragraphs.py
ENCODING: utf-8
=====

```

```

from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel

```

```

class FetchParagraphGenerator:
 """
 Generates complete FETCH paragraphs for cursor-based IDMS set walking.

 Guarantees:
 - FETCH statement has END-EXEC.
 - SQLCODE handling is emitted after FETCH.
 - INTO list matches every selected column in the cursor.
 """

 def __init__(
 self,
 schema: SchemaModel
):
 self.schema = schema

 def paragraph_name(
 self,
 set_name: str
) -> str:
 return f"FETCH-{set_name}".upper()

 def generate(
 self,
 used_sets: list[str]
) -> str:
 print(
 "USING FETCH_PARAGRAPH_GENERATOR VERSION COMPLETE-SQLCODE-2026-07-01"
)

 paragraphs: list[str] = []
 seen: set[str] = set()

 for set_name in used_sets:
 set_name = set_name.upper()

 if set_name in seen:

```



```

 continue

 seen.add(set_name)

 if set_name not in self.schema.relationships:
 continue

 paragraph = self._generate_one(
 set_name
)

 if paragraph.strip():
 paragraphs.append(
 paragraph
)

 return "\n\n".join(
 paragraphs
)

def _generate_one(
 self,
 set_name: str
) -> str:

 relationship = self.schema.relationships[
 set_name
]

 child_record_name = self._physical_record_name(
 relationship.child_record
)

 child_record = self.schema.records.get(
 child_record_name
)

 if child_record is None:
 return ""

 into_items: list[str] = []

 for column_name, column in child_record.fields.items():
 host = (
 f":{Naming.hv(relationship.child_record, column_name)}"
)

 if column.nullable:
 host += (
 f":{Naming.ni(relationship.child_record, column_name)}"
)

 into_items.append(
 host
)

 lines: list[str] = [
 f"{self.paragraph_name(set_name)}.",
 " EXEC SQL",
 f" FETCH {Naming.cursor(set_name)}"
]

 for index, item in enumerate(into_items):
 suffix = "," if index < len(into_items) - 1 else ""

 if index == 0:
 lines.append(
 f" INTO {item}{suffix}"
)
 else:
 lines.append(
 f" {item}{suffix}"
)

 lines.extend(
 [

```

```

 " END-EXEC.",
 "",
 " IF SQLCODE NOT = 0 AND SQLCODE NOT = 100",
 " PERFORM SQL-ERROR",
 " END-IF."
]
)

return "\n".join(
 lines
)

```

```

def _physical_record_name(
 self,
 logical_record_name: str
) -> str:

 logical_record_name = logical_record_name.upper()

 mapped = self.schema.record_table_map.get(
 logical_record_name
)

 if mapped and mapped in self.schema.records:
 return mapped

 normalized = logical_record_name.replace(
 "-",
 " "
)

 mapped = self.schema.record_table_map.get(
 normalized
)

 if mapped and mapped in self.schema.records:
 return mapped

 if logical_record_name in self.schema.records:
 return logical_record_name

 if normalized in self.schema.records:
 return normalized

 return logical_record_name

```

```

=====
FILE: src\idms_db2_converter\generators\formatting.py
ENCODING: utf-8
=====

```

```

def comma_block(
 items: list[str],
 first_prefix: str = "",
 next_prefix: str = "",
) -> list[str]:
 """
 Format a list of SQL items as comma-separated lines.

```

```

 Example:
 comma_block(
 items=["EMP_ID", "EMP_NAME", "DEPT_ID"],
 first_prefix="SELECT ",
 next_prefix=" "
)

```

```

 Returns:
 [
 "SELECT EMP_ID,",
 " EMP_NAME,",
 " DEPT_ID"
]
 """

```

```

 if not items:

```

```

 return []

 lines: list[str] = []

 for index, item in enumerate(items):
 prefix = first_prefix if index == 0 else next_prefix

 suffix = "," if index < len(items) - 1 else ""

 lines.append(
 f"{prefix}{item}{suffix}"
)

 return lines

```

```

=====
FILE: src\idms_db2_converter\generators\host_variables.py
ENCODING: utf-8
=====

```

```
import re
```

```

from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel

```

```
class HostVariableGenerator:
```

```
 """
```

```
 Generates COBOL host variables for DB2 embedded SQL.
```

```
 Supports:
```

- CHAR(n)
- VARCHAR(n)
- DECIMAL(p)
- DECIMAL(p,s)
- DATE
- TIME
- TIMESTAMP

```
 Schema Listing numeric rule:
```

- Numeric fields come through as DECIMAL.
  - SMALLINT, INTEGER, BIGINT are normalized defensively to DECIMAL precision.
- ```
    """
```

```
    def generate(
```

```
        self,
        schema: SchemaModel,
        used_records: list[str],

```

```
) -> str:
```

```

        lines: list[str] = [
            "",
            "      EXEC SQL",
            "      INCLUDE SQLCA",
            "      END-EXEC.",
            "      EXEC SQL",
            "      BEGIN DECLARE SECTION",
            "      END-EXEC.",
            "",
        ]

```

```
        null_indicators: list[str] = []
```

```
        emitted_null_indicators: set[str] = set()
```

```

        physical_records = self.resolve_used_records(
            schema=schema,
            used_records=used_records,
        )

```

```

        for logical_record_name, physical_record_name in physical_records:
            if physical_record_name not in schema.records:
                continue

```

```
            record = schema.records[physical_record_name]
```

```
            logical_group_name = Naming.normalize(
```

```

        logical_record_name,
    )

    lines.append(
        f"      01 HV-{logical_group_name}."
    )

    for column_name, column in record.fields.items():
        host_variable = Naming.hv(
            logical_record_name,
            column_name,
        )

        datatype, length, scale = self.normalize_db2_datatype(
            datatype=getattr(column, "datatype", None),
            length=getattr(column, "length", None),
            scale=getattr(column, "scale", None),
        )

        lines.extend(
            self.column_to_cobol(
                hv=host_variable,
                datatype=datatype,
                length=length,
                scale=scale,
            )
        )

        if getattr(column, "nullable", True):
            null_indicator = Naming.ni(
                logical_record_name,
                column_name,
            )

            if null_indicator not in emitted_null_indicators:
                emitted_null_indicators.add(
                    null_indicator,
                )
                null_indicators.append(
                    f"      01 {null_indicator:<45} PIC S9(4) COMP."
                )

        lines.append("")

    if null_indicators:
        lines.extend(
            null_indicators,
        )

    lines.extend(
        [
            "      EXEC SQL",
            "      END DECLARE SECTION",
            "      END-EXEC.",
        ]
    )

    return "\n".join(lines)

def resolve_used_records(
    self,
    schema: SchemaModel,
    used_records: list[str],
) -> list[tuple[str, str]]:
    result: list[tuple[str, str]] = []
    seen: set[tuple[str, str]] = set()

    for logical_record_name in used_records:
        if not logical_record_name:
            continue

        physical_record_name = self.physical_record_name(
            schema=schema,
            logical_record_name=logical_record_name,
        )

```

```

        key = (
            logical_record_name.upper(),
            physical_record_name.upper(),
        )

        if key in seen:
            continue

        seen.add(
            key,
        )

        result.append(
            (
                logical_record_name.upper(),
                physical_record_name,
            )
        )

    return result

def physical_record_name(
    self,
    schema: SchemaModel,
    logical_record_name: str,
) -> str:
    logical_record_name = logical_record_name.upper()

    mapped = schema.record_table_map.get(
        logical_record_name,
    )

    if mapped and mapped in schema.records:
        return mapped

    normalized = logical_record_name.replace(
        "-",
        "_",
    )

    mapped = schema.record_table_map.get(
        normalized,
    )

    if mapped and mapped in schema.records:
        return mapped

    if logical_record_name in schema.records:
        return logical_record_name

    if normalized in schema.records:
        return normalized

    hyphen_name = logical_record_name.replace(
        "-",
        "_",
    )

    if hyphen_name in schema.records:
        return hyphen_name

    return logical_record_name

def normalize_db2_datatype(
    self,
    datatype: str | None,
    length: int | None,
    scale: int | None,
) -> tuple[str, int | None, int | None]:
    value = str(datatype or "").strip().upper()

    if not value:
        return "CHAR", length, scale

    decimal_match = re.match(
        r"^(DECIMAL|NUMERIC|DEC)\s*\$(\d+)\s*(?:,\s*(\d+)\s*)?\$",
    
```

```

        value,
        flags=re.IGNORECASE,
    )

    if decimal_match:
        precision = int(decimal_match.group(2))
        parsed_scale = int(decimal_match.group(3) or 0)

        return "DECIMAL", precision, parsed_scale

    char_match = re.match(
        r"^(CHAR|CHARACTER)\s*\s*(\d+)\s*$",
        value,
        flags=re.IGNORECASE,
    )

    if char_match:
        return "CHAR", int(char_match.group(2)), scale

    varchar_match = re.match(
        r"^(VARCHAR|LONG VARCHAR)\s*\s*(\d+)\s*$",
        value,
        flags=re.IGNORECASE,
    )

    if varchar_match:
        return "VARCHAR", int(varchar_match.group(2)), scale

    if value in {"VARCHAR", "LONG VARCHAR"}:
        return "VARCHAR", self.safe_int(length, 255), scale

    if value in {"CHAR", "CHARACTER"}:
        return "CHAR", self.safe_int(length, 1), scale

    if value in {"DECIMAL", "NUMERIC", "DEC"}:
        return "DECIMAL", self.safe_int(length, 18), self.safe_int(scale, 0)

    if value == "SMALLINT":
        return "DECIMAL", 4, 0

    if value in {"INTEGER", "INT"}:
        return "DECIMAL", 9, 0

    if value == "BIGINT":
        return "DECIMAL", 18, 0

    if value == "DATE":
        return "DATE", None, None

    if value == "TIME":
        return "TIME", None, None

    if value == "TIMESTAMP":
        return "TIMESTAMP", None, None

    if value in {"DOUBLE", "FLOAT", "REAL"}:
        return value, None, None

    return value, length, scale

def column_to_cobol(
    self,
    hv: str,
    datatype: str | None,
    length: int | None,
    scale: int | None,
) -> list[str]:
    datatype = (
        datatype
        or "CHAR"
    ).upper()

    if datatype in {
        "CHAR",
        "CHARACTER",
    }:

```

```

        actual_length = self.safe_int(
            length,
            1,
        )

        return [
            f"          05 {hv:<45} PIC X({actual_length})."
        ]

if datatype in {
    "VARCHAR",
    "LONG VARCHAR",
}:
    actual_length = self.safe_int(
        length,
        255,
    )

    return [
        f"          05 {hv:<45} PIC X({actual_length})."
    ]

if datatype == "DATE":
    return [
        f"          05 {hv:<45} PIC X(10).",
    ]

if datatype == "TIME":
    return [
        f"          05 {hv:<45} PIC X(8).",
    ]

if datatype == "TIMESTAMP":
    return [
        f"          05 {hv:<45} PIC X(26).",
    ]

if datatype in {
    "DECIMAL",
    "NUMERIC",
    "DEC",
}:
    return self.decimal_to_cobol(
        hv=hv,
        precision=length,
        scale=scale,
    )

if datatype == "SMALLINT":
    return self.decimal_to_cobol(
        hv=hv,
        precision=4,
        scale=0,
    )

if datatype in {
    "INTEGER",
    "INT",
}:
    return self.decimal_to_cobol(
        hv=hv,
        precision=9,
        scale=0,
    )

if datatype == "BIGINT":
    return self.decimal_to_cobol(
        hv=hv,
        precision=18,
        scale=0,
    )

if datatype in {
    "DOUBLE",
    "FLOAT",
    "REAL",

```

```

    }:
        return [
            f"          05 {hv:<45} COMP-2."
        ]

    actual_length = self.safe_int(
        length,
        255,
    )

    return [
        f"          05 {hv:<45} PIC X({actual_length})."
    ]

def decimal_to_cobol(
    self,
    hv: str,
    precision: int | None,
    scale: int | None,
) -> list[str]:
    actual_precision = self.safe_int(
        precision,
        18,
    )

    actual_scale = self.safe_int(
        scale,
        0,
    )

    integer_digits = max(
        actual_precision - actual_scale,
        1,
    )

    if actual_scale > 0:
        return [
            f"          05 {hv:<45} PIC S9({integer_digits})V9({actual_scale}) COMP-3."
        ]

    return [
        f"          05 {hv:<45} PIC S9({actual_precision}) COMP-3."
    ]

def safe_int(
    self,
    value,
    default: int,
) -> int:
    try:
        if value is None:
            return default

        text = str(value).strip()

        if not text:
            return default

        return int(text)
    except Exception:
        return default

```

```

=====
FILE: src\idms_db2_converter\generators\naming.py
ENCODING: utf-8
=====

```

```
import re
```

```

class Naming:
    @staticmethod
    def normalize(
        value: str | None,
    ) -> str:

```



```

if not value:
    return ""

normalized = str(value).strip().upper()
normalized = normalized.replace("_", "-")
normalized = normalized.replace(" ", "-")
normalized = re.sub(
    r"^[^A-Z0-9-]",
    "-",
    normalized,
)
normalized = re.sub(
    r"-+",
    "-",
    normalized,
)

return normalized.strip("-")

@staticmethod
def hv(
    record: str,
    field: str,
) -> str:
    record_name = Naming.normalize(
        record,
    )

    field_name = Naming.normalize(
        field,
    )

    return f"HV-{record_name}-{field_name}"

@staticmethod
def ni(
    record: str,
    field: str,
) -> str:
    record_name = Naming.normalize(
        record,
    )

    field_name = Naming.normalize(
        field,
    )

    return f"NI-{record_name}-{field_name}"

@staticmethod
def cursor(
    set_name: str,
) -> str:
    return f"C-{Naming.normalize(set_name)}"

```

```

=====
FILE: src\idms_db2_converter\generators\sql_snippets.py
ENCODING: utf-8
=====

```

```

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.generators.formatting import comma_block
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel

```

```

class SqlSnippets:
    """
    Generates embedded SQL snippets with full SELECT and INTO lists.

    Composite-key support:
    - OBTAIN CALC uses all effective primary keys.
    - FIND FIRST / first child lookup uses all parent_keys and child_fks.
    - OBTAIN OWNER uses all parent_keys and child_fks.
    - Backward compatible with parent_key / child_fk / primary_key.
    """

```

Nullable FK support:

- nullable_fk_indicator_for_set() finds the first nullable child FK and returns its generated null indicator.

"""

```
def __init__(
    self,
    schema: SchemaModel,
):
    self.schema = schema

def select_for_record_by_pk(
    self,
    record_name: str,
) -> str:
    physical_record_name = self.physical_record_name(
        record_name,
    )

    record = self.schema.records.get(
        physical_record_name,
    )

    if not record:
        raise ConversionError(
            f"Record {record_name} missing from schema."
        )

    primary_keys = self.effective_primary_keys(
        record=record,
    )

    if not primary_keys:
        raise ConversionError(
            f"Record {record_name} has no primary key."
        )

    columns = list(record.fields.keys())

    into_items = [
        self.host_with_indicator(
            logical_record_name=record_name,
            column_name=column_name,
            nullable=record.fields[column_name].nullable,
        )
        for column_name in columns
    ]

    where_items = [
        f"{primary_key} = :{Naming.hv(record_name, primary_key)}"
        for primary_key in primary_keys
    ]

    lines = [
        "EXEC SQL",
    ]

    lines.extend(
        comma_block(
            items=columns,
            first_prefix="SELECT ",
            next_prefix="      ",
        )
    )

    lines.extend(
        comma_block(
            items=into_items,
            first_prefix="INTO ",
            next_prefix="      ",
        )
    )

    lines.append(
        f"FROM {physical_record_name}"
    )
```

```

    )

    lines.extend(
        self.where_lines(
            where_items=where_items,
        )
    )

    lines.append(
        "END-EXEC."
    )

    return "\n".join(lines)

def select_first_child_for_set(
    self,
    set_name: str,
) -> str:
    set_name = set_name.upper()

    relationship = self.schema.relationships.get(
        set_name,
    )

    if not relationship:
        raise ConversionError(
            f"Set {set_name} missing from schema relationships."
        )

    parent_keys = self.effective_parent_keys(
        relationship=relationship,
    )

    child_fks = self.effective_child_fks(
        relationship=relationship,
    )

    self.validate_key_pairing(
        set_name=set_name,
        parent_keys=parent_keys,
        child_fks=child_fks,
    )

    child_record_name = self.physical_record_name(
        relationship.child_record,
    )

    child_record = self.schema.records.get(
        child_record_name,
    )

    if not child_record:
        raise ConversionError(
            f"Child record {relationship.child_record} missing from schema."
        )

    columns = list(child_record.fields.keys())

    into_items = [
        self.host_with_indicator(
            logical_record_name=relationship.child_record,
            column_name=column_name,
            nullable=child_record.fields[column_name].nullable,
        )
        for column_name in columns
    ]

    where_items = []

    for parent_key, child_fk in zip(parent_keys, child_fks):
        parent_host = Naming.hv(
            relationship.parent_record,
            parent_key,
        )

        where_items.append(

```

```

        f"{child_fk} = :{parent_host}"
    )

order_by = relationship.order_by or child_fks

lines = [
    "EXEC SQL",
]

lines.extend(
    comma_block(
        items=columns,
        first_prefix="SELECT ",
        next_prefix="      ",
    )
)

lines.extend(
    comma_block(
        items=into_items,
        first_prefix="INTO ",
        next_prefix="      ",
    )
)

lines.append(
    f"FROM {child_record_name}"
)

lines.extend(
    self.where_lines(
        where_items=where_items,
    )
)

if order_by:
    lines.append(
        f"ORDER BY {' ', ' '.join(order_by)}"
    )

lines.append(
    "FETCH FIRST 1 ROW ONLY"
)

lines.append(
    "END-EXEC."
)

return "\n".join(lines)

def select_for_owner(
    self,
    set_name: str,
) -> str:
    set_name = set_name.upper()

    relationship = self.schema.relationships.get(
        set_name,
    )

    if not relationship:
        raise ConversionError(
            f"Set {set_name} missing from schema relationships."
        )

    parent_keys = self.effective_parent_keys(
        relationship=relationship,
    )

    child_fks = self.effective_child_fks(
        relationship=relationship,
    )

    self.validate_key_pairing(
        set_name=set_name,
        parent_keys=parent_keys,

```

```

        child_fks=child_fks,
    )

    parent_record_name = self.physical_record_name(
        relationship.parent_record,
    )

    parent_record = self.schema.records.get(
        parent_record_name,
    )

    if not parent_record:
        raise ConversionError(
            f"Parent record {relationship.parent_record} missing from schema."
        )

    columns = list(parent_record.fields.keys())

    into_items = [
        self.host_with_indicator(
            logical_record_name=relationship.parent_record,
            column_name=column_name,
            nullable=parent_record.fields[column_name].nullable,
        )
        for column_name in columns
    ]

    where_items = []

    for parent_key, child_fk in zip(parent_keys, child_fks):
        child_host = Naming.hv(
            relationship.child_record,
            child_fk,
        )

        where_items.append(
            f"{parent_key} = :{child_host}"
        )

    lines = [
        "EXEC SQL",
    ]

    lines.extend(
        comma_block(
            items=columns,
            first_prefix="SELECT ",
            next_prefix="      ",
        )
    )

    lines.extend(
        comma_block(
            items=into_items,
            first_prefix="INTO ",
            next_prefix="      ",
        )
    )

    lines.append(
        f"FROM {parent_record_name}"
    )

    lines.extend(
        self.where_lines(
            where_items=where_items,
        )
    )

    lines.append(
        "END-EXEC."
    )

    return "\n".join(lines)

def fetch_for_set(

```

```

        self,
        set_name: str,
    ) -> str:
        set_name = set_name.upper()

        relationship = self.schema.relationships.get(
            set_name,
        )

        if not relationship:
            raise ConversionError(
                f"Set {set_name} missing from schema relationships."
            )

        child_record_name = self.physical_record_name(
            relationship.child_record,
        )

        child_record = self.schema.records.get(
            child_record_name,
        )

        if not child_record:
            raise ConversionError(
                f"Child record {relationship.child_record} missing from schema."
            )

        into_items = [
            self.host_with_indicator(
                logical_record_name=relationship.child_record,
                column_name=column_name,
                nullable=column.nullable,
            )
            for column_name, column in child_record.fields.items()
        ]

        lines = [
            "EXEC SQL",
            f"FETCH {Naming.cursor(set_name)}",
        ]

        lines.extend(
            comma_block(
                items=into_items,
                first_prefix="INTO ",
                next_prefix="      ",
            )
        )

        lines.append(
            "END-EXEC."
        )

        return "\n".join(lines)

def nullable_fk_indicator_for_set(
    self,
    set_name: str,
) -> str | None:
    set_name = set_name.upper()

    relationship = self.schema.relationships.get(
        set_name,
    )

    if not relationship:
        return None

    child_record_name = self.physical_record_name(
        relationship.child_record,
    )

    child_record = self.schema.records.get(
        child_record_name,
    )

```

```

if not child_record:
    return None

child_fks = self.effective_child_fks(
    relationship=relationship,
)

for child_fk in child_fks:
    column = child_record.fields.get(
        child_fk,
    )

    if not column:
        continue

    if getattr(column, "nullable", True):
        return Naming.ni(
            relationship.child_record,
            child_fk,
        )

return None

def effective_primary_keys(
    self,
    record,
) -> list[str]:
    if hasattr(record, "effective_primary_keys"):
        keys = record.effective_primary_keys()
    else:
        keys = list(getattr(record, "primary_keys", []) or [])

        if getattr(record, "primary_key", None):
            if record.primary_key not in keys:
                keys.append(record.primary_key)

    return [
        key
        for key in keys
        if key
    ]

def effective_parent_keys(
    self,
    relationship,
) -> list[str]:
    if hasattr(relationship, "effective_parent_keys"):
        keys = relationship.effective_parent_keys()
    else:
        keys = list(getattr(relationship, "parent_keys", []) or [])

        if getattr(relationship, "parent_key", None):
            if relationship.parent_key not in keys:
                keys.append(relationship.parent_key)

    return [
        key
        for key in keys
        if key
    ]

def effective_child_fks(
    self,
    relationship,
) -> list[str]:
    if hasattr(relationship, "effective_child_fks"):
        keys = relationship.effective_child_fks()
    else:
        keys = list(getattr(relationship, "child_fks", []) or [])

        if getattr(relationship, "child_fk", None):
            if relationship.child_fk not in keys:
                keys.append(relationship.child_fk)

    return [
        key
    ]

```

```

        for key in keys
        if key
    ]

def validate_key_pairing(
    self,
    set_name: str,
    parent_keys: list[str],
    child_fks: list[str],
) -> None:
    if not parent_keys:
        raise ConversionError(
            f"Set {set_name} missing parent_key or parent_keys."
        )

    if not child_fks:
        raise ConversionError(
            f"Set {set_name} missing child_fk or child_fks."
        )

    if len(parent_keys) != len(child_fks):
        raise ConversionError(
            f"Set {set_name} has mismatched composite keys: "
            f"{len(parent_keys)} parent key(s), {len(child_fks)} child FK(s)."
        )

def where_lines(
    self,
    where_items: list[str],
) -> list[str]:
    lines = []

    for index, item in enumerate(where_items):
        if index == 0:
            lines.append(
                f"WHERE {item}"
            )
        else:
            lines.append(
                f" AND {item}"
            )

    return lines

def physical_record_name(
    self,
    logical_record_name: str,
) -> str:
    logical_record_name = logical_record_name.upper()

    mapped = self.schema.record_table_map.get(
        logical_record_name,
    )

    if mapped and mapped in self.schema.records:
        return mapped

    normalized = logical_record_name.replace(
        "-",
        "_",
    )

    mapped = self.schema.record_table_map.get(
        normalized,
    )

    if mapped and mapped in self.schema.records:
        return mapped

    if logical_record_name in self.schema.records:
        return logical_record_name

    if normalized in self.schema.records:
        return normalized

    return logical_record_name

```



```

def host_with_indicator(
    self,
    logical_record_name: str,
    column_name: str,
    nullable: bool,
) -> str:
    host = f":{Naming.hv(logical_record_name, column_name)}"

    if nullable:
        host += f" :{Naming.ni(logical_record_name, column_name)}"

    return host

```

```

=====
FILE: src\idms_db2_converter\models.py
ENCODING: utf-8
=====

```

```

from dataclasses import dataclass, field
from typing import Dict, List, Optional

```

```

@dataclass
class Column:
    name: str
    datatype: str
    length: Optional[int] = None
    scale: Optional[int] = None
    nullable: bool = True
    primary_key: bool = False
    generated: bool = False
    source_kind: str = ""

```

```

@dataclass
class Record:
    name: str
    primary_key: Optional[str] = None
    fields: Dict[str, Column] = field(default_factory=dict)

    # New composite PK support.
    # Existing code can continue using primary_key.
    primary_keys: List[str] = field(default_factory=list)

    def effective_primary_keys(self) -> List[str]:
        keys: List[str] = []

        if self.primary_keys:
            keys.extend(self.primary_keys)

        if self.primary_key and self.primary_key not in keys:
            keys.append(self.primary_key)

        return keys

    def set_primary_keys(self, keys: List[str]) -> None:
        cleaned: List[str] = []

        for key in keys or []:
            if not key:
                continue

            normalized = str(key).upper()

            if normalized in cleaned:
                continue

            cleaned.append(normalized)

        self.primary_keys = cleaned
        self.primary_key = cleaned[0] if cleaned else None

        for key in cleaned:
            if key in self.fields:

```

```

        self.fields[key].primary_key = True
        self.fields[key].nullable = False

```

```
@dataclass
```

```

class Relationship:
    set_name: str
    parent_record: str
    child_record: str
    cardinality: Optional[str] = None

    # Backward-compatible single key fields.
    parent_key: Optional[str] = None
    child_fk: Optional[str] = None

    # New composite relationship support.
    parent_keys: List[str] = field(default_factory=list)
    child_fks: List[str] = field(default_factory=list)

    order_by: List[str] = field(default_factory=list)

    def effective_parent_keys(self) -> List[str]:
        keys: List[str] = []

        if self.parent_keys:
            keys.extend(self.parent_keys)

        if self.parent_key and self.parent_key not in keys:
            keys.append(self.parent_key)

        return keys

    def effective_child_fks(self) -> List[str]:
        keys: List[str] = []

        if self.child_fks:
            keys.extend(self.child_fks)

        if self.child_fk and self.child_fk not in keys:
            keys.append(self.child_fk)

        return keys

```

```
@dataclass
```

```

class SchemaModel:
    records: Dict[str, Record] = field(default_factory=dict)
    relationships: Dict[str, Relationship] = field(default_factory=dict)

    field_map: Dict[str, dict] = field(default_factory=dict)
    record_table_map: Dict[str, str] = field(default_factory=dict)
    calc_key_map: Dict[str, dict] = field(default_factory=dict)

    # Existing Phase 2 metadata maps.
    set_ordering_map: Dict[str, dict] = field(default_factory=dict)
    navigation_intent: Dict[str, dict] = field(default_factory=dict)
    nullable_fk_map: Dict[str, dict] = field(default_factory=dict)
    date_part_map: Dict[str, dict] = field(default_factory=dict)
    output_semantics: Dict[str, dict] = field(default_factory=dict)
    paragraph_operation_graph: Dict[str, list] = field(default_factory=dict)
    validation_messages: List[str] = field(default_factory=list)

    # New key metadata from Phase 1.
    table_key_map: Dict[str, dict] = field(default_factory=dict)

    schema_source: Optional[str] = None

    def has_schema_objects(self) -> bool:
        return bool(self.records or self.relationships)

    def is_empty(self) -> bool:
        return not self.has_schema_objects()

    def add_validation_message(self, message: str) -> None:
        if not hasattr(self, "validation_messages"):
            self.validation_messages = []

```

```
self.validation_messages.append(message)
```

```
@dataclass
class CobolAnalysis:
    program_id: Optional[str] = None

    idms_records: List[str] = field(default_factory=list)
    obtain_calc_records: List[str] = field(default_factory=list)
    obtain_next: List[tuple[str, str]] = field(default_factory=list)
    obtain_owner_sets: List[str] = field(default_factory=list)
    find_first_sets: List[str] = field(default_factory=list)

    store_records: List[str] = field(default_factory=list)
    modify_records: List[str] = field(default_factory=list)
    erase_records: List[str] = field(default_factory=list)
    ready_update_areas: List[str] = field(default_factory=list)

    field_references: List[str] = field(default_factory=list)
    idms_fields: List[str] = field(default_factory=list)
```

```
@dataclass
class Paragraph:
    name: str
    text: str
```

```
=====
FILE: src\idms_db2_converter\parsers\__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_converter\parsers\canonical_parser.py
ENCODING: utf-8
=====
```

```
import json

from typing import Any

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import Column, Record, Relationship, SchemaModel
```

```
class CanonicalParser:
    """
    Parses canonical JSON into SchemaModel.

    Fixes:
    - Avoids calling .upper() on None.
    - Handles fields with null datatype.
    - Skips incomplete relationships safely.
    - Supports primary_key and primary_keys.
    - Preserves field_map when present.
    """

    DEFAULT_DATATYPE = "CHAR"

    def parse(
        self,
        text: str,
    ) -> SchemaModel:
        if not text or not text.strip():
            return SchemaModel()

        try:
            payload = json.loads(text)
        except json.JSONDecodeError as exc:
            raise ConversionError(f"Invalid canonical JSON: {exc}") from exc
```

```

if not isinstance(payload, dict):
    raise ConversionError("Invalid canonical JSON: root must be an object.")

schema = SchemaModel()

self._parse_records(
    payload=payload,
    schema=schema,
)

self._parse_relationships(
    payload=payload,
    schema=schema,
)

self._parse_field_map(
    payload=payload,
    schema=schema,
)

return schema

def _parse_records(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    records = payload.get("records") or []

    if isinstance(records, dict):
        records = list(records.values())

    for record_json in records:
        if not isinstance(record_json, dict):
            continue

        record_name = self._upper_or_empty(
            self._first_non_empty(
                record_json,
                [
                    "name",
                    "record",
                    "record_name",
                    "table",
                    "table_name",
                ],
            )
        )

        if not record_name:
            continue

        primary_keys = self._extract_primary_keys(record_json)
        primary_key = primary_keys[0] if primary_keys else None

        fields: dict[str, Column] = {}

        field_items = (
            record_json.get("fields")
            or record_json.get("columns")
            or record_json.get("db2_columns")
            or []
        )

        if isinstance(field_items, dict):
            field_items = list(field_items.values())

        for field_json in field_items:
            if not isinstance(field_json, dict):
                continue

            field_name = self._upper_or_empty(
                self._first_non_empty(
                    field_json,
                    [
                        "name",

```

```

        "field",
        "column",
        "column_name",
    ],
)
)

if not field_name:
    continue

datatype = self._upper_or_empty(
    self._first_non_empty(
        field_json,
        [
            "datatype",
            "data_type",
            "type",
            "db2_datatype",
        ],
    )
)

if not datatype:
    datatype = self.DEFAULT_DATATYPE

nullable = field_json.get("nullable")

if nullable is None:
    nullable = field_name != primary_key

primary_key_flag = bool(field_json.get("primary_key", False))

if field_name in primary_keys:
    primary_key_flag = True
    nullable = False

fields[field_name] = Column(
    name=field_name,
    datatype=datatype,
    length=field_json.get("length"),
    scale=field_json.get("scale"),
    nullable=bool(nullable),
    primary_key=primary_key_flag,
    generated=bool(field_json.get("generated", False)),
    source_kind=str(field_json.get("source_kind", "") or ""),
)

record = Record(
    name=record_name,
    primary_key=primary_key,
    fields=fields,
    primary_keys=primary_keys,
)

if hasattr(record, "set_primary_keys"):
    record.set_primary_keys(primary_keys)

schema.records[record_name] = record

def _parse_relationships(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    relationships = (
        payload.get("relationships")
        or payload.get("sets")
        or payload.get("db2_relationships")
        or []
    )

    if isinstance(relationships, dict):
        relationships = list(relationships.values())

    for rel_json in relationships:
        if not isinstance(rel_json, dict):

```

```

        continue

    set_name = self._upper_or_empty(
        self._first_non_empty(
            rel_json,
            [
                "set_name",
                "name",
                "set",
                "relationship",
            ],
        )
    )

    parent_record = self._upper_or_empty(
        self._first_non_empty(
            rel_json,
            [
                "parent_record",
                "owner_record",
                "parent",
                "owner",
                "parent_table",
            ],
        )
    )

    child_record = self._upper_or_empty(
        self._first_non_empty(
            rel_json,
            [
                "child_record",
                "member_record",
                "child",
                "member",
                "child_table",
            ],
        )
    )

    if not set_name or not parent_record or not child_record:
        continue

    parent_keys = self._extract_key_list(
        source=rel_json,
        plural_keys=[
            "parent_keys",
            "owner_keys",
        ],
        single_keys=[
            "parent_key",
            "owner_key",
            "parent_pk",
        ],
    )

    child_fks = self._extract_key_list(
        source=rel_json,
        plural_keys=[
            "child_fks",
            "member_fks",
            "foreign_keys",
        ],
        single_keys=[
            "child_fk",
            "member_fk",
            "foreign_key",
            "child_pk",
        ],
    )

    parent_key = parent_keys[0] if parent_keys else None
    child_fk = child_fks[0] if child_fks else None

    if not parent_key:
        parent_key = self._resolve_parent_key(

```

```

        schema=schema,
        parent_record=parent_record,
    )

    if parent_key:
        parent_keys = [parent_key]

    if not child_fk:
        child_fk = self._resolve_child_fk(
            schema=schema,
            parent_record=parent_record,
            child_record=child_record,
            parent_keys=parent_keys,
        )

    if child_fk:
        child_fks = [child_fk]

    order_by = [
        self._upper_or_empty(value)
        for value in rel_json.get("order_by", []) or []
        if self._upper_or_empty(value)
    ]

    if not order_by and child_fks:
        order_by = child_fks.copy()

    schema.relationships[set_name] = Relationship(
        set_name=set_name,
        parent_record=parent_record,
        child_record=child_record,
        cardinality=rel_json.get("cardinality") or "1:N",
        parent_key=parent_key,
        child_fk=child_fk,
        parent_keys=parent_keys,
        child_fks=child_fks,
        order_by=order_by,
    )

def _parse_field_map(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    field_map = payload.get("field_map") or {}

    if not isinstance(field_map, dict):
        return

    parsed: dict[str, dict] = {}

    for key, value in field_map.items():
        normalized_key = self._upper_or_empty(key)

        if not normalized_key:
            continue

        if isinstance(value, dict):
            parsed[normalized_key] = value
        else:
            parsed[normalized_key] = {
                "host": str(value),
            }

    schema.field_map = parsed

def _extract_primary_keys(
    self,
    source: dict,
) -> list[str]:
    keys: list[str] = []

    raw_primary_keys = source.get("primary_keys")

    if isinstance(raw_primary_keys, list):
        for key in raw_primary_keys:

```

```

        normalized = self._upper_or_empty(key)

        if normalized and normalized not in keys:
            keys.append(normalized)

    elif raw_primary_keys:
        normalized = self._upper_or_empty(raw_primary_keys)

        if normalized and normalized not in keys:
            keys.append(normalized)

    primary_key = self._upper_or_empty(
        source.get("primary_key")
    )

    if primary_key and primary_key not in keys:
        keys.append(primary_key)

    return keys

def _extract_key_list(
    self,
    source: dict,
    plural_keys: list[str],
    single_keys: list[str],
) -> list[str]:
    result: list[str] = []

    for key in plural_keys:
        values = source.get(key)

        if not values:
            continue

        if not isinstance(values, list):
            values = [values]

        for value in values:
            normalized = self._upper_or_empty(value)

            if normalized and normalized not in result:
                result.append(normalized)

    for key in single_keys:
        value = source.get(key)
        normalized = self._upper_or_empty(value)

        if normalized and normalized not in result:
            result.append(normalized)

    return result

def _resolve_parent_key(
    self,
    schema: SchemaModel,
    parent_record: str,
) -> str | None:
    record = schema.records.get(parent_record)

    if not record:
        return None

    if hasattr(record, "effective_primary_keys"):
        primary_keys = record.effective_primary_keys()

        if primary_keys:
            return primary_keys[0]

    return record.primary_key

def _resolve_child_fk(
    self,
    schema: SchemaModel,
    parent_record: str,
    child_record: str,
    parent_keys: list[str] | None = None,

```



```

) -> str | None:
    parent = schema.records.get(parent_record)
    child = schema.records.get(child_record)

    if not parent or not child:
        return None

    keys = parent_keys or []

    if not keys:
        if hasattr(parent, "effective_primary_keys"):
            keys = parent.effective_primary_keys()

        if not keys and parent.primary_key:
            keys = [parent.primary_key]

    for key in keys:
        if key in child.fields:
            return key

    return None

def _first_non_empty(
    self,
    source: dict,
    keys: list[str],
) -> Any:
    for key in keys:
        value = source.get(key)

        if value is None:
            continue

        if isinstance(value, str):
            value = value.strip()

        if not value:
            continue

    return value

    return None

def _upper_or_empty(
    self,
    value: Any,
) -> str:
    if value is None:
        return ""

    text = str(value).strip()

    if not text:
        return ""

    return text.upper()

def _upper_or_none(
    self,
    value: str | None,
) -> str | None:
    normalized = self._upper_or_empty(value)

    return normalized or None

```

```

=====
FILE: src\idms_db2_converter\parsers\cobol_parser.py
ENCODING: utf-8
=====

```

```

import re

from idms_db2_converter.models import CobolAnalysis, Paragraph

```

```

class CobolParser:
    """
    Parses IDMS COBOL.

    Captures:
    - PROGRAM-ID
    - COPY IDMS RECORD usage
    - OBTAIN CALC record usage
    - OBTAIN NEXT record WITHIN set
    - OBTAIN OWNER WITHIN set
    - FIND FIRST WITHIN set
    - STORE record usage
    - MODIFY record usage
    - ERASE record usage
    - READY AREA ... USAGE-MODE IS UPDATE
    - Paragraphs
    - IDMS schema-suffixed field references in PROCEDURE DIVISION

    Important:
    - This parser does not hard-code record suffix to record-name mappings.
    - Record/field inference should come from schema metadata, not from demo data.
    """

    PROGRAM_ID = re.compile(
        r"^\s*PROGRAM-ID\s*([A-Z0-9-]+\s*\.\s*)?",
        re.IGNORECASE | re.MULTILINE,
    )

    COPY_RECORD = re.compile(
        r"^\s*(?:01\s+)?COPY\s+IDMS\s+RECORD\s+([A-Z0-9-]+\s*\.\s*)?",
        re.IGNORECASE | re.MULTILINE,
    )

    OBTAIN_CALC = re.compile(
        r"^\s*OBTAIN\s+CALC\s+([A-Z0-9-]+\s*\.\s*)?",
        re.IGNORECASE | re.MULTILINE,
    )

    OBTAIN_NEXT = re.compile(
        r"^\s*OBTAIN\s+NEXT\s+([A-Z0-9-]+\s*\.\s*)?\s+WITHIN\s+([A-Z0-9-]+\s*\.\s*)?",
        re.IGNORECASE | re.MULTILINE,
    )

    OBTAIN_OWNER = re.compile(
        r"^\s*OBTAIN\s+OWNER\s+WITHIN\s+([A-Z0-9-]+\s*\.\s*)?",
        re.IGNORECASE | re.MULTILINE,
    )

    FIND_FIRST = re.compile(
        r"^\s*FIND\s+FIRST\s+WITHIN\s+([A-Z0-9-]+\s*\.\s*)?",
        re.IGNORECASE | re.MULTILINE,
    )

    STORE_RECORD = re.compile(
        r"^\s*STORE\s+([A-Z0-9-]+\s*\.\s*)?",
        re.IGNORECASE | re.MULTILINE,
    )

    MODIFY_RECORD = re.compile(
        r"^\s*MODIFY\s+([A-Z0-9-]+\s*\.\s*)?",
        re.IGNORECASE | re.MULTILINE,
    )

    ERASE_RECORD = re.compile(
        r"^\s*ERASE\s+([A-Z0-9-]+\s*\.\s*)?",
        re.IGNORECASE | re.MULTILINE,
    )

    READY_UPDATE_AREA = re.compile(
        r"""
        ^\s*READY\s+AREA\s+([A-Z0-9-]+\s*\.\s*)
        (?:\n\s*USAGE-MODE\s+IS\s+UPDATE\s*\.\s*)?
        """,
        re.IGNORECASE | re.MULTILINE | re.VERBOSE,
    )

```

```

PARAGRAPH_HEADER = re.compile(
    r"^\s*([A-Z0-9-]+)\.\s*$",
    re.IGNORECASE | re.MULTILINE,
)

IDMS_SCHEMA_FIELD = re.compile(
    r"\b[A-Z][A-Z0-9]*(?:-[A-Z0-9-]+)*-\d{4}\b",
    re.IGNORECASE,
)

PROCEDURE_DIVISION = re.compile(
    r"^\s*PROCEDURE\s+DIVISION\.\s*$",
    re.IGNORECASE | re.MULTILINE,
)

def parse(
    self,
    cobol: str,
) -> CobolAnalysis:
    source = self.remove_comment_lines(
        cobol,
    )

    program_match = self.PROGRAM_ID.search(
        source,
    )

    analysis = CobolAnalysis(
        program_id=self._upper_or_none(
            program_match.group(1)
            if program_match
            else None
        ),
        idms_records=self._unique(
            self.COPY_RECORD.findall(source),
        ),
        obtain_calc_records=self._unique(
            self.OBTAIN_CALC.findall(source),
        ),
        obtain_next=[
            (
                record.upper(),
                set_name.upper(),
            )
            for record, set_name in self.OBTAIN_NEXT.findall(source)
        ],
        obtain_owner_sets=self._unique(
            self.OBTAIN_OWNER.findall(source),
        ),
        find_first_sets=self._unique(
            self.FIND_FIRST.findall(source),
        ),
        store_records=self._unique(
            self.STORE_RECORD.findall(source),
        ),
        modify_records=self._unique(
            self.MODIFY_RECORD.findall(source),
        ),
        erase_records=self._unique(
            self.ERASE_RECORD.findall(source),
        ),
        ready_update_areas=self._unique(
            self.READY_UPDATE_AREA.findall(source),
        ),
    )

    self._attach_field_references(
        analysis=analysis,
        cobol=source,
    )

    return analysis

def paragraphs(
    self,
    cobol: str,

```

```

) -> list[Paragraph]:
    source = self.remove_comment_lines(
        cobol,
    )

    matches = list(
        self.PARAGRAPH_HEADER.finditer(source),
    )

    paragraphs: list[Paragraph] = []

    for index, match in enumerate(matches):
        name = match.group(1).upper()
        start = match.start()

        if index + 1 < len(matches):
            end = matches[index + 1].start()
        else:
            end = len(source)

        paragraphs.append(
            Paragraph(
                name=name,
                text=source[start:end],
            )
        )

    return paragraphs

def remove_comment_lines(
    self,
    cobol: str,
) -> str:
    lines: list[str] = []

    for line in cobol.splitlines():
        stripped = line.strip()

        if not stripped:
            lines.append(line)
            continue

        if stripped.startswith("*"):
            continue

        lines.append(line)

    return "\n".join(lines)

def _attach_field_references(
    self,
    analysis: CobolAnalysis,
    cobol: str,
) -> None:
    procedure_text = self._procedure_division_text(
        cobol,
    )

    field_references = self._unique(
        self.IDMS_SCHEMA_FIELD.findall(
            procedure_text,
        )
    )

    try:
        analysis.field_references = field_references
    except Exception:
        pass

    try:
        analysis.idms_fields = field_references
    except Exception:
        pass

    record_names = set(
        analysis.idms_records,

```

```

)

for record in analysis.obtain_calc_records:
    record_names.add(
        record,
    )

for record, _ in analysis.obtain_next:
    record_names.add(
        record,
    )

for record in getattr(
    analysis,
    "store_records",
    [],
):
    record_names.add(
        record,
    )

for record in getattr(
    analysis,
    "modify_records",
    [],
):
    record_names.add(
        record,
    )

for record in getattr(
    analysis,
    "erase_records",
    [],
):
    record_names.add(
        record,
    )

try:
    analysis.idms_records = sorted(
        record_names,
    )
except Exception:
    pass

def _procedure_division_text(
    self,
    cobol: str,
) -> str:
    match = self.PROCEDURE_DIVISION.search(
        cobol,
    )

    if not match:
        return cobol

    return cobol[match.start():]

def _unique(
    self,
    values,
) -> list[str]:
    seen = set()
    result: list[str] = []

    for value in values:
        upper_value = value.upper()

        if upper_value not in seen:
            seen.add(
                upper_value,
            )

            result.append(
                upper_value,
            )

```

```

        )

        return result

def _upper_or_none(
    self,
    value: str | None,
) -> str | None:
    return value.upper() if value else None

=====
FILE: src\idms_db2_converter\parsers\dclgen_parser.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_db2_converter\parsers\ddl_parser.py
ENCODING: utf-8
=====

import re

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import (
    Column,
    Record,
    Relationship,
    SchemaModel,
)

class DDLParser:
    """
    Robust DB2 DDL parser for Phase 2.

    Supports:
    - CREATE TABLE with table name and opening parenthesis on separate lines.
    - CREATE TABLE with blank lines before opening parenthesis.
    - CHAR(n), VARCHAR(n), DECIMAL(p), DECIMAL(p,s).
    - DATE, TIME, TIMESTAMP.
    - Composite primary keys.
    - Composite foreign keys.
    - ALTER TABLE ADD FOREIGN KEY.
    - Schema-qualified table names.
    - Quoted identifiers.

    Schema Listing datatype rule:
    - SMALLINT, INTEGER, BIGINT are normalized to DECIMAL equivalents.
    - CHAR(n), VARCHAR(n), DECIMAL(p,s) must preserve length/precision/scale.
    """

    DEBUG = True

    CONSTRAINT_STARTERS = {
        "CONSTRAINT",
        "PRIMARY",
        "FOREIGN",
        "UNIQUE",
        "CHECK",
    }

    INLINE_PRIMARY_KEY = re.compile(
        r"\bPRIMARY\s+KEY\b",
        re.IGNORECASE,
    )

    INLINE_NOT_NULL = re.compile(
        r"\bNOT\s+NULL\b",
        re.IGNORECASE,
    )

    TABLE_PRIMARY_KEY = re.compile(

```

```

r"\bPRIMARY\s+KEY\s*$(?P<columns>.*?)$",
re.IGNORECASE | re.DOTALL,
)

TABLE_FOREIGN_KEY = re.compile(
r"""
\bFOREIGN\s+KEY\s*
$(?P<child_cols>.*?)$
\s+REFERENCES\s+
(?P<parent_table>(?:"[^"]+"| [A-Z0-9_]+)(?:\.(?:"[^"]+"| [A-Z0-9_]+))?)
\s*
$(?P<parent_cols>.*?)$
""",
re.IGNORECASE | re.DOTALL | re.VERBOSE,
)

ALTER_TABLE_FK = re.compile(
r"""
\bALTER\s+TABLE\s+
(?P<child_table>(?:"[^"]+"| [A-Z0-9_]+)(?:\.(?:"[^"]+"| [A-Z0-9_]+))?)
.*?
\bFOREIGN\s+KEY\s*
$(?P<child_cols>.*?)$
\s+REFERENCES\s+
(?P<parent_table>(?:"[^"]+"| [A-Z0-9_]+)(?:\.(?:"[^"]+"| [A-Z0-9_]+))?)
\s*
$(?P<parent_cols>.*?)$
""",
re.IGNORECASE | re.DOTALL | re.VERBOSE,
)

def parse(
    self,
    ddl: str,
) -> SchemaModel:
    self._debug(
        "USING DDL_PARSER VERSION FIXED-DATATYPE-PARENS-COMPOSITE-2026-08-05"
    )

    schema = SchemaModel()
    schema.schema_source = "DDL"

    if ddl is None:
        ddl = ""

    self._debug(
        f"DDL INPUT LENGTH: {len(ddl)}"
    )

    ddl = self.remove_comments(
        ddl=ddl,
    )

    create_statements = self._extract_create_table_statements(
        ddl=ddl,
    )

    self._debug(
        f"DDL PARSER CREATE TABLE COUNT: {len(create_statements)}"
    )

    for statement in create_statements:
        table_name = statement["table_name"]
        body = statement["body"]

        self._debug(
            f"DDL PARSER START TABLE: {table_name}"
        )

        record = Record(
            name=table_name,
            primary_key=None,
            primary_keys=[],
            fields={},
        )

```

```

schema.record_table_map[table_name] = table_name

pending_foreign_keys = []

entries = self.split_entries(
    body=body,
)

self._debug(
    f"DDL PARSER TABLE {table_name} ENTRY COUNT: {len(entries)}"
)

for entry in entries:
    entry = entry.strip()

    if not entry:
        continue

    first_word = self._first_word(
        entry=entry,
    )

    if first_word in self.CONSTRAINT_STARTERS:
        self._parse_constraint_entry(
            record=record,
            entry=entry,
            pending_foreign_keys=pending_foreign_keys,
        )
        continue

    column = self._parse_column_entry(
        entry=entry,
        table_name=table_name,
    )

    if column is None:
        continue

    record.fields[column.name] = column

    if self.INLINE_PRIMARY_KEY.search(entry):
        self._set_primary_keys(
            record=record,
            primary_keys=[column.name],
        )

schema.records[table_name] = record

for foreign_key in pending_foreign_keys:
    self._add_relationship(
        schema=schema,
        child_table=table_name,
        child_fks=foreign_key["child_fks"],
        parent_table=foreign_key["parent_table"],
        parent_keys=foreign_key["parent_keys"],
    )

self._debug_record(
    record=record,
)

self._parse_alter_table_foreign_keys(
    ddl=ddl,
    schema=schema,
)

self._validate(
    schema=schema,
)

if "CREATE TABLE" in ddl.upper() and not schema.records:
    raise ConversionError(
        "DDL text was provided and contains CREATE TABLE, but no tables were parsed."
    )

return schema

```



```

def remove_comments(
    self,
    ddl: str,
) -> str:
    ddl = re.sub(
        r"--.*?$",
        "",
        ddl,
        flags=re.MULTILINE,
    )

    ddl = re.sub(
        r"/\*.*?\*/",
        "",
        ddl,
        flags=re.DOTALL,
    )

    return ddl

def _extract_create_table_statements(
    self,
    ddl: str,
) -> list[dict[str, str]]:
    statements = []
    upper = ddl.upper()
    marker = "CREATE TABLE"
    position = 0

    while True:
        start = upper.find(
            marker,
            position,
        )

        if start < 0:
            break

        name_start = start + len(marker)

        while name_start < len(ddl) and ddl[name_start].isspace():
            name_start += 1

        if name_start >= len(ddl):
            break

        table_name, name_end = self._read_identifier(
            text=ddl,
            start=name_start,
        )

        table_name = self._normalize_name(
            table_name,
        )

        if not table_name:
            position = name_start + 1
            continue

        open_index = ddl.find(
            "(",
            name_end,
        )

        if open_index < 0:
            position = name_end
            continue

        close_index = self._find_matching_parenthesis(
            text=ddl,
            open_index=open_index,
        )

        if close_index is None:
            raise ConversionError(

```

```

        f"CREATE TABLE {table_name} has no matching closing parenthesis."
    )

    body = ddl[open_index + 1 : close_index]

    statements.append(
        {
            "table_name": table_name,
            "body": body,
        }
    )

    position = close_index + 1

    return statements

def _read_identififier(
    self,
    text: str,
    start: int,
) -> tuple[str, int]:
    if start >= len(text):
        return "", start

    if text[start] == "'":
        end = start + 1

        while end < len(text):
            if text[end] == "'":
                return text[start : end + 1], end + 1

            end += 1

        return text[start:], len(text)

    end = start

    while end < len(text):
        char = text[end]

        if char.isspace() or char == "(":
            break

        end += 1

    return text[start:end], end

def _find_matching_parenthesis(
    self,
    text: str,
    open_index: int,
) -> int | None:
    depth = 0
    in_single_quote = False
    in_double_quote = False

    for index in range(open_index, len(text)):
        char = text[index]

        if char == '"' and not in_double_quote:
            in_single_quote = not in_single_quote
            continue

        if char == "'" and not in_single_quote:
            in_double_quote = not in_double_quote
            continue

        if in_single_quote or in_double_quote:
            continue

        if char == "(":
            depth += 1
            continue

        if char == ")":
            depth -= 1

```

```

        if depth == 0:
            return index

    return None

def split_entries(
    self,
    body: str,
) -> list[str]:
    entries = []
    current = []
    depth = 0
    in_single_quote = False
    in_double_quote = False

    for char in body:
        if char == '"' and not in_double_quote:
            in_single_quote = not in_single_quote
            current.append(char)
            continue

        if char == "'" and not in_single_quote:
            in_double_quote = not in_double_quote
            current.append(char)
            continue

        if not in_single_quote and not in_double_quote:
            if char == "(":
                depth += 1
                current.append(char)
                continue

            if char == ")":
                depth -= 1
                current.append(char)
                continue

            if char == "," and depth == 0:
                entry = "".join(current).strip()

                if entry:
                    entries.append(entry)

                current = []
                continue

            current.append(char)

    entry = "".join(current).strip()

    if entry:
        entries.append(entry)

    return entries

def _parse_constraint_entry(
    self,
    record: Record,
    entry: str,
    pending_foreign_keys: list[dict],
) -> None:
    normalized_entry = self._remove_constraint_name(
        entry=entry,
    )

    primary_key_match = self.TABLE_PRIMARY_KEY.search(
        normalized_entry,
    )

    if primary_key_match:
        primary_keys = self._column_list(
            primary_key_match.group("columns"),
        )

        if primary_keys:

```

```

        self._set_primary_keys(
            record=record,
            primary_keys=primary_keys,
        )

    return

foreign_key_match = self.TABLE_FOREIGN_KEY.search(
    normalized_entry,
)

if foreign_key_match:
    child_fks = self._column_list(
        foreign_key_match.group("child_cols"),
    )

    parent_table = self._normalize_name(
        foreign_key_match.group("parent_table"),
    )

    parent_keys = self._column_list(
        foreign_key_match.group("parent_cols"),
    )

    if child_fks and parent_table and parent_keys:
        self._validate_key_pairing(
            context=f"FK in table {record.name}",
            child_fks=child_fks,
            parent_keys=parent_keys,
        )

        pending_foreign_keys.append(
            {
                "child_fks": child_fks,
                "parent_table": parent_table,
                "parent_keys": parent_keys,
            }
        )

    return

def _remove_constraint_name(
    self,
    entry: str,
) -> str:
    tokens = entry.strip().split()

    if len(tokens) >= 3 and tokens[0].upper() == "CONSTRAINT":
        return " ".join(tokens[2:])

    return entry

def _parse_column_entry(
    self,
    entry: str,
    table_name: str,
) -> Column | None:
    entry = entry.strip()

    if not entry:
        return None

    match = re.match(
        r'^(?P<name>"[^"]+"|[A-Z0-9_]+\s+(?P<datatype>.+))$',
        entry,
        flags=re.IGNORECASE | re.DOTALL,
    )

    if not match:
        self._debug(
            f"DDL PARSER SKIPPED ENTRY IN {table_name}: no column match. ENTRY={repr(entry)}"
        )
        return None

    column_name = self._normalize_name(
        match.group("name"),
    )

```

```

)

datatype_text = match.group("datatype").strip()

datatype, length, scale = self._parse_datatype(
    datatype_text,
)

if not datatype:
    self._debug_unrecognized_datatype(
        table_name=table_name,
        column_name=column_name,
        datatype_text=datatype_text,
    )
    return None

nullable = not bool(
    self.INLINE_NOT_NULL.search(entry)
    or self.INLINE_PRIMARY_KEY.search(entry)
)

primary_key = bool(
    self.INLINE_PRIMARY_KEY.search(entry)
)

return Column(
    name=column_name,
    datatype=datatype,
    length=length,
    scale=scale,
    nullable=nullable,
    primary_key=primary_key,
)

def _parse_datatype(
    self,
    text: str,
) -> tuple[str | None, int | None, int | None]:
    normalized = self._normalize_datatype_text(
        text,
    )

    if not normalized:
        return None, None, None

    leading = self._leading_word(
        normalized,
    )

    if leading in {"CHAR", "CHARACTER"}:
        length = self._first_parenthesized_int(
            normalized,
        )

        if length is None:
            length = 1

        return "CHAR", length, None

    if leading in {"VARCHAR", "VARCHAR2"}:
        length = self._first_parenthesized_int(
            normalized,
        )

        if length is None:
            length = 255

        return "VARCHAR", length, None

    if normalized.startswith("LONG VARCHAR"):
        length = self._first_parenthesized_int(
            normalized,
        )

        if length is None:
            length = 255

```

```

        return "VARCHAR", length, None

    if leading in {"DECIMAL", "NUMERIC", "DEC"}:
        precision, scale = self._first_parenthesized_pair(
            normalized,
        )

        if precision is None:
            precision = 18

        if scale is None:
            scale = 0

        return "DECIMAL", precision, scale

    if leading in {"INTEGER", "INT"}:
        return "DECIMAL", 9, 0

    if leading == "BIGINT":
        return "DECIMAL", 18, 0

    if leading == "SMALLINT":
        return "DECIMAL", 4, 0

    if leading == "DATE":
        return "DATE", None, None

    if leading == "TIMESTAMP":
        return "TIMESTAMP", None, None

    if leading == "TIME":
        return "TIME", None, None

    if leading in {"DOUBLE", "FLOAT", "REAL"}:
        return leading, None, None

    return None, None, None

def _first_parenthesized_int(
    self,
    text: str,
) -> int | None:
    value = str(text or "")

    open_index = value.find("(")

    if open_index < 0:
        return None

    close_index = value.find(")", open_index + 1)

    if close_index < 0:
        return None

    content = value[open_index + 1 : close_index].strip()

    if "," in content:
        content = content.split(",", 1)[0].strip()

    if not content.isdigit():
        return None

    return int(content)

def _first_parenthesized_pair(
    self,
    text: str,
) -> tuple[int | None, int | None]:
    value = str(text or "")

    open_index = value.find("(")

    if open_index < 0:
        return None, None

```

```

close_index = value.find('"', open_index + 1)

if close_index < 0:
    return None, None

content = value[open_index + 1 : close_index].strip()

if not content:
    return None, None

parts = [
    part.strip()
    for part in content.split(",")
]

if not parts:
    return None, None

if not parts[0].isdigit():
    return None, None

precision = int(parts[0])
scale = None

if len(parts) > 1 and parts[1].isdigit():
    scale = int(parts[1])

return precision, scale

def _normalize_datatype_text(
    self,
    text: str,
) -> str:
    value = str(text or "").strip().upper()

    stop_patterns = [
        r"\bNOT\s+NULL\b",
        r"\bNULL\b",
        r"\bPRIMARY\s+KEY\b",
        r"\bGENERATED\b",
        r"\bDEFAULT\b",
        r"\bCONSTRAINT\b",
        r"\bREFERENCES\b",
        r"\bCHECK\b",
        r"\bUNIQUE\b",
    ]

    stop_positions = []

    for pattern in stop_patterns:
        match = re.search(
            pattern,
            value,
            flags=re.IGNORECASE,
        )

        if match:
            stop_positions.append(
                match.start(),
            )

    if stop_positions:
        value = value[: min(stop_positions)].strip()

    value = re.sub(
        r"\s+",
        " ",
        value,
    )

    return value

def _leading_word(
    self,
    text: str,
) -> str:

```

```

        match = re.match(
            r"([A-Z0-9]+)",
            text.strip().upper(),
        )

        if not match:
            return ""

        return match.group(1)

def _parse_alter_table_foreign_keys(
    self,
    ddl: str,
    schema: SchemaModel,
) -> None:
    for match in self.ALTER_TABLE_FK.finditer(ddl):
        child_table = self._normalize_name(
            match.group("child_table"),
        )

        child_fks = self._column_list(
            match.group("child_cols"),
        )

        parent_table = self._normalize_name(
            match.group("parent_table"),
        )

        parent_keys = self._column_list(
            match.group("parent_cols"),
        )

        if not child_table:
            continue

        if child_table not in schema.records:
            continue

        if not parent_table or parent_table not in schema.records:
            continue

        if not child_fks or not parent_keys:
            continue

        self._validate_key_pairing(
            context=f"ALTER TABLE FK {child_table} -> {parent_table}",
            child_fks=child_fks,
            parent_keys=parent_keys,
        )

        self._add_relationship(
            schema=schema,
            child_table=child_table,
            child_fks=child_fks,
            parent_table=parent_table,
            parent_keys=parent_keys,
        )

def _add_relationship(
    self,
    schema: SchemaModel,
    child_table: str,
    child_fks: list[str],
    parent_table: str,
    parent_keys: list[str],
) -> None:
    if child_table not in schema.records:
        raise ConversionError(
            f"FK child table {child_table} is not present in schema."
        )

    if parent_table not in schema.records:
        raise ConversionError(
            f"FK parent table {parent_table} is not present in schema."
        )

```



```

self._validate_key_pairing(
    context=f"FK {child_table} -> {parent_table}",
    child_fks=child_fks,
    parent_keys=parent_keys,
)

child = schema.records[child_table]
parent = schema.records[parent_table]

for child_fk in child_fks:
    if child_fk not in child.fields:
        raise ConversionError(
            f"FK column {child_table}.{child_fk} is not declared as a column."
        )

for parent_key in parent_keys:
    if parent_key not in parent.fields:
        raise ConversionError(
            f"Referenced column {parent_table}.{parent_key} is not declared as a column."
        )

if parent_keys and not getattr(parent, "primary_keys", []):
    self._set_primary_keys(
        record=parent,
        primary_keys=parent_keys,
    )

set_name = f"{parent_table}_{child_table}"

schema.relationships[set_name] = Relationship(
    set_name=set_name,
    parent_record=parent_table,
    child_record=child_table,
    cardinality="1:N",
    parent_key=parent_keys[0] if parent_keys else None,
    child_fk=child_fks[0] if child_fks else None,
    parent_keys=parent_keys,
    child_fks=child_fks,
    order_by=child_fks.copy(),
)

def _set_primary_keys(
    self,
    record: Record,
    primary_keys: list[str],
) -> None:
    cleaned = []

    for key in primary_keys or []:
        normalized = self._normalize_name(
            key,
        )

        if not normalized:
            continue

        if normalized in cleaned:
            continue

        cleaned.append(
            normalized,
        )

    if hasattr(record, "set_primary_keys"):
        record.set_primary_keys(
            keys=cleaned,
        )
    else:
        record.primary_keys = cleaned
        record.primary_key = cleaned[0] if cleaned else None

    for key in cleaned:
        if key in record.fields:
            record.fields[key].primary_key = True
            record.fields[key].nullable = False

```

```

def _validate_key_pairing(
    self,
    context: str,
    child_fks: list[str],
    parent_keys: list[str],
) -> None:
    if len(child_fks) != len(parent_keys):
        raise ConversionError(
            f"{context} has mismatched composite key column counts: "
            f"{len(child_fks)} child FK column(s), "
            f"{len(parent_keys)} parent key column(s)."
        )

def _column_list(
    self,
    text: str,
) -> list[str]:
    if not text:
        return []

    result = []

    for item in text.split(","):
        normalized = self._normalize_name(
            item,
        )

        if not normalized:
            continue

        if normalized in result:
            continue

        result.append(
            normalized,
        )

    return result

def _validate(
    self,
    schema: SchemaModel,
) -> None:
    errors = []

    for record in schema.records.values():
        primary_keys = []

        if hasattr(record, "effective_primary_keys"):
            primary_keys = record.effective_primary_keys()
        else:
            primary_keys = list(getattr(record, "primary_keys", []) or [])

            if getattr(record, "primary_key", None):
                if record.primary_key not in primary_keys:
                    primary_keys.append(record.primary_key)

        for primary_key in primary_keys:
            if primary_key not in record.fields:
                errors.append(
                    f"{record.name} primary key {primary_key} is not declared as a column."
                )

        for field_name, field in record.fields.items():
            if not field.datatype:
                errors.append(
                    f"{record.name}.{field_name} has no datatype."
                )

    if errors:
        raise ConversionError(
            "\n".join(errors),
        )

def _first_word(
    self,

```

```

        entry: str,
    ) -> str:
        tokens = entry.strip().split()

        if not tokens:
            return ""

        return self._normalize_name(
            tokens[0],
        )

def _normalize_name(
    self,
    value: str | None,
) -> str:
    if not value:
        return ""

    value = str(value).strip()
    value = value.strip('\'')
    value = value.strip('"')
    value = value.strip("`")
    value = value.strip("[")
    value = value.strip("]")

    if "." in value:
        value = value.split(".")[1]

    value = value.upper()
    value = value.replace("-", "_")
    value = re.sub(
        r"^[^A-Z0-9_]",
        "_",
        value,
    )
    value = re.sub(
        r"_+",
        "_",
        value,
    )

    return value.strip("_")

def _debug(
    self,
    message: str,
) -> None:
    if self.DEBUG:
        print(message)

def _debug_record(
    self,
    record: Record,
) -> None:
    if not self.DEBUG:
        return

    primary_keys = list(getattr(record, "primary_keys", []) or [])

    if not primary_keys and getattr(record, "primary_key", None):
        primary_keys = [record.primary_key]

    print(
        f"DDL PARSER TABLE {record.name}: "
        f"{len(record.fields)} columns, PKS={primary_keys}"
    )

    for field_name, field in record.fields.items():
        print(
            f" COLUMN {field_name}: "
            f"{field.datatype} ({field.length}, {field.scale}) "
            f"NULLABLE={field.nullable}"
        )

def _debug_unrecognized_datatype(
    self,

```

```

        table_name: str,
        column_name: str,
        datatype_text: str,
    ) -> None:
        if not self.DEBUG:
            return

        normalized = self._normalize_datatype_text(
            datatype_text,
        )

        print(
            "DDL PARSER SKIPPED COLUMN "
            f"{table_name}.{column_name}: "
            f"raw={repr(datatype_text)} "
            f"normalized={repr(normalized)} "
            f"leading={repr(self._leading_word(normalized))}"
        )

```

```

=====
FILE: src\idms_db2_converter\parsers\idms_schema_parser.py
ENCODING: utf-8
=====

```

```
import re
```

```

from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import Column, Record, Relationship, SchemaModel

```

```

class IdmsSchemaParser:
    RECORD_HEADER = re.compile(
        r"^RECORD NAME\.\s+([A-Z0-9-]+)",
        re.IGNORECASE | re.MULTILINE,
    )

    LOCATION_CALC = re.compile(
        r"LOCATION MODE\.\s+CALC USING\s+([A-Z0-9-]+)",
        re.IGNORECASE,
    )

    DBKEY_SET_ROLE = re.compile(
        r"^\s+([A-Z0-9-]+)\s+(?:INDEX\s+)?(OWNER|MEMBER)\b",
        re.IGNORECASE | re.MULTILINE,
    )

    DATA_LINE_PREFIX = re.compile(
        r"^\s*(0[2-5])\s+([A-Z0-9-]+)\s+(.*)$",
        re.IGNORECASE,
    )

    PICTURE_FIND = re.compile(
        r"S?9$\d+$V9$\d+$|"
        r"S?9$\d+$V9+|"
        r"S?9+V9$\d+$|"
        r"S?9+V9+|"
        r"S?V9$\d+$|"
        r"S?V9+|"
        r"S?9$\d+$|"
        r"S?9+|"
        r"X$\d+$|"
        r"X+",
        re.IGNORECASE,
    )

    DATE_PARTS = {
        "YEAR": {
            "substring_start": 1,
            "substring_length": 4,
        },
        "MONTH": {
            "substring_start": 5,
            "substring_length": 2,
        },
        "DAY": {

```

```

        "substring_start": 7,
        "substring_length": 2,
    },
}

def parse(self, text: str) -> SchemaModel:
    schema = SchemaModel()
    schema.schema_source = "IDMS_SCHEMA"

    blocks = self._record_blocks(text)
    set_roles: dict[str, dict[str, str]] = {}

    for record_name, block in blocks:
        self._parse_record(schema, record_name, block)
        self._collect_set_roles(set_roles, record_name, block)

    self._build_relationships(schema, set_roles)

    if hasattr(schema, "add_validation_message"):
        schema.add_validation_message("Schema built from IDMS schema listing.")

    return schema

def _record_blocks(self, text: str) -> list[tuple[str, str]]:
    matches = list(self.RECORD_HEADER.finditer(text))
    result = []

    for index, match in enumerate(matches):
        record_name = match.group(1).upper()
        start = match.start()
        end = matches[index + 1].start() if index + 1 < len(matches) else len(text)

        result.append((record_name, text[start:end]))

    return result

def _parse_record(
    self,
    schema: SchemaModel,
    record_name: str,
    block: str,
) -> None:
    primary_key = None

    calc_match = self.LOCATION_CALC.search(block)

    if calc_match:
        primary_key = self._normalize_column(calc_match.group(1))

    record = Record(
        name=record_name,
        primary_key=primary_key,
        fields={},
    )

    schema.record_table_map[record_name] = self._normalize_table(record_name)

    active_date_groups: dict[int, dict[str, str]] = {}

    for line in block.splitlines():
        parsed = self._parse_data_item_line(line)

        if not parsed:
            continue

        level = parsed["level"]
        raw_field_name = parsed["field_name"]
        body = parsed["body"]
        storage_length = parsed["length"]

        self._close_inactive_date_groups(active_date_groups, level)

        if raw_field_name == "FILLER":
            continue

        usage = self._extract_usage(body)

```

```

picture = self._extract_picture(body)
column_name = self._normalize_column(raw_field_name)

if self._is_date_group(raw_field_name, picture, storage_length):
    if column_name not in record.fields:
        record.fields[column_name] = Column(
            name=column_name,
            datatype="CHAR",
            length=8,
            scale=None,
            nullable=False,
        )

    host = Naming.hv(record_name, column_name)
    self._add_field_map(schema, raw_field_name, host)

    active_date_groups[level] = {
        "raw_field_name": raw_field_name,
        "column_name": column_name,
        "host": host,
    }

    continue

date_group = self._nearest_date_group(active_date_groups)

if date_group and self._is_date_part(raw_field_name):
    self._add_date_part_map(
        schema=schema,
        idms_field=raw_field_name,
        host=date_group["host"],
    )
    continue

if not picture:
    continue

datatype, column_length, scale = self._map_picture(
    picture=picture,
    usage=usage,
    storage_length=storage_length,
)

if column_name not in record.fields:
    record.fields[column_name] = Column(
        name=column_name,
        datatype=datatype,
        length=column_length,
        scale=scale,
        nullable=False,
    )

self._add_field_map(
    schema=schema,
    idms_field=raw_field_name,
    host=Naming.hv(record_name, column_name),
)

if primary_key and primary_key not in record.fields:
    record.fields[primary_key] = Column(
        name=primary_key,
        datatype="CHAR",
        length=12,
        scale=None,
        nullable=False,
    )

schema.records[record_name] = record

def _parse_data_item_line(self, line: str) -> dict | None:
    match = self.DATA_LINE_PREFIX.match(line)

    if not match:
        return None

    level = int(match.group(1))

```

```

field_name = match.group(2).upper()
remainder = match.group(3).strip()

if field_name == "FILLER":
    return None

parts = remainder.split()

if len(parts) < 2:
    return None

if not parts[-1].isdigit() or not parts[-2].isdigit():
    return None

length = int(parts[-1])
start = int(parts[-2])
body = " ".join(parts[:-2])

return {
    "level": level,
    "field_name": field_name,
    "body": body,
    "start": start,
    "length": length,
}

def _collect_set_roles(
    self,
    set_roles: dict[str, dict[str, str]],
    record_name: str,
    block: str,
) -> None:
    for match in self.DBKEY_SET_ROLE.finditer(block):
        set_name = match.group(1).upper()
        role = match.group(2).upper()

        if set_name == "CALC":
            continue

        set_roles.setdefault(set_name, {})

        if role == "OWNER":
            set_roles[set_name]["owner"] = record_name

        if role == "MEMBER":
            set_roles[set_name]["member"] = record_name

def _build_relationships(
    self,
    schema: SchemaModel,
    set_roles: dict[str, dict[str, str]],
) -> None:
    for set_name, roles in set_roles.items():
        owner = roles.get("owner")
        member = roles.get("member")

        if not owner or not member:
            continue

        if owner not in schema.records or member not in schema.records:
            continue

        owner_record = schema.records[owner]
        member_record = schema.records[member]

        parent_key = owner_record.primary_key

        if not parent_key:
            continue

        child_fk = parent_key

        if child_fk not in member_record.fields:
            parent_column = owner_record.fields.get(parent_key)

            member_record.fields[child_fk] = Column(

```

```

        name=child_fk,
        datatype=parent_column.datatype if parent_column else "CHAR",
        length=parent_column.length if parent_column else 12,
        scale=parent_column.scale if parent_column else None,
        nullable=True,
    )

    schema.relationships[set_name] = Relationship(
        set_name=set_name,
        parent_record=owner,
        child_record=member,
        cardinality="1:N",
        parent_key=parent_key,
        child_fk=child_fk,
        order_by=[child_fk],
    )

def _extract_usage(self, body: str) -> str:
    upper = body.upper()

    if "COMP-3" in upper:
        return "COMP-3"

    if re.search(r"\bCOMP\b", upper):
        return "COMP"

    return "DISPLAY"

def _extract_picture(self, body: str) -> str | None:
    upper = body.upper()
    matches = list(self.PICTURE_FIND.finditer(upper))

    if not matches:
        return None

    valid_matches = []

    for match in matches:
        value = match.group(0).upper()
        start = match.start()
        end = match.end()

        before = upper[start - 1] if start > 0 else " "
        after = upper[end] if end < len(upper) else " "

        if before.isalnum() or before in {"-", "_":
            continue

        if after.isalnum() or after in {"-", "_":
            continue

        valid_matches.append(value)

    if not valid_matches:
        return None

    return valid_matches[-1]

def _map_picture(
    self,
    picture: str,
    usage: str,
    storage_length: int,
) -> tuple[str, int | None, int | None]:
    picture = picture.upper()
    usage = usage.upper()

    if picture.startswith("X"):
        length_match = re.search(r"X$(\d+)$", picture)

        if length_match:
            return "CHAR", int(length_match.group(1)), None

        return "CHAR", len(picture), None

    if "9" in picture:

```



```

        precision = self._numeric_precision(picture)
        scale = self._numeric_scale(picture)

        if usage == "DISPLAY":
            return "CHAR", storage_length, None

        if usage == "COMP":
            return "INTEGER", None, None

        return "DECIMAL", precision, scale

    return "CHAR", storage_length, None

def _numeric_precision(self, picture: str) -> int:
    picture = picture.upper().replace("S", "")

    if "V" in picture:
        before_v, after_v = picture.split("V", 1)
    else:
        before_v = picture
        after_v = ""

    return self._count_9_digits(before_v) + self._count_9_digits(after_v)

def _numeric_scale(self, picture: str) -> int:
    picture = picture.upper().replace("S", "")

    if "V" not in picture:
        return 0

    after_v = picture.split("V", 1)[1]

    return self._count_9_digits(after_v)

def _count_9_digits(self, value: str) -> int:
    total = 0

    for match in re.finditer(r"9$(\d+)$|9", value):
        if match.group(1):
            total += int(match.group(1))
        else:
            total += 1

    return total

def _is_date_group(
    self,
    raw_field_name: str,
    picture: str | None,
    storage_length: int,
) -> bool:
    normalized = self._normalize_column(raw_field_name)

    return (
        picture is None
        and normalized.endswith("DATE")
        and storage_length == 8
    )

def _is_date_part(self, raw_field_name: str) -> bool:
    normalized = self._normalize_column(raw_field_name)

    return (
        normalized.endswith("_YEAR")
        or normalized.endswith("_MONTH")
        or normalized.endswith("_DAY")
    )

def _add_date_part_map(
    self,
    schema: SchemaModel,
    idms_field: str,
    host: str,
) -> None:
    normalized = self._normalize_column(idms_field)

```

```

        if normalized.endswith("_YEAR"):
            part = "YEAR"
        elif normalized.endswith("_MONTH"):
            part = "MONTH"
        elif normalized.endswith("_DAY"):
            part = "DAY"
        else:
            return

        schema.date_part_map[idms_field.upper()] = {
            "host": host,
            "substring_start": self.DATE_PARTS[part]["substring_start"],
            "substring_length": self.DATE_PARTS[part]["substring_length"],
        }

    def _add_field_map(
        self,
        schema: SchemaModel,
        idms_field: str,
        host: str,
    ) -> None:
        schema.field_map[idms_field.upper()] = {
            "host": host,
        }

    def _close_inactive_date_groups(
        self,
        active_date_groups: dict[int, dict[str, str]],
        current_level: int,
    ) -> None:
        closed_levels = [
            level
            for level in active_date_groups
            if level >= current_level
        ]

        for level in closed_levels:
            del active_date_groups[level]

    def _nearest_date_group(
        self,
        active_date_groups: dict[int, dict[str, str]],
    ) -> dict[str, str] | None:
        if not active_date_groups:
            return None

        nearest_level = max(active_date_groups.keys())

        return active_date_groups[nearest_level]

    def _normalize_table(self, record_name: str) -> str:
        return record_name.upper().replace("-", "_")

    def _normalize_column(self, field_name: str) -> str:
        value = field_name.upper()
        value = re.sub(r"-\\d{4}$", "", value)

        return value.replace("-", "_")

```

```

=====
FILE: src\idms_db2_converter\parsers\phase2_metadata_parser.py
ENCODING: utf-8
=====

```

```

import json
import re

from typing import Any

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import (
    Column,
    Record,
    Relationship,
    SchemaModel,

```

)

```
class Phase2MetadataParser:
    """
    Parses Phase 2 metadata JSON produced by Phase 1.

    Critical rule:
    - If an existing DDL schema is passed into parse_into_schema(), DDL physical
      columns must win for datatype, length, scale, nullable, and generated flags.
    - Phase 2 metadata enriches maps and relationships but must not downgrade
      physical DDL metadata.

    This prevents host variables such as:
    - PIC X(1) instead of PIC X(45)
    - PIC S9(18) COMP-3 instead of PIC S9(4) COMP
    """

    DEFAULT_DATATYPE = "CHAR"

    def parse_as_schema(
        self,
        text: str | None,
    ) -> SchemaModel:
        schema = SchemaModel()

        return self.parse_into_schema(
            text=text,
            schema=schema,
        )

    def parse_into_schema(
        self,
        text: str | None,
        schema: SchemaModel,
    ) -> SchemaModel:
        if schema is None:
            schema = SchemaModel()

        if not text or not str(text).strip():
            return schema

        try:
            payload = json.loads(text)
        except json.JSONDecodeError as exc:
            raise ConversionError(f"Invalid Phase 2 metadata JSON: {exc}") from exc

        if not isinstance(payload, dict):
            raise ConversionError("Invalid Phase 2 metadata JSON: root must be an object.")

        preserve_existing_physical = self._has_existing_physical_schema(
            schema=schema,
        )

        self._parse_records(
            payload=payload,
            schema=schema,
            preserve_existing_physical=preserve_existing_physical,
        )

        self._merge_record_table_map(
            payload=payload,
            schema=schema,
        )

        self._parse_field_map(
            payload=payload,
            schema=schema,
        )

        self._merge_calc_key_map(
            payload=payload,
            schema=schema,
        )

        self._merge_table_key_map(
```

```

        payload=payload,
        schema=schema,
    )

    self._parse_relationships(
        payload=payload,
        schema=schema,
    )

    self._merge_set_ordering_map(
        payload=payload,
        schema=schema,
    )

    self._merge_navigation_intent(
        payload=payload,
        schema=schema,
    )

    self._merge_nullable_fk_map(
        payload=payload,
        schema=schema,
    )

    self._merge_date_part_map(
        payload=payload,
        schema=schema,
    )

    self._merge_output_semantics(
        payload=payload,
        schema=schema,
    )

    self._merge_paragraph_operation_graph(
        payload=payload,
        schema=schema,
    )

    self._merge_validation_messages(
        payload=payload,
        schema=schema,
    )

    self._apply_key_flags(
        schema=schema,
    )

    if preserve_existing_physical:
        schema.schema_source = "DDL+PHASE2_METADATA"
    else:
        schema.schema_source = "PHASE2_METADATA"

    return schema

def _has_existing_physical_schema(
    self,
    schema: SchemaModel,
) -> bool:
    for record in getattr(schema, "records", {}).values():
        if getattr(record, "fields", None):
            return True

    return False

def _parse_records(
    self,
    payload: dict,
    schema: SchemaModel,
    preserve_existing_physical: bool,
) -> None:
    record_items = (
        payload.get("records")
        or payload.get("tables")
        or payload.get("db2_tables")
        or []
    )

```

```

)

if isinstance(record_items, dict):
    record_items = list(record_items.values())

for record_json in record_items:
    if not isinstance(record_json, dict):
        continue

    record_name = self.normalize_name(
        self._get_first_non_empty(
            source=record_json,
            keys=[
                "name",
                "record",
                "table",
                "table_name",
                "record_name",
            ],
        )
    )

    if not record_name:
        continue

    primary_keys = self._extract_primary_keys(
        source=record_json,
    )

    fields = self._parse_fields(
        record_json=record_json,
        primary_keys=primary_keys,
    )

    primary_key = primary_keys[0] if primary_keys else None

    if record_name in schema.records:
        existing_record = schema.records[record_name]

        if primary_keys:
            self._set_record_primary_keys(
                record=existing_record,
                primary_keys=primary_keys,
            )
        elif primary_key and not existing_record.primary_key:
            existing_record.primary_key = primary_key

        for field_name, metadata_field in fields.items():
            if field_name in existing_record.fields:
                if preserve_existing_physical:
                    self._merge_non_physical_column_flags(
                        existing_column=existing_record.fields[field_name],
                        metadata_column=metadata_field,
                    )
                else:
                    existing_record.fields[field_name] = metadata_field
            else:
                existing_record.fields[field_name] = metadata_field
    else:
        record = Record(
            name=record_name,
            primary_key=primary_key,
            primary_keys=primary_keys,
            fields=fields,
        )

        self._set_record_primary_keys(
            record=record,
            primary_keys=primary_keys,
        )

        schema.records[record_name] = record

def _parse_fields(
    self,

```

```

        record_json: dict,
        primary_keys: list[str],
    ) -> dict[str, Column]:
        field_items = (
            record_json.get("fields")
            or record_json.get("columns")
            or record_json.get("db2_columns")
            or []
        )

        if isinstance(field_items, dict):
            field_items = list(field_items.values())

        fields: dict[str, Column] = {}

        for field_json in field_items:
            if not isinstance(field_json, dict):
                continue

            field_name = self.normalize_name(
                self._get_first_non_empty(
                    source=field_json,
                    keys=[
                        "name",
                        "field",
                        "column",
                        "column_name",
                    ],
                )
            )

            if not field_name:
                continue

            datatype, length, scale = self._parse_metadata_datatype(
                field_json=field_json,
            )

            if not datatype:
                datatype = self.DEFAULT_DATATYPE

            nullable = field_json.get("nullable")

            if nullable is None:
                nullable = field_name not in primary_keys

            primary_key_flag = (
                bool(field_json.get("primary_key", False))
                or field_name in primary_keys
            )

            if primary_key_flag:
                nullable = False

            fields[field_name] = Column(
                name=field_name,
                datatype=datatype,
                length=length,
                scale=scale,
                nullable=bool(nullable),
                primary_key=primary_key_flag,
                generated=bool(field_json.get("generated", False)),
                source_kind=str(field_json.get("source_kind", "") or ""),
            )

        return fields

    def _parse_metadata_datatype(
        self,
        field_json: dict,
    ) -> tuple[str, int | None, int | None]:
        raw_datatype = self._get_first_non_empty(
            source=field_json,
            keys=[
                "datatype",
                "data_type",
            ],
        )

```

```

        "type",
        "db2_datatype",
    ],
)

datatype_text = self.normalize_datatype(
    raw_datatype,
)

length = self._safe_int_or_none(
    field_json.get("length"),
)

scale = self._safe_int_or_none(
    field_json.get("scale"),
)

if not datatype_text:
    return "", length, scale

decimal_match = re.match(
    r"^(DECIMAL|NUMERIC|DEC)\s*\$(\d+)\s*(?:,\s*(\d+)\s*)?$$",
    datatype_text,
)

if decimal_match:
    return (
        "DECIMAL",
        int(decimal_match.group(2)),
        int(decimal_match.group(3) or 0),
    )

char_match = re.match(
    r"^(VARCHAR|CHAR|CHARACTER|LONG VARCHAR)\s*\$(\d+)\s*$$",
    datatype_text,
)

if char_match:
    datatype_name = char_match.group(1)

    if datatype_name == "CHARACTER":
        datatype_name = "CHAR"

    return (
        datatype_name,
        int(char_match.group(2)),
        scale,
    )

if datatype_text == "VARCHAR":
    return "VARCHAR", length, scale

if datatype_text == "LONG VARCHAR":
    return "LONG VARCHAR", length, scale

if datatype_text in {"CHAR", "CHARACTER"}:
    return "CHAR", length, scale

if datatype_text in {"DECIMAL", "NUMERIC", "DEC"}:
    return "DECIMAL", length, scale

if datatype_text in {"INTEGER", "INT"}:
    return "INTEGER", 9, 0

if datatype_text == "SMALLINT":
    return "SMALLINT", 4, 0

if datatype_text == "BIGINT":
    return "BIGINT", 18, 0

if datatype_text == "DATE":
    return "DATE", None, None

if datatype_text == "TIME":
    return "TIME", None, None

```

```

        if datatype_text == "TIMESTAMP":
            return "TIMESTAMP", None, None

        return datatype_text, length, scale

def _merge_non_physical_column_flags(
    self,
    existing_column: Column,
    metadata_column: Column,
) -> None:
    if getattr(metadata_column, "primary_key", False):
        existing_column.primary_key = True
        existing_column.nullable = False

    if getattr(metadata_column, "generated", False):
        existing_column.generated = True

    if getattr(metadata_column, "source_kind", ""):
        existing_column.source_kind = metadata_column.source_kind

def _set_record_primary_keys(
    self,
    record,
    primary_keys: list[str],
) -> None:
    cleaned: list[str] = []

    for key in primary_keys or []:
        normalized = self.normalize_name(key)

        if not normalized:
            continue

        if normalized in cleaned:
            continue

        cleaned.append(normalized)

    if hasattr(record, "set_primary_keys"):
        record.set_primary_keys(
            keys=cleaned,
        )
    else:
        record.primary_keys = cleaned
        record.primary_key = cleaned[0] if cleaned else None

    for key in cleaned:
        if key in record.fields:
            record.fields[key].primary_key = True
            record.fields[key].nullable = False

def _parse_relationships(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    relationship_items = (
        payload.get("relationships")
        or payload.get("sets")
        or payload.get("db2_relationships")
        or []
    )

    if isinstance(relationship_items, dict):
        relationship_items = list(relationship_items.values())

    for rel_json in relationship_items:
        if not isinstance(rel_json, dict):
            continue

        set_name = self.normalize_name(
            self._get_first_non_empty(
                source=rel_json,
                keys=[
                    "set_name",
                    "name",
                ]
            )
        )

```



```

        "set",
        "relationship",
    ],
)
)

parent_record = self.normalize_name(
    self._get_first_non_empty(
        source=rel_json,
        keys=[
            "parent_record",
            "owner_record",
            "parent",
            "owner",
            "parent_table",
        ],
    )
)

child_record = self.normalize_name(
    self._get_first_non_empty(
        source=rel_json,
        keys=[
            "child_record",
            "member_record",
            "child",
            "member",
            "child_table",
        ],
    )
)

if not set_name or not parent_record or not child_record:
    continue

parent_keys = self._extract_key_list(
    source=rel_json,
    plural_keys=[
        "parent_keys",
        "owner_keys",
    ],
    single_keys=[
        "parent_key",
        "owner_key",
        "parent_pk",
    ],
)

child_fks = self._extract_key_list(
    source=rel_json,
    plural_keys=[
        "child_fks",
        "member_fks",
        "foreign_keys",
    ],
    single_keys=[
        "child_fk",
        "member_fk",
        "foreign_key",
        "child_pk",
    ],
)

if not parent_keys:
    parent_key = self._resolve_parent_key(
        schema=schema,
        parent_record=parent_record,
    )

    if parent_key:
        parent_keys = [parent_key]

if not child_fks:
    child_fk = self._resolve_child_fk(
        schema=schema,
        parent_record=parent_record,

```

```

        child_record=child_record,
        parent_keys=parent_keys,
    )

    if child_fk:
        child_fks = [child_fk]

    order_by = [
        self.normalize_name(value)
        for value in rel_json.get("order_by", []) or []
        if self.normalize_name(value)
    ]

    if not order_by and child_fks:
        order_by = child_fks.copy()

    parent_key_single = parent_keys[0] if parent_keys else None
    child_fk_single = child_fks[0] if child_fks else None

    if set_name in schema.relationships:
        rel = schema.relationships[set_name]
        rel.parent_record = parent_record or rel.parent_record
        rel.child_record = child_record or rel.child_record
        rel.parent_key = parent_key_single or rel.parent_key
        rel.child_fk = child_fk_single or rel.child_fk
        rel.parent_keys = parent_keys or getattr(rel, "parent_keys", [])
        rel.child_fks = child_fks or getattr(rel, "child_fks", [])
        rel.order_by = order_by or rel.order_by
        rel.cardinality = rel_json.get("cardinality") or rel.cardinality or "1:N"
        continue

    schema.relationships[set_name] = Relationship(
        set_name=set_name,
        parent_record=parent_record,
        child_record=child_record,
        cardinality=rel_json.get("cardinality") or "1:N",
        parent_key=parent_key_single,
        child_fk=child_fk_single,
        parent_keys=parent_keys,
        child_fks=child_fks,
        order_by=order_by,
    )

def _parse_field_map(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    value = payload.get("field_map") or {}

    if not isinstance(value, dict):
        return

    for key, item in value.items():
        if isinstance(item, dict):
            parsed_item = item
        else:
            parsed_item = {
                "host": str(item),
            }

    aliases = self._lookup_aliases(key)

    if isinstance(parsed_item, dict):
        aliases.extend(self._lookup_aliases(parsed_item.get("legacy_field")))
        aliases.extend(self._lookup_aliases(parsed_item.get("column")))

    for alias in self._unique_values(aliases):
        if not alias:
            continue

        schema.field_map[alias] = parsed_item

def _merge_record_table_map(
    self,
    payload: dict,

```

```

        schema: SchemaModel,
    ) -> None:
        value = payload.get("record_table_map") or {}

        if not isinstance(value, dict):
            return

        for key, item in value.items():
            normalized_key = self.normalize_name(key)
            normalized_value = self.normalize_name(item)

            if not normalized_key or not normalized_value:
                continue

            schema.record_table_map[normalized_key] = normalized_value

            hyphen_key = self.normalize_cobol_name(key)

            if hyphen_key:
                schema.record_table_map.setdefault(
                    hyphen_key,
                    normalized_value,
                )

def _merge_calc_key_map(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    value = payload.get("calc_key_map") or {}

    if not isinstance(value, dict):
        return

    for key, item in value.items():
        normalized_key = self.normalize_name(key)

        if normalized_key:
            schema.calc_key_map[normalized_key] = item

            hyphen_key = self.normalize_cobol_name(key)

            if hyphen_key:
                schema.calc_key_map.setdefault(
                    hyphen_key,
                    item,
                )

def _merge_table_key_map(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    value = payload.get("table_key_map") or {}

    if not isinstance(value, dict):
        return

    for key, item in value.items():
        normalized_key = self.normalize_name(key)

        if normalized_key:
            schema.table_key_map[normalized_key] = item

            hyphen_key = self.normalize_cobol_name(key)

            if hyphen_key:
                schema.table_key_map.setdefault(
                    hyphen_key,
                    item,
                )

def _merge_set_ordering_map(
    self,
    payload: dict,
    schema: SchemaModel,

```

```

) -> None:
    value = payload.get("set_ordering_map") or {}

    if not isinstance(value, dict):
        return

    for key, item in value.items():
        normalized_key = self.normalize_name(key)

        if normalized_key:
            schema.set_ordering_map[normalized_key] = item

        relationship = schema.relationships.get(normalized_key)

        if relationship is None:
            continue

        if isinstance(item, dict):
            parent_keys = self._extract_key_list(
                source=item,
                plural_keys=["parent_keys"],
                single_keys=["parent_key"],
            )

            child_fks = self._extract_key_list(
                source=item,
                plural_keys=["child_fks"],
                single_keys=["child_fk"],
            )

            if parent_keys:
                relationship.parent_keys = parent_keys
                relationship.parent_key = parent_keys[0]

            if child_fks:
                relationship.child_fks = child_fks
                relationship.child_fk = child_fks[0]

            order_by = item.get("order_by") or []

            if order_by:
                relationship.order_by = [
                    self.normalize_name(order_item)
                    for order_item in order_by
                    if order_item
                ]

def _merge_navigation_intent(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    value = payload.get("navigation_intent") or {}

    if isinstance(value, dict):
        schema.navigation_intent = value

def _merge_nullable_fk_map(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    value = payload.get("nullable_fk_map") or {}

    if isinstance(value, dict):
        schema.nullable_fk_map = value

def _merge_date_part_map(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    value = payload.get("date_part_map") or {}

    if not isinstance(value, dict):
        return

```

```

for key, item in value.items():
    aliases = self._lookup_aliases(key)

    if isinstance(item, dict):
        aliases.extend(self._lookup_aliases(item.get("column")))
        aliases.extend(self._lookup_aliases(item.get("legacy_field")))

    for alias in self._unique_values(aliases):
        if alias:
            schema.date_part_map[alias] = item

def _merge_output_semantics(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    value = payload.get("output_semantics") or {}

    if isinstance(value, dict):
        schema.output_semantics = value

def _merge_paragraph_operation_graph(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    value = payload.get("paragraph_operation_graph") or {}

    if isinstance(value, dict):
        schema.paragraph_operation_graph = value

def _merge_validation_messages(
    self,
    payload: dict,
    schema: SchemaModel,
) -> None:
    messages = payload.get("validation_messages") or []

    if not isinstance(messages, list):
        return

    schema.validation_messages.extend(str(message) for message in messages)

def _apply_key_flags(
    self,
    schema: SchemaModel,
) -> None:
    for record in schema.records.values():
        primary_keys = []

        if hasattr(record, "effective_primary_keys"):
            primary_keys = record.effective_primary_keys()

        if not primary_keys:
            primary_keys = list(getattr(record, "primary_keys", []) or [])

        if not primary_keys and getattr(record, "primary_key", None):
            primary_keys = [record.primary_key]

        cleaned_primary_keys = []

        for primary_key in primary_keys:
            normalized_primary_key = self.normalize_name(primary_key)

            if not normalized_primary_key:
                continue

            if normalized_primary_key not in cleaned_primary_keys:
                cleaned_primary_keys.append(normalized_primary_key)

            if normalized_primary_key not in record.fields:
                continue

            record.fields[normalized_primary_key].primary_key = True
            record.fields[normalized_primary_key].nullable = False

```

```

        if cleaned_primary_keys:
            record.primary_keys = cleaned_primary_keys
            record.primary_key = cleaned_primary_keys[0]

def _extract_primary_keys(
    self,
    source: dict,
) -> list[str]:
    return self._extract_key_list(
        source=source,
        plural_keys=["primary_keys", "pk_columns"],
        single_keys=["primary_key", "pk"],
    )

def _extract_key_list(
    self,
    source: dict,
    plural_keys: list[str],
    single_keys: list[str],
) -> list[str]:
    result: list[str] = []

    for key in plural_keys:
        values = source.get(key)

        if not values:
            continue

        if not isinstance(values, list):
            values = [values]

        for value in values:
            normalized = self.normalize_name(value)

            if normalized and normalized not in result:
                result.append(normalized)

    for key in single_keys:
        value = source.get(key)
        normalized = self.normalize_name(value)

        if normalized and normalized not in result:
            result.append(normalized)

    return result

def _resolve_parent_key(
    self,
    schema: SchemaModel,
    parent_record: str,
) -> str | None:
    record = schema.records.get(parent_record)

    if not record:
        return None

    if hasattr(record, "effective_primary_keys"):
        primary_keys = record.effective_primary_keys()

        if primary_keys:
            return primary_keys[0]

    return record.primary_key

def _resolve_child_fk(
    self,
    schema: SchemaModel,
    parent_record: str,
    child_record: str,
    parent_keys: list[str],
) -> str | None:
    parent = schema.records.get(parent_record)
    child = schema.records.get(child_record)

    if not parent or not child:

```

```

        return None

    keys = parent_keys or []

    if not keys:
        if hasattr(parent, "effective_primary_keys"):
            keys = parent.effective_primary_keys()

        if not keys and parent.primary_key:
            keys = [parent.primary_key]

    for key in keys:
        if key in child.fields:
            return key

    return None

def _get_first_non_empty(
    self,
    source: dict,
    keys: list[str],
) -> Any:
    for key in keys:
        value = source.get(key)

        if value is None:
            continue

        if isinstance(value, str):
            value = value.strip()

        if not value:
            continue

        return value

    return None

def _lookup_aliases(
    self,
    value: Any,
) -> list[str]:
    if value is None:
        return []

    original = str(value or "").strip().upper()

    if not original:
        return []

    normalized = self.normalize_name(original)
    cobol = self.normalize_cobol_name(original)
    no_numeric_suffix = self.remove_numeric_suffix(normalized)
    no_numeric_suffix_cobol = self.remove_numeric_suffix(cobol)
    no_generated_suffix = self.remove_generated_suffix(normalized)
    no_generated_suffix_cobol = self.remove_generated_suffix(cobol)
    compact = self.compact_name(no_generated_suffix or no_numeric_suffix or normalized)

    return self._unique_values(
        [
            original,
            normalized,
            cobol,
            no_numeric_suffix,
            no_numeric_suffix_cobol,
            no_generated_suffix,
            no_generated_suffix_cobol,
            compact,
        ]
    )

def normalize_name(
    self,
    value: Any,
) -> str:
    if value is None:

```

```

        return ""

text = str(value).strip().upper()

if not text:
    return ""

text = text.replace("-", "_")
text = text.replace(" ", "_")
text = text.replace("\u00a0", "_")
text = text.replace("\t", "_")
text = self._remove_wrapping_quotes(text)
text = self._strip_table_qualifier(text)
text = self._collapse_separator(text, "_")

return text.strip("_")

def normalize_cobol_name(
    self,
    value: Any,
) -> str:
    if value is None:
        return ""

text = str(value).strip().upper()

if not text:
    return ""

text = text.replace("_", "-")
text = text.replace(" ", "-")
text = text.replace("\u00a0", "-")
text = text.replace("\t", "-")
text = self._remove_wrapping_quotes(text)
text = self._strip_table_qualifier(text)
text = self._collapse_separator(text, "-")

return text.strip("-")

def normalize_datatype(
    self,
    value: Any,
) -> str:
    if value is None:
        return ""

text = str(value).strip().upper()

if not text:
    return ""

return text

def remove_numeric_suffix(
    self,
    value: Any,
) -> str:
    text = self.normalize_name(value)

    if not text:
        return ""

text = re.sub(r"_?[0-9]{4}$", "", text)
text = re.sub(r"[0-9]{4}$", "", text)
text = re.sub(r"[_\-\s]+$", "", text)

return text

def remove_generated_suffix(
    self,
    value: Any,
) -> str:
    text = self.normalize_name(value)

    if not text:
        return ""

```



```

text = re.sub(r"_479[A-Z0-9]+$", "", text)
text = re.sub(r"479[A-Z0-9]+$", "", text)
text = re.sub(r"[_\-\s]+$", "", text)

return text

def compact_name(
    self,
    value: Any,
) -> str:
    if value is None:
        return ""

    return "".join(char for char in str(value or "").upper() if char.isalnum())

def _unique_values(
    self,
    values: list[str],
) -> list[str]:
    result: list[str] = []
    seen: set[str] = set()

    for value in values:
        normalized = str(value or "").strip().upper()

        if not normalized:
            continue

        if normalized in seen:
            continue

        seen.add(normalized)
        result.append(normalized)

    return result

def _collapse_separator(
    self,
    value: str,
    separator: str,
) -> str:
    if separator == "_":
        value = "".join(char if char.isalnum() else "_" for char in value)

        while "__" in value:
            value = value.replace("__", "_")

        return value

    value = "".join(char if char.isalnum() else "-" for char in value)

    while "--" in value:
        value = value.replace("--", "-")

    return value

def _remove_wrapping_quotes(
    self,
    value: str,
) -> str:
    text = value.strip()

    for quote in ['"', "'", "`", "[", "]"]:
        text = text.strip(quote)

    return text

def _strip_table_qualifier(
    self,
    value: str,
) -> str:
    if "." not in value:
        return value

    return value.split(".")[1]

```

```

def _safe_int_or_none(
    self,
    value,
) -> int | None:
    try:
        if value is None:
            return None

        text = str(value).strip()

        if not text:
            return None

        return int(text)
    except Exception:
        return None

```

```

=====
FILE: src\idms_db2_converter\services\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_converter\services\conversion_service.py
ENCODING: utf-8
=====

```

```

from copy import deepcopy
from time import perf_counter
from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import SchemaModel
from idms_db2_converter.parsers.cobol_parser import CobolParser
from idms_db2_converter.parsers.canonical_parser import CanonicalParser
from idms_db2_converter.parsers.ddl_parser import DDLParser
from idms_db2_converter.parsers.phase2_metadata_parser import Phase2MetadataParser
from idms_db2_converter.parsers.idms_schema_parser import IdmsSchemaParser
from idms_db2_converter.services.relationship_resolver import RelationshipResolver
from idms_db2_converter.services.schema_merger import SchemaMerger
from idms_db2_converter.transformers.retrieval_transformer import RetrievalTransformer
from idms_db2_converter.validators.output_validator import OutputValidator
from idms_db2_converter.validators.schema_validator import SchemaValidator

```

```

class ConversionService:
    DEBUG_SCHEMA = False

    def convert_retrieval(
        self,
        cobol_text: str,
        canonical_json: str,
        phase2_metadata_json: str | None = None,
        ddl_text: str | None = None,
        idms_schema_text: str | None = None,
        relationship_overrides_json: str | None = None,
        target_program: str | None = None,
    ) -> tuple[str, list[str]]:
        timings: list[str] = []
        total_start = perf_counter()
        step_start = perf_counter()

        def add_timing(
            label: str,
        ) -> None:
            nonlocal step_start

            elapsed = perf_counter() - step_start
            message = f"Phase 2 - {label}: {elapsed:.2f} seconds"

            print(
                message,
            )

```

```

        timings.append(
            message,
        )

        step_start = perf_counter()

analysis = CobolParser().parse(
    cobol_text,
)

add_timing(
    "COBOL parse",
)

schema_builder = (
    getattr(
        self,
        "_build_schema_from_available_sources",
        None,
    )
    or getattr(
        self,
        "build_schema_from_available_sources",
    )
)

schema = schema_builder(
    canonical_json=canonical_json,
    phase2_metadata_json=phase2_metadata_json,
    ddl_text=ddl_text,
    idms_schema_text=idms_schema_text,
)

add_timing(
    "Build schema from available sources",
)

if self.DEBUG_SCHEMA:
    self.debug_schema(
        schema,
    )

add_timing(
    "Debug schema",
)

if (
    relationship_overrides_json
    and relationship_overrides_json.strip()
):
    schema = RelationshipResolver().apply_overrides(
        schema=schema,
        relationship_overrides_json=relationship_overrides_json,
    )

add_timing(
    "Apply relationship overrides",
)

schema = RelationshipResolver().resolve_cobol_sets(
    schema=schema,
    analysis=analysis,
)

add_timing(
    "Resolve COBOL sets",
)

schema_errors = SchemaValidator().validate(
    schema,
    analysis,
)

add_timing(
    "Schema validation",

```

```

)

if schema_errors:
    raise ConversionError(
        "\n".join(schema_errors),
    )

converted = RetrievalTransformer(
    schema,
    analysis,
).transform(
    cobol=cobol_text,
    target_program=target_program,
)

add_timing(
    "Retrieval transformer",
)

output_errors = OutputValidator().validate(
    converted,
)

add_timing(
    "Output validation",
)

validation_messages = []

validation_messages.extend(
    getattr(
        schema,
        "validation_messages",
        [],
    )
)

validation_messages.extend(
    output_errors,
)

total_elapsed = perf_counter() - total_start

timings.append(
    f"Phase 2 - TOTAL: {total_elapsed:.2f} seconds",
)

print("=" * 80)
print("PHASE 2 TIMINGS")
print("=" * 80)

for timing in timings:
    print(
        timing,
    )

validation_messages.extend(
    timings,
)

return converted, validation_messages

def _build_schema_from_available_sources(
    self,
    canonical_json: str | None,
    phase2_metadata_json: str | None,
    ddl_text: str | None,
    idms_schema_text: str | None,
) -> SchemaModel:
    canonical_has_text = bool(canonical_json and canonical_json.strip())
    metadata_has_text = bool(phase2_metadata_json and phase2_metadata_json.strip())
    ddl_has_text = bool(ddl_text and ddl_text.strip())
    idms_schema_has_text = bool(idms_schema_text and idms_schema_text.strip())

    canonical_schema = SchemaModel()
    metadata_schema = SchemaModel()

```

```

ddl_schema = SchemaModel()
idms_schema = SchemaModel()

if canonical_has_text:
    canonical_schema = CanonicalParser().parse(canonical_json)
    canonical_schema.schema_source = "CANONICAL"

if metadata_has_text:
    metadata_schema = Phase2MetadataParser().parse_as_schema(
        phase2_metadata_json
    )
    metadata_schema.schema_source = "PHASE2_METADATA"

if ddl_has_text:
    ddl_schema = DDLParser().parse(ddl_text)

    if self._has_schema_objects(ddl_schema):
        ddl_schema.schema_source = "DDL"
    elif self._looks_like_idms_schema(ddl_text):
        idms_schema = IdmsSchemaParser().parse(ddl_text)
        idms_schema.schema_source = "IDMS_SCHEMA"
    elif "CREATE TABLE" in ddl_text.upper():
        raise ConversionError(
            "DDL text was provided and contains CREATE TABLE, but no tables were parsed."
        )

if idms_schema_has_text:
    idms_schema = IdmsSchemaParser().parse(idms_schema_text)
    idms_schema.schema_source = "IDMS_SCHEMA"

if self._has_schema_objects(ddl_schema):
    base_schema = ddl_schema
    base_schema.schema_source = "DDL"

    if self._has_schema_objects(canonical_schema):
        base_schema = self._merge_non_physical_metadata(
            physical_schema=base_schema,
            overlay=canonical_schema,
        )

    if metadata_has_text:
        base_schema = Phase2MetadataParser().parse_into_schema(
            text=phase2_metadata_json,
            schema=base_schema,
        )

    if self._has_schema_objects(idms_schema):
        base_schema = SchemaMerger().merge_physical_with_idms(
            physical_schema=base_schema,
            idms_schema=idms_schema,
        )

    return base_schema

if self._has_schema_objects(canonical_schema):
    base_schema = canonical_schema

    if metadata_has_text:
        base_schema = Phase2MetadataParser().parse_into_schema(
            text=phase2_metadata_json,
            schema=base_schema,
        )

    if self._has_schema_objects(idms_schema):
        base_schema = self._merge_schema(base_schema, idms_schema)

    return base_schema

if self._has_schema_objects(metadata_schema):
    base_schema = metadata_schema

    if self._has_schema_objects(idms_schema):
        base_schema = self._merge_schema(base_schema, idms_schema)

    return base_schema

```

```

if self._has_schema_objects(idms_schema):
    return idms_schema

raise ConversionError(
    "No usable schema source found. Provide canonical model JSON, "
    "Phase 2 metadata JSON with records/relationships, DB2 DDL, "
    "or IDMS schema listing."
)

def _merge_schema(
    self,
    base: SchemaModel,
    overlay: SchemaModel,
) -> SchemaModel:
    merged = deepcopy(base)

    for record_name, overlay_record in overlay.records.items():
        if record_name in merged.records:
            merged.records[record_name].primary_key = (
                overlay_record.primary_key
                or merged.records[record_name].primary_key
            )

            for field_name, overlay_field in overlay_record.fields.items():
                merged.records[record_name].fields[field_name] = overlay_field
            else:
                merged.records[record_name] = overlay_record

    for set_name, overlay_rel in overlay.relationships.items():
        if set_name not in merged.relationships:
            merged.relationships[set_name] = overlay_rel
            continue

        rel = merged.relationships[set_name]
        rel.parent_record = overlay_rel.parent_record or rel.parent_record
        rel.child_record = overlay_rel.child_record or rel.child_record
        rel.parent_key = overlay_rel.parent_key or rel.parent_key
        rel.child_fk = overlay_rel.child_fk or rel.child_fk
        rel.cardinality = overlay_rel.cardinality or rel.cardinality

        if overlay_rel.order_by:
            rel.order_by = overlay_rel.order_by

    self._merge_dict(merged.field_map, overlay.field_map)
    self._merge_dict(merged.record_table_map, overlay.record_table_map)
    self._merge_dict(merged.calc_key_map, overlay.calc_key_map)
    self._merge_dict(merged.set_ordering_map, overlay.set_ordering_map)
    self._merge_dict(merged.navigation_intent, overlay.navigation_intent)
    self._merge_dict(merged.nullable_fk_map, overlay.nullable_fk_map)
    self._merge_dict(merged.date_part_map, overlay.date_part_map)
    self._merge_dict(merged.output_semantics, overlay.output_semantics)
    self._merge_dict(
        merged.paragraph_operation_graph,
        overlay.paragraph_operation_graph,
    )

    if hasattr(merged, "validation_messages") and hasattr(
        overlay,
        "validation_messages",
    ):
        merged.validation_messages.extend(overlay.validation_messages)

    if not getattr(merged, "schema_source", None):
        merged.schema_source = getattr(overlay, "schema_source", None)

    return merged

def _merge_non_physical_metadata(
    self,
    physical_schema: SchemaModel,
    overlay: SchemaModel,
) -> SchemaModel:
    merged = deepcopy(physical_schema)

    self._merge_dict(merged.field_map, overlay.field_map)

```

```

for key, value in overlay.record_table_map.items():
    if key not in merged.record_table_map:
        merged.record_table_map[key] = value

self._merge_dict(merged.calc_key_map, overlay.calc_key_map)
self._merge_dict(merged.set_ordering_map, overlay.set_ordering_map)
self._merge_dict(merged.navigation_intent, overlay.navigation_intent)
self._merge_dict(merged.nullable_fk_map, overlay.nullable_fk_map)
self._merge_dict(merged.date_part_map, overlay.date_part_map)
self._merge_dict(merged.output_semantics, overlay.output_semantics)
self._merge_dict(
    merged.paragraph_operation_graph,
    overlay.paragraph_operation_graph,
)

for set_name, relationship in overlay.relationships.items():
    if set_name not in merged.relationships:
        merged.relationships[set_name] = relationship

if hasattr(merged, "validation_messages") and hasattr(
    overlay,
    "validation_messages",
):
    merged.validation_messages.extend(overlay.validation_messages)

merged.schema_source = "DDL+CANONICAL_METADATA"

return merged

def _merge_dict(self, target: dict, source: dict) -> None:
    for key, value in source.items():
        target[key] = value

def _has_schema_objects(self, schema: SchemaModel) -> bool:
    if hasattr(schema, "has_schema_objects"):
        return schema.has_schema_objects()

    return bool(schema.records or schema.relationships)

def _looks_like_idms_schema(self, text: str | None) -> bool:
    if not text:
        return False

    upper = text.upper()

    return (
        "RECORD NAME" in upper
        and "DBKEY POSITIONS" in upper
        and "DATA ITEM" in upper
    )

def _debug_schema(self, schema: SchemaModel) -> None:
    print("FINAL SCHEMA SOURCE:", getattr(schema, "schema_source", None))
    print("RECORD COUNT:", len(schema.records))
    print("RELATIONSHIP COUNT:", len(schema.relationships))
    print("RECORD TABLE MAP COUNT:", len(schema.record_table_map))
    print("FIELD MAP COUNT:", len(schema.field_map))
    print("DATE PART MAP COUNT:", len(schema.date_part_map))

    for record_name in sorted(schema.records.keys()):
        record = schema.records[record_name]
        print(f"RECORD: {record_name} PK={record.primary_key}")

        for field_name in sorted(record.fields.keys()):
            field = record.fields[field_name]
            print(
                "  FIELD:",
                field_name,
                field.datatype,
                field.length,
                field.scale,
                "NULLABLE=",
                field.nullable,
            )

    print("RELATIONSHIPS:")

```

```

for set_name in sorted(schema.relationships.keys()):
    rel = schema.relationships[set_name]
    print(
        "  REL:",
        set_name,
        rel.parent_record,
        rel.parent_key,
        "->",
        rel.child_record,
        rel.child_fk,
    )

```

```

=====
FILE: src\idms_db2_converter\services\relationship_resolver.py
ENCODING: utf-8
=====

```

```
import json
```

```

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import CobolAnalysis, Relationship, SchemaModel

```

```
class RelationshipResolver:
```

```
    """
```

```
    Resolves COBOL IDMS set names to schema relationships.
```

```
    Composite-key support:
```

- Supports parent_keys and child_fks in override JSON.
- Supports parent_key and child_fk for backward compatibility.
- Resolves existing relationships using effective_parent_keys/effective_child_fks.
- Preserves parent_key/child_fk as first pair for older code.

```
    """
```

```

def apply_overrides(
    self,
    schema: SchemaModel,
    relationship_overrides_json: str,
) -> SchemaModel:
    try:
        payload = json.loads(
            relationship_overrides_json,
        )
    except json.JSONDecodeError as exc:
        raise ConversionError(f"Invalid relationship override JSON: {exc}") from exc

    self._apply_record_aliases(
        schema=schema,
        payload=payload,
    )

    for item in payload.get("relationships", []) or []:
        if not isinstance(item, dict):
            continue

        self._apply_relationship_override(
            schema=schema,
            item=item,
        )

    return schema

def resolve_cobol_sets(
    self,
    schema: SchemaModel,
    analysis: CobolAnalysis,
) -> SchemaModel:
    references = self._collect_set_references(
        analysis,
    )

    for set_name, child_records in references.items():
        if set_name in schema.relationships:
            continue

```



```

        matched = self._find_unambiguous_relationship(
            schema=schema,
            set_name=set_name,
            child_records=child_records,
        )

        if not matched:
            continue

        parent_keys = self._effective_parent_keys(
            matched,
        )

        child_fks = self._effective_child_fks(
            matched,
        )

        schema.relationships[set_name] = Relationship(
            set_name=set_name,
            parent_record=matched.parent_record,
            child_record=matched.child_record,
            cardinality=matched.cardinality,
            parent_key=parent_keys[0] if parent_keys else matched.parent_key,
            child_fk=child_fks[0] if child_fks else matched.child_fk,
            parent_keys=parent_keys,
            child_fks=child_fks,
            order_by=matched.order_by or child_fks,
        )

        schema.add_validation_message(
            f"Set {set_name} resolved from existing schema relationship "
            f"{matched.parent_record}-{matched.child_record}."
        )

    return schema

def _collect_set_references(
    self,
    analysis: CobolAnalysis,
) -> dict[str, set[str]]:
    references: dict[str, set[str]] = {}

    for record_name, set_name in getattr(analysis, "obtain_next", []) or []:
        references.setdefault(set_name, set()).add(record_name)

    for set_name in getattr(analysis, "obtain_owner_sets", []) or []:
        references.setdefault(set_name, set())

    for set_name in getattr(analysis, "find_first_sets", []) or []:
        references.setdefault(set_name, set())

    return references

def _apply_record_aliases(
    self,
    schema: SchemaModel,
    payload: dict,
) -> None:
    for item in payload.get("record_aliases", []) or []:
        if not isinstance(item, dict):
            continue

        record_name = self._upper_or_none(item.get("record"))
        table_name = self._upper_or_none(item.get("table"))

        if not record_name or not table_name:
            continue

        if table_name not in schema.records:
            raise ConversionError(
                f"Record alias {record_name} references unknown table {table_name}."
            )

        schema.records[record_name] = schema.records[table_name]
        schema.record_table_map[record_name] = table_name

```

```

def _apply_relationship_override(
    self,
    schema: SchemaModel,
    item: dict,
) -> None:
    set_name = self._upper_or_none(
        item.get("set_name")
        or item.get("name")
        or item.get("relationship")
    )

    if not set_name:
        return

    parent_record = self._upper_or_none(
        item.get("parent_record")
        or item.get("owner_record")
        or item.get("parent_table")
    )

    child_record = self._upper_or_none(
        item.get("child_record")
        or item.get("member_record")
        or item.get("child_table")
    )

    parent_keys = self._extract_key_list(
        source=item,
        plural_keys=[
            "parent_keys",
            "owner_keys",
        ],
        single_keys=[
            "parent_key",
            "owner_key",
            "parent_pk",
        ],
    )

    child_fks = self._extract_key_list(
        source=item,
        plural_keys=[
            "child_fks",
            "member_fks",
            "foreign_keys",
        ],
        single_keys=[
            "child_fk",
            "member_fk",
            "foreign_key",
            "child_pk",
        ],
    )

    order_by = [
        self._normalize_name(value)
        for value in item.get("order_by", []) or []
        if self._normalize_name(value)
    ]

    if set_name in schema.relationships:
        rel = schema.relationships[set_name]

        if parent_record:
            rel.parent_record = parent_record

        if child_record:
            rel.child_record = child_record

        if parent_keys:
            rel.parent_keys = parent_keys
            rel.parent_key = parent_keys[0]

        if child_fks:
            rel.child_fks = child_fks

```

```

        rel.child_fk = child_fks[0]

    if order_by:
        rel.order_by = order_by

    self._validate_relationship_keys(
        schema=schema,
        relationship=rel,
        set_name=set_name,
    )

    return

if parent_record and child_record:
    self._create_explicit_relationship(
        schema=schema,
        set_name=set_name,
        parent_record=parent_record,
        child_record=child_record,
        parent_keys=parent_keys,
        child_fks=child_fks,
        order_by=order_by,
    )
    return

matched = self._find_relationship_by_keys(
    schema=schema,
    parent_keys=parent_keys,
    child_fks=child_fks,
)

if matched:
    matched_parent_keys = parent_keys or self._effective_parent_keys(matched)
    matched_child_fks = child_fks or self._effective_child_fks(matched)

    schema.relationships[set_name] = Relationship(
        set_name=set_name,
        parent_record=matched.parent_record,
        child_record=matched.child_record,
        cardinality=matched.cardinality,
        parent_key=matched_parent_keys[0] if matched_parent_keys else matched.parent_key,
        child_fk=matched_child_fks[0] if matched_child_fks else matched.child_fk,
        parent_keys=matched_parent_keys,
        child_fks=matched_child_fks,
        order_by=order_by or matched.order_by or matched_child_fks,
    )
    return

raise ConversionError(
    f"Override references set {set_name}, but it could not be resolved. "
    "Provide parent_record and child_record, or provide keys matching a DDL foreign key."
)

def _create_explicit_relationship(
    self,
    schema: SchemaModel,
    set_name: str,
    parent_record: str,
    child_record: str,
    parent_keys: list[str],
    child_fks: list[str],
    order_by: list[str],
) -> None:
    if parent_record not in schema.records:
        raise ConversionError(
            f"Set {set_name} parent_record {parent_record} is not present in schema."
        )

    if child_record not in schema.records:
        raise ConversionError(
            f"Set {set_name} child_record {child_record} is not present in schema."
        )

    parent = schema.records[parent_record]
    child = schema.records[child_record]

```

```

resolved_parent_keys = parent_keys or self._effective_primary_keys(parent)

if not resolved_parent_keys:
    raise ConversionError(
        f"Set {set_name} cannot resolve parent_keys."
    )

resolved_child_fks = child_fks or self._resolve_child_fks(
    parent_keys=resolved_parent_keys,
    child=child,
)

if not resolved_child_fks:
    raise ConversionError(
        f"Set {set_name} cannot resolve child_fks."
    )

if len(resolved_parent_keys) != len(resolved_child_fks):
    raise ConversionError(
        f"Set {set_name} has mismatched composite key counts: "
        f"{len(resolved_parent_keys)} parent key(s), "
        f"{len(resolved_child_fks)} child FK(s)."
    )

for parent_key in resolved_parent_keys:
    if parent_key not in parent.fields:
        raise ConversionError(
            f"Set {set_name} parent key {parent_key} is not present in {parent_record}."
        )

for child_fk in resolved_child_fks:
    if child_fk not in child.fields:
        raise ConversionError(
            f"Set {set_name} child FK {child_fk} is not present in {child_record}."
        )

schema.relationships[set_name] = Relationship(
    set_name=set_name,
    parent_record=parent_record,
    child_record=child_record,
    cardinality="1:N",
    parent_key=resolved_parent_keys[0],
    child_fk=resolved_child_fks[0],
    parent_keys=resolved_parent_keys,
    child_fks=resolved_child_fks,
    order_by=order_by or resolved_child_fks.copy(),
)

def _find_relationship_by_keys(
    self,
    schema: SchemaModel,
    parent_keys: list[str],
    child_fks: list[str],
) -> Relationship | None:
    if not parent_keys and not child_fks:
        return None

    matches = []

    for rel in schema.relationships.values():
        rel_parent_keys = self._effective_parent_keys(rel)
        rel_child_fks = self._effective_child_fks(rel)

        if parent_keys and rel_parent_keys != parent_keys:
            continue

        if child_fks and rel_child_fks != child_fks:
            continue

        matches.append(rel)

    if len(matches) == 1:
        return matches[0]

    if len(matches) > 1:
        raise ConversionError(

```

```

        "Relationship override is ambiguous. Add parent_record and child_record."
    )

    return None

def _find_unambiguous_relationship(
    self,
    schema: SchemaModel,
    set_name: str,
    child_records: set[str],
) -> Relationship | None:
    candidates = list(schema.relationships.values())

    if child_records:
        filtered = [
            rel
            for rel in candidates
            if rel.child_record in child_records
        ]

        if filtered:
            candidates = filtered

    if len(candidates) == 1:
        return candidates[0]

    scored = []

    for rel in candidates:
        score = self._score_relationship(
            set_name=set_name,
            rel=rel,
        )

        if score > 0:
            scored.append(
                (
                    score,
                    rel,
                )
            )

    if not scored:
        return None

    scored.sort(
        key=lambda item: item[0],
        reverse=True,
    )

    top_score = scored[0][0]

    top_matches = [
        rel
        for score, rel in scored
        if score == top_score
    ]

    if len(top_matches) == 1:
        return top_matches[0]

    return None

def _score_relationship(
    self,
    set_name: str,
    rel: Relationship,
) -> int:
    tokens = [
        token
        for token in set_name.replace("_", "-").split("-")
        if token
    ]

    searchable_values = [
        rel.parent_record,

```

```

        rel.child_record,
        rel.parent_key or "",
        rel.child_fk or "",
    ]

    searchable_values.extend(
        self._effective_parent_keys(rel)
    )

    searchable_values.extend(
        self._effective_child_fks(rel)
    )

    score = 0

    for token in tokens:
        for value in searchable_values:
            if self._token_matches(token, value):
                score += 1

    return score

def _validate_relationship_keys(
    self,
    schema: SchemaModel,
    relationship: Relationship,
    set_name: str,
) -> None:
    if relationship.parent_record not in schema.records:
        raise ConversionError(
            f"Set {set_name} parent_record {relationship.parent_record} is not present in schema."
        )

    if relationship.child_record not in schema.records:
        raise ConversionError(
            f"Set {set_name} child_record {relationship.child_record} is not present in schema."
        )

    parent = schema.records[relationship.parent_record]
    child = schema.records[relationship.child_record]

    parent_keys = self._effective_parent_keys(relationship)
    child_fks = self._effective_child_fks(relationship)

    if not parent_keys:
        parent_keys = self._effective_primary_keys(parent)
        relationship.parent_keys = parent_keys
        relationship.parent_key = parent_keys[0] if parent_keys else None

    if not child_fks:
        child_fks = self._resolve_child_fks(
            parent_keys=parent_keys,
            child=child,
        )
        relationship.child_fks = child_fks
        relationship.child_fk = child_fks[0] if child_fks else None

    if len(parent_keys) != len(child_fks):
        raise ConversionError(
            f"Set {set_name} has mismatched composite key counts: "
            f"{len(parent_keys)} parent key(s), {len(child_fks)} child FK(s)."
        )

    for parent_key in parent_keys:
        if parent_key not in parent.fields:
            raise ConversionError(
                f"Set {set_name} parent key {parent_key} is not present in {relationship.parent_record}."
            )

    for child_fk in child_fks:
        if child_fk not in child.fields:
            raise ConversionError(
                f"Set {set_name} child FK {child_fk} is not present in {relationship.child_record}."
            )

    if not relationship.order_by:

```

```

        relationship.order_by = child_fks.copy()

def _resolve_child_fks(
    self,
    parent_keys: list[str],
    child,
) -> list[str]:
    child_fks = []

    for parent_key in parent_keys:
        if parent_key in child.fields:
            child_fks.append(parent_key)
            continue

    parent_base = self._remove_record_suffix(parent_key)

    matched = None

    for child_field_name in child.fields:
        child_base = self._remove_record_suffix(child_field_name)

        if child_base == parent_base:
            matched = child_field_name
            break

    if matched:
        child_fks.append(matched)

    if len(child_fks) == len(parent_keys):
        return child_fks

    return []

def _effective_primary_keys(
    self,
    record,
) -> list[str]:
    if hasattr(record, "effective_primary_keys"):
        keys = record.effective_primary_keys()
    else:
        keys = list(getattr(record, "primary_keys", []) or [])

    if getattr(record, "primary_key", None):
        if record.primary_key not in keys:
            keys.append(record.primary_key)

    return [
        key
        for key in keys
        if key
    ]

def _effective_parent_keys(
    self,
    relationship,
) -> list[str]:
    if hasattr(relationship, "effective_parent_keys"):
        keys = relationship.effective_parent_keys()
    else:
        keys = list(getattr(relationship, "parent_keys", []) or [])

    if getattr(relationship, "parent_key", None):
        if relationship.parent_key not in keys:
            keys.append(relationship.parent_key)

    return [
        key
        for key in keys
        if key
    ]

def _effective_child_fks(
    self,
    relationship,
) -> list[str]:
    if hasattr(relationship, "effective_child_fks"):

```

```

        keys = relationship.effective_child_fks()
    else:
        keys = list(getattr(relationship, "child_fks", []) or [])

        if getattr(relationship, "child_fk", None):
            if relationship.child_fk not in keys:
                keys.append(relationship.child_fk)

    return [
        key
        for key in keys
        if key
    ]

def _extract_key_list(
    self,
    source: dict,
    plural_keys: list[str],
    single_keys: list[str],
) -> list[str]:
    values: list[str] = []

    for key in plural_keys:
        raw_value = source.get(key)

        if raw_value is None:
            continue

        if isinstance(raw_value, list):
            for item in raw_value:
                normalized = self._normalize_name(item)

                if normalized and normalized not in values:
                    values.append(normalized)
        else:
            normalized = self._normalize_name(raw_value)

            if normalized and normalized not in values:
                values.append(normalized)

    for key in single_keys:
        raw_value = source.get(key)

        if raw_value is None:
            continue

        normalized = self._normalize_name(raw_value)

        if normalized and normalized not in values:
            values.append(normalized)

    return values

def _token_matches(
    self,
    token: str,
    value: str,
) -> bool:
    token = token.upper()
    value = value.upper()

    return token in value or value in token

def _upper_or_none(
    self,
    value: str | None,
) -> str | None:
    return value.upper() if value else None

def _normalize_name(
    self,
    value,
) -> str:
    if value is None:
        return ""

```



```

text = str(value).strip().upper()

if not text:
    return ""

text = text.replace("-", "_")
text = text.replace(" ", "_")

while "__" in text:
    text = text.replace("__", "_")

return text.strip("_")

def _remove_record_suffix(
    self,
    value: str,
) -> str:
    text = self._normalize_name(value)

    text = text.replace("-", "_")
    text = text.replace(" ", "_")

    text = text.upper()

    import re

    text = re.sub(
        r"_[0-9]{4}$",
        "",
        text,
    )

    text = re.sub(
        r"_479[A-Z0-9]+$",
        "",
        text,
    )

    return text

```

```

=====
FILE: src\idms_db2_converter\services\schema_merger.py
ENCODING: utf-8
=====

```

```

from copy import deepcopy
import re

```

```

from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import Relationship, SchemaModel

```

```

class SchemaMerger:
    """
    Merge DB2 DDL schema with IDMS schema listing.

    Rules:
    - DDL wins for physical column definitions.
    - IDMS wins for legacy field names, IDMS set names, and field_map.
    """

    def merge_physical_with_idms(
        self,
        physical_schema: SchemaModel,
        idms_schema: SchemaModel,
    ) -> SchemaModel:
        merged = deepcopy(physical_schema)

        if not getattr(merged, "schema_source", None):
            merged.schema_source = "DDL"

        merged.schema_source = f"{merged.schema_source}+IDMS_SCHEMA"

        self._build_record_table_map(merged, idms_schema)
        self._build_field_map_from_idms_to_physical(merged, idms_schema)

```

```

self._build_date_part_map_from_idms_to_physical(merged, idms_schema)
self._preserve_idms_set_names(merged, idms_schema)
self._merge_validation_messages(merged, idms_schema)

return merged

def _build_record_table_map(
    self,
    merged: SchemaModel,
    idms_schema: SchemaModel,
) -> None:
    for idms_record_name in idms_schema.records:
        physical_record_name = self._find_matching_record(
            schema=merged,
            record_name=idms_record_name,
        )

        if physical_record_name:
            merged.record_table_map[idms_record_name] = physical_record_name
        else:
            merged.record_table_map[idms_record_name] = self._table_name(
                idms_record_name
            )

def _build_field_map_from_idms_to_physical(
    self,
    merged: SchemaModel,
    idms_schema: SchemaModel,
) -> None:
    for idms_record_name in idms_schema.records:
        physical_record_name = merged.record_table_map.get(idms_record_name)

        if not physical_record_name:
            continue

        if physical_record_name not in merged.records:
            continue

        physical_record = merged.records[physical_record_name]

        for idms_field_name, meta in idms_schema.field_map.items():
            idms_host = str(meta.get("host", "")).upper()

            if not self._host_belongs_to_record(idms_host, idms_record_name):
                continue

            normalized_idms_field = self._normalize_field_name(idms_field_name)

            physical_column = self._find_physical_column(
                physical_record=physical_record,
                normalized_idms_field=normalized_idms_field,
                idms_host=idms_host,
                idms_record_name=idms_record_name,
            )

            if not physical_column:
                continue

            merged.field_map[idms_field_name.upper()] = {
                "host": Naming.hv(idms_record_name, physical_column)
            }

def _build_date_part_map_from_idms_to_physical(
    self,
    merged: SchemaModel,
    idms_schema: SchemaModel,
) -> None:
    for idms_field_name, meta in idms_schema.date_part_map.items():
        idms_host = str(meta.get("host", "")).upper()

        idms_record_name = self._find_record_for_host(
            host=idms_host,
            record_names=list(idms_schema.records.keys()),
        )

        if not idms_record_name:

```

```

        continue

    physical_record_name = merged.record_table_map.get(idms_record_name)

    if not physical_record_name:
        continue

    if physical_record_name not in merged.records:
        continue

    physical_record = merged.records[physical_record_name]

    idms_column = self._column_from_host(idms_host, idms_record_name)

    if not idms_column:
        continue

    physical_column = self._find_physical_column(
        physical_record=physical_record,
        normalized_idms_field=idms_column,
        idms_host=idms_host,
        idms_record_name=idms_record_name,
    )

    if not physical_column:
        continue

    column = physical_record.fields[physical_column]

    substring_start = meta.get("substring_start")
    substring_length = meta.get("substring_length")

    if column.datatype.upper() == "DATE":
        part = self._date_part_name(idms_field_name)

        if part == "YEAR":
            substring_start = 3
            substring_length = 2
        elif part == "MONTH":
            substring_start = 6
            substring_length = 2
        elif part == "DAY":
            substring_start = 9
            substring_length = 2

    merged.date_part_map[idms_field_name.upper()] = {
        "host": Naming.hv(idms_record_name, physical_column),
        "substring_start": substring_start,
        "substring_length": substring_length,
    }

def _preserve_idms_set_names(
    self,
    merged: SchemaModel,
    idms_schema: SchemaModel,
) -> None:
    for set_name, idms_rel in idms_schema.relationships.items():
        idms_parent = idms_rel.parent_record
        idms_child = idms_rel.child_record

        physical_parent = merged.record_table_map.get(idms_parent)
        physical_child = merged.record_table_map.get(idms_child)

        if not physical_parent or not physical_child:
            continue

        ddl_rel = self._find_relationship_by_records(
            schema=merged,
            parent_record=physical_parent,
            child_record=physical_child,
        )

        if ddl_rel:
            parent_key = ddl_rel.parent_key
            child_fk = ddl_rel.child_fk
            order_by = ddl_rel.order_by

```

```

else:
    parent_key = self._find_matching_column_name(
        record=merged.records[physical_parent],
        name=idms_rel.parent_key or "",
    )

    child_fk = self._find_matching_column_name(
        record=merged.records[physical_child],
        name=idms_rel.child_fk or idms_rel.parent_key or "",
    )

    order_by = [child_fk] if child_fk else []

if not parent_key or not child_fk:
    continue

merged.relationships[set_name] = Relationship(
    set_name=set_name,
    parent_record=idms_parent,
    child_record=idms_child,
    cardinality=idms_rel.cardinality or "1:N",
    parent_key=parent_key,
    child_fk=child_fk,
    order_by=order_by or [child_fk],
)

def _find_physical_column(
    self,
    physical_record,
    normalized_idms_field: str,
    idms_host: str,
    idms_record_name: str,
) -> str | None:
    direct = self._find_matching_column_name(
        record=physical_record,
        name=normalized_idms_field,
    )

    if direct:
        return direct

    from_host = self._column_from_host(idms_host, idms_record_name)

    if from_host:
        direct = self._find_matching_column_name(
            record=physical_record,
            name=from_host,
        )

        if direct:
            return direct

    singular = self._singularize(normalized_idms_field)

    if singular != normalized_idms_field:
        direct = self._find_matching_column_name(
            record=physical_record,
            name=singular,
        )

        if direct:
            return direct

    return self._find_flattened_occurs_column(
        physical_record=physical_record,
        name=normalized_idms_field,
    )

def _find_flattened_occurs_column(
    self,
    physical_record,
    name: str,
) -> str | None:
    candidates = [
        column_name
        for column_name in physical_record.fields

```

```

        if column_name.upper().startswith(name.upper() + "_")
    ]

    if not candidates:
        singular = self._singularize(name)
        candidates = [
            column_name
            for column_name in physical_record.fields
            if column_name.upper().startswith(singular.upper() + "_")
        ]

    if not candidates:
        return None

    return sorted(candidates)[0]

def _find_matching_record(
    self,
    schema: SchemaModel,
    record_name: str,
) -> str | None:
    record_name = record_name.upper()

    if record_name in schema.records:
        return record_name

    normalized = self._normalize_record_name(record_name)

    for candidate in schema.records:
        if self._normalize_record_name(candidate) == normalized:
            return candidate

    return None

def _find_matching_column_name(
    self,
    record,
    name: str,
) -> str | None:
    normalized = self._normalize_field_name(name)

    if normalized in record.fields:
        return normalized

    for column_name in record.fields:
        if column_name.upper() == normalized:
            return column_name

    return None

def _find_relationship_by_records(
    self,
    schema: SchemaModel,
    parent_record: str,
    child_record: str,
) -> Relationship | None:
    normalized_parent = self._normalize_record_name(parent_record)
    normalized_child = self._normalize_record_name(child_record)

    for relationship in schema.relationships.values():
        if (
            self._normalize_record_name(relationship.parent_record)
            == normalized_parent
            and self._normalize_record_name(relationship.child_record)
            == normalized_child
        ):
            return relationship

    return None

def _host_belongs_to_record(
    self,
    host: str,
    record_name: str,
) -> bool:
    normalized_host = self._normalize_token(host)

```

```

        normalized_record = self._normalize_token(record_name)

        return normalized_host.startswith(f"HV_{normalized_record}_")

def _find_record_for_host(
    self,
    host: str,
    record_names: list[str],
) -> str | None:
    normalized_host = self._normalize_token(host)

    for record_name in sorted(record_names, key=len, reverse=True):
        normalized_record = self._normalize_token(record_name)

        if normalized_host.startswith(f"HV_{normalized_record}_"):
            return record_name

    return None

def _column_from_host(
    self,
    host: str,
    record_name: str,
) -> str | None:
    normalized_host = self._normalize_token(host)
    normalized_record = self._normalize_token(record_name)

    prefix = f"HV_{normalized_record}_"

    if not normalized_host.startswith(prefix):
        return None

    return normalized_host[len(prefix):].upper()

def _date_part_name(
    self,
    idms_field_name: str,
) -> str | None:
    normalized = self._normalize_field_name(idms_field_name)

    if normalized.endswith("_YEAR"):
        return "YEAR"

    if normalized.endswith("_MONTH"):
        return "MONTH"

    if normalized.endswith("_DAY"):
        return "DAY"

    return None

def _normalize_token(
    self,
    value: str,
) -> str:
    return value.upper().replace("-", "_")

def _normalize_record_name(
    self,
    value: str,
) -> str:
    return value.upper().replace("-", "_")

def _normalize_field_name(
    self,
    value: str,
) -> str:
    value = value.upper()
    value = re.sub(r"\d{4}$", "", value)
    return value.replace("-", "_")

def _singularize(
    self,
    value: str,
) -> str:
    value = value.upper()

```

```

        if value.endswith("IES"):
            return value[:-3] + "Y"

        if value.endswith("S"):
            return value[:-1]

    return value

def _table_name(
    self,
    record_name: str,
) -> str:
    return record_name.upper().replace("-", "_")

def _merge_validation_messages(
    self,
    merged: SchemaModel,
    overlay: SchemaModel,
) -> None:
    if hasattr(merged, "validation_messages") and hasattr(
        overlay,
        "validation_messages",
    ):
        merged.validation_messages.extend(overlay.validation_messages)

```

```

=====
FILE: src\idms_db2_converter\transformers\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_converter\transformers\cobol_cleanup.py
ENCODING: utf-8
=====

```

```
import re
```

```

class CobolCleanup:
    """
    Final cleanup pass for generated COBOL.

    Goals:
    - Preserve DATA DIVISION and WORKING-STORAGE declarations.
    - Normalize all EXEC SQL blocks, including declarations before PROCEDURE DIVISION.
    - Normalize PROCEDURE DIVISION formatting.
    - Fix IF / ELSE / ELSE IF / END-IF indentation.
    - Fix PERFORM ... UNTIL continuation indentation.
    - Fix READ ... AT END indentation.
    - Fix paragraph header concatenation.
    - Avoid changing business logic.
    """

    EXEC_SQL_START = re.compile(
        r"^\s*EXEC\s+SQL\b",
        re.IGNORECASE,
    )

    EXEC_SQL_END = re.compile(
        r"^\s*END-EXEC\.\?\s*$",
        re.IGNORECASE,
    )

    PROCEDURE_DIVISION = re.compile(
        r"^\s*PROCEDURE\s+DIVISION\.",
        re.IGNORECASE | re.MULTILINE,
    )

    DATA_DIVISION = re.compile(
        r"^\s*DATA\s+DIVISION\.",
        re.IGNORECASE | re.MULTILINE,
    )

```

```

)

PARAGRAPH_HEADER = re.compile(
    r"^\s*[A-Z0-9-]+\.\s*$",
    re.IGNORECASE,
)

COMMENT_LINE = re.compile(
    r"^\s*\*",
)

DEBUG_DISPLAY_LINE = re.compile(
    r"^\s*D\s+DISPLAY\b",
    re.IGNORECASE,
)

def clean(
    self,
    text: str,
) -> str:
    if not text:
        return text

    text = self.fix_continue_concatenation(
        text,
    )

    text = self.fix_paragraph_header_concatenation(
        text,
    )

    text = self.repair_read_at_end(
        text,
    )

    text = self.remove_end_processing_sqlcode_guard(
        text,
    )

    text = self.normalize_all_exec_sql_blocks(
        text,
    )

    text = self.normalize_procedure_division(
        text,
    )

    text = self.fix_db2_check_status(
        text,
    )

    text = self.force_procedure_keyword_indentation(
        text,
    )

    text = self.normalize_perform_until(
        text,
    )

    text = self.normalize_else_if(
        text,
    )

    text = self.normalize_then_lines(
        text,
    )

    text = self.normalize_nested_move_perform_after_if(
        text,
    )

    text = self.normalize_sql_spacing(
        text,
    )

    text = self.normalize_blank_lines(

```



```

        text,
    )

    text = self.trim_trailing_spaces(
        text,
    )

    return text

def normalize_all_exec_sql_blocks(
    self,
    text: str,
) -> str:
    lines = text.splitlines()
    result = []
    in_exec_sql = False

    for line in lines:
        stripped = line.strip()

        if not stripped:
            if in_exec_sql:
                result.append("")
            else:
                result.append(line.rstrip())
                continue

        if self.EXEC_SQL_START.match(line):
            in_exec_sql = True
            result.append("        EXEC SQL")
            continue

        if in_exec_sql:
            if self.EXEC_SQL_END.match(line):
                result.append("        END-EXEC.")
                in_exec_sql = False
                continue

            result.append(
                self.normalize_sql_line(
                    stripped,
                )
            )
            continue

        result.append(
            line.rstrip()
        )

    return "\n".join(result)

def normalize_procedure_division(
    self,
    text: str,
) -> str:
    match = self.PROCEDURE_DIVISION.search(
        text,
    )

    if not match:
        return text

    before = text[: match.start()]
    procedure = text[match.start() :]

    lines = procedure.splitlines()
    result = []
    in_exec_sql = False

    for line in lines:
        stripped = line.strip()

        if not stripped:
            result.append("")
            continue

```

```

        if self.COMMENT_LINE.match(line):
            result.append(stripped)
            continue

        if self.EXEC_SQL_START.match(line):
            in_exec_sql = True
            result.append("        EXEC SQL")
            continue

        if in_exec_sql:
            if self.EXEC_SQL_END.match(line):
                result.append("        END-EXEC.")
                in_exec_sql = False
                continue

            result.append(
                self.normalize_sql_line(
                    stripped,
                )
            )
            continue

        result.append(
            self.normalize_procedure_line(
                stripped,
            )
        )

    return before + "\n".join(result)

def normalize_sql_line(
    self,
    stripped: str,
) -> str:
    upper = stripped.upper()

    if not stripped:
        return ""

    if upper == "EXEC SQL":
        return "        EXEC SQL"

    if upper in {"END-EXEC", "END-EXEC."}:
        return "        END-EXEC."

    if upper == "COMMIT":
        return "        COMMIT"

    if upper in {"(", ")"}:
        return "        " + stripped

    if stripped.startswith(":"):
        return "        " + stripped

    if re.match(
        r"^[A-Z0-9_]+,?$",
        stripped,
        flags=re.IGNORECASE,
    ):
        return "        " + stripped

    if re.match(
        r"^(DECLARE|FETCH|OPEN|CLOSE|INSERT|UPDATE|DELETE|INCLUDE|BEGIN|END)\b",
        stripped,
        flags=re.IGNORECASE,
    ):
        return "        " + stripped

    if re.match(
        r"^SELECT\b",
        stripped,
        flags=re.IGNORECASE,
    ):
        return "        " + stripped

    if re.match(

```

```

        r"^(INTO|FROM|WHERE|ORDER\s+BY|FETCH\s+FIRST|SET|VALUES)\b",
        stripped,
        flags=re.IGNORECASE,
    ):
        return "          " + stripped

    if re.match(
        r"^AND\b",
        stripped,
        flags=re.IGNORECASE,
    ):
        return "          " + stripped

    if re.match(
        r"^[A-Z0-9_]+\s*=",
        stripped,
        flags=re.IGNORECASE,
    ):
        return "          " + stripped

    return "          " + stripped

def normalize_procedure_line(
    self,
    stripped: str,
) -> str:
    upper = stripped.upper()

    if upper == "PROCEDURE DIVISION.":
        return "PROCEDURE DIVISION."

    if upper in {"END-IF", "END-IF."}:
        return "          END-IF."

    if upper in {"END-PERFORM", "END-PERFORM."}:
        return "          END-PERFORM."

    if upper == "ELSE":
        return "          ELSE"

    if upper.startswith("ELSE IF "):
        return "          " + stripped

    if upper.startswith("IF "):
        return "          " + stripped

    if upper.startswith("THEN "):
        return "          " + stripped

    if upper.startswith("PERFORM "):
        return "          " + stripped

    if upper.startswith("UNTIL "):
        return "          " + stripped

    if upper.startswith("MOVE "):
        return "          " + stripped

    if upper.startswith("READ "):
        return "          " + stripped

    if upper.startswith("AT END "):
        return "          " + stripped

    if upper.startswith("WRITE "):
        return "          " + stripped

    if upper.startswith("OPEN "):
        return "          " + stripped

    if upper.startswith("CLOSE "):
        return "          " + stripped

    if upper.startswith("DISPLAY "):
        return "          " + stripped

```

```

        if upper in {"CONTINUE", "CONTINUE."}:
            return "        CONTINUE."

        if upper in {"EXIT", "EXIT."}:
            return "        EXIT."

        if upper in {"GOBACK", "GOBACK."}:
            return "        GOBACK."

        if self.DEBUG_DISPLAY_LINE.match(stripped):
            return "        " + stripped

        if self.PARAGRAPH_HEADER.match(stripped):
            return stripped.upper()

        return "        " + stripped

def normalize_sql_spacing(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"(?im)^\s{12,}COMMIT\s*$",
        "        COMMIT",
        text,
    )

    text = re.sub(
        r"(?im)^\s{12,}VALUES\s*$",
        "        VALUES",
        text,
    )

    return text

def normalize_perform_until(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"(?im)^\s*PERFORM\s+([A-Z0-9- ]+)\s+THRU\s+([A-Z0-9- ]+)\s*\n\s*UNTIL\s+(.+)?\s*$",
        r"        PERFORM \1 THRU \2\n        UNTIL \3.",
        text,
    )

    text = re.sub(
        r"(?im)^\s*PERFORM\s+([A-Z0-9- ]+)\s*\n\s*UNTIL\s+(.+)?\s*$",
        r"        PERFORM \1\n        UNTIL \2.",
        text,
    )

    return text

def normalize_else_if(
    self,
    text: str,
) -> str:
    return re.sub(
        r"(?im)^\s*ELSE\s+IF\s+(.+)$",
        r"        ELSE IF \1",
        text,
    )

def normalize_then_lines(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"(?im)^\s*THEN\s+MOVE\s+(.+)$",
        r"        THEN MOVE \1",
        text,
    )

    text = re.sub(
        r"(?im)^\s*THEN\s+ADD\s+(.+)$",
        r"        THEN ADD \1",
        text,
    )

```

```

        text,
    )

    text = re.sub(
        r"(?im)^\s*THEN\s+PERFORM\s+(.+)$",
        r"        THEN PERFORM \1",
        text,
    )

    return text

def normalize_nested_move_perform_after_if(
    self,
    text: str,
) -> str:
    lines = text.splitlines()
    result = []

    previous_control = False

    for line in lines:
        stripped = line.strip()
        upper = stripped.upper()

        if not stripped:
            result.append(line)
            continue

        if re.match(
            r"^\s*(IF|ELSE|ELSE IF)\b",
            line,
            flags=re.IGNORECASE,
        ):
            previous_control = True
            result.append(line)
            continue

        if previous_control and re.match(
            r"^(MOVE|PERFORM|CONTINUE|DISPLAY)\b",
            upper,
            flags=re.IGNORECASE,
        ):
            result.append("        " + stripped)
            previous_control = False
            continue

        previous_control = False
        result.append(line)

    return "\n".join(result)

def fix_db2_check_status(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"(?im)^\s*DB2-CHECK-STATUS\.\s*$",
        "DB2-CHECK-STATUS.",
        text,
    )

    text = re.sub(
        r"(?im)^\s*IF\s+SQLCODE\s+NOT\s+=\s+\s*\s*$",
        "        IF SQLCODE NOT = 0",
        text,
    )

    text = re.sub(
        r"(?im)^\s*DISPLAY\s+'DB2 SQL ERROR SQLCODE='\s+SQLCODE\s*$",
        "        DISPLAY 'DB2 SQL ERROR SQLCODE=' SQLCODE",
        text,
    )

    return text

def force_procedure_keyword_indentation(

```

```

        self,
        text: str,
    ) -> str:
        match = self.PROCEDURE_DIVISION.search(
            text,
        )

        if not match:
            return text

        before = text[: match.start()]
        procedure = text[match.start() :]

        replacements = [
            (
                r"(?im)^\s*END-IF\.\?\s*$",
                "        END-IF.",
            ),
            (
                r"(?im)^\s*ELSE\s*$",
                "        ELSE",
            ),
            (
                r"(?im)^\s*GOBACK\.\?\s*$",
                "        GOBACK.",
            ),
            (
                r"(?im)^\s*EXIT\.\?\s*$",
                "        EXIT.",
            ),
            (
                r"(?im)^\s*CONTINUE\.\?\s*$",
                "        CONTINUE.",
            ),
            (
                r"(?im)^\s*PERFORM\s+SQL-ERROR\.\?\s*$",
                "        PERFORM SQL-ERROR",
            ),
        ]

        for pattern, replacement in replacements:
            procedure = re.sub(
                pattern,
                replacement,
                procedure,
            )

        return before + procedure

def repair_read_at_end(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"(?im)^\s*READ\s+([A-Z0-9-])+\s*$\n\s*AT\s+END\s+MOVE\s+(.+) \s+TO\s+([A-Z0-9-])+\s*$",
        r"        READ \1\n                AT END MOVE \2 TO \3.",
        text,
    )

    return text

def remove_end_processing_sqlcode_guard(
    self,
    text: str,
) -> str:
    pattern = re.compile(
        r"""
        ^\s*END-PROCESSING\.\s*$
        \s*IF\s+SQLCODE\s+NOT\s+=\s+\d+\s+AND\s+SQLCODE\s+NOT\s+=\s+\d+\s*$
        \s*PERFORM\s+SQL-ERROR\s*$
        \s*END-IF\.\?\s*$
        """,
        re.IGNORECASE | re.MULTILINE | re.VERBOSE,
    )

    return pattern.sub(

```

```

        "END-PROCESSING.",
        text,
    )

def fix_continue_concatenation(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"CONTINUE\.\s*([A-Z0-9-]+\.)",
        r"CONTINUE.\n1",
        text,
        flags=re.IGNORECASE,
    )

    return text

def fix_paragraph_header_concatenation(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"(\.)\s+([A-Z0-9-]+\.)",
        r"1\n2",
        text,
    )

    return text

def normalize_blank_lines(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"\n{3,}",
        "\n\n",
        text,
    )

    text = re.sub(
        r"\n\s+\n",
        "\n\n",
        text,
    )

    text = re.sub(
        r"(?m)^(END-PROCESSING\.)\n\s*\n\s*(CLOSE\b)",
        r"1\n        \2",
        text,
    )

    text = re.sub(
        r"(?m)^(SQL-ERROR\.)\n\s*\n",
        r"1\n",
        text,
    )

    text = re.sub(
        r"(?m)^(SQL-ERROR-EXIT\.)\n\s*\n",
        r"1\n",
        text,
    )

    return text

def trim_trailing_spaces(
    self,
    text: str,
) -> str:
    return re.sub(
        r"[ \t]+$",
        "",
        text,
        flags=re.MULTILINE,
    )

```

```
=====
FILE: src\idms_db2_converter\transformers\dml_transformer.py
ENCODING: utf-8
=====
```

```
import re

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import CobolAnalysis, SchemaModel
```

```
class DmlTransformer:
    """
    Converts IDMS update DML to DB2 embedded SQL.

    Supported mappings:
    - STORE record -> INSERT
    - MODIFY record -> UPDATE
    - ERASE record -> DELETE
    - READY AREA ... USAGE-MODE IS UPDATE -> CONTINUE
    - FINISH -> COMMIT for update programs

    Composite-key support:
    - UPDATE WHERE uses all effective primary keys.
    - DELETE WHERE uses all effective primary keys.
    - UPDATE excludes all primary key columns from SET list.
    - Single primary key remains fully supported.
    """

    STORE_RECORD = re.compile(
        r"^\s*STORE\s+([A-Z0-9-]+)\.?\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    MODIFY_RECORD = re.compile(
        r"^\s*MODIFY\s+([A-Z0-9-]+)\.?\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    ERASE_RECORD = re.compile(
        r"^\s*ERASE\s+([A-Z0-9-]+)\.?\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    READY_UPDATE_BLOCK = re.compile(
        r"""
        ^\s*READY\s+AREA\s+[A-Z0-9-]+\.\s*$
        (?:\n\s*USAGE-MODE\s+IS\s+UPDATE\.\s*$)?
        (?:\n\s*PERFORM\s+IDMS-STATUS\.\s*$)?
        """,
        re.IGNORECASE | re.MULTILINE | re.VERBOSE,
    )

    READY_AREA_LINE = re.compile(
        r"^\s*READY\s+AREA\s+[A-Z0-9-]+\.\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    USAGE_MODE_UPDATE_LINE = re.compile(
        r"^\s*USAGE-MODE\s+IS\s+UPDATE\.\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    FINISH_LINE = re.compile(
        r"^\s*FINISH\.\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    IDMS_STATUS_PERFORM = re.compile(
        r"^\s*PERFORM\s+IDMS-STATUS\.\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    DB2_CHECK_STATUS_PERFORM = re.compile(
```



```

        r"^\\s*PERFORM\\s+DB2-CHECK-STATUS\\.?\\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    PARAGRAPH_HEADER = re.compile(
        r"^\\s*([A-Z0-9-]+)\\.\\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    MOVE_TO_TARGET = re.compile(
        r"^\\s*MOVE\\s+\\.?\\s+TO\\s+([A-Z0-9-]+)\\.?\\s*$",
        re.IGNORECASE,
    )

    def __init__(
        self,
        schema: SchemaModel,
        analysis: CobolAnalysis,
    ) -> None:
        self.schema = schema
        self.analysis = analysis

    def has_update_dml(
        self,
    ) -> bool:
        return bool(
            getattr(self.analysis, "store_records", [])
            or getattr(self.analysis, "modify_records", [])
            or getattr(self.analysis, "erase_records", [])
            or getattr(self.analysis, "ready_update_areas", [])
        )

    def transform(
        self,
        text: str,
    ) -> str:
        text = self.replace_ready_update_blocks(
            text,
        )

        text = self.replace_store(
            text,
        )

        text = self.replace_modify(
            text,
        )

        text = self.replace_erase(
            text,
        )

        text = self.replace_finish_with_commit(
            text,
        )

        text = self.remove_idms_status_performs(
            text,
        )

        text = self.insert_db2_check_status_paragraph(
            text,
        )

        return text

    def replace_ready_update_blocks(
        self,
        text: str,
    ) -> str:
        text = self.READY_UPDATE_BLOCK.sub(
            "        CONTINUE.",
            text,
        )

        text = self.READY_AREA_LINE.sub(

```

```

        "        CONTINUE.",
        text,
    )

    text = self.USAGE_MODE_UPDATE_LINE.sub(
        "",
        text,
    )

    return text

def replace_store(
    self,
    text: str,
) -> str:
    def replacement(match: re.Match) -> str:
        record_name = match.group(1).upper()

        return self.insert_sql(
            record_name,
        )

    return self.STORE_RECORD.sub(
        replacement,
        text,
    )

def replace_modify(
    self,
    text: str,
) -> str:
    matches = list(
        self.MODIFY_RECORD.finditer(text),
    )

    if not matches:
        return text

    pieces: list[str] = []
    last_position = 0

    for match in matches:
        record_name = match.group(1).upper()

        pieces.append(
            text[last_position : match.start()],
        )

        changed_columns = self.changed_columns_before_modify(
            text=text,
            modify_position=match.start(),
            logical_record_name=record_name,
        )

        pieces.append(
            self.update_sql(
                logical_record_name=record_name,
                changed_columns=changed_columns,
            )
        )

        last_position = match.end()

    pieces.append(
        text[last_position:],
    )

    return "".join(pieces)

def replace_erase(
    self,
    text: str,
) -> str:
    def replacement(match: re.Match) -> str:
        record_name = match.group(1).upper()

```

```

        return self.delete_sql(
            record_name,
        )

    return self.ERASE_RECORD.sub(
        replacement,
        text,
    )

def replace_finish_with_commit(
    self,
    text: str,
) -> str:
    return self.FINISH_LINE.sub(
        self.commit_sql(),
        text,
    )

def remove_idms_status_performs(
    self,
    text: str,
) -> str:
    return self.IDMS_STATUS_PERFORM.sub(
        "",
        text,
    )

def insert_sql(
    self,
    logical_record_name: str,
) -> str:
    physical_record_name = self.physical_record_name(
        logical_record_name,
    )

    record = self.schema.records.get(
        physical_record_name,
    )

    if not record:
        raise ConversionError(
            f"STORE references missing record {logical_record_name}."
        )

    columns = list(record.fields.keys())

    value_items = [
        self.host_with_indicator(
            logical_record_name=logical_record_name,
            column_name=column_name,
            nullable=record.fields[column_name].nullable,
        )
        for column_name in columns
    ]

    lines: list[str] = [
        "      EXEC SQL",
        f"      INSERT INTO {physical_record_name}",
        "      (",
    ]

    lines.extend(
        self.sql_list_lines(
            items=columns,
            indent="      ",
        )
    )

    lines.extend(
        [
            "      )",
            "      VALUES",
            "      (",
        ]
    )

```

```

        lines.extend(
            self.sql_list_lines(
                items=value_items,
                indent="        ",
            )
        )

        lines.extend(
            [
                "    )",
                "    END-EXEC.",
                "    PERFORM DB2-CHECK-STATUS.",
            ]
        )

        return "\n".join(lines)

def update_sql(
    self,
    logical_record_name: str,
    changed_columns: list[str],
) -> str:
    physical_record_name = self.physical_record_name(
        logical_record_name,
    )

    record = self.schema.records.get(
        physical_record_name,
    )

    if not record:
        raise ConversionError(
            f"MODIFY references missing record {logical_record_name}."
        )

    primary_keys = self.resolve_primary_keys(
        logical_record_name=logical_record_name,
        physical_record_name=physical_record_name,
    )

    if not primary_keys:
        raise ConversionError(
            f"MODIFY record {logical_record_name} has no primary key."
        )

    primary_key_set = set(primary_keys)

    update_columns = [
        column_name
        for column_name in changed_columns
        if column_name in record.fields
        and column_name not in primary_key_set
    ]

    if not update_columns:
        update_columns = [
            column_name
            for column_name in record.fields.keys()
            if column_name not in primary_key_set
        ]

    if not update_columns:
        raise ConversionError(
            f"MODIFY record {logical_record_name} has no update columns."
        )

    set_items = []

    for column_name in update_columns:
        column = record.fields[column_name]

        host = self.host_with_indicator(
            logical_record_name=logical_record_name,
            column_name=column_name,
            nullable=column.nullable,
        )

```

```

        set_items.append(
            f"{column_name} = {host}"
        )

    where_items = self.where_items_for_primary_keys(
        logical_record_name=logical_record_name,
        primary_keys=primary_keys,
    )

    lines: list[str] = [
        "      EXEC SQL",
        f"      UPDATE {physical_record_name}",
    ]

    lines.extend(
        self.sql_assignment_lines(
            keyword="SET",
            items=set_items,
            indent="      ",
        )
    )

    lines.extend(
        self.where_lines(
            where_items=where_items,
            indent="      ",
        )
    )

    lines.extend(
        [
            "      END-EXEC.",
            "      PERFORM DB2-CHECK-STATUS.",
        ]
    )

    return "\n".join(lines)

def delete_sql(
    self,
    logical_record_name: str,
) -> str:
    physical_record_name = self.physical_record_name(
        logical_record_name,
    )

    record = self.schema.records.get(
        physical_record_name,
    )

    if not record:
        raise ConversionError(
            f"ERASE references missing record {logical_record_name}."
        )

    primary_keys = self.resolve_primary_keys(
        logical_record_name=logical_record_name,
        physical_record_name=physical_record_name,
    )

    if not primary_keys:
        raise ConversionError(
            f"ERASE record {logical_record_name} has no primary key."
        )

    where_items = self.where_items_for_primary_keys(
        logical_record_name=logical_record_name,
        primary_keys=primary_keys,
    )

    lines = [
        "      EXEC SQL",
        f"      DELETE FROM {physical_record_name}",
    ]

```

```

        lines.extend(
            self.where_lines(
                where_items=where_items,
                indent="        ",
            )
        )

        lines.extend(
            [
                "            END-EXEC.",
                "            PERFORM DB2-CHECK-STATUS.",
            ]
        )

        return "\n".join(lines)

def commit_sql(
    self,
) -> str:
    return "\n".join(
        [
            "        EXEC SQL",
            "        COMMIT",
            "        END-EXEC.",
            "        PERFORM DB2-CHECK-STATUS.",
        ]
    )

def insert_db2_check_status_paragraph(
    self,
    text: str,
) -> str:
    if re.search(
        r"^s*DB2-CHECK-STATUS\.\s*$",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    ):
        return text

    paragraph = "\n".join(
        [
            "",
            "DB2-CHECK-STATUS.",
            "    IF SQLCODE NOT = 0",
            "        DISPLAY 'DB2 SQL ERROR SQLCODE=' SQLCODE",
            "    END-IF.",
        ]
    )

    if re.search(
        r"\n\s*MAIN-LINE\.",
        text,
        flags=re.IGNORECASE,
    ):
        return re.sub(
            r"(?=\n\s*MAIN-LINE\.)",
            paragraph,
            text,
            count=1,
            flags=re.IGNORECASE,
        )

    return text.rstrip() + paragraph

def changed_columns_before_modify(
    self,
    text: str,
    modify_position: int,
    logical_record_name: str,
) -> list[str]:
    paragraph_text = self.current_paragraph_text(
        text=text,
        position=modify_position,
    )

    physical_record_name = self.physical_record_name(

```

```

        logical_record_name,
    )

    record = self.schema.records.get(
        physical_record_name,
    )

    if not record:
        return []

    changed_columns: list[str] = []
    seen: set[str] = set()

    for line in paragraph_text.splitlines():
        match = self.MOVE_TO_TARGET.match(
            line,
        )

        if not match:
            continue

        target = match.group(1).upper()

        column_name = self.column_from_move_target(
            logical_record_name=logical_record_name,
            physical_record_name=physical_record_name,
            target=target,
        )

        if not column_name:
            continue

        if column_name not in record.fields:
            continue

        if column_name in seen:
            continue

        seen.add(
            column_name,
        )

        changed_columns.append(
            column_name,
        )

    return changed_columns

def current_paragraph_text(
    self,
    text: str,
    position: int,
) -> str:
    paragraph_start = 0

    for match in self.PARAGRAPH_HEADER.finditer(
        text,
        0,
        position,
    ):
        paragraph_start = match.start()

    return text[paragraph_start:position]

def column_from_move_target(
    self,
    logical_record_name: str,
    physical_record_name: str,
    target: str,
) -> str | None:
    record = self.schema.records.get(
        physical_record_name,
    )

    if not record:
        return None

```

```

target = target.upper().rstrip(".")
target_as_column = self.normalize_column_token(
    target,
)

direct = self.find_matching_column(
    record_fields=list(record.fields.keys()),
    candidate=target_as_column,
)

if direct:
    return direct

field_map_result = self.column_from_field_map(
    logical_record_name=logical_record_name,
    target=target,
    record_fields=list(record.fields.keys()),
)

if field_map_result:
    return field_map_result

host_result = self.column_from_host_variable(
    logical_record_name=logical_record_name,
    target=target,
    record_fields=list(record.fields.keys()),
)

if host_result:
    return host_result

return None

def column_from_field_map(
    self,
    logical_record_name: str,
    target: str,
    record_fields: list[str],
) -> str | None:
    field_map = getattr(
        self.schema,
        "field_map",
        {},
    )

    meta = field_map.get(
        target.upper(),
    )

    if not meta:
        meta = field_map.get(
            target.upper().replace("-", "_"),
        )

    if not isinstance(meta, dict):
        return None

    column = meta.get(
        "column",
    )

    if column:
        matched = self.find_matching_column(
            record_fields=record_fields,
            candidate=self.normalize_column_token(str(column)),
        )

        if matched:
            return matched

    host = meta.get(
        "host",
    )

    if host:

```



```

        return self.column_from_host_variable(
            logical_record_name=logical_record_name,
            target=str(host).upper(),
            record_fields=record_fields,
        )

    return None

def column_from_host_variable(
    self,
    logical_record_name: str,
    target: str,
    record_fields: list[str],
) -> str | None:
    normalized_target = target.upper().replace("_", "-")
    normalized_record = logical_record_name.upper().replace("_", "-")
    prefix = f"HV-{normalized_record}-"

    if not normalized_target.startswith(prefix):
        return None

    column_part = normalized_target[len(prefix) :]
    candidate = self.normalize_column_token(
        column_part,
    )

    return self.find_matching_column(
        record_fields=record_fields,
        candidate=candidate,
    )

def find_matching_column(
    self,
    record_fields: list[str],
    candidate: str,
) -> str | None:
    candidate = candidate.upper()

    for field_name in record_fields:
        if field_name.upper() == candidate:
            return field_name

    candidate_hyphen = candidate.replace("_", "-")

    for field_name in record_fields:
        if field_name.upper().replace("_", "-") == candidate_hyphen:
            return field_name

    candidate_compact = self.compact_name(
        candidate,
    )

    for field_name in record_fields:
        field_compact = self.compact_name(
            field_name,
        )

        if field_compact == candidate_compact:
            return field_name

    return None

def resolve_primary_keys(
    self,
    logical_record_name: str,
    physical_record_name: str,
) -> list[str]:
    record = self.schema.records.get(
        physical_record_name,
    )

    if not record:
        return []

    record_fields = list(
        record.fields.keys(),
    )

```

```

)

calc_keys = self.primary_keys_from_calc_key_map(
    logical_record_name=logical_record_name,
    physical_record_name=physical_record_name,
    record_fields=record_fields,
)

if calc_keys:
    return calc_keys

if hasattr(record, "effective_primary_keys"):
    effective = record.effective_primary_keys()
else:
    effective = list(getattr(record, "primary_keys", []) or [])

    if getattr(record, "primary_key", None):
        if record.primary_key not in effective:
            effective.append(record.primary_key)

matched_effective = []

for key in effective:
    matched = self.find_matching_column(
        record_fields=record_fields,
        candidate=self.normalize_column_token(key),
    )

    if matched:
        matched_effective.append(
            matched,
        )

if matched_effective:
    return matched_effective

preferred_id = self.preferred_id_column(
    logical_record_name=logical_record_name,
    record_fields=record_fields,
)

if preferred_id:
    return [preferred_id]

if record_fields:
    return [record_fields[0]]

return []

def primary_keys_from_calc_key_map(
    self,
    logical_record_name: str,
    physical_record_name: str,
    record_fields: list[str],
) -> list[str]:
    calc_key_map = getattr(
        self.schema,
        "calc_key_map",
        {},
    )

    possible_keys = [
        logical_record_name.upper(),
        logical_record_name.upper().replace("-", "_"),
        physical_record_name.upper(),
        physical_record_name.upper().replace("-", "_"),
    ]

    for key in possible_keys:
        meta = calc_key_map.get(
            key,
        )

        if not isinstance(meta, dict):
            continue

```

```

raw_keys = []

if meta.get("primary_keys"):
    raw_keys.extend(
        meta.get("primary_keys") or [],
    )

for single_key_name in [
    "key",
    "primary_key",
    "column",
]:
    value = meta.get(
        single_key_name,
    )

    if value:
        raw_keys.append(
            value,
        )

result = []
seen = set()

for raw_key in raw_keys:
    matched = self.find_matching_column(
        record_fields=record_fields,
        candidate=self.normalize_column_token(str(raw_key)),
    )

    if not matched:
        continue

    if matched in seen:
        continue

    seen.add(
        matched,
    )

    result.append(
        matched,
    )

if result:
    return result

return []

def preferred_id_column(
    self,
    logical_record_name: str,
    record_fields: list[str],
) -> str | None:
    id_columns = [
        field_name
        for field_name in record_fields
        if re.search(
            r"^(^|_)ID(_|$)",
            field_name.upper(),
        )
    ]

    if not id_columns:
        return None

    logical_prefix = logical_record_name.upper().replace("-", "_").split("_")[0]

    for field_name in id_columns:
        first_token = field_name.upper().split("_")[0]

        if first_token.startswith(
            logical_prefix[:3],
        ):
            return field_name

```

```

        return id_columns[0]

def where_items_for_primary_keys(
    self,
    logical_record_name: str,
    primary_keys: list[str],
) -> list[str]:
    return [
        f"{primary_key} = :{Naming.hv(logical_record_name, primary_key)}"
        for primary_key in primary_keys
    ]

def where_lines(
    self,
    where_items: list[str],
    indent: str,
) -> list[str]:
    lines = []

    for index, item in enumerate(where_items):
        if index == 0:
            lines.append(
                f"{indent}WHERE {item}"
            )
        else:
            lines.append(
                f"{indent} AND {item}"
            )

    return lines

def sql_assignment_lines(
    self,
    keyword: str,
    items: list[str],
    indent: str,
) -> list[str]:
    lines = []

    for index, item in enumerate(items):
        suffix = "," if index < len(items) - 1 else ""

        if index == 0:
            lines.append(
                f"{indent}{keyword} {item}{suffix}"
            )
        else:
            lines.append(
                f"{indent}    {item}{suffix}"
            )

    return lines

def sql_list_lines(
    self,
    items: list[str],
    indent: str,
) -> list[str]:
    lines = []

    for index, item in enumerate(items):
        suffix = "," if index < len(items) - 1 else ""

        lines.append(
            f"{indent}{item}{suffix}"
        )

    return lines

def physical_record_name(
    self,
    logical_record_name: str,
) -> str:
    logical_record_name = logical_record_name.upper()

    mapped = self.schema.record_table_map.get(

```

```

        logical_record_name,
    )

    if mapped and mapped in self.schema.records:
        return mapped

    normalized = logical_record_name.replace(
        "-",
        "_",
    )

    mapped = self.schema.record_table_map.get(
        normalized,
    )

    if mapped and mapped in self.schema.records:
        return mapped

    if logical_record_name in self.schema.records:
        return logical_record_name

    if normalized in self.schema.records:
        return normalized

    return logical_record_name

def host_with_indicator(
    self,
    logical_record_name: str,
    column_name: str,
    nullable: bool,
) -> str:
    host = f":{Naming.hv(logical_record_name, column_name)}"

    if nullable:
        host += f" :{Naming.ni(logical_record_name, column_name)}"

    return host

def normalize_column_token(
    self,
    value: str,
) -> str:
    return (
        str(value or "")
        .upper()
        .strip()
        .rstrip(".")
        .replace("-", "_")
        .replace(" ", "_")
    )

def compact_name(
    self,
    value: str,
) -> str:
    return re.sub(
        r"^[A-Z0-9]",
        "",
        str(value or "").upper(),
    )

```

```

=====
FILE: src\idms_db2_converter\transformers\field_rewriter.py
ENCODING: utf-8
=====

```

```
import re
```

```

class FieldRewriter:
    """
    Rewrites legacy IDMS COBOL field references in PROCEDURE DIVISION to DB2
    host variables.

```

Handles cases where Phase 2 metadata field_map is incomplete because Schema Listing conversion changed physical DB2 names.

Examples:

- DEPT-ID-0410 -> HV-DEPARTMENT-DEPT-ID-479MENT
- DEPT-NAME-0410 -> HV-DEPARTMENT-DEPT-NAME-479MENT
- EMP-ID-0415 -> HV-EMPLOYEE-EMP-ID-479OYEE
- EMP-LAST-NAME-0415 -> HV-EMPLOYEE-EMP-LASTNAME-479OYEE
- START-YEAR-0415 -> HV-EMPLOYEE-DA-STARTDATE-479OYEE(3:2)
- OFFICE-ZIP-FIRST-FIVE-0450 -> HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE

Safety:

- Does not rewrite DATA DIVISION.
- Does not rewrite EXEC SQL blocks.
- Does not rewrite comments.
- Does not rewrite existing HV-* or NI-* variables.
- Does not rewrite quoted literals.
- Does not rewrite standalone paragraph headers or IDMS command lines.

""

```
PROCEDURE_DIVISION = re.compile(
    r"^\s*PROCEDURE\s+DIVISION\.",
    re.IGNORECASE | re.MULTILINE,
)
```

```
EXEC_SQL_START = re.compile(
    r"^\s*EXEC\s+SQL\b",
    re.IGNORECASE,
)
```

```
EXEC_SQL_END = re.compile(
    r"^\s*END-EXEC\.\s*$",
    re.IGNORECASE,
)
```

```
COMMENT_LINE = re.compile(
    r"^\s*\*",
)
```

```
COBOL_IDENTIFIER = re.compile(
    r"\b[A-Z][A-Z0-9]*(?:-[A-Z0-9]+)*\b",
    re.IGNORECASE,
)
```

```
STANDALONE_DOTTED_LINE = re.compile(
    r"^\s*[A-Z0-9]+\.\s*$",
    re.IGNORECASE,
)
```

```
RESERVED_WORDS = {
    "ACCEPT",
    "ADD",
    "AFTER",
    "ALL",
    "AND",
    "ARE",
    "AREA",
    "AT",
    "BEFORE",
    "BIND",
    "BY",
    "CALL",
    "CLOSE",
    "COMMIT",
    "CONNECT",
    "CONTINUE",
    "DATA",
    "DELETE",
    "DISCONNECT",
    "DISPLAY",
    "DIVISION",
    "ELSE",
    "END",
    "END-EXEC",
    "END-IF",
    "END-PERFORM",
}
```

```

"ENVIRONMENT",
"ERASE",
"EVALUATE",
"EXEC",
"EXIT",
"FETCH",
"FILE",
"FIND",
"FINISH",
"FROM",
"GET",
"GOBACK",
"IDENTIFICATION",
"IF",
"IN",
"INCLUDE",
"INTO",
"IS",
"KEEP",
"MODIFY",
"MOVE",
"NEXT",
"NOT",
"OBTAIN",
"OF",
"OPEN",
"OR",
"ORDER",
"PERFORM",
"PROCEDURE",
"READ",
"READY",
"RECORD",
"SECTION",
"SELECT",
"SET",
"SQL",
"SQLCA",
"SQLCODE",
"SPACES",
"STOP",
"STORE",
"THEN",
"THRU",
"TO",
"UNTIL",
"UPDATE",
"VALUE",
"WHEN",
"WHERE",
"WRITE",
}

DATE_PARTS = {
    "YEAR": {
        "substring_start": 3,
        "substring_length": 2,
    },
    "MONTH": {
        "substring_start": 6,
        "substring_length": 2,
    },
    "DAY": {
        "substring_start": 9,
        "substring_length": 2,
    },
}

def __init__(
    self,
    schema,
) -> None:
    self.schema = schema

    self.field_map = getattr(
        schema,

```

```

        "field_map",
        {},
    ) or {}

    self.date_part_map = getattr(
        schema,
        "date_part_map",
        {},
    ) or {}

    self.calc_key_map = getattr(
        schema,
        "calc_key_map",
        {},
    ) or {}

    self.records = getattr(
        schema,
        "records",
        {},
    ) or {}

    self.record_table_map = getattr(
        schema,
        "record_table_map",
        {},
    ) or {}

    self.suffix_record_map = self._build_suffix_record_map()
    self.host_candidates = self._build_host_candidates()
    self.rewrite_map = self._build_rewrite_map()

def rewrite(
    self,
    text: str,
) -> str:
    if not text:
        return text

    match = self.PROCEDURE_DIVISION.search(text)

    if not match:
        return text

    before_procedure = text[: match.start()]
    procedure_text = text[match.start() :]

    return before_procedure + self._rewrite_procedure_text(
        procedure_text,
    )

def _rewrite_procedure_text(
    self,
    text: str,
) -> str:
    result_lines = []
    in_exec_sql = False

    for line in text.splitlines():
        if self.EXEC_SQL_START.match(line):
            in_exec_sql = True
            result_lines.append(line)
            continue

        if in_exec_sql:
            result_lines.append(line)

            if self.EXEC_SQL_END.match(line):
                in_exec_sql = False

            continue

        if self.COMMENT_LINE.match(line):
            result_lines.append(line)
            continue

```



```

        result_lines.append(
            self._rewrite_line(line),
        )

    return "\n".join(result_lines)

def _rewrite_line(
    self,
    line: str,
) -> str:
    if not line.strip():
        return line

    if self.STANDALONE_DOTTED_LINE.match(line):
        return line

    protected_ranges = self._protected_ranges(line)

    def replace_token(match):
        token = match.group(0)

        if self._is_protected(
            start=match.start(),
            end=match.end(),
            ranges=protected_ranges,
        ):
            return token

        replacement = self._replacement_for_token(token)

        if not replacement:
            return token

        return replacement

    return self.COBOL_IDENTIFIER.sub(
        replace_token,
        line,
    )

def _replacement_for_token(
    self,
    token: str,
):
    normalized = self._normalize_cobol_name(token)

    if not normalized:
        return None

    if normalized in self.RESERVED_WORDS:
        return None

    if normalized.startswith("HV-"):
        return None

    if normalized.startswith("NI-"):
        return None

    direct = self.rewrite_map.get(normalized)

    if direct:
        return direct

    underscore_key = normalized.replace("-", "_")

    direct = self.rewrite_map.get(underscore_key)

    if direct:
        return direct

    suffix_removed = self._remove_numeric_suffix(normalized)

    if suffix_removed and suffix_removed != normalized:
        direct = self.rewrite_map.get(suffix_removed)

        if direct:

```

```

        return direct

    direct = self.rewrite_map.get(
        suffix_removed.replace("-", "_"),
    )

    if direct:
        return direct

    compact = self._compact_name(suffix_removed)

    direct = self.rewrite_map.get(compact)

    if direct:
        return direct

    inferred = self._infer_replacement_from_schema(
        token=normalized,
    )

    if inferred:
        self._add_rewrite_aliases(
            rewrite_map=self.rewrite_map,
            key=normalized,
            value=inferred,
            overwrite=True,
        )

        return inferred

    return None

def _build_rewrite_map(
    self,
):
    rewrite_map = {}

    self._merge_field_map(
        rewrite_map,
    )

    self._merge_date_part_map(
        rewrite_map,
    )

    self._merge_calc_key_map(
        rewrite_map,
    )

    self._merge_schema_fallback_map(
        rewrite_map,
    )

    return rewrite_map

def _merge_field_map(
    self,
    rewrite_map,
) -> None:
    for legacy_field, metadata in self.field_map.items():
        if not isinstance(metadata, dict):
            continue

        host = metadata.get("host")

        if not host:
            continue

        value = self._normalize_host_name(
            str(host),
        )

        if not value:
            continue

        keys = [

```

```

        str(legacy_field),
        metadata.get("legacy_field"),
        metadata.get("column"),
    ]

    for key_value in keys:
        if not key_value:
            continue

        self._add_rewrite_aliases(
            rewrite_map=rewrite_map,
            key=str(key_value),
            value=value,
            overwrite=False,
        )

def _merge_date_part_map(
    self,
    rewrite_map,
) -> None:
    for legacy_field, metadata in self.date_part_map.items():
        if not isinstance(metadata, dict):
            continue

        host = metadata.get("host")

        if not host:
            continue

        host_name = self._normalize_host_name(
            str(host),
        )

        if not host_name:
            continue

        substring_start = metadata.get("substring_start")
        substring_length = metadata.get("substring_length")

        if substring_start and substring_length:
            value = f"{host_name}({substring_start}:{substring_length})"
        else:
            value = host_name

        self._add_rewrite_aliases(
            rewrite_map=rewrite_map,
            key=str(legacy_field),
            value=value,
            overwrite=True,
        )

def _merge_calc_key_map(
    self,
    rewrite_map,
) -> None:
    for record_name, metadata in self.calc_key_map.items():
        if not isinstance(metadata, dict):
            continue

        host = metadata.get("host")
        key = (
            metadata.get("key")
            or metadata.get("primary_key")
            or metadata.get("column")
        )

        if not key:
            continue

        value = None

        if host:
            value = self._normalize_host_name(
                str(host),
            )

```

```

        if value:
            self._add_rewrite_aliases(
                rewrite_map=rewrite_map,
                key=str(key),
                value=value,
                overwrite=True,
            )

        suffix = self._extract_numeric_suffix(str(key))

        if suffix:
            record = self._normalize_db2_name(str(record_name))

            if record:
                self.suffix_record_map[suffix] = record

def _merge_schema_fallback_map(
    self,
    rewrite_map,
) -> None:
    for candidate in self.host_candidates:
        record_name = candidate["record"]
        column_name = candidate["column"]
        host = candidate["host"]

        column_base = self._remove_generated_suffix(
            self._normalize_cobol_name(column_name),
        )

        if not column_base:
            continue

        self._add_rewrite_aliases(
            rewrite_map=rewrite_map,
            key=column_base,
            value=host,
            overwrite=False,
        )

        compact_base = self._compact_name(column_base)

        if compact_base:
            rewrite_map.setdefault(
                compact_base,
                host,
            )

        record_suffix = self._suffix_for_record(record_name)

        if record_suffix:
            self._add_rewrite_aliases(
                rewrite_map=rewrite_map,
                key=f"{column_base}-{record_suffix}",
                value=host,
                overwrite=True,
            )

def _infer_replacement_from_schema(
    self,
    token: str,
):
    parsed = self._parse_legacy_token(
        token,
    )

    base = parsed["base"]
    suffix = parsed["suffix"]

    if not base:
        return None

    date_replacement = self._infer_date_part_replacement(
        base=base,
        suffix=suffix,
    )

```

```

    if date_replacement:
        return date_replacement

    candidates = self._candidate_hosts_for_base(
        base=base,
        suffix=suffix,
    )

    if not candidates:
        return None

    if len(candidates) == 1:
        selected = candidates[0]
    else:
        selected = self._choose_best_candidate(
            base=base,
            suffix=suffix,
            candidates=candidates,
        )

    if not selected:
        return None

    if suffix:
        self.suffix_record_map[suffix] = selected["record"]

    return selected["host"]

def _infer_date_part_replacement(
    self,
    base: str,
    suffix,
):
    tokens = [
        token
        for token in base.replace("_", "-").split("-")
        if token
    ]

    if len(tokens) < 2:
        return None

    part = None
    part_index = None

    for index, token in enumerate(tokens):
        resolved_part = self._date_part_type(
            token=token,
            tokens=tokens,
        )

        if resolved_part:
            part = resolved_part
            part_index = index
            break

    if not part or part_index is None:
        return None

    base_tokens = tokens[:part_index] + tokens[part_index + 1 :]
    date_tokens = tokens.copy()
    date_tokens[part_index] = "DATE"

    joined_base = "-".join(base_tokens)
    compact_base = self._compact_name(joined_base)

    candidate_bases = self._unique_values(
        [
            "-".join(date_tokens),
            joined_base,
            "DA-" + compact_base + "DATE",
            "DA-" + joined_base + "DATE",
            compact_base + "DATE",
        ]
    )

```

```

candidates = []

for candidate_base in candidate_bases:
    candidates.extend(
        self._candidate_hosts_for_base(
            base=candidate_base,
            suffix=suffix,
            date_only=True,
        )
    )

if not candidates:
    return None

if len(candidates) == 1:
    selected = candidates[0]
else:
    selected = self._choose_best_candidate(
        base=joined_base,
        suffix=suffix,
        candidates=candidates,
    )

if not selected:
    return None

if suffix:
    self.suffix_record_map[suffix] = selected["record"]

meta = self.DATE_PARTS[part]

return (
    f"{selected['host']}" +
    f"{meta['substring_start']}:{meta['substring_length']}"
)

def _candidate_hosts_for_base(
    self,
    base: str,
    suffix,
    date_only: bool = False,
):
    base_normalized = self._normalize_cobol_name(base)
    base_compact = self._compact_name(base_normalized)

    result = []
    suffix_record = self.suffix_record_map.get(suffix or "")

    for candidate in self.host_candidates:
        record_name = candidate["record"]
        column_base = candidate["base"]
        column_compact = candidate["compact"]
        datatype = candidate.get("datatype", "")

        if date_only and datatype != "DATE":
            continue

        if suffix_record and record_name != suffix_record:
            continue

        if column_base == base_normalized:
            result.append(candidate)
            continue

        if column_compact == base_compact:
            result.append(candidate)
            continue

        if base_compact and column_compact.endswith(base_compact):
            result.append(candidate)
            continue

        if base_compact and base_compact in column_compact:
            result.append(candidate)
            continue

```

```

        return self._dedupe_candidates(result)

def _choose_best_candidate(
    self,
    base: str,
    suffix,
    candidates,
):
    if not candidates:
        return None

    suffix_record = self.suffix_record_map.get(suffix or "")

    if suffix_record:
        filtered = [
            candidate
            for candidate in candidates
            if candidate["record"] == suffix_record
        ]

        if len(filtered) == 1:
            return filtered[0]

        if filtered:
            candidates = filtered

    base_tokens = [
        token
        for token in base.replace("_", "-").split("-")
        if token
    ]

    first_token = base_tokens[0] if base_tokens else ""
    base_compact = self._compact_name(base)

    scored = []

    for candidate in candidates:
        score = 0

        record_name = candidate["record"]
        record_prefix = candidate.get("record_prefix", "")
        column_base = candidate["base"]
        column_compact = candidate["compact"]

        if base_compact == column_compact:
            score += 100

        if first_token and first_token == record_prefix:
            score += 40

        if first_token and column_base.startswith(first_token + "-"):
            score += 20

        if first_token and record_name.startswith(first_token):
            score += 10

        if base_compact and column_compact.endswith(base_compact):
            score += 5

        scored.append((score, candidate))

    scored.sort(
        key=lambda item: item[0],
        reverse=True,
    )

    if not scored:
        return None

    top_score = scored[0][0]

    top = [
        candidate
        for score, candidate in scored
        if score == top_score
    ]

```

```

]

if len(top) == 1:
    return top[0]

return None

def _build_suffix_record_map(
    self,
):
    result = {}

    for key, metadata in self.field_map.items():
        if not isinstance(metadata, dict):
            continue

        suffix = self._extract_numeric_suffix(str(key))

        if not suffix:
            legacy = metadata.get("legacy_field")

            if legacy:
                suffix = self._extract_numeric_suffix(str(legacy))

        record = metadata.get("record") or metadata.get("table")

        if suffix and record:
            result[suffix] = self._normalize_db2_name(str(record))

    for key, metadata in self.date_part_map.items():
        if not isinstance(metadata, dict):
            continue

        suffix = self._extract_numeric_suffix(str(key))
        record = metadata.get("record") or metadata.get("table")

        if suffix and record:
            result[suffix] = self._normalize_db2_name(str(record))

    for record_name, metadata in self.calc_key_map.items():
        if not isinstance(metadata, dict):
            continue

        key = (
            metadata.get("key")
            or metadata.get("primary_key")
            or metadata.get("column")
        )

        suffix = self._extract_numeric_suffix(str(key or ""))

        if suffix:
            result[suffix] = self._normalize_db2_name(str(record_name))

    return result

def _build_host_candidates(
    self,
):
    candidates = []

    for logical_record_name, physical_record_name in self._logical_physical_records():
        record = self.records.get(physical_record_name)

        if not record:
            continue

        record_prefix = self._dominant_record_prefix(record)

        for column_name, column in getattr(record, "fields", {}).items():
            column_text = self._normalize_cobol_name(str(column_name))
            base = self._remove_generated_suffix(column_text)

            host = self._host_variable(
                record_name=logical_record_name,
                column_name=str(column_name),
            )

```



```

    )

    candidates.append(
        {
            "record": self._normalize_db2_name(logical_record_name),
            "physical_record": self._normalize_db2_name(physical_record_name),
            "column": column_text,
            "base": base,
            "compact": self._compact_name(base),
            "host": host,
            "datatype": str(getattr(column, "datatype", "") or "").upper(),
            "record_prefix": record_prefix,
        }
    )

    return self._dedupe_candidates(candidates)

def _logical_physical_records(
    self,
):
    pairs = []

    for logical_record_name, physical_record_name in self.record_table_map.items():
        logical = self._normalize_db2_name(str(logical_record_name))
        physical = self._normalize_db2_name(str(physical_record_name))

        if physical in self.records:
            pairs.append((logical, physical))

    for record_name in self.records.keys():
        normalized_record = self._normalize_db2_name(str(record_name))
        pairs.append((normalized_record, normalized_record))

    seen = set()
    result = []

    for pair in pairs:
        if pair in seen:
            continue

        seen.add(pair)
        result.append(pair)

    return result

def _dominant_record_prefix(
    self,
    record,
) -> str:
    counts = {}

    for column_name in getattr(record, "fields", {}).keys():
        base = self._remove_generated_suffix(
            self._normalize_cobol_name(str(column_name)),
        )

        tokens = [
            token
            for token in base.split("-")
            if token
        ]

        if not tokens:
            continue

        first = tokens[0]
        counts[first] = counts.get(first, 0) + 1

    if not counts:
        return ""

    return sorted(
        counts.items(),
        key=lambda item: item[1],
        reverse=True,
    )[0][0]

```

```

def _add_rewrite_aliases(
    self,
    rewrite_map,
    key: str,
    value: str,
    overwrite: bool,
) -> None:
    normalized = self._normalize_cobol_name(key)
    underscore = normalized.replace("-", "_")
    suffix_removed = self._remove_numeric_suffix(normalized)
    generated_removed = self._remove_generated_suffix(normalized)
    underscore_suffix_removed = suffix_removed.replace("-", "_")
    compact = self._compact_name(suffix_removed)

    aliases = [
        normalized,
        underscore,
        suffix_removed,
        generated_removed,
        underscore_suffix_removed,
        compact,
    ]

    for alias in aliases:
        if not alias:
            continue

        if overwrite:
            rewrite_map[alias] = value
        else:
            rewrite_map.setdefault(alias, value)

def _protected_ranges(
    self,
    line: str,
):
    ranges = []

    for match in re.finditer(
        r"\bHV-[A-Z0-9-]+(?:$[0-9]+:[0-9]+$)?",
        line,
        flags=re.IGNORECASE,
    ):
        ranges.append((match.start(), match.end()))

    for match in re.finditer(
        r"\bNI-[A-Z0-9-]+\b",
        line,
        flags=re.IGNORECASE,
    ):
        ranges.append((match.start(), match.end()))

    for match in re.finditer(r"'[^']*'", line):
        ranges.append((match.start(), match.end()))

    for match in re.finditer(r'"[^"]*"', line):
        ranges.append((match.start(), match.end()))

    return ranges

def _is_protected(
    self,
    start: int,
    end: int,
    ranges,
) -> bool:
    for range_start, range_end in ranges:
        if start >= range_start and end <= range_end:
            return True

    return False

def _parse_legacy_token(
    self,
    token: str,

```

```

):
    normalized = self._normalize_cobol_name(token)
    suffix = self._extract_numeric_suffix(normalized)
    base = self._remove_numeric_suffix(normalized)

    return {
        "normalized": normalized,
        "base": base,
        "suffix": suffix,
    }

def _date_part_type(
    self,
    token: str,
    tokens,
):
    token = str(token or "").upper()
    has_dy_dm_dd = "DY" in tokens and "DM" in tokens and "DD" in tokens

    if token in {"YEAR", "YR", "Y", "YY", "YYYY"}:
        return "YEAR"

    if token in {"MONTH", "MON", "MO", "M", "MM", "DM"}:
        return "MONTH"

    if token in {"DAY", "D", "DD"}:
        return "DAY"

    if token == "DY":
        if has_dy_dm_dd:
            return "YEAR"

        return "DAY"

    return None

def _suffix_for_record(
    self,
    record_name: str,
):
    normalized_record = self._normalize_db2_name(record_name)

    for suffix, mapped_record in self.suffix_record_map.items():
        if mapped_record == normalized_record:
            return suffix

    return None

def _host_variable(
    self,
    record_name: str,
    column_name: str,
) -> str:
    return (
        "HV-"
        + self._normalize_cobol_name(record_name)
        + "-"
        + self._normalize_cobol_name(column_name)
    )

def _normalize_cobol_name(
    self,
    value: str,
) -> str:
    normalized = str(value or "").strip().upper()

    if not normalized:
        return ""

    normalized = normalized.replace("_", "-")
    normalized = normalized.replace(" ", "-")
    normalized = re.sub(r"^[A-Z0-9-]", "-", normalized)
    normalized = re.sub(r"-+", "-", normalized)

    return normalized.strip("-")

```

```

def _normalize_db2_name(
    self,
    value: str,
) -> str:
    normalized = str(value or "").strip().upper()

    if not normalized:
        return ""

    normalized = normalized.replace("-", "_")
    normalized = normalized.replace(" ", "_")
    normalized = re.sub(r"^[A-Z0-9_]", "_", normalized)
    normalized = re.sub(r"_+", "_", normalized)

    return normalized.strip("_")

def _normalize_host_name(
    self,
    value: str,
) -> str:
    normalized = str(value or "").strip().upper()

    if not normalized:
        return ""

    normalized = normalized.lstrip(":").strip()
    normalized = normalized.replace("_", "-")
    normalized = normalized.replace(" ", "-")
    normalized = re.sub(r"^[A-Z0-9\-\(\):]", "-", normalized)
    normalized = re.sub(r"--+", "-", normalized)

    return normalized.strip("-")

def _remove_numeric_suffix(
    self,
    value: str,
) -> str:
    normalized = self._normalize_cobol_name(value)

    return re.sub(r"-d{4}$", "", normalized)

def _remove_generated_suffix(
    self,
    value: str,
) -> str:
    normalized = self._normalize_cobol_name(value)
    normalized = re.sub(r"-479[A-Z0-9]+$", "", normalized)
    normalized = re.sub(r"-d{4}$", "", normalized)

    return normalized

def _extract_numeric_suffix(
    self,
    value: str,
):
    normalized = self._normalize_cobol_name(value)
    match = re.search(r"-(d{4})$", normalized)

    if not match:
        return None

    return match.group(1)

def _compact_name(
    self,
    value: str,
) -> str:
    return re.sub(r"^[A-Z0-9]", "", str(value or "").upper())

def _unique_values(
    self,
    values,
):
    result = []
    seen = set()

```

```

for value in values:
    normalized = str(value or "").strip().upper()

    if not normalized:
        continue

    if normalized in seen:
        continue

    seen.add(normalized)
    result.append(normalized)

return result

def _dedupe_candidates(
    self,
    candidates,
):
    result = []
    seen = set()

    for candidate in candidates:
        key = (
            candidate.get("record", ""),
            candidate.get("column", ""),
        )

        if key in seen:
            continue

        seen.add(key)
        result.append(candidate)

    return result

```

```

=====
FILE: src\idms_db2_converter\transformers\metadata_helpers.py
ENCODING: utf-8
=====

```

```

from idms_db2_converter.generators.cobol_builder import Move
from idms_db2_converter.models import SchemaModel

```

```

class MetadataHelpers:
    def __init__(self, schema: SchemaModel):
        self.schema = schema

    def lookup_by_name(self, name: str) -> dict | None:
        for item in self.schema.output_semantics.get("lookups", []):
            if item.get("name", "").upper() == name.upper():
                return item

        return None

    def lookup_by_owner_set(self, owner_set: str) -> dict | None:
        for item in self.schema.output_semantics.get("lookups", []):
            if item.get("owner_set", "").upper() == owner_set.upper():
                return item

        return None

    def lookup_by_first_member_set(self, first_member_set: str) -> dict | None:
        for item in self.schema.output_semantics.get("lookups", []):
            if item.get("first_member_set", "").upper() == first_member_set.upper():
                return item

        return None

    def moves_to_nodes(self, moves: list[dict]) -> list[Move]:
        nodes = []

        for move in moves:
            source = self.map_source(move["from"])
            target = move["to"]

```

```

        nodes.append(Move(source=source, target=target))

    return nodes

def map_source(self, source: str) -> str:
    source_upper = source.upper()

    if source_upper in self.schema.date_part_map:
        meta = self.schema.date_part_map[source_upper]
        return f"{meta['host']}({meta['substring_start']}:{meta['substring_length']})"

    if source_upper in self.schema.field_map:
        return self.schema.field_map[source_upper].get("host", source)

    return source

=====
FILE: src\idms_db2_converter\transformers\operation_graph_transformer.py
ENCODING: utf-8
=====

import re

from idms_db2_converter.generators.cobol_builder import (
    Continue,
    ExecSql,
    IfBlock,
    Move,
    Perform,
    PerformUntil,
    PreRenderedBlock,
    ReadAtEndMove,
    Write,
    render_nodes,
    render_paragraph,
)
from idms_db2_converter.generators.fetch_paragraphs import FetchParagraphGenerator
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.generators.sql_snippets import SqlSnippets
from idms_db2_converter.models import SchemaModel
from idms_db2_converter.transformers.metadata_helpers import MetadataHelpers

class OperationGraphTransformer:
    def __init__(self, schema: SchemaModel):
        self.schema = schema
        self.sql = SqlSnippets(schema)
        self.meta = MetadataHelpers(schema)
        self.fetch_generator = FetchParagraphGenerator(schema)
        self.used_cursor_sets: set[str] = set()

    def has_graph(self) -> bool:
        return bool(self.schema.paragraph_operation_graph)

    def transform(self, text: str) -> str:
        for paragraph_name, operations in self.schema.paragraph_operation_graph.items():
            rendered = self._render_paragraph(paragraph_name, operations)
            text = self._replace_paragraph(text, paragraph_name, rendered)

        return text

    def _replace_paragraph(self, text: str, paragraph_name: str, replacement: str) -> str:
        pattern = re.compile(
            rf"^\s*{re.escape(paragraph_name)}\.\s*\n.*?(?=(^\s*[A-Z0-9-]+\.\s*$)",
            re.IGNORECASE | re.MULTILINE | re.DOTALL,
        )

        if pattern.search(text):
            return pattern.sub(replacement + "\n", text, count=1)

        return text + "\n" + replacement + "\n"

    def _render_paragraph(self, paragraph_name: str, operations: list[dict]) -> str:
        nodes = self._render_operations(operations)
        return render_paragraph(paragraph_name, nodes)

```

```

def _render_operations(self, operations: list[dict]) -> list:
    nodes = []

    for operation in operations:
        op_type = operation.get("type", "").upper()

        if op_type == "MOVE":
            nodes.append(
                Move(
                    source=self.meta.map_source(operation["from"]),
                    target=operation["to"],
                )
            )

        elif op_type == "MOVE_DATE_PART":
            nodes.append(
                Move(
                    source=self.meta.map_source(operation["from"]),
                    target=operation["to"],
                )
            )

        elif op_type == "PERFORM":
            nodes.append(Perform(paragraph=operation["paragraph"]))

        elif op_type == "WRITE":
            nodes.append(
                Write(
                    record=operation["record"],
                    source=operation.get("source"),
                )
            )

        elif op_type == "READ_NEXT_INPUT":
            at_end_move = operation.get("at_end_move", {})
            nodes.append(
                ReadAtEndMove(
                    file_name=operation["file"],
                    source=at_end_move.get("from", "Y"),
                    target=at_end_move.get("to", "EOF-SW"),
                )
            )

        elif op_type == "CONTINUE":
            nodes.append(Continue())

        elif op_type == "OBTAIN_CALC":
            nodes.append(self._node_obtain_calc(operation))

        elif op_type == "CURSOR_LOOP":
            nodes.append(self._node_cursor_loop(operation))

        elif op_type == "FIRST_MEMBER_TO_OWNER_LOOKUP":
            nodes.extend(self._nodes_first_member_to_owner_lookup(operation))

        elif op_type == "OWNER_LOOKUP":
            nodes.extend(self._nodes_owner_lookup(operation))

    return nodes

def _node_obtain_calc(self, operation: dict):
    record = operation["record"].upper()
    select_sql = self.sql.select_for_record_by_pk(record)

    not_found_nodes = self._render_operations(operation.get("not_found", []))
    success_nodes = self._render_operations(operation.get("success", []))

    lines = []
    lines.extend(ExecSql(select_sql).render(indent=7))
    lines.append("")
    lines.extend(
        IfBlock(
            condition="SQLCODE = 100",
            then_body=not_found_nodes,
            else_body=[

```

```

        IfBlock(
            condition="SQLCODE NOT = 0",
            then_body=[Perform("SQL-ERROR")],
            else_body=success_nodes,
        )
    ],
).render(indent=7, final=True)
)

return PreRenderedBlock(lines)

def _node_cursor_loop(self, operation: dict):
    set_name = operation["set"].upper()
    self.used_cursor_sets.add(set_name)

    cursor_name = Naming.cursor(set_name)
    fetch_paragraph = self.fetch_generator.paragraph_name(set_name)

    process_paragraph = operation["process_paragraph"]
    exit_paragraph = operation.get("exit_paragraph")
    empty_nodes = self._render_operations(operation.get("empty", []))
    before_loop_nodes = self._render_operations(operation.get("before_loop", []))

    perform_target = process_paragraph
    if exit_paragraph:
        perform_target = f"{{process_paragraph}} THRU {{exit_paragraph}}"

    cursor_open = ExecSql(
        "\n".join(
            [
                "EXEC SQL",
                f"      OPEN {cursor_name}",
                "END-EXEC.",
            ]
        )
    )

    cursor_close = ExecSql(
        "\n".join(
            [
                "EXEC SQL",
                f"      CLOSE {cursor_name}",
                "END-EXEC.",
            ]
        )
    )

    loop_nodes = before_loop_nodes + [
        PerformUntil(
            condition="SQLCODE = 100",
            body=[
                Perform(perform_target),
                Perform(fetch_paragraph),
            ],
        )
    ]

    return IfBlock(
        condition="SQLCODE NOT = 0",
        then_body=[Perform("SQL-ERROR")],
        else_body=[
            Perform(fetch_paragraph),
            IfBlock(
                condition="SQLCODE = 100",
                then_body=empty_nodes,
                else_body=loop_nodes,
            ),
            cursor_close,
        ],
    ).with_prefix(cursor_open)

def _nodes_first_member_to_owner_lookup(self, operation: dict) -> list:
    lookup_name = operation.get("name")
    first_member_set = operation["first_member_set"].upper()
    owner_set = operation["owner_set"].upper()

```



```

        ],
    ),
],
)
]

return [
    owner_select,
    IfBlock(
        condition="SQLCODE = 0",
        then_body=success_nodes,
        else_body=[
            IfBlock(
                condition="SQLCODE = 100",
                then_body=not_found_nodes,
                else_body=[Perform("SQL-ERROR")],
            )
        ],
    ),
]

```

```

=====
FILE: src\idms_db2_converter\transformers\paragraph_rewriter.py
ENCODING: utf-8
=====

```

```
import re
```

```

from idms_db2_converter.generators.fetch_paragraphs import FetchParagraphGenerator
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.generators.sql_snippets import SqlSnippets
from idms_db2_converter.models import SchemaModel
from idms_db2_converter.transformers.structured_blocks import StructuredBlocks

```

```
class ParagraphRewriter:
```

```
    """
```

```
    Rewrites IDMS retrieval paragraphs to DB2 SQL blocks.
```

```
    Supports:
```

```

    - OBTAIN CALC -> keyed SELECT
    - OBTAIN NEXT WITHIN set -> cursor fetch / loop
    - FIND FIRST WITHIN set -> first child SELECT
    - OBTAIN OWNER WITHIN set -> owner SELECT
    - Nullable child FK guard before owner SELECT
    """

```

```

def __init__(
    self,
    schema: SchemaModel,
):
    self.schema = schema
    self.sql = SqlSnippets(schema)
    self.fetch_generator = FetchParagraphGenerator(schema)
    self.controlled_loop_sets: set[str] = set()

```

```

def rewrite(
    self,
    text: str,
) -> str:
    text = self._rewrite_cursor_loop_blocks(text)
    text = self._rewrite_find_first_owner_chain_blocks(text)
    text = self._rewrite_empty_else_obtain_owner_blocks(text)
    text = self._rewrite_empty_else_find_first_blocks(text)
    text = self._rewrite_obtain_calc_if_blocks(text)
    text = self._rewrite_remaining_obtain_calc(text)
    text = self._rewrite_remaining_obtain_next(text)
    text = self._rewrite_remaining_find_first(text)
    text = self._rewrite_remaining_obtain_owner(text)
    text = self._rewrite_status_tokens(text)

```

```
    return text
```

```

def _rewrite_cursor_loop_blocks(
    self,

```

```

        text: str,
    ) -> str:
        for set_name in self.schema.relationships:
            text = self._rewrite_cursor_loop_for_set(text, set_name)

        return text

def _rewrite_cursor_loop_for_set(
    self,
    text: str,
    set_name: str,
) -> str:
    pattern = re.compile(
        rf"""
        IF\s+{re.escape(set_name)}\s+IS\s+NOT\s+EMPTY\s+THEN
        (?P<before_loop>.*?)
        PERFORM\s+(?P<walk>[A-Z0-9-]+\s+THRU\s+(?P<exit>[A-Z0-9-]+\s+
        \s+UNTIL\s+DB-END-OF-SET
        \s+ELSE
        (?P<empty_body>.*?)
        (?=
            \. \s* READ
            |
            \n\s*[A-Z0-9-]+\s+\.
            |
            \Z
        )
        """,
        re.IGNORECASE | re.DOTALL | re.VERBOSE,
    )

    def replace(match: re.Match) -> str:
        self.controlled_loop_sets.add(set_name.upper())

        return StructuredBlocks.cursor_loop_block(
            cursor_name=Naming.cursor(set_name),
            fetch_paragraph=self.fetch_generator.paragraph_name(set_name),
            empty_body=self._clean_block(match.group("empty_body")),
            before_loop_body=self._clean_block(match.group("before_loop")),
            walk_paragraph=match.group("walk").upper(),
            walk_exit=match.group("exit").upper(),
            indent=7,
        )

    return pattern.sub(replace, text)

def _rewrite_find_first_owner_chain_blocks(
    self,
    text: str,
) -> str:
    for first_set in self.schema.relationships:
        for owner_set in self.schema.relationships:
            text = self._rewrite_find_first_owner_chain(
                text=text,
                first_set=first_set,
                owner_set=owner_set,
            )

    return text

def _rewrite_find_first_owner_chain(
    self,
    text: str,
    first_set: str,
    owner_set: str,
) -> str:
    pattern = re.compile(
        rf"""
        IF\s+{re.escape(first_set)}\s+IS\s+EMPTY
        (?P<first_empty>.*?)
        ELSE\s+
        FIND\s+FIRST\s+WITHIN\s+{re.escape(first_set)}\s+\.?
        (?P<after_find>.*?)
        IF\s+NOT\s+{re.escape(owner_set)}\s+MEMBER
        (?P<owner_missing>.*?)
        ELSE\s+
        """
    )

```

```

OBTAIN\s+OWNER\s+WITHIN\s+{re.escape(owner_set)}\.\.?
(?P<success>.*?)
(?:=
    \n\s*IF\s+[A-Z0-9-]+\s+IS\s+EMPTY
    |
    \n\s*MOVE\s+EMP-DETAIL-LINE
    |
    \n\s*PERFORM\s+
    |
    \n\s*[A-Z0-9-]+-EXIT\.\.
    |
    \Z
)
""",
re.IGNORECASE | re.DOTALL | re.VERBOSE,
)

def replace(match: re.Match) -> str:
    first_empty = self._remove_idms_status_lines(
        self._clean_block(match.group("first_empty"))
    )

    owner_missing = self._remove_idms_status_lines(
        self._clean_block(match.group("owner_missing"))
    )

    success = self._remove_idms_status_lines(
        self._clean_block(match.group("success"))
    )

    if not success:
        success = "CONTINUE"

    return StructuredBlocks.select_first_then_owner_chain(
        first_select_sql=self.sql.select_first_child_for_set(first_set),
        owner_select_sql=self.sql.select_for_owner(owner_set),
        first_not_found_body=first_empty,
        owner_not_found_body=owner_missing,
        success_body=success,
        indent=7,
    )

return pattern.sub(replace, text)

def _rewrite_empty_else_obtain_owner_blocks(
    self,
    text: str,
) -> str:
    for set_name in self.schema.relationships:
        pattern = re.compile(
            rf"""
            IF\s+{re.escape(set_name)}\s+IS\s+EMPTY
            (?P<empty_body>.*?)
            ELSE\s+
            OBTAIN\s+OWNER\s+WITHIN\s+{re.escape(set_name)}\.\.?
            (?P<success_body>.*?)
            (?:=
                \n\s*MOVE\s+EMP-DETAIL-LINE
                |
                \n\s*PERFORM\s+
                |
                \n\s*[A-Z0-9-]+-EXIT\.\.
                |
                \Z
            )
            """,
            re.IGNORECASE | re.DOTALL | re.VERBOSE,
        )

    def replace(match: re.Match, set_name=set_name) -> str:
        empty_body = self._remove_idms_status_lines(
            self._clean_block(match.group("empty_body"))
        )

        success_body = self._remove_idms_status_lines(
            self._clean_block(match.group("success_body"))
        )

```

```

    )

    if not success_body:
        success_body = "CONTINUE"

    indicator = self.sql.nullable_fk_indicator_for_set(set_name)

    if indicator:
        return StructuredBlocks.nullable_owner_block(
            null_indicator=indicator,
            null_body=empty_body,
            owner_select_sql=self.sql.select_for_owner(set_name),
            success_body=success_body,
            fallback_body=empty_body,
            indent=7,
        )

    return StructuredBlocks.owner_select_block(
        owner_select_sql=self.sql.select_for_owner(set_name),
        success_body=success_body,
        fallback_body=empty_body,
        indent=7,
    )

    text = pattern.sub(replace, text)

    return text

def _rewrite_empty_else_find_first_blocks(
    self,
    text: str,
) -> str:
    for set_name in self.schema.relationships:
        pattern = re.compile(
            rf"""
            IF\s+{re.escape(set_name)}\s+IS\s+EMPTY
            (?P<empty_body>.*?)
            ELSE\s+
            FIND\s+FIRST\s+WITHIN\s+{re.escape(set_name)}\s+\.
            (?P<success_body>.*?)
            (?=
                \n\s*IF\s+[A-Z0-9-]+\s+IS\s+EMPTY
                |
                \n\s*PERFORM\s+
                |
                \n\s*MOVE\s+EMP-DETAIL-LINE
                |
                \n\s*[A-Z0-9-]+\s+-EXIT\
                |
                \Z
            )
            """,
            re.IGNORECASE | re.DOTALL | re.VERBOSE,
        )

    def replace(match: re.Match, set_name=set_name) -> str:
        empty_body = self._remove_idms_status_lines(
            self._clean_block(match.group("empty_body"))
        )

        success_body = self._remove_idms_status_lines(
            self._clean_block(match.group("success_body"))
        )

        if not success_body:
            success_body = "CONTINUE"

        return StructuredBlocks.sqlcode_select_block(
            sql=self.sql.select_first_child_for_set(set_name),
            not_found_body=empty_body,
            success_body=success_body,
            indent=7,
        )

    text = pattern.sub(replace, text)

```

```

        return text

def _rewrite_obtain_calc_if_blocks(
    self,
    text: str,
) -> str:
    pattern = re.compile(
        r"""
        OBTAIN\s+CALC\s+(?P<record>[A-Z0-9-]+\)\.?.?
        \s+
        IF\s+DB-REC-NOT-FOUND\s+THEN
        (?P<not_found>.*?)
        ELSE
        (?P<success>.*?)
        (?=\n\s*READ\s+|\Z)
        """,
        re.IGNORECASE | re.DOTALL | re.VERBOSE,
    )

    def replace(match: re.Match) -> str:
        record = match.group("record").upper()

        not_found = self._remove_idms_status_lines(
            self._clean_block(match.group("not_found"))
        )

        success = self._remove_idms_status_lines(
            self._clean_block(match.group("success"))
        )

        return StructuredBlocks.sqlcode_select_block(
            sql=self.sql.select_for_record_by_pk(record),
            not_found_body=not_found,
            success_body=success,
            indent=7,
        )

    return pattern.sub(replace, text)

def _rewrite_remaining_obtain_calc(
    self,
    text: str,
) -> str:
    pattern = re.compile(
        r"^\\s*OBTAIN\s+CALC\s+([A-Z0-9-]+\)\.?.?\\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    return pattern.sub(
        lambda match: self.sql.select_for_record_by_pk(match.group(1).upper()),
        text,
    )

def _rewrite_remaining_obtain_next(
    self,
    text: str,
) -> str:
    pattern = re.compile(
        r"^\\s*OBTAIN\s+NEXT\s+([A-Z0-9-]+\)\s+WITHIN\s+([A-Z0-9-]+\)\.?.?\\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    def replace(match: re.Match) -> str:
        set_name = match.group(2).upper()

        if set_name in self.controlled_loop_sets:
            return "CONTINUE."

        return f"PERFORM {self.fetch_generator.paragraph_name(set_name)}"

    return pattern.sub(replace, text)

def _rewrite_remaining_find_first(
    self,
    text: str,
) -> str:

```

```

pattern = re.compile(
    r"^s*FIND\s+FIRST\s+WITHIN\s+([A-Z0-9-]+)\.?\s*$",
    re.IGNORECASE | re.MULTILINE,
)

return pattern.sub(
    lambda match: self.sql.select_first_child_for_set(match.group(1).upper()),
    text,
)

def _rewrite_remaining_obtain_owner(
    self,
    text: str,
) -> str:
    pattern = re.compile(
        r"^s*OBTAIN\s+OWNER\s+WITHIN\s+([A-Z0-9-]+)\.?\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    def replace(match: re.Match) -> str:
        set_name = match.group(1).upper()
        indicator = self.sql.nullable_fk_indicator_for_set(set_name)
        owner_select = self.sql.select_for_owner(set_name)

        if not indicator:
            return owner_select

        return "\n".join(
            [
                f"        IF {indicator} < 0",
                "        MOVE 100 TO SQLCODE",
                "        ELSE",
                *[
                    "        " + line.strip()
                    if line.strip()
                    else ""
                    for line in owner_select.splitlines()
                ],
                "        END-IF.",
            ]
        )

    return pattern.sub(replace, text)

def _rewrite_status_tokens(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"\bDB-REC-NOT-FOUND\b",
        "SQLCODE = 100",
        text,
        flags=re.IGNORECASE,
    )

    text = re.sub(
        r"\bDB-END-OF-SET\b",
        "SQLCODE = 100",
        text,
        flags=re.IGNORECASE,
    )

    text = re.sub(
        r"IF\s+NOT\s+[A-Z0-9-]+\s+MEMBER",
        "IF SQLCODE = 100",
        text,
        flags=re.IGNORECASE,
    )

    return text

def _remove_idms_status_lines(
    self,
    text: str,
) -> str:
    text = re.sub(

```

```

        r"^\\s*PERFORM\\s+IDMS-STATUS\\.?\\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

    text = re.sub(
        r"^\\s*IF SQLCODE NOT = 0 AND SQLCODE NOT = 100\\s*"
        r"\\n\\s*PERFORM SQL-ERROR\\s*"
        r"\\n\\s*END-IF\\.?\\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

    return text.strip()

def _clean_block(
    self,
    value: str,
) -> str:
    value = value.strip()

    if value.endswith("."):
        value = value[:-1].rstrip()

    return value

```

```

=====
FILE: src\idms_db2_converter\transformers\retrieval_transformer.py
ENCODING: utf-8
=====

```

```

import re

from idms_db2_converter.generators.cursors import CursorGenerator
from idms_db2_converter.generators.fetch_paragraphs import FetchParagraphGenerator
from idms_db2_converter.generators.host_variables import HostVariableGenerator
from idms_db2_converter.models import CobolAnalysis, SchemaModel
from idms_db2_converter.transformers.cobol_cleanup import CobolCleanup
from idms_db2_converter.transformers.dml_transformer import DmlTransformer
from idms_db2_converter.transformers.field_rewriter import FieldRewriter
from idms_db2_converter.transformers.operation_graph_transformer import (
    OperationGraphTransformer,
)
from idms_db2_converter.transformers.paragraph_rewriter import ParagraphRewriter

```

```

class RetrievalTransformer:
    """
    Transforms IDMS retrieval/update COBOL into DB2 embedded SQL COBOL.

    Retrieval behavior:
    - OBTAIN CALC becomes SELECT.
    - OBTAIN NEXT set navigation becomes cursor processing.
    - FIND FIRST lookup sets do not become declared cursors.
    - OBTAIN OWNER becomes keyed SELECT.
    - Existing generated DB2 blocks are removed before regeneration.
    - Existing incomplete FETCH paragraphs are removed and regenerated.
    - INIT-BIND-READY is renamed to INIT-DB2-SETUP.
    - PERFORM IDMS-STATUS lines are removed.
    - FINISH executable lines are removed unless already converted by DML logic.

    Update behavior:
    - STORE, MODIFY, ERASE are delegated to DmlTransformer.
    - DmlTransformer is called only when update DML exists.

    Host-variable optimization:
    - Does not add every schema relationship record.
    - Adds only records actually used by COBOL DML, set navigation,
      or field references.
    """

    PROCEDURE_DIVISION = re.compile(
        r"^\\s*PROCEDURE\\s+DIVISION\\.",

```



```

    re.IGNORECASE | re.MULTILINE,
)

def __init__(
    self,
    schema: SchemaModel,
    analysis: CobolAnalysis,
) -> None:
    self.schema = schema
    self.analysis = analysis

def transform(
    self,
    cobol: str,
    target_program: str | None = None,
) -> str:
    text = cobol

    if target_program and self.analysis.program_id:
        text = self._rename_program(
            text=text,
            source=self.analysis.program_id,
            target=target_program,
        )

    text = self._remove_existing_generated_db2_block(text)
    text = self._remove_idms_control_section(text)
    text = self._remove_schema_section(text)
    text = self._remove_idms_copybooks(text)

    operation_graph_transformer = OperationGraphTransformer(
        self.schema,
    )

    if operation_graph_transformer.has_graph():
        text = operation_graph_transformer.transform(text)

        used_cursor_sets = sorted(
            operation_graph_transformer.used_cursor_sets,
        )

    else:
        paragraph_rewriter = ParagraphRewriter(
            self.schema,
        )

        text = paragraph_rewriter.rewrite(text)

        used_cursor_sets = sorted(
            paragraph_rewriter.controlled_loop_sets,
        )

    used_cursor_sets = self._ensure_cursor_sets(
        used_cursor_sets,
    )

    dml_transformer = DmlTransformer(
        schema=self.schema,
        analysis=self.analysis,
    )

    if dml_transformer.has_update_dml():
        text = dml_transformer.transform(text)

    text = self._remove_existing_fetch_paragraphs(
        text=text,
        used_cursor_sets=used_cursor_sets,
    )

    text = self._insert_sql_declarations(
        text=text,
        used_cursor_sets=used_cursor_sets,
    )

    text = self._insert_fetch_paragraphs(
        text=text,

```

```

        used_cursor_sets=used_cursor_sets,
    )

    text = FieldRewriter(
        self.schema,
    ).rewrite(text)

    text = self._replace_init_bind_ready(text)
    text = self._remove_idms_status_perform(text)
    text = self._replace_finish(text)
    text = self._remove_bind_ready_lines(text)
    text = self._clean_end_processing(text)
    text = self._remove_idms_abort_paragraphs(text)
    text = self._remove_obsolete_set_walk_guard(text)
    text = self._cleanup_orphan_sqlcode_blocks(text)
    text = self._fix_paragraph_header_concatenation(text)
    text = self._normalize_formatting(text)

    text = CobolCleanup().clean(text)

    text = self._add_sql_error_paragraph(text)

    text = CobolCleanup().clean(text)

    return text

def _rename_program(
    self,
    text: str,
    source: str,
    target: str,
) -> str:
    return re.sub(
        rf"(PROGRAM-ID\\.\\s*){re.escape(source)}(\\.\\.)?",
        rf"\\1{target}.",
        text,
        flags=re.IGNORECASE,
    )

def _remove_existing_generated_db2_block(
    self,
    text: str,
) -> str:
    pattern = re.compile(
        r"""
        \n\s*
        (?EJECT\s*)?
        \*\s*
        \n\s*\*\s*DB2\s+SQLCA,\s+HOST\s+VARIABLES,\s+AND\s+Cursors\s*
        .*?
        (?=\n\s*PROCEDURE\s+DIVISION\.)
        """,
        re.IGNORECASE | re.DOTALL | re.VERBOSE,
    )

    return pattern.sub(
        "\n",
        text,
    )

def _remove_idms_control_section(
    self,
    text: str,
) -> str:
    return re.sub(
        r"\n\s*IDMS-CONTROL\s+SECTION\.\.\.*?(?=\n\s*DATA\s+DIVISION\.)",
        "\n",
        text,
        flags=re.IGNORECASE | re.DOTALL,
    )

def _remove_schema_section(
    self,
    text: str,
) -> str:
    return re.sub(

```

```

        r"\n\s*SCHEMA\s+SECTION\..*?(?=\n\s*FILE\s+SECTION\.)",
        "\n",
        text,
        flags=re.IGNORECASE | re.DOTALL,
    )

def _remove_idms_copybooks(
    self,
    text: str,
) -> str:
    return re.sub(
        r"^\\s*(?:01\s+)?COPY\s+IDMS\s+.*?\.\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

def _ensure_cursor_sets(
    self,
    used_cursor_sets: list[str],
) -> list[str]:
    result = set(
        item.upper()
        for item in used_cursor_sets
        if item
    )

    for _, set_name in getattr(self.analysis, "obtain_next", []):
        if set_name:
            result.add(
                set_name.upper(),
            )

    return sorted(result)

def _insert_sql_declarations(
    self,
    text: str,
    used_cursor_sets: list[str],
) -> str:
    used_records = self._used_records(
        used_cursor_sets=used_cursor_sets,
    )

    host_variables = HostVariableGenerator().generate(
        self.schema,
        used_records,
    )

    cursors = CursorGenerator().generate(
        self.schema,
        used_cursor_sets,
    )

    block = "\n".join(
        [
            "",
            "*****",
            " * DB2 SQLCA, HOST VARIABLES, AND CURSORS",
            "*****",
            host_variables,
            cursors,
            "",
        ]
    )

    return re.sub(
        r"(?=\n\s*PROCEDURE\s+DIVISION\.)",
        block,
        text,
        count=1,
        flags=re.IGNORECASE,
    )

def _insert_fetch_paragraphs(
    self,

```

```

        text: str,
        used_cursor_sets: list[str],
    ) -> str:
        if not used_cursor_sets:
            return text

        fetch_paragraphs = FetchParagraphGenerator(
            self.schema,
        ).generate(
            used_cursor_sets,
        )

        if not fetch_paragraphs.strip():
            return text

        return re.sub(
            r"(?=\n\s*MAIN-LINE\.)",
            "\n" + fetch_paragraphs + "\n",
            text,
            count=1,
            flags=re.IGNORECASE,
        )

def _remove_existing_fetch_paragraphs(
    self,
    text: str,
    used_cursor_sets: list[str],
) -> str:
    if not used_cursor_sets:
        return text

    for set_name in used_cursor_sets:
        fetch_name = f"FETCH-{set_name}".upper()

        pattern = re.compile(
            rf"""
            ^\s*{re.escape(fetch_name)}\.\s*
            .*?
            (?=
                ^\s*MAIN-LINE\.\s*$
                |
                ^\s*[A-Z0-9-]+\.\s*$
                |
                \Z
            )
            """,
            re.IGNORECASE | re.MULTILINE | re.DOTALL | re.VERBOSE,
        )

        text = pattern.sub(
            "",
            text,
        )

    return text

def _replace_init_bind_ready(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"\bPERFORM\s+INIT-BIND-READY\b",
        "PERFORM INIT-DB2-SETUP",
        text,
        flags=re.IGNORECASE,
    )

    pattern = re.compile(
        rf"""
        ^\s*INIT-BIND-READY\.\s*
        .*?
        (?=
            ^\s*INIT-FILES\.\s*$
            |
            \Z
        )
        """
    )

```

```

        """
        re.IGNORECASE | re.MULTILINE | re.DOTALL | re.VERBOSE,
    )

    replacement = (
        "\n"
        "INIT-DB2-SETUP.\n"
        "    CONTINUE.\n"
    )

    return pattern.sub(
        replacement,
        text,
        count=1,
    )

def _remove_idms_status_perform(
    self,
    text: str,
) -> str:
    return re.sub(
        r"^\\s*PERFORM\\s+IDMS-STATUS\\.?.\\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

def _replace_finish(
    self,
    text: str,
) -> str:
    return re.sub(
        r"^\\s*FINISH\\.?.\\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

def _remove_bind_ready_lines(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"^\\s*BIND\\s+[A-Z0-9-]+\\.?.\\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

    text = re.sub(
        r"^\\s*READY\\.?.\\s*$",
        "",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    )

    return text

def _clean_end_processing(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"(\n\\s*END-PROCESSING\\.\\s*)\\n\\s*CONTINUE\\.\\s*",
        r"\1",
        text,
        flags=re.IGNORECASE,
    )

    return text

def _remove_idms_abort_paragraphs(
    self,
    text: str,
) -> str:
    return re.sub(

```

```

        r"\n\s*IDMS-ABORT\.\s*(?:\n\s*EXIT\.\s*)?\n\s*IDMS-ABORT-EXIT\.\s*(?:\n\s*EXIT\.\s*)?",
        "\n",
        text,
        flags=re.IGNORECASE | re.DOTALL,
    )

def _remove_obsolete_set_walk_guard(
    self,
    text: str,
) -> str:
    pattern = re.compile(
        r"""
        \n\s*CONTINUE\.\s*
        \n\s*IF\s+SQLCODE\s*=\s*100\s*
        \n\s*GO\s+TO\s+[A-Z0-9-]+-EXIT\s*
        \n\s*ELSE\s*
        """,
        re.IGNORECASE | re.VERBOSE,
    )

    return pattern.sub(
        "\n        CONTINUE.\n",
        text,
    )

def _cleanup_orphan_sqlcode_blocks(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"""
        (\n\s*CONTINUE\.\s*)
        \n\s*IF\s+SQLCODE\s+NOT\s*=\s*0\s+AND\s+SQLCODE\s+NOT\s*=\s*100\s*
        \n\s*PERFORM\s+SQL-ERROR\s*
        \n\s*END-IF\.?
        """,
        r"\1",
        text,
        flags=re.IGNORECASE | re.VERBOSE,
    )

    text = re.sub(
        r"""
        (\n\s*END-PROCESSING\.\s*)
        \s*IF\s+SQLCODE\s+NOT\s*=\s*0\s+AND\s+SQLCODE\s+NOT\s*=\s*100\s*
        \n\s*PERFORM\s+SQL-ERROR\s*
        \n\s*END-IF\.?
        """,
        r"\1",
        text,
        flags=re.IGNORECASE | re.VERBOSE,
    )

    return text

def _fix_paragraph_header_concatenation(
    self,
    text: str,
) -> str:
    text = re.sub(
        r"(\b[A-Z0-9-]+\.)\s*(IF\s+SQLCODE)",
        r"\1\n        \2",
        text,
        flags=re.IGNORECASE,
    )

    text = re.sub(
        r"(\b[A-Z0-9-]+\.)\s*(EXEC\s+SQL)",
        r"\1\n        \2",
        text,
        flags=re.IGNORECASE,
    )

    return text

def _normalize_formatting(

```

```

        self,
        text: str,
    ) -> str:
        text = re.sub(
            r"\n{3,}",
            "\n\n",
            text,
        )

        text = re.sub(
            r"[\t]+$",
            "",
            text,
            flags=re.MULTILINE,
        )

        return text

def _add_sql_error_paragraph(
    self,
    text: str,
) -> str:
    if re.search(
        r"^\\s*SQL-ERROR\\.",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    ):
        return text

    return text.rstrip() + """

SQL-ERROR.
MOVE SPACES TO ERR-LINE.
MOVE 'SQL ERROR' TO ERR-MESS-OUT.
MOVE ERR-DETAIL-LINE TO ERR-LINE.
PERFORM U200-WRITE-ERR-LINE.

SQL-ERROR-EXIT.
EXIT.
"""

def _used_records(
    self,
    used_cursor_sets: list[str] | None = None,
) -> list[str]:
    records: set[str] = set()

    for record in getattr(self.analysis, "idms_records", []):
        self._add_record(
            records=records,
            record_name=record,
        )

    for record in getattr(self.analysis, "obtain_calc_records", []):
        self._add_record(
            records=records,
            record_name=record,
        )

    for record, _ in getattr(self.analysis, "obtain_next", []):
        self._add_record(
            records=records,
            record_name=record,
        )

    for record in getattr(self.analysis, "store_records", []):
        self._add_record(
            records=records,
            record_name=record,
        )

    for record in getattr(self.analysis, "modify_records", []):
        self._add_record(
            records=records,
            record_name=record,
        )

```

```

    for record in getattr(self.analysis, "erase_records", []):
        self._add_record(
            records=records,
            record_name=record,
        )

    self._add_relationship_records_for_used_sets(
        records=records,
        used_cursor_sets=used_cursor_sets or [],
    )

    self._add_field_reference_records(
        records=records,
    )

    return sorted(records)

def _add_record(
    self,
    records: set[str],
    record_name: str | None,
) -> None:
    if not record_name:
        return

    value = str(record_name).strip().upper()

    if not value:
        return

    records.add(
        value,
    )

def _used_set_names(
    self,
    used_cursor_sets: list[str],
) -> set[str]:
    set_names: set[str] = set()

    for set_name in used_cursor_sets:
        if set_name:
            set_names.add(
                set_name.upper(),
            )

    for _, set_name in getattr(self.analysis, "obtain_next", []):
        if set_name:
            set_names.add(
                set_name.upper(),
            )

    for set_name in getattr(self.analysis, "obtain_owner_sets", []):
        if set_name:
            set_names.add(
                set_name.upper(),
            )

    for set_name in getattr(self.analysis, "find_first_sets", []):
        if set_name:
            set_names.add(
                set_name.upper(),
            )

    return set_names

def _add_relationship_records_for_used_sets(
    self,
    records: set[str],
    used_cursor_sets: list[str],
) -> None:
    used_set_names = self._used_set_names(
        used_cursor_sets=used_cursor_sets,
    )

```



```

if not used_set_names:
    return

for set_name in used_set_names:
    relationship = self.schema.relationships.get(
        set_name,
    )

    if not relationship:
        continue

    self._add_record(
        records=records,
        record_name=relationship.parent_record,
    )

    self._add_record(
        records=records,
        record_name=relationship.child_record,
    )

def _add_field_reference_records(
    self,
    records: set[str],
) -> None:
    field_references = (
        getattr(self.analysis, "field_references", [])
        or getattr(self.analysis, "idms_fields", [])
        or []
    )

    for field_name in field_references:
        for record_name in self._record_names_from_field_reference(
            field_name=field_name,
        ):
            self._add_record(
                records=records,
                record_name=record_name,
            )

def _record_names_from_field_reference(
    self,
    field_name: str,
) -> list[str]:
    result: list[str] = []

    if not field_name:
        return result

    field_map = getattr(
        self.schema,
        "field_map",
        {},
    )

    meta = field_map.get(
        field_name.upper(),
    )

    if not meta:
        return result

    record = (
        meta.get("record")
        or meta.get("table")
        or meta.get("record_name")
        or meta.get("table_name")
    )

    if record:
        result.append(
            str(record).upper(),
        )

    host = meta.get(
        "host",
    )

```

```

    )

    if host:
        host_record = self._record_name_from_host(
            str(host),
        )

        if host_record:
            result.append(
                host_record,
            )

    column_record = meta.get(
        "logical_record",
    )

    if column_record:
        result.append(
            str(column_record).upper(),
        )

    return result

def _record_name_from_host(
    self,
    host: str,
) -> str | None:
    normalized_host = self._normalize_name(
        host,
    )

    if not normalized_host.startswith("HV-"):
        return None

    candidates: list[str] = []

    candidates.extend(
        list(getattr(self.schema, "record_table_map", {}).keys()),
    )

    candidates.extend(
        list(getattr(self.schema, "records", {}).keys()),
    )

    candidates = sorted(
        set(
            str(candidate).upper()
            for candidate in candidates
            if candidate
        ),
        key=len,
        reverse=True,
    )

    for candidate in candidates:
        normalized_candidate = self._normalize_name(
            candidate,
        )

        prefix = f"HV-{normalized_candidate}-"

        if normalized_host.startswith(prefix):
            return candidate.upper()

    return None

def _normalize_name(
    self,
    value: str,
) -> str:
    return (
        str(value)
        .strip()
        .upper()
        .replace("_", "-")
        .replace(" ", "-")
    )

```

)

```
=====
FILE: src\idms_db2_converter\transformers\structured_blocks.py
ENCODING: utf-8
=====
```

```
class StructuredBlocks:
    @staticmethod
    def indent(block: str, spaces: int) -> str:
        pad = " " * spaces
        lines = []

        for line in block.splitlines():
            stripped = line.strip()
            if stripped:
                lines.append(pad + stripped)
            else:
                lines.append("")

        return "\n".join(lines)

    @staticmethod
    def sqlcode_select_block(
        sql: str,
        not_found_body: str,
        success_body: str,
        indent: int = 7,
    ) -> str:
        pad = " " * indent
        inner = " " * (indent + 3)
        inner2 = " " * (indent + 6)

        return "\n".join(
            [
                StructuredBlocks.indent(sql, indent),
                "",
                f"{pad}IF SQLCODE = 100",
                StructuredBlocks.indent(not_found_body, indent + 3),
                f"{pad}ELSE",
                f"{inner}IF SQLCODE NOT = 0",
                f"{inner2}PERFORM SQL-ERROR",
                f"{inner}ELSE",
                StructuredBlocks.indent(success_body, indent + 6),
                f"{inner}END-IF",
                f"{pad}END-IF.",
            ]
        )

    @staticmethod
    def select_first_then_owner_chain(
        first_select_sql: str,
        owner_select_sql: str,
        first_not_found_body: str,
        owner_not_found_body: str,
        success_body: str,
        indent: int = 7,
    ) -> str:
        pad = " " * indent
        inner = " " * (indent + 3)
        inner2 = " " * (indent + 6)
        inner3 = " " * (indent + 9)

        return "\n".join(
            [
                StructuredBlocks.indent(first_select_sql, indent),
                "",
                f"{pad}IF SQLCODE = 100",
                StructuredBlocks.indent(first_not_found_body, indent + 3),
                f"{pad}ELSE",
                f"{inner}IF SQLCODE NOT = 0",
                f"{inner2}PERFORM SQL-ERROR",
                f"{inner}ELSE",
                StructuredBlocks.indent(owner_select_sql, indent + 6),
                "",
            ]
        )
```

```

        f"{inner2}IF SQLCODE = 100",
        StructuredBlocks.indent(owner_not_found_body, indent + 9),
        f"{inner2}ELSE",
        f"{inner3}IF SQLCODE NOT = 0",
        f"{inner3}    PERFORM SQL-ERROR",
        f"{inner3}ELSE",
        StructuredBlocks.indent(success_body, indent + 12),
        f"{inner3}END-IF",
        f"{inner2}END-IF",
        f"{inner}END-IF",
        f"{pad}END-IF.",
    ]
)

@staticmethod
def nullable_owner_block(
    null_indicator: str,
    null_body: str,
    owner_select_sql: str,
    success_body: str,
    fallback_body: str,
    indent: int = 7,
) -> str:
    pad = " " * indent
    inner = " " * (indent + 3)

    return "\n".join(
        [
            f"{pad}IF {null_indicator} < 0",
            StructuredBlocks.indent(null_body, indent + 3),
            f"{pad}ELSE",
            StructuredBlocks.indent(owner_select_sql, indent + 3),
            "",
            f"{inner}IF SQLCODE = 0",
            StructuredBlocks.indent(success_body, indent + 6),
            f"{inner}ELSE",
            StructuredBlocks.indent(fallback_body, indent + 6),
            f"{inner}END-IF",
            f"{pad}END-IF.",
        ]
    )

@staticmethod
def owner_select_block(
    owner_select_sql: str,
    success_body: str,
    fallback_body: str,
    indent: int = 7,
) -> str:
    pad = " " * indent
    inner = " " * (indent + 3)

    return "\n".join(
        [
            StructuredBlocks.indent(owner_select_sql, indent),
            "",
            f"{pad}IF SQLCODE = 0",
            StructuredBlocks.indent(success_body, indent + 3),
            f"{pad}ELSE",
            StructuredBlocks.indent(fallback_body, indent + 3),
            f"{pad}END-IF.",
        ]
    )

@staticmethod
def cursor_loop_block(
    cursor_name: str,
    fetch_paragraph: str,
    empty_body: str,
    before_loop_body: str,
    walk_paragraph: str,
    walk_exit: str,
    indent: int = 7,
) -> str:
    pad = " " * indent
    inner = " " * (indent + 3)

```

```

inner2 = " " * (indent + 6)

return "\n".join(
    [
        f"{pad}EXEC SQL",
        f"{pad}      OPEN {cursor_name}",
        f"{pad}END-EXEC.",
        "",
        f"{pad}IF SQLCODE NOT = 0",
        f"{inner}PERFORM SQL-ERROR",
        f"{pad}ELSE",
        f"{inner}PERFORM {fetch_paragraph}",
        "",
        f"{inner}IF SQLCODE = 100",
        StructuredBlocks.indent(empty_body, indent + 6),
        f"{inner}ELSE",
        StructuredBlocks.indent(before_loop_body, indent + 6),
        "",
        f"{inner2}PERFORM UNTIL SQLCODE = 100",
        f"{inner2}  PERFORM {walk_paragraph} THRU {walk_exit}",
        f"{inner2}  PERFORM {fetch_paragraph}",
        f"{inner2}END-PERFORM",
        f"{inner}END-IF",
        "",
        f"{inner}EXEC SQL",
        f"{inner}      CLOSE {cursor_name}",
        f"{inner}END-EXEC",
        f"{pad}END-IF",
    ]
)

```

```

=====
FILE: src\idms_db2_converter\validators\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_converter\validators\output_validator.py
ENCODING: utf-8
=====

```

```
import re
```

```
class OutputValidator:
    """
```

Validates generated DB2 COBOL output.

This validator is generic:

- No application record names are hard-coded.
- No demo set names are hard-coded.
- Paragraph names like READY-AREA are not treated as IDMS READY commands.
- Multiline SQL column lists are not falsely flagged as empty parentheses.

```
"""
```

```

FORBIDDEN_EXECUTABLE_PATTERNS = [
    r"^\\s*OBTAIN\\b",
    r"^\\s*FIND\\b",
    r"^\\s*READY(?:\\s|\\.|$)",
    r"^\\s*BIND(?:\\s|\\.|$)",
    r"^\\s*FINISH\\.\\s*$",
    r"^\\s*KEEP\\b",
    r"^\\s*STORE\\b",
    r"^\\s*MODIFY\\b",
    r"^\\s*ERASE\\b",
    r"^\\s*CONNECT\\b",
    r"^\\s*DISCONNECT\\b",
    r"^\\s*GET\\b",
    r"^\\s*IF\\s+DB-[A-Z0-9-]+\\b",
    r"^\\s*PERFORM\\s+IDMS-STATUS\\.\\s*$",
    r"^\\s*USAGE-MODE\\s+IS\\s+UPDATE\\.\\s*$",

```

```
]

```

```

REQUIRED = [
    "EXEC SQL",
    "SQLCA",
    "END-EXEC",
]

EXEC_SQL_START = re.compile(
    r"^\\s*EXEC\\s+SQL\\b",
    re.IGNORECASE,
)

EXEC_SQL_END = re.compile(
    r"^\\s*END-EXEC\\.\\s*$",
    re.IGNORECASE,
)

COMMENT_LINE = re.compile(
    r"^\\s*\\*",
)

PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^\\s*[A-Z0-9-]+\\.\\s*$",
    re.IGNORECASE,
)

UNCONVERTED_IDMS_FIELD_PATTERN = re.compile(
    r"\\b[A-Z][A-Z0-9]*(?:-[A-Z0-9]+)*-\\d{4}\\b",
    re.IGNORECASE,
)

HOST_VARIABLE_PATTERN = re.compile(
    r"\\b(?:HV|NI)-[A-Z0-9-]*(?:\\d+:\\d+)?\\b",
    re.IGNORECASE,
)

def validate(
    self,
    text: str,
) -> list[str]:
    errors: list[str] = []

    executable_text = self.executable_text(
        text,
    )

    upper_executable = executable_text.upper()
    upper_all = text.upper()

    self.validate_forbidden_idms_patterns(
        upper=upper_executable,
        errors=errors,
    )

    self.validate_required_db2_tokens(
        upper=upper_all,
        errors=errors,
    )

    self.validate_no_nested_host_variables(
        upper=upper_all,
        errors=errors,
    )

    self.validate_no_rewritten_report_fields(
        upper=upper_all,
        errors=errors,
    )

    self.validate_no_unconverted_idms_schema_fields(
        text=executable_text,
        errors=errors,
    )

    self.validate_sql_blocks_closed(
        text=text,

```

```

        errors=errors,
    )

    self.validate_sql_punctuation(
        text=text,
        errors=errors,
    )

    return errors

def executable_text(
    self,
    text: str,
) -> str:
    lines: list[str] = []
    in_exec_sql = False
    in_data_division = False
    in_procedure_division = False

    for line in text.splitlines():
        stripped = line.strip()
        upper = stripped.upper()

        if not stripped:
            continue

        if self.COMMENT_LINE.match(line):
            continue

        if upper.startswith("DATA DIVISION"):
            in_data_division = True
            in_procedure_division = False
            continue

        if upper.startswith("PROCEDURE DIVISION"):
            in_data_division = False
            in_procedure_division = True
            lines.append(line)
            continue

        if self.EXEC_SQL_START.match(line):
            in_exec_sql = True
            lines.append(line)

            if self.EXEC_SQL_END.match(line):
                in_exec_sql = False

            continue

        if in_exec_sql:
            lines.append(line)

            if self.EXEC_SQL_END.match(line):
                in_exec_sql = False

            continue

        if in_data_division:
            continue

        if not in_procedure_division:
            continue

        lines.append(line)

    return "\n".join(lines)

def validate_forbidden_idms_patterns(
    self,
    upper: str,
    errors: list[str],
) -> None:
    for pattern in self.FORBIDDEN_EXECUTABLE_PATTERNS:
        if re.search(
            pattern,
            upper,

```

```

        flags=re.IGNORECASE | re.MULTILINE,
    ):
        errors.append(
            f"Forbidden IDMS executable pattern remains: {pattern}"
        )

def validate_required_db2_tokens(
    self,
    upper: str,
    errors: list[str],
) -> None:
    for token in self.REQUIRED:
        if token not in upper:
            errors.append(
                f"Required DB2 token missing: {token}"
            )

def validate_no_nested_host_variables(
    self,
    upper: str,
    errors: list[str],
) -> None:
    if re.search(
        r"\bHV-[A-Z0-9-]+-HV-[A-Z0-9-]+\b",
        upper,
    ):
        errors.append(
            "Converted output contains nested host variables like HV-...-HV-...."
        )

    if re.search(
        r"\bNI-[A-Z0-9-]+-NI-[A-Z0-9-]+\b",
        upper,
    ):
        errors.append(
            "Converted output contains nested null indicators like NI-...-NI-...."
        )

def validate_no_rewritten_report_fields(
    self,
    upper: str,
    errors: list[str],
) -> None:
    bad_patterns = [
        r"\bHV-[A-Z0-9-]+-IN\b",
        r"\bHV-[A-Z0-9-]+-OUT\b",
        r"\bNI-[A-Z0-9-]+-IN\b",
        r"\bNI-[A-Z0-9-]+-OUT\b",
    ]

    for pattern in bad_patterns:
        if re.search(
            pattern,
            upper,
        ):
            errors.append(
                "Converted output appears to have rewritten input/output report fields."
            )
    return

def validate_no_unconverted_idms_schema_fields(
    self,
    text: str,
    errors: list[str],
) -> None:
    lines = text.splitlines()
    in_exec_sql = False

    for line in lines:
        if self.EXEC_SQL_START.match(line):
            in_exec_sql = True

        if self.EXEC_SQL_END.match(line):
            in_exec_sql = False

        continue

```



```

        if in_exec_sql:
            if self.EXEC_SQL_END.match(line):
                in_exec_sql = False

            continue

        if self.COMMENT_LINE.match(line):
            continue

        if self.PARAGRAPH_HEADER_PATTERN.match(line):
            continue

        scrubbed = self.remove_host_variables(
            line,
        )

        matches = self.UNCONVERTED_IDMS_FIELD_PATTERN.findall(
            scrubbed,
        )

        for match in matches:
            upper_match = match.upper()

            if upper_match.startswith("HV-"):
                continue

            if upper_match.startswith("NI-"):
                continue

            errors.append(
                f"Unconverted IDMS schema field remains in procedure logic: {match}"
            )

    def remove_host_variables(
        self,
        line: str,
    ) -> str:
        return self.HOST_VARIABLE_PATTERN.sub(
            "",
            line,
        )

    def validate_sql_blocks_closed(
        self,
        text: str,
        errors: list[str],
    ) -> None:
        start_count = 0
        end_count = 0

        for line in text.splitlines():
            if self.EXEC_SQL_START.match(line):
                start_count += 1

            if self.EXEC_SQL_END.match(line):
                end_count += 1

        if start_count != end_count:
            errors.append(
                f"EXEC SQL / END-EXEC block mismatch: EXEC SQL={start_count}, END-EXEC={end_count}"
            )

    def validate_sql_punctuation(
        self,
        text: str,
        errors: list[str],
    ) -> None:
        sql_blocks = self.extract_sql_blocks(
            text,
        )

        for block in sql_blocks:
            self.validate_sql_commas(
                block=block,
                errors=errors,
            )

```

```

    )

def extract_sql_blocks(
    self,
    text: str,
) -> list[str]:
    blocks: list[str] = []
    current: list[str] = []
    in_exec_sql = False

    for line in text.splitlines():
        if self.EXEC_SQL_START.match(line):
            in_exec_sql = True
            current = [
                line,
            ]

            if self.EXEC_SQL_END.match(line):
                blocks.append(
                    "\n".join(current),
                )

                current = []
                in_exec_sql = False

            continue

        if in_exec_sql:
            current.append(
                line,
            )

            if self.EXEC_SQL_END.match(line):
                blocks.append(
                    "\n".join(current),
                )

                current = []
                in_exec_sql = False

    return blocks

def validate_sql_commas(
    self,
    block: str,
    errors: list[str],
) -> None:
    if re.search(
        r",\s*,",
        block,
    ):
        errors.append(
            "SQL block contains duplicate commas."
        )

    if re.search(
        r",\s*(FROM|WHERE|END-EXEC)",
        block,
        flags=re.IGNORECASE,
    ):
        errors.append(
            "SQL block contains trailing comma before FROM, WHERE, or END-EXEC."
        )

```

```

=====
FILE: src\idms_db2_converter\validators\schema_validator.py
ENCODING: utf-8
=====

```

```

import re

from idms_db2_converter.models import (
    CobolAnalysis,
    Column,
    Relationship,

```

```

    SchemaModel,
)

class SchemaValidator:
    """
    Validates and enriches schema references used by COBOL.

    Composite-key support:
    - Uses record.effective_primary_keys() when available.
    - Uses relationship.effective_parent_keys() and effective_child_fks().
    - Preserves parent_key / child_fk as first key for backward compatibility.
    - Preserves parent_keys / child_fks for composite relationships.
    - Adds missing parent/child key columns when inferred.
    """

    def validate(
        self,
        schema: SchemaModel,
        analysis: CobolAnalysis,
    ) -> list[str]:
        errors: list[str] = []

        self._ensure_primary_keys(schema)

        for record in getattr(analysis, "idms_records", []) or []:
            if record not in schema.records:
                errors.append(
                    f"COBOL references record {record}, but schema does not define it."
                )

        for record in getattr(analysis, "obtain_calc_records", []) or []:
            if record not in schema.records:
                errors.append(
                    f"OBTAIN CALC references missing record {record}."
                )
            continue

        self._infer_primary_key_for_record(
            schema=schema,
            record_name=record,
        )

        primary_keys = self._effective_primary_keys(
            schema.records[record],
        )

        if not primary_keys:
            errors.append(
                f"OBTAIN CALC record {record} has no primary key."
            )

        for primary_key in primary_keys:
            if primary_key not in schema.records[record].fields:
                errors.append(
                    f"OBTAIN CALC record {record} primary key {primary_key} is not declared as a column."
                )

        for record_name, set_name in getattr(analysis, "obtain_next", []) or []:
            self._validate_set(
                schema=schema,
                set_name=set_name,
                errors=errors,
                expected_child_record=record_name,
            )

        for set_name in getattr(analysis, "obtain_owner_sets", []) or []:
            self._validate_set(
                schema=schema,
                set_name=set_name,
                errors=errors,
                expected_child_record=None,
            )

        for set_name in getattr(analysis, "find_first_sets", []) or []:
            self._validate_set(

```

```

        schema=schema,
        set_name=set_name,
        errors=errors,
        expected_child_record=None,
    )

    return errors

def _ensure_primary_keys(
    self,
    schema: SchemaModel,
) -> None:
    for record_name in list(schema.records.keys()):
        self._infer_primary_key_for_record(
            schema=schema,
            record_name=record_name,
        )

def _infer_primary_key_for_record(
    self,
    schema: SchemaModel,
    record_name: str,
) -> None:
    if record_name not in schema.records:
        return

    record = schema.records[record_name]

    primary_keys = self._effective_primary_keys(record)

    if primary_keys:
        cleaned = []

        for key in primary_keys:
            if key not in record.fields:
                continue

            if key in cleaned:
                continue

            cleaned.append(key)
            record.fields[key].nullable = False
            record.fields[key].primary_key = True

        if cleaned:
            self._set_primary_keys(
                record=record,
                keys=cleaned,
            )
            return

    candidate = self._find_primary_key_candidate(
        record_name=record_name,
        fields=set(record.fields.keys()),
    )

    if candidate:
        self._set_primary_keys(
            record=record,
            keys=[candidate],
        )
        return

    if record.fields:
        first_field = next(iter(record.fields.keys()))
        self._set_primary_keys(
            record=record,
            keys=[first_field],
        )

def _find_primary_key_candidate(
    self,
    record_name: str,
    fields: set[str],
) -> str | None:
    normalized_record = self._normalize(record_name)

```

```

compact_record = self._compact(normalized_record)

direct_candidates = [
    f"{normalized_record}_ID",
    f"{normalized_record}_CODE",
]

for candidate in direct_candidates:
    if candidate in fields:
        return candidate

for field in fields:
    normalized_field = self._normalize(field)
    compact_field = self._compact(normalized_field)

    if normalized_field.endswith("_ID") and compact_record in compact_field:
        return field

    if normalized_field.endswith("_CODE") and compact_record in compact_field:
        return field

id_fields = [
    field
    for field in fields
    if self._normalize(field).endswith("_ID")
]

if len(id_fields) == 1:
    return id_fields[0]

code_fields = [
    field
    for field in fields
    if self._normalize(field).endswith("_CODE")
]

if len(code_fields) == 1:
    return code_fields[0]

return None

def _validate_set(
    self,
    schema: SchemaModel,
    set_name: str,
    errors: list[str],
    expected_child_record: str | None,
) -> None:
    set_name = set_name.upper()

    if set_name not in schema.relationships:
        inferred = self._infer_relationship(
            schema=schema,
            set_name=set_name,
            expected_child_record=expected_child_record,
        )

        if inferred:
            schema.relationships[set_name] = inferred
        else:
            errors.append(
                f"COBOL references set {set_name}, but schema relationships do not define it."
            )
            return

    rel = schema.relationships[set_name]

    if rel.parent_record not in schema.records:
        errors.append(
            f"Set {set_name} parent record {rel.parent_record} is missing."
        )
        return

    if rel.child_record not in schema.records:
        errors.append(
            f"Set {set_name} child record {rel.child_record} is missing."

```

```

    )
    return

parent_record = schema.records[rel.parent_record]
child_record = schema.records[rel.child_record]

self._infer_primary_key_for_record(
    schema=schema,
    record_name=rel.parent_record,
)

parent_keys = self._effective_parent_keys(rel)

if not parent_keys:
    parent_keys = self._effective_primary_keys(parent_record)

if not parent_keys:
    errors.append(
        f"Set {set_name} has no parent_key or parent_keys."
    )
    return

child_fks = self._effective_child_fks(rel)

if not child_fks:
    child_fks = self._infer_child_fks(
        schema=schema,
        parent_record=rel.parent_record,
        child_record=rel.child_record,
        parent_keys=parent_keys,
    )

if not child_fks:
    child_fks = parent_keys.copy()

if len(parent_keys) != len(child_fks):
    errors.append(
        f"Set {set_name} has mismatched composite key counts: "
        f"{len(parent_keys)} parent key(s), {len(child_fks)} child FK(s)."
    )
    return

self._set_relationship_keys(
    relationship=rel,
    parent_keys=parent_keys,
    child_fks=child_fks,
)

for parent_key in parent_keys:
    if parent_key not in parent_record.fields:
        self._add_field_like_parent_key(
            schema=schema,
            target_record=rel.parent_record,
            source_record=rel.parent_record,
            source_key=parent_key,
            target_field=parent_key,
            nullable=False,
        )

for parent_key, child_fk in zip(parent_keys, child_fks):
    if child_fk not in child_record.fields:
        self._add_field_like_parent_key(
            schema=schema,
            target_record=rel.child_record,
            source_record=rel.parent_record,
            source_key=parent_key,
            target_field=child_fk,
            nullable=True,
        )

if not rel.order_by:
    rel.order_by = child_fks.copy()

def _infer_relationship(
    self,
    schema: SchemaModel,

```

```

        set_name: str,
        expected_child_record: str | None,
    ) -> Relationship | None:
        child_record = self._infer_child_record(
            schema=schema,
            set_name=set_name,
            expected_child_record=expected_child_record,
        )

        if not child_record:
            return None

        parent_record = self._infer_parent_record(
            schema=schema,
            set_name=set_name,
            child_record=child_record,
        )

        if not parent_record:
            return None

        self._infer_primary_key_for_record(
            schema=schema,
            record_name=parent_record,
        )

        parent = schema.records[parent_record]
        parent_keys = self._effective_primary_keys(parent)

        if not parent_keys:
            return None

        child_fks = self._infer_child_fks(
            schema=schema,
            parent_record=parent_record,
            child_record=child_record,
            parent_keys=parent_keys,
        )

        if not child_fks:
            child_fks = parent_keys.copy()

        if len(parent_keys) != len(child_fks):
            return None

        for parent_key, child_fk in zip(parent_keys, child_fks):
            if child_fk not in schema.records[child_record].fields:
                self._add_field_like_parent_key(
                    schema=schema,
                    target_record=child_record,
                    source_record=parent_record,
                    source_key=parent_key,
                    target_field=child_fk,
                    nullable=True,
                )

        return Relationship(
            set_name=set_name,
            parent_record=parent_record,
            child_record=child_record,
            cardinality="1:N",
            parent_key=parent_keys[0] if parent_keys else None,
            child_fk=child_fks[0] if child_fks else None,
            parent_keys=parent_keys,
            child_fks=child_fks,
            order_by=child_fks.copy(),
        )

    def _infer_child_record(
        self,
        schema: SchemaModel,
        set_name: str,
        expected_child_record: str | None,
    ) -> str | None:
        if expected_child_record and expected_child_record in schema.records:
            return expected_child_record

```

```

set_tokens = self._tokens(set_name)

best_score = 0
best_record = None

for record_name in schema.records:
    score = self._score_record_against_tokens(
        record_name,
        set_tokens,
    )

    if score > best_score:
        best_score = score
        best_record = record_name

return best_record if best_score > 0 else None

def _infer_parent_record(
    self,
    schema: SchemaModel,
    set_name: str,
    child_record: str,
) -> str | None:
    set_tokens = self._tokens(set_name)

    best_score = 0
    best_record = None

    for record_name in schema.records:
        if record_name == child_record:
            continue

        score = self._score_record_against_tokens(
            record_name,
            set_tokens,
        )

        if score > best_score:
            best_score = score
            best_record = record_name

    return best_record if best_score > 0 else None

def _infer_child_fks(
    self,
    schema: SchemaModel,
    parent_record: str,
    child_record: str,
    parent_keys: list[str],
) -> list[str]:
    result: list[str] = []
    child = schema.records[child_record]

    for parent_key in parent_keys:
        child_fk = self._infer_child_fk(
            schema=schema,
            parent_record=parent_record,
            child_record=child_record,
            parent_key=parent_key,
        )

        if child_fk:
            result.append(child_fk)

    if len(result) == len(parent_keys):
        return result

    parent_base_keys = {
        self._remove_record_suffix(parent_key): parent_key
        for parent_key in parent_keys
        if parent_key
    }

    result = []

```



```

for parent_base_key, parent_key in parent_base_keys.items():
    matched_child_fk = None

    for child_field_name in child.fields:
        child_base_name = self._remove_record_suffix(child_field_name)

        if child_base_name == parent_base_key:
            matched_child_fk = child_field_name
            break

    if matched_child_fk:
        result.append(matched_child_fk)

if len(result) == len(parent_keys):
    return result

return []

def _infer_child_fk(
    self,
    schema: SchemaModel,
    parent_record: str,
    child_record: str,
    parent_key: str | None,
) -> str | None:
    if not parent_key:
        return None

    child = schema.records[child_record]
    parent_key_norm = self._normalize(parent_key)

    if parent_key in child.fields:
        return parent_key

    if parent_key_norm in child.fields:
        return parent_key_norm

    parent_tokens = self._tokens(parent_record)
    key_suffixes = ["_ID", "_CODE"]

    for field_name in child.fields:
        normalized_field = self._normalize(field_name)

        if normalized_field == parent_key_norm:
            return field_name

        for token in parent_tokens:
            if normalized_field.startswith(token + "_"):
                for suffix in key_suffixes:
                    if normalized_field.endswith(suffix):
                        return field_name

    parent_compact = self._compact(parent_record)

    for field_name in child.fields:
        compact_field = self._compact(field_name)

        if parent_compact and parent_compact in compact_field:
            if self._compact(parent_key) in compact_field:
                return field_name

    return None

def _add_field_like_parent_key(
    self,
    schema: SchemaModel,
    target_record: str,
    source_record: str,
    source_key: str,
    target_field: str,
    nullable: bool,
) -> None:
    if target_record not in schema.records:
        return

    if source_record not in schema.records:

```

```

        return

    source = schema.records[source_record]
    target = schema.records[target_record]

    source_column = source.fields.get(source_key)

    target.fields[target_field] = Column(
        name=target_field,
        datatype=source_column.datatype if source_column else "CHAR",
        length=source_column.length if source_column else 20,
        scale=source_column.scale if source_column else None,
        nullable=nullable,
        primary_key=False,
    )

def _effective_primary_keys(
    self,
    record,
) -> list[str]:
    if hasattr(record, "effective_primary_keys"):
        keys = record.effective_primary_keys()
    else:
        keys = list(getattr(record, "primary_keys", []) or [])

        if getattr(record, "primary_key", None):
            if record.primary_key not in keys:
                keys.append(record.primary_key)

    return [
        key
        for key in keys
        if key
    ]

def _effective_parent_keys(
    self,
    relationship,
) -> list[str]:
    if hasattr(relationship, "effective_parent_keys"):
        keys = relationship.effective_parent_keys()
    else:
        keys = list(getattr(relationship, "parent_keys", []) or [])

        if getattr(relationship, "parent_key", None):
            if relationship.parent_key not in keys:
                keys.append(relationship.parent_key)

    return [
        key
        for key in keys
        if key
    ]

def _effective_child_fks(
    self,
    relationship,
) -> list[str]:
    if hasattr(relationship, "effective_child_fks"):
        keys = relationship.effective_child_fks()
    else:
        keys = list(getattr(relationship, "child_fks", []) or [])

        if getattr(relationship, "child_fk", None):
            if relationship.child_fk not in keys:
                keys.append(relationship.child_fk)

    return [
        key
        for key in keys
        if key
    ]

def _set_primary_keys(
    self,
    record,

```

```

        keys: list[str],
    ) -> None:
        cleaned: list[str] = []

        for key in keys or []:
            if not key:
                continue

            normalized = str(key).upper()

            if normalized in cleaned:
                continue

            cleaned.append(normalized)

        if hasattr(record, "set_primary_keys"):
            record.set_primary_keys(cleaned)
        else:
            record.primary_keys = cleaned
            record.primary_key = cleaned[0] if cleaned else None

            for key in cleaned:
                if key in record.fields:
                    record.fields[key].primary_key = True
                    record.fields[key].nullable = False

        for key in cleaned:
            if key in record.fields:
                record.fields[key].primary_key = True
                record.fields[key].nullable = False

def _set_relationship_keys(
    self,
    relationship,
    parent_keys: list[str],
    child_fks: list[str],
) -> None:
    relationship.parent_keys = parent_keys.copy()
    relationship.child_fks = child_fks.copy()
    relationship.parent_key = parent_keys[0] if parent_keys else None
    relationship.child_fk = child_fks[0] if child_fks else None

def _score_record_against_tokens(
    self,
    record_name: str,
    tokens: list[str],
) -> int:
    score = 0
    normalized_record = self._normalize(record_name)
    compact_record = self._compact(record_name)

    for token in tokens:
        normalized_token = self._normalize(token)
        compact_token = self._compact(token)

        if normalized_token == normalized_record:
            score += 5
            continue

        if normalized_token in normalized_record:
            score += 2

        if compact_token and compact_token in compact_record:
            score += 2

    return score

def _tokens(
    self,
    value: str,
) -> list[str]:
    normalized = self._normalize(value)

    return [
        token
        for token in normalized.split("_")
    ]

```

```

        if token
    ]

def _normalize(
    self,
    value: str,
) -> str:
    return str(value or "").upper().replace("-", "_").replace(" ", "_")

def _compact(
    self,
    value: str,
) -> str:
    return re.sub(
        r"^[^A-Z0-9]",
        "",
        self._normalize(value),
    )

def _remove_record_suffix(
    self,
    value: str,
) -> str:
    normalized = self._normalize(value)

    normalized = re.sub(
        r"_?[0-9]{4}$",
        "",
        normalized,
    )

    normalized = re.sub(
        r"_479[A-Z0-9]+$",
        "",
        normalized,
    )

    return normalized

```

```

=====
FILE: src\idms_modernizer\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_modernizer\app.py
ENCODING: utf-8
=====

```

```

from pathlib import Path
from time import perf_counter

```

```

import streamlit as st

```

```

from idms_modernizer.core.config import settings
from idms_modernizer.core.utils import ensure_directory
from idms_modernizer.core.constants import (
    ER_DIAGRAM_FILE,
    PHASE2_METADATA_FILE,
)
from idms_modernizer.parsers.cobol_pdf_extractor import CobolPdfExtractor
from idms_modernizer.services.metadata_service import MetadataService
from idms_modernizer.services.canonical_schema_builder import (
    CanonicalSchemaBuilder,
)
from idms_modernizer.services.canonical_reconciler import (
    CanonicalReconciler,
)
from idms_modernizer.services.db2_model_builder import DB2ModelBuilder
from idms_modernizer.services.phase2_metadata_generator import (
    Phase2MetadataGenerator,
)

```

```

from idms_modernizer.services.phase2_conversion_bridge import (
    Phase2ConversionBridge,
)
from idms_modernizer.services.er_diagram_generator import (
    generate_er_diagram,
)
from idms_modernizer.services.excel_sheet_mapping_service import (
    ExcelSheetMappingService,
)
from idms_modernizer.generators.ddl_generator import DDLGenerator

```

```

def initialize_directories() -> None:
    ensure_directory(
        settings.input_dir,
    )

    ensure_directory(
        settings.output_dir,
    )

```

```

def initialize_session_state() -> None:
    defaults = {
        "generated": False,
        "schema_path": "",
        "cobol_pdf_path": "",
        "metadata": None,
        "canonical_schema": None,
        "db2_model": None,
        "ddl_text": "",
        "canonical_json": "",
        "phase2_metadata_json": "",
        "cobol_text": "",
        "converted_cobol": "",
        "relationships": [],
        "all_sets": [],
        "record_count": 0,
        "set_count": 0,
        "relationship_count": 0,
        "validation_messages": [],
        "generation_warnings": [],
        "generation_timings": [],
        "er_png_path": "",
        "er_image_bytes": None,
        "output_files": {},
        "excel_mapping_rows": [],
    }

    for key, value in defaults.items():
        if key not in st.session_state:
            st.session_state[key] = value

```

```

def reset_outputs() -> None:
    st.session_state.generated = False
    st.session_state.schema_path = ""
    st.session_state.cobol_pdf_path = ""
    st.session_state.metadata = None
    st.session_state.canonical_schema = None
    st.session_state.db2_model = None
    st.session_state.ddl_text = ""
    st.session_state.canonical_json = ""
    st.session_state.phase2_metadata_json = ""
    st.session_state.cobol_text = ""
    st.session_state.converted_cobol = ""
    st.session_state.relationships = []
    st.session_state.all_sets = []
    st.session_state.record_count = 0
    st.session_state.set_count = 0
    st.session_state.relationship_count = 0
    st.session_state.validation_messages = []
    st.session_state.generation_warnings = []
    st.session_state.generation_timings = []
    st.session_state.er_png_path = ""
    st.session_state.er_image_bytes = None

```

```

st.session_state.output_files = {}
st.session_state.excel_mapping_rows = []

def save_uploaded_file(
    uploaded_file,
    target_folder: str,
) -> str:
    target_dir = Path(
        target_folder,
    )

    target_dir.mkdir(
        parents=True,
        exist_ok=True,
    )

    target_path = target_dir / uploaded_file.name

    with open(
        target_path,
        "wb",
    ) as output_file:
        output_file.write(
            uploaded_file.getbuffer(),
        )

    return str(
        target_path,
    )

def write_output_file(
    output_dir: Path,
    file_name: str,
    content: str,
    encoding: str = "utf-8",
) -> Path:
    output_dir.mkdir(
        parents=True,
        exist_ok=True,
    )

    path = output_dir / file_name

    path.write_text(
        content,
        encoding=encoding,
    )

    return path

def log_timing(
    timings: list[str],
    label: str,
    step_start: float,
) -> float:
    elapsed = perf_counter() - step_start

    timings.append(
        f"{label}: {elapsed:.2f} seconds",
    )

    return perf_counter()

def build_sets_summary(
    metadata,
) -> list[str]:
    all_sets = set()

    for record in metadata.records:
        for membership in record.set_memberships:
            set_name = getattr(
                membership,

```

```

        "set_name",
        "",
    )

    if set_name:
        all_sets.add(
            set_name,
        )

    return sorted(
        all_sets,
    )

def build_relationship_rows(
    relationships,
) -> list[dict[str, str]]:
    rows = []

    for relationship in relationships:
        rows.append(
            {
                "Owner Record": getattr(
                    relationship,
                    "owner_record",
                    "",
                ),
                "Member Record": getattr(
                    relationship,
                    "member_record",
                    "",
                ),
                "Set Name": getattr(
                    relationship,
                    "set_name",
                    "",
                ),
            }
        )

    return rows

def build_mapping_row_count_by_table(
    rows: list[dict],
    include_total: bool = True,
) -> list[dict[str, int | str]]:
    """
    Builds table-level counts from the actual Excel Sheet Mapping rows.

    This count matches the detailed mapping table because it counts the
    generated mapping rows, not only DB2 physical columns.

    Output column order:
    - Column Field Count
    - Table Name
    """
    table_counts: dict[str, int] = {}

    for row in rows or []:
        table_name = (
            row.get("New DB2 Record")
            or row.get("Cobol Record IDMS")
            or ""
        )

        table_name = str(
            table_name,
        ).strip()

        if not table_name:
            continue

        table_counts[table_name] = table_counts.get(
            table_name,
            0,

```

```

        ) + 1

output_rows: list[dict[str, int | str]] = []

total_count = 0

for table_name in sorted(table_counts):
    count = table_counts[table_name]

    output_rows.append(
        {
            "Column Field Count": count,
            "Table Name": table_name,
        }
    )

    total_count += count

if include_total:
    output_rows.append(
        {
            "Column Field Count": total_count,
            "Table Name": "TOTAL",
        }
    )

return output_rows

def build_db2_physical_column_count_rows(
    db2_model,
    include_total: bool = True,
) -> list[dict[str, int | str]]:
    """
    Builds table-level physical DB2 column counts.

    This is separate from mapping row counts because COBOL YEAR, MONTH,
    and DAY fields can map to one DB2 DATE column.
    """
    table_rows: list[dict[str, int | str]] = []

    if db2_model is None:
        return table_rows

    total_count = 0

    for table in getattr(
        db2_model,
        "tables",
        [],
    ) or []:
        table_name = getattr(
            table,
            "name",
            "",
        )

        columns = getattr(
            table,
            "columns",
            [],
        ) or []

        count = len(
            columns,
        )

        table_rows.append(
            {
                "Column Field Count": count,
                "Table Name": table_name,
            }
        )

        total_count += count

```



```

    if include_total:
        table_rows.append(
            {
                "Column Field Count": total_count,
                "Table Name": "TOTAL",
            }
        )

    return table_rows

def generate_outputs(
    schema_pdf,
    cobol_pdf,
) -> None:
    total_start = perf_counter()
    step_start = perf_counter()
    timings: list[str] = []

    reset_outputs()

    warnings = []

    schema_path = save_uploaded_file(
        schema_pdf,
        settings.input_dir,
    )

    cobol_pdf_path = save_uploaded_file(
        cobol_pdf,
        settings.input_dir,
    )

    step_start = log_timing(
        timings,
        "Save uploaded files",
        step_start,
    )

    metadata_service = MetadataService()

    metadata = metadata_service.build_metadata(
        schema_path,
    )

    step_start = log_timing(
        timings,
        "Build metadata from schema PDF",
        step_start,
    )

    canonical_builder = CanonicalSchemaBuilder()

    canonical_schema = canonical_builder.build(
        metadata=metadata,
    )

    step_start = log_timing(
        timings,
        "Build canonical schema",
        step_start,
    )

    canonical_reconciler = CanonicalReconciler()

    canonical_schema = canonical_reconciler.reconcile(
        canonical_schema=canonical_schema,
        metadata=metadata,
    )

    step_start = log_timing(
        timings,
        "Reconcile canonical schema",
        step_start,
    )

```

```

db2_builder = DB2ModelBuilder()

db2_model = db2_builder.build(
    canonical_schema,
)

step_start = log_timing(
    timings,
    "Build DB2 model",
    step_start,
)

ddl_generator = DDLGenerator()

ddl_text = ddl_generator.generate(
    db2_model,
)

step_start = log_timing(
    timings,
    "Generate DB2 DDL",
    step_start,
)

excel_mapping_service = ExcelSheetMappingService()

excel_mapping_rows = excel_mapping_service.build(
    metadata=metadata,
    db2_model=db2_model,
)

step_start = log_timing(
    timings,
    "Generate Excel Sheet Mapping rows",
    step_start,
)

phase2_metadata_generator = Phase2MetadataGenerator()

phase2_metadata_json = phase2_metadata_generator.generate_json(
    canonical_schema=canonical_schema,
    db2_model=db2_model,
    metadata=metadata,
)

step_start = log_timing(
    timings,
    "Generate Phase 2 metadata JSON",
    step_start,
)

canonical_json = canonical_schema.model_dump_json(
    indent=2,
)

step_start = log_timing(
    timings,
    "Generate canonical JSON",
    step_start,
)

cobol_extractor = CobolPdfExtractor()

cobol_text = cobol_extractor.extract_text(
    cobol_pdf_path,
)

step_start = log_timing(
    timings,
    "Extract COBOL PDF text",
    step_start,
)

converted_cobol = ""
validation_messages = []

```

```

try:
    phase2_bridge = Phase2ConversionBridge()

    converted_cobol, validation_messages = phase2_bridge.convert(
        cobol_text=cobol_text,
        canonical_json=canonical_json,
        phase2_metadata_json=phase2_metadata_json,
        ddl_text=ddl_text,
        idms_schema_text=None,
        relationship_overrides_json=None,
        target_program=None,
        use_ddl=True,
        use_idms_schema=False,
        use_overrides=False,
    )

except Exception as exc:
    converted_cobol = ""
    validation_messages = [
        f"Phase 2 COBOL conversion failed: {exc}",
    ]

step_start = log_timing(
    timings,
    "Phase 2 COBOL conversion",
    step_start,
)

relationships = metadata.relationships

all_sets = build_sets_summary(
    metadata,
)

step_start = log_timing(
    timings,
    "Build relationship and set summaries",
    step_start,
)

output_dir = Path(
    settings.output_dir,
)

output_files = {}

output_files["ddl"] = write_output_file(
    output_dir=output_dir,
    file_name="db2_ddl.sql",
    content=ddl_text,
)

output_files["canonical"] = write_output_file(
    output_dir=output_dir,
    file_name="canonical_model.json",
    content=canonical_json,
)

output_files["phase2_metadata"] = write_output_file(
    output_dir=output_dir,
    file_name=PHASE2_METADATA_FILE,
    content=phase2_metadata_json,
)

output_files["converted_cobol"] = write_output_file(
    output_dir=output_dir,
    file_name="converted_db2_cobol.cbl",
    content=converted_cobol,
)

step_start = log_timing(
    timings,
    "Write output files",
    step_start,
)

```

```

er_png_path = ""
er_image_bytes = None

try:
    er_path = output_dir / ER_DIAGRAM_FILE

    er_png_path = generate_er_diagram(
        metadata,
        str(
            er_path,
        ),
    )

    with open(
        er_png_path,
        "rb",
    ) as image_file:
        er_image_bytes = image_file.read()

    output_files["er_diagram"] = Path(
        er_png_path,
    )

except Exception as exc:
    warnings.append(
        "ER diagram was not generated. "
        "Graphviz may not be installed or configured. "
        f"Details: {exc}"
    )

step_start = log_timing(
    timings,
    "ER diagram generation",
    step_start,
)

st.session_state.generated = True
st.session_state.schema_path = schema_path
st.session_state.cobol_pdf_path = cobol_pdf_path
st.session_state.metadata = metadata
st.session_state.canonical_schema = canonical_schema
st.session_state.db2_model = db2_model
st.session_state.ddl_text = ddl_text
st.session_state.canonical_json = canonical_json
st.session_state.phase2_metadata_json = phase2_metadata_json
st.session_state.cobol_text = cobol_text
st.session_state.converted_cobol = converted_cobol
st.session_state.relationships = relationships
st.session_state.all_sets = all_sets
st.session_state.record_count = len(
    metadata.records,
)
st.session_state.set_count = len(
    all_sets,
)
st.session_state.relationship_count = len(
    relationships,
)
st.session_state.validation_messages = validation_messages
st.session_state.generation_warnings = warnings
st.session_state.er_png_path = er_png_path
st.session_state.er_image_bytes = er_image_bytes
st.session_state.output_files = output_files
st.session_state.excel_mapping_rows = excel_mapping_rows

total_elapsed = perf_counter() - total_start

timings.append(
    f"TOTAL: {total_elapsed:.2f} seconds",
)

st.session_state.generation_timings = timings

def render_download_button(
    label: str,

```

```

    path,
    file_name: str,
    mime: str,
) -> None:
    if not path:
        st.button(
            label,
            disabled=True,
            use_container_width=True,
        )
        return

    with open(
        path,
        "rb",
    ) as file:
        st.download_button(
            label=label,
            data=file,
            file_name=file_name,
            mime=mime,
            use_container_width=True,
        )

def render_download_buttons() -> None:
    if not st.session_state.generated:
        return

    st.markdown("### Download Outputs")

    output_files = st.session_state.output_files

    col1, col2, col3, col4, col5 = st.columns(
        5,
    )

    with col1:
        render_download_button(
            label="DB2 DDL",
            path=output_files.get(
                "ddl",
            ),
            file_name="db2_ddl.sql",
            mime="text/sql",
        )

    with col2:
        render_download_button(
            label="Phase Metadata",
            path=output_files.get(
                "phase2_metadata",
            ),
            file_name=PHASE2_METADATA_FILE,
            mime="application/json",
        )

    with col3:
        render_download_button(
            label="Canonical JSON",
            path=output_files.get(
                "canonical",
            ),
            file_name="canonical_model.json",
            mime="application/json",
        )

    with col4:
        if st.session_state.er_image_bytes:
            st.download_button(
                label="ER Diagram",
                data=st.session_state.er_image_bytes,
                file_name=ER_DIAGRAM_FILE,
                mime="image/png",
                use_container_width=True,
            )

```

```

else:
    st.button(
        "ER Diagram",
        disabled=True,
        use_container_width=True,
    )

with col5:
    render_download_button(
        label="DB2 COBOL",
        path=output_files.get(
            "converted_cobol",
        ),
        file_name="converted_db2_cobol.cbl",
        mime="text/plain",
    )

def total_timing_only() -> str:
    for timing in st.session_state.generation_timings:
        if timing.startswith(
            "TOTAL:",
        ):
            return timing

    return ""

def get_total_generation_time() -> str:
    total_timing = total_timing_only()

    if not total_timing:
        return ""

    return total_timing.replace(
        "TOTAL:",
        "",
    ).strip()

def render_status_panel() -> None:
    if not st.session_state.generated:
        return

    total_time = get_total_generation_time()

    if total_time:
        st.success(
            f"Generation completed in {total_time}",
        )

    if st.session_state.generation_warnings:
        with st.expander(
            "Generation Warnings",
            expanded=False,
        ):
            for warning in st.session_state.generation_warnings:
                st.warning(
                    warning,
                )

    if not st.session_state.validation_messages:
        st.success(
            "COBOL conversion validation passed.",
        )
    else:
        st.warning(
            "COBOL conversion completed with validation messages. "
            "Open the Validation tab for details."
        )

def render_main_tab() -> None:
    st.markdown("## Generate Modernization Outputs")

    st.info(

```

```

        "Upload the Schema Listing PDF and COBOL Code PDF to generate DB2 "
        "DDL, metadata, ER diagram, converted DB2 COBOL, and sheet mapping."
    )

col1, col2 = st.columns(
    2,
)

with col1:
    schema_pdf = st.file_uploader(
        "Schema Listing PDF",
        type=["pdf"],
        key="schema_listing_pdf",
    )

with col2:
    cobol_pdf = st.file_uploader(
        "COBOL Code PDF",
        type=["pdf"],
        key="cobol_code_pdf",
    )

generate_clicked = st.button(
    "Generate Outputs",
    type="primary",
    use_container_width=True,
)

if generate_clicked:
    if schema_pdf is None:
        st.error(
            "Please upload the Schema Listing PDF.",
        )
        st.stop()

    if cobol_pdf is None:
        st.error(
            "Please upload the COBOL Code PDF.",
        )
        st.stop()

    with st.spinner(
        "Processing schema, generating DB2 artifacts, and converting COBOL..."
    ):
        try:
            generate_outputs(
                schema_pdf=schema_pdf,
                cobol_pdf=cobol_pdf,
            )

            st.success(
                "Generation completed successfully.",
            )

        except Exception as exc:
            st.error(
                "Generation failed.",
            )
            st.exception(
                exc,
            )
            st.stop()

if st.session_state.generated:
    st.markdown("### Output Status")

    col_a, col_b = st.columns(
        2,
    )

    with col_a:
        st.write(
            f"Schema PDF: `{st.session_state.schema_path}`",
        )

    with col_b:

```

```

        st.write(
            f"COBOL PDF: `{st.session_state.cobol_pdf_path}`",
        )

    render_status_panel()
    render_download_buttons()

def render_metadata_overview_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first.",
        )
        return

    st.markdown("## Metadata Overview")

    st.markdown("### Summary")

    col1, col2, col3, col4 = st.columns(
        4,
    )

    with col1:
        st.metric(
            "Records",
            st.session_state.record_count,
        )

    with col2:
        st.metric(
            "Sets",
            st.session_state.set_count,
        )

    with col3:
        st.metric(
            "Relationships",
            st.session_state.relationship_count,
        )

    with col4:
        st.metric(
            "Sheet Mapping Rows",
            len(
                st.session_state.excel_mapping_rows,
            ),
        )

    st.divider()

    st.markdown("### Detected Sets")

    st.json(
        st.session_state.all_sets,
    )

    st.divider()

    st.markdown("### Detected Relationships")

    relationship_rows = build_relationship_rows(
        st.session_state.relationships,
    )

    if relationship_rows:
        st.dataframe(
            relationship_rows,
            use_container_width=True,
            hide_index=True,
        )
    else:
        st.info(
            "No relationships detected.",
        )

```



```

st.divider()

st.markdown("### Table List and Mapping Row Counts")

mapping_count_rows = build_mapping_row_count_by_table(
    rows=st.session_state.excel_mapping_rows,
    include_total=True,
)

if mapping_count_rows:
    st.dataframe(
        mapping_count_rows,
        use_container_width=True,
        hide_index=True,
    )
else:
    st.info(
        "No table mapping row counts available.",
    )

st.caption(
    "This count is based on actual Excel Sheet Mapping rows. "
    "The TOTAL matches the detailed mapping table."
)

st.divider()

st.markdown("### DB2 Physical Column Counts")

physical_count_rows = build_db2_physical_column_count_rows(
    db2_model=st.session_state.db2_model,
    include_total=True,
)

if physical_count_rows:
    st.dataframe(
        physical_count_rows,
        use_container_width=True,
        hide_index=True,
    )
else:
    st.info(
        "No DB2 physical column counts available.",
    )

st.caption(
    "This count is based on DB2 physical columns. It can differ from "
    "mapping row count when COBOL YEAR, MONTH, and DAY fields map to "
    "one DB2 DATE column."
)

def render_ddl_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first.",
        )
        return

    st.markdown("## DB2 DDL Preview")

    st.code(
        st.session_state.ddl_text,
        language="sql",
    )

def render_phase2_metadata_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first.",
        )
        return

    st.markdown("## Phase 2 Metadata Preview")

```

```

st.code(
    st.session_state.phase2_metadata_json,
    language="json",
)

def render_excel_sheet_mapping_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first.",
        )
        return

    st.markdown("## Excel Sheet Mapping")

    rows = st.session_state.excel_mapping_rows

    if not rows:
        st.info(
            "No mapping rows generated.",
        )
        return

    st.markdown("### Sheet Map Abstract")

    abstract_rows = build_mapping_row_count_by_table(
        rows=rows,
        include_total=True,
    )

    if abstract_rows:
        st.dataframe(
            abstract_rows,
            use_container_width=True,
            hide_index=True,
        )
    else:
        st.info(
            "No table abstract rows available.",
        )

    st.caption(
        "The TOTAL row matches the number of detailed mapping rows below."
    )

    st.markdown("### Sheet Mapping Details")

    st.dataframe(
        rows,
        use_container_width=True,
        hide_index=True,
    )

def render_er_diagram_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first.",
        )
        return

    st.markdown("## ER Diagram")

    if st.session_state.er_image_bytes:
        st.image(
            st.session_state.er_image_bytes,
        )
    else:
        st.warning(
            "ER diagram was not generated. Graphviz may not be installed "
            "or configured."
        )

def render_converted_cobol_tab() -> None:
    if not st.session_state.generated:

```

```

        st.info(
            "Generate outputs from the Main tab first.",
        )
        return

st.markdown("## Converted DB2 COBOL")

if not st.session_state.converted_cobol:
    st.warning(
        "Converted COBOL output is empty or conversion failed.",
    )

st.text_area(
    "Final DB2 COBOL Code",
    value=st.session_state.converted_cobol,
    height=750,
)

def render_validation_tab() -> None:
    if not st.session_state.generated:
        st.info(
            "Generate outputs from the Main tab first.",
        )
        return

    st.markdown("## Validation")

    if st.session_state.generation_timings:
        st.markdown("### Generation Timings")

        st.code(
            "\n".join(
                st.session_state.generation_timings,
            )
        )

    if st.session_state.validation_messages:
        st.markdown("### Validation Messages")

        for message in st.session_state.validation_messages:
            st.warning(
                message,
            )
    else:
        st.success(
            "No warnings or validation messages.",
        )

def main() -> None:
    st.set_page_config(
        page_title="IDMS DB2 Modernizer",
        layout="wide",
    )

    initialize_directories()
    initialize_session_state()

    st.title(
        "IDMS > DB2 Modernizer",
    )

    st.caption(
        "Generate DB2 DDL, Phase 2 metadata, canonical JSON, ER diagram, "
        "converted DB2 COBOL, and Excel-style sheet mapping from uploaded PDFs."
    )

    tabs = st.tabs(
        [
            "Main",
            "Metadata Overview",
            "DB2 DDL",
            "Phase 2 Metadata",
            "Excel Sheet Mapping",
            "ER Diagram",
        ]
    )

```

```

        "Converted DB2 COBOL",
        "Validation",
    ]
)

with tabs[0]:
    render_main_tab()

with tabs[1]:
    render_metadata_overview_tab()

with tabs[2]:
    render_ddl_tab()

with tabs[3]:
    render_phase2_metadata_tab()

with tabs[4]:
    render_excel_sheet_mapping_tab()

with tabs[5]:
    render_er_diagram_tab()

with tabs[6]:
    render_converted_cobol_tab()

with tabs[7]:
    render_validation_tab()

if __name__ == "__main__":
    main()

=====
FILE: src\idms_modernizer\core\__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_modernizer\core\config.py
ENCODING: utf-8
=====

from pathlib import Path

from pydantic_settings import BaseSettings

class Settings(BaseSettings):

    APP_NAME: str = (
        "IDMS DB2 Modernizer"
    )

    BASE_DIR: Path = Path.cwd()

    INPUT_DIR: str = "data/input"

    OUTPUT_DIR: str = "data/output"

    TEMP_DIR: str = "data/temp"

    DEBUG: bool = True

    @property
    def input_dir(self) -> str:
        return self.INPUT_DIR

    @property
    def output_dir(self) -> str:
        return self.OUTPUT_DIR

```

```

@property
def temp_dir(self) -> str:
    return self.TEMP_DIR

class Config:
    env_file = ".env"

settings = Settings()

=====
FILE: src\idms_modernizer\core\constants.py
ENCODING: utf-8
=====

SUPPORTED_FILE_TYPES = [
    "pdf"
]

MAX_UPLOAD_SIZE_MB = 100

DB2_DDL_FILE = (
    "db2_ddl.sql"
)

ER_DIAGRAM_FILE = (
    "er_diagram.png"
)

EXCEL_MODEL_FILE = (
    "db2_model.xlsx"
)

REPORT_FILE = (
    "migration_report.pdf"
)

SUPPORTED_FILE_TYPES = [
    "pdf"
]

MAX_UPLOAD_SIZE_MB = 100

DB2_DDL_FILE = (
    "db2_ddl.sql"
)

ER_DIAGRAM_FILE = (
    "er_diagram.png"
)

EXCEL_MODEL_FILE = (
    "db2_model.xlsx"
)

REPORT_FILE = (
    "migration_report.pdf"
)

CANONICAL_MODEL_FILE = (
    "canonical_model.json"
)

PHASE2_METADATA_FILE = (
    "phase2_metadata.json"
)

=====
FILE: src\idms_modernizer\core\exceptions.py
ENCODING: utf-8
=====

class ModernizerException(
    Exception

```

```
):  
    pass
```

```
class PdfExtractionException(  
    ModernizerException  
):  
    pass
```

```
class SchemaParsingException(  
    ModernizerException  
):  
    pass
```

```
class ValidationException(  
    ModernizerException  
):  
    pass
```

```
=====
```

```
FILE: src\idms_modernizer\core\logger.py
```

```
ENCODING: utf-8
```

```
=====
```

```
import logging
```

```
def get_logger(  
    logger_name: str  
) -> logging.Logger:  
  
    logger = logging.getLogger(  
        logger_name  
    )  
  
    logger.setLevel(  
        logging.INFO  
    )  
  
    if not logger.handlers:  
  
        handler = (  
            logging.StreamHandler()  
        )  
  
        formatter = (  
            logging.Formatter(  
                "%(asctime)s | %(levelname)s | %(message)s"  
            )  
        )  
  
        handler.setFormatter(  
            formatter  
        )  
  
        logger.addHandler(  
            handler  
        )  
  
    return logger
```

```
=====
```

```
FILE: src\idms_modernizer\core\utils.py
```

```
ENCODING: utf-8
```

```
=====
```

```
from pathlib import Path
```

```
def ensure_directory(  
    directory: str  
) -> None:
```

```

    Path(
        directory
    ).mkdir(
        parents=True,
        exist_ok=True
    )

def normalize_text(
    text: str
) -> str:

    return " ".join(
        text.split()
    )

=====
FILE: src\idms_modernizer\domain\__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_modernizer\domain\canonical_models.py
ENCODING: utf-8
=====

from pydantic import BaseModel, Field

class CanonicalField(BaseModel):
    name: str
    datatype: str | None = None
    length: int | None = None
    scale: int | None = None
    occurs: bool = False
    occurs_max: int | None = None

class CanonicalRecord(BaseModel):
    name: str

    # Backward-compatible single PK.
    primary_key: str | None = None

    # Composite PK support.
    # When CALC points to a group, physical child fields are stored here.
    primary_keys: list[str] = Field(default_factory=list)

    record_id: str | None = None
    location_mode: str | None = None
    fields: list[CanonicalField] = Field(default_factory=list)

class CanonicalSet(BaseModel):
    name: str
    owner_record: str | None = None
    member_record: str | None = None
    order_mode: str | None = None

class CanonicalRelationship(BaseModel):
    set_name: str
    parent_record: str
    child_record: str
    cardinality: str = "1:N"

class CanonicalSchema(BaseModel):
    records: list[CanonicalRecord] = Field(default_factory=list)
    sets: list[CanonicalSet] = Field(default_factory=list)
    relationships: list[CanonicalRelationship] = Field(default_factory=list)

```

```
=====
FILE: src\idms_modernizer\domain\db2_models.py
ENCODING: utf-8
=====
```

```
from dataclasses import dataclass, field
```

```
@dataclass
class DB2Column:
    name: str
    datatype: str
    nullable: bool = True
    primary_key: bool = False
    generated: bool = False
    source_kind: str = ""
```

```
@dataclass
class DB2ForeignKey:
    column_name: str
    reference_table: str
    reference_column: str
    set_name: str = ""
```

```
@dataclass
class DB2Table:
    name: str
    columns: list[DB2Column] = field(default_factory=list)
    foreign_keys: list[DB2ForeignKey] = field(default_factory=list)
    primary_key: str | None = None
    primary_keys: list[str] = field(default_factory=list)
```

```
@dataclass
class DB2Model:
    tables: list[DB2Table] = field(default_factory=list)
```

```
=====
FILE: src\idms_modernizer\domain\document_models.py
ENCODING: utf-8
=====
```

```
from pydantic import BaseModel, Field
```

```
class DocumentLine(BaseModel):
    line_number: int
    text: str
```

```
class DocumentPage(BaseModel):
    page_number: int
    lines: list[DocumentLine] = Field(
        default_factory=list
    )
```

```
class DocumentModel(BaseModel):
    pages: list[DocumentPage] = Field(
        default_factory=list
    )
```

```
=====
FILE: src\idms_modernizer\domain\models.py
ENCODING: utf-8
=====
```

```
from pydantic import BaseModel
```

```
from idms_modernizer.domain.db2_models import Table
```



```

from idms_modernizer.domain.relationship_models import (
    Relationship
)
from idms_modernizer.domain.schema_models import (
    SchemaMetadata
)

```

```

class ConversionContext(
    BaseModel
):
    schema_metadata: (
        SchemaMetadata | None
    ) = None

    relationships: (
        list[Relationship]
    ) = []

    db2_tables: (
        list[Table]
    ) = []

```

```

=====
FILE: src\idms_modernizer\domain\relationship_models.py
ENCODING: utf-8
=====

```

```

from pydantic import BaseModel

```

```

class SetMembership(BaseModel):

```

```

    set_name: str

```

```

    role: str

```

```

class Relationship(BaseModel):

```

```

    set_name: str

```

```

    owner_record: str

```

```

    member_record: str

```

```

    cardinality: str = "1:N"

```

```

=====
FILE: src\idms_modernizer\domain\schema_models.py
ENCODING: utf-8
=====

```

```

from dataclasses import dataclass, field

```

```

from pydantic import BaseModel, Field

```

```

from idms_modernizer.domain.relationship_models import (
    Relationship,
    SetMembership,
)

```

```

class DataField(BaseModel):

```

```

    name: str

```

```

    level: int | None = None

```

```

    datatype: str | None = None

```

```

    length: int | None = None

```

```

    scale: int | None = None

```

```

    picture: str | None = None

```

```

    start_position: int | None = None

```

```

end_position: int | None = None
basetype: str | None = None

has_child: bool = False
is_group: bool = False

occurs: bool = False
occurs_min: int | None = None
occurs_max: int | None = None

raw_line: str | None = None
rest: str | None = None

@dataclass
class Record:
    name: str

    # Physical / DDL-safe fields.
    # Used by canonical schema, DB2 model, DDL, Phase 2 metadata, and COBOL conversion.
    fields: list[DataField] = field(
        default_factory=list,
    )

    # Excel-only complete field list.
    # Includes group fields, leaf fields, outer date groups, key groups, attributes, OCCURS groups.
    mapping_fields: list[DataField] = field(
        default_factory=list,
    )

    set_memberships: list[SetMembership] = field(
        default_factory=list,
    )

    primary_key: str | None = None
    cobol_zone: str | None = None

class SetDefinition(BaseModel):
    name: str
    owner_record: str | None = None
    member_record: str | None = None

class SchemaMetadata(BaseModel):
    records: list[Record] = Field(
        default_factory=list,
    )

    sets: list[SetDefinition] = Field(
        default_factory=list,
    )

    relationships: list[Relationship] = Field(
        default_factory=list,
    )

=====
FILE: src\idms_modernizer\domain\section_models.py
ENCODING: utf-8
=====

from dataclasses import dataclass, field

@dataclass
class RecordSection:
    record_name: str
    lines: list[str] = field(
        default_factory=list
    )

```

```
=====
FILE: src\idms_modernizer\extractors\cobol_zone_extractor.py
ENCODING: utf-8
=====
```

```
import re
```

```
class CobolZoneExtractor:
```

```
    """
    Extracts the IDMS region / zone from a schema record section.
```

```
    Examples:
```

```
        WITHIN INS-DEMO-REGION OFFSET 5 PGS FOR 20 PGS
        WITHIN
        EMP-DEMO-REGION OFFSET 5 PGS FOR 45 PGS
```

```
    Output:
```

```
        INS-DEMO-REGION
        EMP-DEMO-REGION
    """
```

```
    DIRECT_WITHIN_PATTERN = re.compile(
        r"\bWITHIN\s+([A-Z][A-Z0-9-]*REGION)\b",
        re.IGNORECASE,
    )
```

```
    REGION_PATTERN = re.compile(
        r"\b([A-Z][A-Z0-9-]*REGION)\b",
        re.IGNORECASE,
    )
```

```
    def extract(
        self,
        lines: list[str],
    ) -> str | None:
        cleaned_lines = [
            self.clean_line(line)
            for line in lines
            if line and self.clean_line(line)
        ]

        if not cleaned_lines:
            return None

        joined_text = " ".join(cleaned_lines)

        direct_match = self.DIRECT_WITHIN_PATTERN.search(
            joined_text,
        )

        if direct_match:
            return direct_match.group(1).upper()

        for index, line in enumerate(cleaned_lines):
            upper_line = line.upper()

            if upper_line == "WITHIN":
                region = self.find_region_after_within(
                    lines=cleaned_lines,
                    start_index=index + 1,
                )

                if region:
                    return region

            if "WITHIN" in upper_line:
                region = self.find_region_in_line(
                    line=line,
                )

                if region:
                    return region

            region = self.find_region_after_within(
                lines=cleaned_lines,
```

```

        start_index=index + 1,
    )

    if region:
        return region

    return None

def find_region_after_within(
    self,
    lines: list[str],
    start_index: int,
) -> str | None:
    for line in lines[start_index : start_index + 5]:
        region = self.find_region_in_line(
            line=line,
        )

        if region:
            return region

    return None

def find_region_in_line(
    self,
    line: str,
) -> str | None:
    match = self.REGION_PATTERN.search(
        line,
    )

    if not match:
        return None

    return match.group(1).upper()

def clean_line(
    self,
    line: str,
) -> str:
    if line is None:
        return ""

    cleaned = line.strip()
    cleaned = cleaned.replace("\u00a0", " ")
    cleaned = re.sub(r"\s+", " ", cleaned)

    return cleaned

```

```

=====
FILE: src\idms_modernizer\extractors\field_extractor.py
ENCODING: utf-8
=====

```

```

import re

from dataclasses import dataclass
from typing import List

from idms_modernizer.domain.schema_models import DataField

@dataclass
class FieldCandidate:
    level: int
    name: str
    rest: str
    original_line: str
    has_child: bool = False
    is_group: bool = False
    occurs: bool = False
    occurs_min: int | None = None
    occurs_max: int | None = None

```

```

class FieldExtractor:
    """
    Extracts IDMS schema fields.

    Generic behavior only:
    - No business field names are hardcoded.
    - No record names are hardcoded.
    - Physical extraction and Sheet Mapping extraction are separated.

    extract():
    - Used for physical / DDL-safe fields.
    - Returns only leaf fields.
    - Excludes FILLER by default because FILLER should not become a DB2 column.

    extract_all():
    - Used for Excel Sheet Mapping.
    - Returns all fields:
      groups, subgroups, leaves, date groups, date parts, OCCURS groups, and FILLER.
    - Includes FILLER by default so Sheet Mapping can show FILLER rows.
    """

    FIELD_NAME_PATTERN = re.compile(
        r"^\s*(?P<level>0[1-9]|[1-4][0-9]|88)\s+"
        r"(?P<name>[A-Z][A-Z0-9-]*|FILLER)\b"
        r"(?P<rest>.*)$",
        re.IGNORECASE,
    )

    OCCURS_PATTERN = re.compile(
        r"\bOCCURS\s+"
        r"(?P<min>[0-9]+)"
        r"(?:\s+TO\s+(?P<max>[0-9]+))?",
        re.IGNORECASE,
    )

    def __init__(
        self,
        include_filler: bool = False,
        auto_number_filler: bool = False,
    ) -> None:
        self.include_filler = include_filler
        self.auto_number_filler = auto_number_filler

    def extract(
        self,
        lines: List[str],
    ) -> List[DataField]:
        """
        Physical / DDL-safe extraction.

        FILLER is excluded here because physical DB2 columns should not be created
        from COBOL FILLER.
        """

        print("USING FIELD EXTRACTOR VERSION LEAF-ONLY-WITH-LEVEL")

        candidates = self.discover_candidates(
            lines=lines,
            include_filler=False,
        )

        leaf_candidates = self.mark_and_filter_leaf_fields(
            candidates=candidates,
        )

        return self.candidates_to_fields(
            candidates=leaf_candidates,
        )

    def extract_all(
        self,
        lines: List[str],
        include_filler: bool = True,
    ) -> List[DataField]:
        """
        Excel Sheet Mapping extraction.

```

```

FILLER is included here so Sheet Mapping can show FILLER rows.
"""

print("USING FIELD EXTRACTOR VERSION ALL-FIELDS-FOR-MAPPING")

candidates = self.discover_candidates(
    lines=lines,
    include_filler=include_filler,
)

self.mark_group_fields(
    candidates=candidates,
)

return self.candidates_to_fields(
    candidates=candidates,
)

def discover_candidates(
    self,
    lines: List[str],
    include_filler: bool | None = None,
) -> List[FieldCandidate]:
    candidates: List[FieldCandidate] = []

    if include_filler is None:
        include_filler = self.include_filler

    filler_index = 0

    for line in lines:
        cleaned_line = self.clean_line(
            line=line,
        )

        if not cleaned_line:
            continue

        match = self.FIELD_NAME_PATTERN.match(
            cleaned_line,
        )

        if not match:
            continue

        level = int(
            match.group("level"),
        )

        name = match.group("name").upper()
        rest = match.group("rest").strip()

        if level == 88:
            continue

        if name == "FILLER":
            if not include_filler:
                continue

            filler_index += 1

            candidate_name = name

            if self.auto_number_filler:
                candidate_name = f"FILLER_{filler_index}"
            else:
                candidate_name = name

            occurs_info = self.parse_occurs(
                text=rest,
            )

            candidate = FieldCandidate(
                level=level,
                name=candidate_name,

```

```

        rest=rest,
        original_line=cleaned_line,
        occurs=occurs_info["occurs"],
        occurs_min=occurs_info["occurs_min"],
        occurs_max=occurs_info["occurs_max"],
    )

    candidates.append(candidate)

return candidates

def mark_group_fields(
    self,
    candidates: List[FieldCandidate],
) -> None:
    for index, candidate in enumerate(candidates):
        candidate.has_child = False
        candidate.is_group = False

        for later_candidate in candidates[index + 1:]:
            if later_candidate.level <= candidate.level:
                break

            if later_candidate.level > candidate.level:
                candidate.has_child = True
                candidate.is_group = True
                break

def mark_and_filter_leaf_fields(
    self,
    candidates: List[FieldCandidate],
) -> List[FieldCandidate]:
    self.mark_group_fields(
        candidates=candidates,
    )

    leaf_candidates: List[FieldCandidate] = []

    for candidate in candidates:
        if candidate.has_child:
            continue

        leaf_candidates.append(candidate)

    return leaf_candidates

def candidates_to_fields(
    self,
    candidates: List[FieldCandidate],
) -> List[DataField]:
    fields: List[DataField] = []

    for candidate in candidates:
        display_name = candidate.name

        if display_name.startswith("FILLER_"):
            display_name = "FILLER"

        field = DataField(
            name=display_name,
            level=candidate.level,
            rest=candidate.rest,
            raw_line=candidate.original_line,
            has_child=candidate.has_child,
            is_group=candidate.is_group,
            occurs=candidate.occurs,
            occurs_min=candidate.occurs_min,
            occurs_max=candidate.occurs_max,
        )

        fields.append(field)

    return fields

def parse_occurs(
    self,

```

```

        text: str,
    ) -> dict:
        match = self.OCCURS_PATTERN.search(
            text or "",
        )

        if not match:
            return {
                "occurs": False,
                "occurs_min": None,
                "occurs_max": None,
            }

        occurs_min = self.safe_int(
            value=match.group("min"),
        )

        occurs_max_raw = match.group("max")

        occurs_max = (
            self.safe_int(
                value=occurs_max_raw,
            )
            if occurs_max_raw is not None
            else occurs_min
        )

        return {
            "occurs": True,
            "occurs_min": occurs_min,
            "occurs_max": occurs_max,
        }

```

```

def clean_line(
    self,
    line: str,
) -> str:
    value = str(line or "").strip()

    if not value:
        return ""

    value = value.replace("\u00a0", " ")
    value = re.sub(r"\s+", " ", value)

    return value.strip()

```

```

def safe_int(
    self,
    value,
) -> int | None:
    try:
        if value is None:
            return None

        return int(value)
    except Exception:
        return None

```

```

=====
FILE: src\idms_modernizer\extractors\owner_member_extractor.py
ENCODING: utf-8
=====

```

```
import re
```

```

from idms_modernizer.domain.relationship_models import (
    SetMembership
)

```

```
class OwnerMemberExtractor:
```

```

    SAME_LINE_PATTERN = re.compile(
        r"^\s*([A-Z0-9-]+\s+)?(?:INDEX\s+)?(OWNER|MEMBER)\b",
    )

```



```

    re.IGNORECASE
)

SET_NAME_PATTERN = re.compile(
    r"^\s*([A-Z][A-Z0-9\-\_])\s*$",
    re.IGNORECASE
)

ROLE_PATTERN = re.compile(
    r"^\s*(OWNER|MEMBER)\b",
    re.IGNORECASE
)

IGNORED_SET_NAMES = {
    "CALC"
}

HEADER_WORDS = {
    "SET",
    "TYPE",
    "NEXT",
    "PRIOR",
    "OWNER",
    "MEMBER"
}

def extract(
    self,
    record_name: str,
    lines: list[str]
) -> list[SetMembership]:

    memberships = []
    seen = set()

    in_dbkey_block = False
    pending_set_name = None

    for raw_line in lines:

        line = raw_line.strip().upper()

        if not line:
            continue

        if "DBKEY POSITIONS" in line:
            in_dbkey_block = True
            pending_set_name = None
            continue

        if in_dbkey_block and line.startswith("DATA ITEM"):
            break

        if not in_dbkey_block:
            continue

        same_line_match = self.SAME_LINE_PATTERN.search(
            line
        )

        if same_line_match:

            set_name = same_line_match.group(1).upper()
            role = same_line_match.group(2).upper()

            pending_set_name = None

            if set_name in self.IGNORED_SET_NAMES:
                continue

            key = (
                set_name,
                role
            )

            if key not in seen:

```

```

        seen.add(key)

        memberships.append(
            SetMembership(
                set_name=set_name,
                role=role
            )
        )

        continue

role_match = self.ROLE_PATTERN.search(
    line
)

if role_match and pending_set_name:

    set_name = pending_set_name
    role = role_match.group(1).upper()

    pending_set_name = None

    if set_name in self.IGNORED_SET_NAMES:
        continue

    key = (
        set_name,
        role
    )

    if key not in seen:
        seen.add(key)

        memberships.append(
            SetMembership(
                set_name=set_name,
                role=role
            )
        )

        continue

set_name_match = self.SET_NAME_PATTERN.search(
    line
)

if set_name_match:

    candidate = set_name_match.group(1).upper()

    if candidate in self.HEADER_WORDS:
        continue

    if candidate in self.IGNORED_SET_NAMES:
        pending_set_name = None
        continue

    pending_set_name = candidate

print(
    f"{record_name}: "
    f"{len(memberships)} memberships"
)

for membership in memberships:
    print(
        f"{record_name} -> "
        f"{membership.set_name} "
        f"{membership.role}"
    )

return memberships

```

```
=====
FILE: src\idms_modernizer\extractors\primary_key_extractor.py
ENCODING: utf-8
=====
```

```
import re

from idms_modernizer.services.name_normalizer import NameNormalizer

class PrimaryKeyExtractor:
    """
    Extracts IDMS CALC primary key metadata.

    Generic behavior only:
    - No hardcoded record names.
    - No hardcoded key names.
    - Handles listing formats with dots, spaces, and split lines.

    Supported examples:
    - LOCATION MODE CALC USING EMP-ID-0415
    - LOCATION MODE..... CALC USING KY-VMBFC DUPLICATES NOT ALLOWED
    - CALC USING KY-VMBFC
    - SET CONTROL ITEM FOR CALC <start> <length>
    """

    CALC_PATTERN = re.compile(
        r"\bCALC\s+(?P<start>[A-Z0-9-])\s+b",
        re.IGNORECASE,
    )

    CONTROL_ITEM_PATTERN = re.compile(
        r"\bSET\s+CONTROL\s+ITEM\s+FOR\s+CALC"
        r"(?:\s+(?P<start>[0-9])\s+(?P<length>[0-9]))?",
        re.IGNORECASE,
    )

    CONTROL_ITEM_FOLLOWING_CALC_PATTERN = re.compile(
        r"\bCALC\s+(?P<start>[0-9])\s+(?P<length>[0-9])\s+b",
        re.IGNORECASE,
    )

    def extract(
        self,
        lines: list[str],
    ) -> str | None:
        cleaned_lines = [
            self.clean_line(line=line)
            for line in lines or []
        ]

        joined_text = " ".join(
            line
            for line in cleaned_lines
            if line
        )

        match = self.CALC_PATTERN.search(joined_text)

        if match:
            return NameNormalizer.normalize(
                match.group(1),
            )

        for line in cleaned_lines:
            match = self.CALC_PATTERN.search(line)

            if match:
                return NameNormalizer.normalize(
                    match.group(1),
                )

        return None

    def extract_control_item(
        self,
```

```

        lines: list[str],
    ) -> dict[str, int] | None:
        cleaned_lines = [
            self.clean_line(line=line)
            for line in lines or []
            if self.clean_line(line=line)
        ]

        joined_text = " ".join(cleaned_lines)

        match = self.CONTROL_ITEM_PATTERN.search(joined_text)

        if match:
            start = match.group("start")
            length = match.group("length")

            if start and length:
                return {
                    "start": int(start),
                    "length": int(length),
                }

        match = self.CONTROL_ITEM_FOLLOWING_CALC_PATTERN.search(joined_text)

        if match:
            return {
                "start": int(match.group("start")),
                "length": int(match.group("length")),
            }

        return None

    def clean_line(
        self,
        line: str,
    ) -> str:
        value = str(line or "").upper()

        value = value.replace(".", " ")
        value = value.replace("=", " ")
        value = value.replace(":", " ")
        value = value.replace("\u00a0", " ")

        value = re.sub(
            r"\s+",
            " ",
            value,
        )

        return value.strip()

```

```

=====
FILE: src\idms_modernizer\generators\ddl_generator.py
ENCODING: utf-8
=====

```

```

class DDLGenerator:
    """
    Generates DB2 DDL.

    Supports:
    - Single primary key
    - Composite primary key through table.primary_keys
    - Multiple FK columns
    - FK constraints emitted after CREATE TABLE
    """

    def generate(
        self,
        db2_model,
    ) -> str:
        ddl: list[str] = []

        for table in getattr(db2_model, "tables", []) or []:
            ddl.extend(

```

```

        self.generate_create_table(
            table=table,
        )
    )

foreign_key_statements = self.generate_foreign_key_statements(
    db2_model=db2_model,
)

if foreign_key_statements:
    ddl.append("")
    ddl.extend(foreign_key_statements)

return "\n".join(ddl)

def generate_create_table(
    self,
    table,
) -> list[str]:
    lines: list[str] = []

    table_name = getattr(table, "name", "")

    lines.append(f"CREATE TABLE {table_name}")
    lines.append("(")

    definitions: list[str] = []
    added_columns: set[str] = set()

    for column in getattr(table, "columns", []) or []:
        column_name = getattr(column, "name", "")

        if not column_name:
            continue

        if column_name in added_columns:
            continue

        datatype = getattr(column, "datatype", "")

        definition = f"    {column_name} {datatype}"

        if not getattr(column, "nullable", True):
            definition += " NOT NULL"

        definitions.append(definition)
        added_columns.add(column_name)

    primary_keys = self.primary_key_columns(
        table=table,
    )

    if primary_keys:
        definitions.append(
            f"    PRIMARY KEY ({', '.join(primary_keys)})"
        )

    lines.append(",\n".join(definitions))
    lines.append(");")
    lines.append("")

    return lines

def primary_key_columns(
    self,
    table,
) -> list[str]:
    primary_keys = list(getattr(table, "primary_keys", []) or [])

    if not primary_keys:
        single_primary_key = getattr(table, "primary_key", None)

        if single_primary_key:
            primary_keys = [single_primary_key]

    if not primary_keys:
```

```

        primary_keys = [
            getattr(column, "name", "")
            for column in getattr(table, "columns", []) or []
            if getattr(column, "primary_key", False)
        ]

    cleaned_primary_keys = []

    for primary_key in primary_keys:
        if not primary_key:
            continue

        if primary_key in cleaned_primary_keys:
            continue

        cleaned_primary_keys.append(primary_key)

    return cleaned_primary_keys

def generate_foreign_key_statements(
    self,
    db2_model,
) -> list[str]:
    statements: list[str] = []
    added_foreign_keys: set[tuple[str, str, str, str]] = set()

    for table in getattr(db2_model, "tables", []) or []:
        table_name = getattr(table, "name", "")

        if not table_name:
            continue

        for foreign_key in getattr(table, "foreign_keys", []) or []:
            child_column = self.get_foreign_key_column_name(
                foreign_key=foreign_key,
            )

            parent_table = self.get_foreign_key_reference_table(
                foreign_key=foreign_key,
            )

            parent_column = self.get_foreign_key_reference_column(
                foreign_key=foreign_key,
            )

            if not child_column or not parent_table or not parent_column:
                continue

            fk_key = (
                table_name,
                child_column,
                parent_table,
                parent_column,
            )

            if fk_key in added_foreign_keys:
                continue

            added_foreign_keys.add(fk_key)

            constraint_name = self.constraint_name(
                table_name=table_name,
                column_name=child_column,
            )

            statements.append(
                f"ALTER TABLE {table_name} "
                f"ADD CONSTRAINT {constraint_name} "
                f"FOREIGN KEY ({child_column}) "
                f"REFERENCES {parent_table} ({parent_column});"
            )

    return statements

def get_foreign_key_column_name(
    self,

```

```

        foreign_key,
    ) -> str:
        return (
            getattr(foreign_key, "column_name", None)
            or getattr(foreign_key, "child_column", None)
            or getattr(foreign_key, "child_fk", None)
            or getattr(foreign_key, "foreign_key", None)
            or ""
        )

    def get_foreign_key_reference_table(
        self,
        foreign_key,
    ) -> str:
        return (
            getattr(foreign_key, "reference_table", None)
            or getattr(foreign_key, "parent_table", None)
            or getattr(foreign_key, "referenced_table", None)
            or ""
        )

    def get_foreign_key_reference_column(
        self,
        foreign_key,
    ) -> str:
        return (
            getattr(foreign_key, "reference_column", None)
            or getattr(foreign_key, "parent_column", None)
            or getattr(foreign_key, "referenced_column", None)
            or getattr(foreign_key, "parent_key", None)
            or ""
        )

    def constraint_name(
        self,
        table_name: str,
        column_name: str,
    ) -> str:
        raw_name = f"FK_{table_name}_{column_name}"

        cleaned = (
            raw_name
            .replace("-", "_")
            .replace(" ", "_")
            .replace(".", "_")
            .upper()
        )

        return cleaned[:128]

```

```

=====
FILE: src\idms_modernizer\graph\relationship_resolver.py
ENCODING: utf-8
=====

```

```

from collections import defaultdict

from idms_modernizer.domain.relationship_models import (
    Relationship
)

class RelationshipResolver:

    def resolve(
        self,
        metadata
    ) -> list[Relationship]:

        owner_map = defaultdict(list)

        member_map = defaultdict(list)

        for record in metadata.records:

```

```

        for membership in record.set_memberships:

            if membership.role == "OWNER":

                owner_map[
                    membership.set_name
                ].append(
                    record.name
                )

            elif membership.role == "MEMBER":

                member_map[
                    membership.set_name
                ].append(
                    record.name
                )

    relationships = []

    all_sets = (
        set(owner_map.keys())
        | set(member_map.keys())
    )

    for set_name in all_sets:

        owners = owner_map.get(
            set_name,
            []
        )

        members = member_map.get(
            set_name,
            []
        )

        for owner in owners:

            for member in members:

                relationships.append(
                    Relationship(
                        set_name=set_name,
                        owner_record=owner,
                        member_record=member
                    )
                )

    return relationships

```

```

=====
FILE: src\idms_modernizer\graph\schema_graph.py
ENCODING: utf-8
=====

```

```
import networkx as nx
```

```
class SchemaGraph:
```

```

    def __init__(self):

        self.graph = nx.DiGraph()

```

```

    def add_record(
        self,
        record_name: str
    ):

```

```

        self.graph.add_node(
            record_name,
            node_type="record"
        )

```



```

def add_relationship(
    self,
    owner: str,
    member: str,
    set_name: str
):

    self.graph.add_edge(
        owner,
        member,
        set_name=set_name
    )

```

```

=====
FILE: src\idms_modernizer\parsers\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_modernizer\parsers\cobol_pdf_extractor.py
ENCODING: utf-8
=====

```

```

import re
import pdfplumber

```

```

class CobolPdfExtractor:
    """
    Extracts COBOL text from PDF.

    Supports:
    - COBOL pasted as text in PDF
    - line-preserving extraction
    - fallback to empty string if extraction fails
    """

    def extract_text(
        self,
        pdf_path: str
    ) -> str:

        lines: list[str] = []

        with pdfplumber.open(pdf_path) as pdf:
            for page in pdf.pages:
                page_text = (
                    page.extract_text()
                    or ""
                )

                for raw_line in page_text.splitlines():
                    cleaned_line = self._clean_line(
                        raw_line
                    )

                    lines.append(
                        cleaned_line
                    )

            return "\n".join(lines).strip()

    def _clean_line(
        self,
        line: str | None
    ) -> str:

        if line is None:
            return ""

        cleaned = str(line).rstrip()

```

```

        cleaned = cleaned.replace(
            "\t",
            " "
        )

        cleaned = re.sub(
            r"[\u00A0]+",
            " ",
            cleaned
        )

    return cleaned

```

```

=====
FILE: src\idms_modernizer\parsers\text_extractor.py
ENCODING: utf-8
=====

```

```

import re
import pdfplumber

```

```

from idms_modernizer.domain.document_models import (
    DocumentLine,
    DocumentModel,
    DocumentPage
)

```

```

class TextExtractor:
    """
    Plain text PDF extractor.

    Important:
    - Uses only page.extract_text().
    - Does not use extract_tables().
    - Does not use extract_words().
    - This keeps the useful rows seen in your debug, such as:
      03 EMP-FIRST-NAME-0415 DISPLAY X(10) 5 10
    """

```

```

    def extract_document(
        self,
        pdf_path: str
    ) -> DocumentModel:

        pages: list[DocumentPage] = []

        with pdfplumber.open(pdf_path) as pdf:
            for page_index, page in enumerate(pdf.pages):
                text = (
                    page.extract_text()
                    or ""
                )

                lines: list[DocumentLine] = []

                for line_index, raw_line in enumerate(
                    text.splitlines()
                ):
                    cleaned_line = self._clean_line(
                        raw_line
                    )

                    if not cleaned_line:
                        continue

                    lines.append(
                        DocumentLine(
                            line_number=line_index,
                            text=cleaned_line
                        )
                    )

                pages.append(
                    DocumentPage(

```

```

        page_number=page_index,
        lines=lines
    )
)

return DocumentModel(
    pages=pages
)

def _clean_line(
    self,
    line: str | None
) -> str:

    if not line:
        return ""

    cleaned = str(line).strip()

    cleaned = re.sub(
        r"<[^>]+>",
        " ",
        cleaned
    )

    cleaned = cleaned.replace(
        "&nbsp;",
        " "
    )

    cleaned = cleaned.replace(
        "\t",
        " "
    )

    cleaned = cleaned.replace(
        "|",
        " "
    )

    cleaned = re.sub(
        r"\s+",
        " ",
        cleaned
    )

    return cleaned.strip()

```

```

=====
FILE: src\idms_modernizer\services\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_modernizer\services\business_datatype_inference.py
ENCODING: utf-8
=====

```

```

class BusinessDatatypeInference:
    """
    Generic datatype inference.

    This class intentionally does not inspect field names.

    Reason:
    Field-name based rules such as PHONE, ZIP, SSN, CODE, ACCOUNT,
    POLICY, or LICENSE are business/domain assumptions and should not be
    hardcoded.

    Datatype should be decided upstream from:
    - IDMS PIC clause
    - USAGE
    """

```

```
- length
- scale
"""
```

```
@staticmethod
def infer(
    field_name: str,
    current_type: str,
) -> str:
    if current_type:
        return current_type.upper()

    return "VARCHAR"
```

```
=====
FILE: src\idms_modernizer\services\canonical_reconciler.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.domain.canonical_models import (
    CanonicalSchema,
    CanonicalField
)
from idms_modernizer.domain.schema_models import SchemaMetadata
from idms_modernizer.services.name_normalizer import NameNormalizer
```

```
class CanonicalReconciler:
    """
    Reconciles canonical schema with IDMS schema metadata.

    IDMS schema is authoritative for:
    - datatype
    - length
    - scale
    - picture

    Canonical schema is authoritative for:
    - normalized record names
    - primary keys
    - relationships
    """

    def reconcile(
        self,
        canonical_schema: CanonicalSchema,
        metadata: SchemaMetadata
    ) -> CanonicalSchema:

        idms_field_lookup = self._build_idms_field_lookup(
            metadata
        )

        for record in canonical_schema.records:
            normalized_record_name = NameNormalizer.normalize(
                record.name
            )

            reconciled_fields: list[CanonicalField] = []

            for field in record.fields:
                normalized_field_name = NameNormalizer.normalize(
                    field.name
                )

                idms_field = idms_field_lookup.get(
                    (
                        normalized_record_name,
                        normalized_field_name
                    )
                )

                if idms_field is None:
                    idms_field = idms_field_lookup.get(
                        (
```

```

        normalized_record_name,
        self._remove_record_suffix(
            normalized_field_name
        )
    )
)

if idms_field is None:
    reconciled_fields.append(field)
    continue

reconciled_fields.append(
    CanonicalField(
        name=field.name,
        datatype=idms_field.datatype,
        length=idms_field.length,
        scale=idms_field.scale,
        occurs=field.occurs,
        occurs_max=field.occurs_max
    )
)

record.fields = reconciled_fields

return canonical_schema

def _build_idms_field_lookup(
    self,
    metadata: SchemaMetadata
) -> dict[tuple[str, str], object]:

    lookup: dict[tuple[str, str], object] = {}

    for record in metadata.records:
        record_name = NameNormalizer.normalize(
            record.name
        )

        for field in record.fields:
            field_name = NameNormalizer.normalize(
                field.name
            )

            lookup[
                (
                    record_name,
                    field_name
                )
            ] = field

            lookup[
                (
                    record_name,
                    self._remove_record_suffix(field_name)
                )
            ] = field

    return lookup

def _remove_record_suffix(
    self,
    field_name: str
) -> str:

    parts = field_name.split("_")

    if (
        len(parts) > 1
        and parts[-1].isdigit()
        and len(parts[-1]) == 4
    ):
        return "_".join(parts[:-1])

    return field_name

```

```
=====
FILE: src\idms_modernizer\services\canonical_schema_builder.py
ENCODING: utf-8
=====
```

```
import re

from idms_modernizer.domain.canonical_models import (
    CanonicalSchema,
    CanonicalRecord,
    CanonicalField,
    CanonicalSet,
    CanonicalRelationship,
)
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.services.date_field_consolidator import DateFieldConsolidator

class CanonicalSchemaBuilder:
    """
    Builds canonical schema from IDMS metadata.

    Generic behavior only:
    - No hardcoded record names.
    - No hardcoded column names.
    - No hardcoded SET names.

    Rules:
    - Physical DB2 columns come from record.fields, after date consolidation.
    - Group detection uses mapping_fields level hierarchy.
    - CALC outer group is never a physical DB2 PK column.
    - CALC intermediate groups are not physical DB2 columns.
    - CALC physical non-date leaf children become actual composite PK columns.
    - Date group/date parts are never physical PK.
    - FILLER is never physical PK.
    - If no CALC exists, DB2MappingService creates ID_RECORD_<record_name> CHAR(20).
    """

    def build(self, metadata) -> CanonicalSchema:
        schema = CanonicalSchema()

        for record in getattr(metadata, "records", []) or []:
            record_name = NameNormalizer.normalize(getattr(record, "name", "") or "")

            primary_key = (
                NameNormalizer.normalize(getattr(record, "primary_key", "") or "")
                if getattr(record, "primary_key", None)
                else None
            )

            canonical_record = CanonicalRecord(
                name=record_name,
                primary_key=primary_key,
                primary_keys=[],
            )

            mapping_fields = (
                getattr(record, "mapping_fields", None)
                or getattr(record, "fields", None)
                or []
            )

            group_names = self.build_group_name_set(fields=mapping_fields)
            date_names = self.build_date_name_set(fields=mapping_fields)

            physical_fields = getattr(record, "fields", []) or []

            normalized_fields = DateFieldConsolidator.consolidate(
                physical_fields,
            )

            added_fields: set[str] = set()

            for field in normalized_fields:
                field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")
```

```

        if not field_name:
            continue

        if self.is_date_part_name(field_name):
            continue

        if field_name in added_fields:
            continue

        if field_name in group_names and field_name not in date_names:
            continue

        if self.is_non_physical_group_field(field=field):
            continue

        adjusted_field = self.adjust_occurs_field(field=field)

        canonical_record.fields.append(
            CanonicalField(
                name=field_name,
                datatype=getattr(adjusted_field, "datatype", None),
                length=getattr(adjusted_field, "length", None),
                scale=getattr(adjusted_field, "scale", None),
                occurs=getattr(adjusted_field, "occurs", False),
                occurs_max=getattr(adjusted_field, "occurs_max", None),
            )
        )

        added_fields.add(field_name)

    self.apply_primary_key_rules(
        canonical_record=canonical_record,
        record=record,
        added_fields=added_fields,
        group_names=group_names,
        date_names=date_names,
    )

    schema.records.append(canonical_record)

self.add_sets_and_relationships(
    schema=schema,
    metadata=metadata,
)

return schema

def apply_primary_key_rules(
    self,
    canonical_record: CanonicalRecord,
    record,
    added_fields: set[str],
    group_names: set[str],
    date_names: set[str],
) -> None:
    primary_key = canonical_record.primary_key

    if not primary_key:
        canonical_record.primary_keys = []
        return

    primary_key_fields = self.resolve_primary_key_fields(
        record=record,
        primary_key=primary_key,
        group_names=group_names,
        date_names=date_names,
    )

    if not primary_key_fields:
        canonical_record.primary_keys = []
        canonical_record.primary_key = None
        return

    canonical_primary_keys: list[str] = []

    for key_field in primary_key_fields:

```

```

key_name = NameNormalizer.normalize(getattr(key_field, "name", "") or "")

if not key_name:
    continue

if key_name in canonical_primary_keys:
    continue

if key_name in date_names:
    continue

if key_name in group_names:
    continue

if self.is_date_part_name(key_name):
    continue

if self.is_date_field(key_field):
    continue

if self.is_filler_field(key_field):
    continue

if not self.has_physical_definition(key_field):
    continue

canonical_primary_keys.append(key_name)

if key_name in added_fields:
    continue

canonical_record.fields.append(
    CanonicalField(
        name=key_name,
        datatype=self.datatype_for_key_field(key_field),
        length=self.length_for_key_field(key_field),
        scale=getattr(key_field, "scale", None),
        occurs=False,
        occurs_max=None,
    )
)

added_fields.add(key_name)

canonical_record.primary_keys = canonical_primary_keys

if canonical_primary_keys:
    canonical_record.primary_key = canonical_primary_keys[0]
else:
    canonical_record.primary_key = None

def resolve_primary_key_fields(
    self,
    record,
    primary_key: str,
    group_names: set[str],
    date_names: set[str],
) -> list:
    mapping_fields = (
        getattr(record, "mapping_fields", None)
        or getattr(record, "fields", None)
        or []
    )

    key_field = self.find_field_for_primary_key(
        record=record,
        primary_key=primary_key,
    )

    if key_field is None:
        return []

    key_name = NameNormalizer.normalize(getattr(key_field, "name", "") or "")

    if key_name in group_names:
        return self.child_physical_fields_for_group(

```



```

        fields=mapping_fields,
        group_field=key_field,
        group_names=group_names,
        date_names=date_names,
    )

    if key_name in date_names:
        return []

    if self.is_date_field(key_field):
        return []

    if self.is_date_part_name(key_name):
        return []

    if self.is_filler_field(key_field):
        return []

    return [key_field]

def child_physical_fields_for_group(
    self,
    fields,
    group_field,
    group_names: set[str],
    date_names: set[str],
) -> list:
    field_list = list(fields or [])

    if not field_list:
        return []

    group_name = NameNormalizer.normalize(getattr(group_field, "name", "") or "")
    group_level = getattr(group_field, "level", None)

    if group_level is None:
        return []

    try:
        group_level_int = int(group_level)
    except Exception:
        return []

    collecting = False
    children = []

    for field in field_list:
        field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")
        field_level = getattr(field, "level", None)

        if field_level is None:
            continue

        try:
            field_level_int = int(field_level)
        except Exception:
            continue

        if not collecting:
            if field_name == group_name:
                collecting = True
            continue

        if field_level_int <= group_level_int:
            break

        if not field_name:
            continue

        if field_name in group_names:
            continue

        if field_name in date_names:
            continue

        if self.is_date_part_name(field_name):

```

```

        continue

    if self.is_date_field(field):
        continue

    if self.is_filler_field(field):
        continue

    if not self.has_physical_definition(field):
        continue

    children.append(field)

    return children

def build_group_name_set(self, fields) -> set[str]:
    result: set[str] = set()
    field_list = list(fields or [])

    for index, field in enumerate(field_list):
        field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")
        field_level = getattr(field, "level", None)

        if not field_name or field_level is None:
            continue

        try:
            field_level_int = int(field_level)
        except Exception:
            continue

        for next_field in field_list[index + 1:]:
            next_level = getattr(next_field, "level", None)

            if next_level is None:
                continue

            try:
                next_level_int = int(next_level)
            except Exception:
                continue

            if next_level_int > field_level_int:
                result.add(field_name)
                break

            if next_level_int <= field_level_int:
                break

    return result

def build_date_name_set(self, fields) -> set[str]:
    result: set[str] = set()
    field_list = list(fields or [])

    for index, field in enumerate(field_list):
        field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")
        field_level = getattr(field, "level", None)

        if not field_name:
            continue

        if self.is_date_part_name(field_name):
            result.add(field_name)
            continue

        if self.is_date_field(field):
            result.add(field_name)
            continue

        if field_level is None:
            continue

        try:
            field_level_int = int(field_level)
        except Exception:

```

```

        continue

    descendants = []

    for next_field in field_list[index + 1:]:
        next_level = getattr(next_field, "level", None)

        if next_level is None:
            continue

        try:
            next_level_int = int(next_level)
        except Exception:
            continue

        if next_level_int <= field_level_int:
            break

        descendants.append(next_field)

    parts_found = set()

    for descendant in descendants:
        descendant_name = NameNormalizer.normalize(
            getattr(descendant, "name", "") or ""
        )

        part = self.date_part_type_from_name(descendant_name)

        if part:
            parts_found.add(part)

    if {"YEAR", "MONTH", "DAY"}.issubset(parts_found):
        result.add(field_name)

        for descendant in descendants:
            descendant_name = NameNormalizer.normalize(
                getattr(descendant, "name", "") or ""
            )

            if self.date_part_type_from_name(descendant_name):
                result.add(descendant_name)

    return result

def date_part_type_from_name(self, field_name: str) -> str | None:
    parsed = DateFieldConsolidator.parse_date_part(field_name=field_name)

    if parsed is None:
        return None

    return parsed.get("part")

def find_field_for_primary_key(self, record, primary_key: str):
    fields = []
    fields.extend(getattr(record, "fields", []) or [])
    fields.extend(getattr(record, "mapping_fields", []) or [])

    normalized_primary_key = NameNormalizer.normalize(primary_key)
    suffix_removed_primary_key = self.remove_record_suffix(normalized_primary_key)

    for field in fields:
        field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")

        if field_name == normalized_primary_key:
            return field

        if self.remove_record_suffix(field_name) == suffix_removed_primary_key:
            return field

    return None

def datatype_for_key_field(self, key_field) -> str:
    datatype = getattr(key_field, "datatype", None)

    if datatype:

```

```

        return datatype

    picture = str(getattr(key_field, "picture", "") or "").upper()

    if "X" in picture:
        return "CHAR"

    if "9" in picture:
        return "DECIMAL"

    return "CHAR"

def length_for_key_field(self, key_field) -> int | None:
    length = getattr(key_field, "length", None)

    if length:
        try:
            return int(length)
        except Exception:
            pass

    return 20

def adjust_occurs_field(self, field):
    occurs = getattr(field, "occurs", False)
    occurs_max = getattr(field, "occurs_max", None)

    if not occurs or not occurs_max:
        return field

    length = getattr(field, "length", None)

    if length is None:
        return field

    try:
        adjusted_length = int(length) * int(occurs_max)
    except Exception:
        return field

    try:
        return field.model_copy(update={"length": adjusted_length})
    except Exception:
        try:
            field.length = adjusted_length
        except Exception:
            pass

    return field

def is_non_physical_group_field(self, field) -> bool:
    if self.is_date_field(field):
        return False

    if getattr(field, "is_group", False):
        return True

    if getattr(field, "has_child", False):
        return True

    return False

def has_physical_definition(self, field) -> bool:
    if getattr(field, "picture", None):
        return True

    if getattr(field, "datatype", None):
        return True

    if getattr(field, "length", None):
        return True

    return False

def is_date_field(self, field) -> bool:
    datatype = str(getattr(field, "datatype", "") or "").upper()

```

```

basetype = str(getattr(field, "basetype", "") or "").upper()

field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")
normalized_name = field_name.replace(" ", "_")

return (
    datatype == "DATE"
    or basetype == "DATE"
    or normalized_name.endswith("_DATE")
    or normalized_name == "DATE"
    or "_DATE_" in normalized_name
)

def is_filler_field(self, field) -> bool:
    return str(getattr(field, "name", "") or "").upper().startswith("FILLER")

def is_date_part_name(self, field_name: str) -> bool:
    return DateFieldConsolidator.parse_date_part(field_name=field_name) is not None

def remove_record_suffix(self, field_name: str) -> str:
    normalized = NameNormalizer.normalize(field_name)
    normalized = normalized.replace(" ", "_")

    return re.sub(r"_[0-9]{4}$", "", normalized)

def add_sets_and_relationships(self, schema: CanonicalSchema, metadata) -> None:
    added_relationships = set()

    for rel in getattr(metadata, "relationships", []) or []:
        set_name = getattr(rel, "set_name", None)
        owner_record = getattr(rel, "owner_record", None)
        member_record = getattr(rel, "member_record", None)

        if not set_name or not owner_record or not member_record:
            continue

        normalized_set_name = NameNormalizer.normalize(set_name)
        normalized_owner = NameNormalizer.normalize(owner_record)
        normalized_member = NameNormalizer.normalize(member_record)

        key = (
            normalized_owner,
            normalized_member,
            normalized_set_name,
        )

        if key in added_relationships:
            continue

        added_relationships.add(key)

        schema.sets.append(
            CanonicalSet(
                name=normalized_set_name,
                owner_record=normalized_owner,
                member_record=normalized_member,
            )
        )

        schema.relationships.append(
            CanonicalRelationship(
                set_name=normalized_set_name,
                parent_record=normalized_owner,
                child_record=normalized_member,
                cardinality="1:N",
            )
        )

```

```

=====
FILE: src\idms_modernizer\services\date_field_consolidator.py
ENCODING: utf-8
=====

```

```
import re
```

```

from idms_modernizer.domain.schema_models import DataField
from idms_modernizer.services.name_normalizer import NameNormalizer

```

```

class DateFieldConsolidator:
    """
    Consolidates legacy date child fields into one DATE field.

    Supports:
    - YEAR / MONTH / DAY
    - YR / MO / DY
    - Y / M / D
    - YY / MM / DD
    - YYYY / MM / DD
    - DY / DM / DD
    """

    YEAR_PARTS = {
        "YEAR",
        "YR",
        "Y",
        "YY",
        "YYYY",
        "DY",
    }

    MONTH_PARTS = {
        "MONTH",
        "MON",
        "MO",
        "M",
        "MM",
        "DM",
    }

    DAY_PARTS = {
        "DAY",
        "D",
        "DD",
    }

    @staticmethod
    def consolidate(
        fields: list[DataField],
    ) -> list[DataField]:
        result: list[DataField] = []
        consumed: set[str] = set()
        added_dates: set[str] = set()

        date_groups = DateFieldConsolidator.collect_date_groups(
            fields=fields,
        )

        for date_name, parts in date_groups.items():
            if not DateFieldConsolidator.has_complete_date_group(
                parts=parts,
            ):
                continue

            normalized_date_name = NameNormalizer.normalize(
                date_name,
            )

            if normalized_date_name in added_dates:
                continue

            template = (
                parts.get("YEAR")
                or parts.get("MONTH")
                or parts.get("DAY")
            )

            result.append(
                DataField(
                    name=date_name,
                    level=DateFieldConsolidator.parent_level(

```

```

        field=template,
    ),
    datatype="DATE",
    length=None,
    scale=None,
    picture=None,
    start_position=DateFieldConsolidator.min_start_position(
        parts=parts,
    ),
    end_position=DateFieldConsolidator.max_end_position(
        parts=parts,
    ),
    basetype="DATE",
    has_child=True,
    is_group=True,
)
)

added_dates.add(
    normalized_date_name,
)

for part_field in parts.values():
    consumed.add(
        NameNormalizer.normalize(
            part_field.name,
        )
    )

for field in fields:
    if not field or not field.name:
        continue

    normalized_name = NameNormalizer.normalize(
        field.name,
    )

    if normalized_name in consumed:
        continue

    if normalized_name in added_dates:
        continue

    result.append(
        field,
    )

return result

@staticmethod
def collect_date_groups(
    fields: list[DataField],
) -> dict[str, dict[str, DataField]]:
    groups: dict[str, dict[str, DataField]] = {}

    for field in fields:
        parsed = DateFieldConsolidator.parse_date_part(
            field_name=field.name,
        )

        if parsed is None:
            continue

        date_name = parsed["date_name"]
        part = parsed["part"]

        if date_name not in groups:
            groups[date_name] = {}

        groups[date_name][part] = field

    return groups

@staticmethod
def parse_date_part(
    field_name: str,

```

```

) -> dict[str, str] | None:
    tokens = DateFieldConsolidator.split_tokens(
        value=field_name,
    )

    if len(tokens) < 2:
        return None

    for index, token in enumerate(tokens):
        part = DateFieldConsolidator.date_part_type(
            token=token,
            tokens=tokens,
        )

        if part is None:
            continue

        date_tokens = tokens.copy()
        date_tokens[index] = "DATE"

        return {
            "date_name": " ".join(date_tokens),
            "part": part,
        }

    return None

@staticmethod
def date_part_type(
    token: str,
    tokens: list[str],
) -> str | None:
    token = token.upper()

    has_dy_dm_dd = (
        "DY" in tokens
        and "DM" in tokens
        and "DD" in tokens
    )

    if token in DateFieldConsolidator.YEAR_PARTS:
        if token == "DY" and not has_dy_dm_dd:
            return "DAY"

        return "YEAR"

    if token in DateFieldConsolidator.MONTH_PARTS:
        return "MONTH"

    if token in DateFieldConsolidator.DAY_PARTS:
        return "DAY"

    return None

@staticmethod
def has_complete_date_group(
    parts: dict[str, DataField],
) -> bool:
    return (
        "YEAR" in parts
        and "MONTH" in parts
        and "DAY" in parts
    )

@staticmethod
def parent_level(
    field: DataField | None,
) -> int | None:
    if field is None:
        return None

    level = getattr(
        field,
        "level",
        None,
    )

```



```

    if level is None:
        return None

    try:
        return max(
            int(level) - 1,
            1,
        )

    except Exception:
        return level

@staticmethod
def min_start_position(
    parts: dict[str, DataField],
) -> int | None:
    values: list[int] = []

    for field in parts.values():
        value = getattr(
            field,
            "start_position",
            None,
        )

        if value is not None:
            values.append(
                int(value),
            )

    if not values:
        return None

    return min(values)

@staticmethod
def max_end_position(
    parts: dict[str, DataField],
) -> int | None:
    values: list[int] = []

    for field in parts.values():
        value = getattr(
            field,
            "end_position",
            None,
        )

        if value is not None:
            values.append(
                int(value),
            )

    if not values:
        return None

    return max(values)

@staticmethod
def split_tokens(
    value: str,
) -> list[str]:
    normalized = NameNormalizer.normalize(
        value,
    )

    return [
        token.upper()
        for token in re.split(
            r"[\s_-]+",
            normalized,
        )
        if token
    ]

```

```
=====
FILE: src\idms_modernizer\services\db2_datatype_mapper.py
ENCODING: utf-8
=====
```

```
import re
```

```
class DB2DatatypeMapper:
```

```
    """
    Generic DB2 datatype mapper for Schema Listing conversion.
```

```
    Rules:
```

- Sheet Mapping and DB2 model must use the same datatype logic.
- Numeric COBOL PIC always maps to DECIMAL.
- Do not emit SMALLINT, INTEGER, or BIGINT from Schema Listing fields.
- Character PIC maps to CHAR / VARCHAR based on length.
- DATE and TIMESTAMP are preserved.

```
    """
```

```
    DEFAULT_CHAR_LENGTH = 1
    DEFAULT_DECIMAL_PRECISION = 18
    DEFAULT_DECIMAL_SCALE = 0
    DEBUG = False
```

```
    @staticmethod
```

```
    def map(
```

```
        field,
```

```
) -> str:
```

```
    datatype = (
        getattr(field, "datatype", None)
        or getattr(field, "data_type", None)
        or getattr(field, "type", None)
        or ""
    )
```

```
    datatype = str(datatype or "").strip().upper()
```

```
    picture = (
        getattr(field, "picture", None)
        or getattr(field, "pic", None)
        or getattr(field, "pic_clause", None)
        or ""
    )
```

```
    picture = str(picture or "").strip()
```

```
    length = DB2DatatypeMapper.first_present(
        getattr(field, "length", None),
        getattr(field, "precision", None),
        getattr(field, "logical_length", None),
        getattr(field, "storage_length", None),
        getattr(field, "physical_length", None),
        getattr(field, "byte_length", None),
        getattr(field, "bytes", None),
    )
```

```
    scale = getattr(field, "scale", None)
```

```
    if datatype == "DATE":
        return "DATE"
```

```
    if datatype in {"TIMESTAMP", "DATETIME"}:
        return "TIMESTAMP"
```

```
    if datatype == "TIME":
        return "TIME"
```

```
    if picture:
        return DB2DatatypeMapper.map_picture(
            picture=picture,
            fallback_length=length,
            fallback_scale=scale,
            fallback_datatype=datatype,
        )
```

```

        return DB2DatatypeMapper.map_without_picture(
            fallback_datatype=datatype,
            fallback_length=length,
            fallback_scale=scale,
        )

    @staticmethod
    def map_datatype(
        field,
    ) -> str:
        return DB2DatatypeMapper.map(
            field=field,
        )

    @staticmethod
    def map_picture(
        picture: str,
        fallback_length=None,
        fallback_scale=None,
        fallback_datatype: str = "",
    ) -> str:
        clean = DB2DatatypeMapper.clean_picture(
            picture=picture,
        )

        if not clean:
            return DB2DatatypeMapper.map_without_picture(
                fallback_datatype=fallback_datatype,
                fallback_length=fallback_length,
                fallback_scale=fallback_scale,
            )

        core = DB2DatatypeMapper.picture_core(
            picture=clean,
        )

        if DB2DatatypeMapper.picture_is_char(core):
            length = DB2DatatypeMapper.char_length(
                picture=core,
                fallback_length=fallback_length,
            )

            if length <= 1:
                return "CHAR(1)"

            return f"VARCHAR({length})"

        if DB2DatatypeMapper.picture_is_decimal(core):
            precision, scale = DB2DatatypeMapper.decimal_precision_scale(
                picture=core,
            )

            return DB2DatatypeMapper.format_decimal(
                precision=precision,
                scale=scale,
                force_scale=True,
            )

        if DB2DatatypeMapper.picture_is_numeric(core):
            precision = DB2DatatypeMapper.numeric_precision(
                picture=core,
                fallback_length=fallback_length,
            )

            return DB2DatatypeMapper.format_decimal(
                precision=precision,
                scale=0,
                force_scale=False,
            )

        return DB2DatatypeMapper.map_without_picture(
            fallback_datatype=fallback_datatype,
            fallback_length=fallback_length,
            fallback_scale=fallback_scale,
        )

```

```

@staticmethod
def map_without_picture(
    fallback_datatype: str,
    fallback_length=None,
    fallback_scale=None,
) -> str:
    datatype = str(fallback_datatype or "").strip().upper()

    if datatype == "DATE":
        return "DATE"

    if datatype in {"TIMESTAMP", "DATETIME"}:
        return "TIMESTAMP"

    if datatype == "TIME":
        return "TIME"

    if datatype == "SMALLINT":
        return "DECIMAL(4)"

    if datatype in {"INTEGER", "INT"}:
        return "DECIMAL(9)"

    if datatype == "BIGINT":
        return "DECIMAL(18)"

    if datatype in {"DECIMAL", "NUMERIC", "COMP", "COMP-3"}:
        precision = DB2DatatypeMapper.safe_int(
            value=fallback_length,
            default=DB2DatatypeMapper.DEFAULT_DECIMAL_PRECISION,
        )

        scale = DB2DatatypeMapper.safe_int(
            value=fallback_scale,
            default=DB2DatatypeMapper.DEFAULT_DECIMAL_SCALE,
        )

        return DB2DatatypeMapper.format_decimal(
            precision=precision,
            scale=scale,
            force_scale=scale > 0,
        )

    if datatype == "CHAR":
        length = DB2DatatypeMapper.safe_int(
            value=fallback_length,
            default=DB2DatatypeMapper.DEFAULT_CHAR_LENGTH,
        )

        return f"CHAR({length})"

    if datatype in {"VARCHAR", "DISPLAY", "TEXT", "ALPHANUMERIC"}:
        length = DB2DatatypeMapper.safe_int(
            value=fallback_length,
            default=DB2DatatypeMapper.DEFAULT_CHAR_LENGTH,
        )

        if length <= 1:
            return "CHAR(1)"

        return f"VARCHAR({length})"

    length = DB2DatatypeMapper.safe_int(
        value=fallback_length,
        default=DB2DatatypeMapper.DEFAULT_CHAR_LENGTH,
    )

    if length <= 1:
        return "CHAR(1)"

    return f"VARCHAR({length})"

@staticmethod
def clean_picture(
    picture: str,

```

```

) -> str:
    text = str(picture or "").strip()
    text = text.replace("\u00a0", " ")
    text = text.replace(".", "")
    text = re.sub(r"\s+", " ", text)

    return text.strip()

@staticmethod
def picture_core(
    picture: str | None,
) -> str:
    text = str(picture or "").upper()
    text = text.replace("PICTURE", "")
    text = text.replace("PIC", "")
    text = text.replace("COMP-3", "")
    text = re.sub(r"\bCOMP\b", "", text)
    text = text.replace("DISPLAY", "")
    text = text.replace("USAGE", "")
    text = text.replace("IS", "")
    text = text.replace(".", "")
    text = re.sub(r"\s+", "", text)

    return text.strip()

@staticmethod
def picture_is_char(
    picture: str,
) -> bool:
    text = str(picture or "").upper()

    if text.startswith("X(") and text.endswith(")"):
        return DB2DatatypeMapper.extract_parenthesized_int(text) is not None

    return bool(text) and set(text) == {"X"}

@staticmethod
def char_length(
    picture: str,
    fallback_length=None,
) -> int:
    text = str(picture or "").upper()

    if text.startswith("X(") and text.endswith(")"):
        parsed = DB2DatatypeMapper.extract_parenthesized_int(text)

        if parsed is not None:
            return parsed

    if re.fullmatch(r"X+", text):
        return len(text)

    return DB2DatatypeMapper.safe_int(
        value=fallback_length,
        default=DB2DatatypeMapper.DEFAULT_CHAR_LENGTH,
    )

@staticmethod
def picture_is_numeric(
    picture: str,
) -> bool:
    text = str(picture or "").upper()

    if text.startswith("S"):
        text = text[1:]

    return "9" in text and "V" not in text

@staticmethod
def picture_is_decimal(
    picture: str,
) -> bool:
    text = str(picture or "").upper()

    if text.startswith("S"):
        text = text[1:]

```

```

        return "9" in text and "V" in text

    @staticmethod
    def numeric_precision(
        picture: str,
        fallback_length=None,
    ) -> int:
        text = str(picture or "").upper()

        if text.startswith("S"):
            text = text[1:]

        if "V" in text:
            precision, _scale = DB2DatatypeMapper.decimal_precision_scale(
                picture=picture,
            )

            return precision

        match = re.fullmatch(r"9$(\d+)$", text)

        if match:
            return int(match.group(1))

        if re.fullmatch(r"9+", text):
            return len(text)

        precision = DB2DatatypeMapper.count_numeric_digits(
            value=text,
        )

        if precision > 0:
            return precision

        return DB2DatatypeMapper.safe_int(
            value=fallback_length,
            default=DB2DatatypeMapper.DEFAULT_DECIMAL_PRECISION,
        )

    @staticmethod
    def decimal_precision_scale(
        picture: str,
    ) -> tuple[int, int]:
        value = str(picture or "").strip().upper()

        if value.startswith("S"):
            value = value[1:]

        if "V" not in value:
            precision = DB2DatatypeMapper.numeric_precision(
                picture=value,
                fallback_length=DB2DatatypeMapper.DEFAULT_DECIMAL_PRECISION,
            )

            return precision, 0

        before_v, after_v = value.split("V", 1)

        integer_digits = DB2DatatypeMapper.count_numeric_digits(
            value=before_v,
        )

        decimal_digits = DB2DatatypeMapper.count_numeric_digits(
            value=after_v,
        )

        precision = integer_digits + decimal_digits
        scale = decimal_digits

        return precision, scale

    @staticmethod
    def count_numeric_digits(
        value: str,
    ) -> int:

```

```

text = str(value or "").strip().upper()

if text.startswith("S"):
    text = text[1:]

total = 0
index = 0

while index < len(text):
    current_character = text[index]

    if current_character not in {"9", "0"}:
        index += 1
        continue

    if index + 1 < len(text) and text[index + 1] == "(":
        close_index = text.find(")", index + 2)

        if close_index != -1:
            number_text = text[index + 2 : close_index].strip()

            if number_text.isdigit():
                total += int(number_text)
                index = close_index + 1
                continue

        total += 1
        index += 1

    return total

@staticmethod
def format_decimal(
    precision: int,
    scale: int,
    force_scale: bool = False,
) -> str:
    actual_precision = DB2DatatypeMapper.safe_int(
        value=precision,
        default=DB2DatatypeMapper.DEFAULT_DECIMAL_PRECISION,
    )

    actual_scale = DB2DatatypeMapper.safe_int(
        value=scale,
        default=DB2DatatypeMapper.DEFAULT_DECIMAL_SCALE,
    )

    if force_scale or actual_scale > 0:
        return f"DECIMAL({actual_precision},{actual_scale})"

    return f"DECIMAL({actual_precision})"

@staticmethod
def extract_parenthesized_int(
    value: str,
) -> int | None:
    match = re.search(r"$(\d+)$", str(value or ""))

    if not match:
        return None

    return int(match.group(1))

@staticmethod
def safe_int(
    value,
    default: int,
) -> int:
    try:
        if value is None:
            return default

        text = str(value).strip()

        if not text:
            return default

```

```

        return int(text)
    except Exception:
        return default

    @staticmethod
    def first_present(
        *values,
    ):
        for value in values:
            if value is None:
                continue

            if isinstance(value, str) and not value.strip():
                continue

            return value

        return None

=====
FILE: src\idms_modernizer\services\db2_generation_service.py
ENCODING: utf-8
=====

from idms_modernizer.generators.ddl_generator import (
    DDLGenerator
)

from idms_modernizer.services.db2_mapping_service import (
    DB2MappingService
)

class DB2GenerationService:

    def __init__(self):
        self.mapper = (
            DB2MappingService()
        )

        self.ddl_generator = (
            DDLGenerator()
        )

    def generate(
        self,
        canonical_schema
    ):
        db2_model = (
            self.mapper.build_db2_model(
                canonical_schema
            )
        )

        ddl = (
            self.ddl_generator.generate(
                db2_model
            )
        )

        return db2_model, ddl

=====
FILE: src\idms_modernizer\services\db2_mapping_service.py
ENCODING: utf-8
=====

import re

from idms_modernizer.domain.canonical_models import CanonicalSchema
from idms_modernizer.domain.db2_models import (

```



```

DB2Column,
DB2ForeignKey,
DB2Model,
DB2Table,
)
from idms_modernizer.services.db2_datatype_mapper import DB2DatatypeMapper

```

```

print("LOADED DB2MappingService VERSION NAMING-FK-DATE-FIX-2026-07-29")

```

```

class DB2MappingService:
    """
    Builds DB2 model from canonical schema.

    Naming rules implemented:

    1. Normal COBOL business field:
        - Start from COBOL business field.
        - Normalize separators to underscore.
        - Apply project abbreviations.
        - Remove generic trailing qualifiers where applicable.
        - Append _479<record-code>.

        Example:
        NR-CIO-FORM-AS in VMBSIC -> NR_CIOFMAS_479BSIC
        CT-RK-TGDSV in VMBFORM  -> CT_RKTGDSV_479FORM
        NR-ID-STOCK in VMBCOUP   -> NR_IDSTOCK_479COUP

    2. Date field:
        - Outer COBOL date field becomes one DB2 DATE column.
        - DB2 date name keeps DA prefix.
        - Append _479<record-code>.

        Example:
        DA-CPTA-GDIFC -> DA_CPTA_479MBFC
        DA-UB-GDIFR  -> DA_UBDATE_479MBFR

    3. Foreign key:
        - FK column reuses exact referenced/master DB2 column name.
        - No SET prefix is added to FK column names.
        - Relationship/set name is metadata only.

        Example:
        Master PK: NR_CIOFMAS_479BSIC
        Child FK : NR_CIOFMAS_479BSIC

    4. Missing PK:
        - If no primary key exists, create generated technical key:
          ID_RECORD_<record-code>
        - Type: CHAR(20)
    """

```

```

TECHNICAL_KEY_DATATYPE = "CHAR(20)"

```

```

GENERIC_TRAILING_TOKENS = {
    "SIC",
    "FC",
    "GDIFC",
    "GDIFR",
    "GDIFAR",
    "GDIF",
    "GDI",
    "REC",
    "RECORD",
}

```

```

PROJECT_ABBREVIATIONS = {
    "FORM": "FM",
    "EVPRCP": "ERCP",
}

```

```

DATE_TOKENS = {
    "DATE",
    "DT",
    "DTE",
}

```

```

        "DA",
        "YYMMDD",
        "YYYYMMDD",
        "YMD",
    }

YEAR_PARTS = {
    "YEAR",
    "YR",
    "Y",
    "YY",
    "YYYY",
    "DY",
}

MONTH_PARTS = {
    "MONTH",
    "MON",
    "MO",
    "M",
    "MM",
    "DM",
}

DAY_PARTS = {
    "DAY",
    "D",
    "DD",
}

def build_db2_model(
    self,
    schema: CanonicalSchema,
) -> DB2Model:
    print("USING DB2MappingService.build_db2_model VERSION NAMING-FK-DATE-FIX-2026-07-29")

    tables: list[DB2Table] = []

    for record in getattr(schema, "records", []) or []:
        record_name = getattr(record, "name", "") or ""
        table_name = self.normalize_table_name(record_name)

        table = DB2Table(
            name=table_name,
            columns=[],
            foreign_keys=[],
            primary_key=None,
            primary_keys=[],
        )

        declared_primary_keys = self.get_record_primary_keys(
            record=record,
        )

        normalized_primary_keys = []

        for primary_key in declared_primary_keys:
            normalized_primary_key = self.normalize_column_name(
                value=primary_key,
                record_name=record_name,
                field=None,
            )

            if normalized_primary_key and normalized_primary_key not in normalized_primary_keys:
                normalized_primary_keys.append(normalized_primary_key)

        table.primary_keys = normalized_primary_keys

        if table.primary_keys:
            table.primary_key = table.primary_keys[0]

        added_columns: set[str] = set()

        for field in getattr(record, "fields", []) or []:
            raw_field_name = getattr(field, "name", "") or ""

```

```

        column_name = self.normalize_column_name(
            value=raw_field_name,
            record_name=record_name,
            field=field,
        )

        if not column_name:
            continue

        if column_name in added_columns:
            continue

        is_primary_key = column_name in table.primary_keys

        column = DB2Column(
            name=column_name,
            datatype=DB2DatatypeMapper.map(field),
            nullable=not is_primary_key,
            primary_key=is_primary_key,
            generated=False,
            source_kind="COBOL",
        )

        table.columns.append(column)
        added_columns.add(column_name)

    self.ensure_record_primary_key(
        table=table,
        record_name=record_name,
    )

    tables.append(table)

model = DB2Model(
    tables=tables,
)

self.apply_relationship_foreign_keys(
    schema=schema,
    model=model,
)

return model

def apply_relationship_foreign_keys(
    self,
    schema: CanonicalSchema,
    model: DB2Model,
) -> None:
    table_lookup = self.build_table_lookup(
        model=model,
    )

    for relationship in getattr(schema, "relationships", []) or []:
        set_name = self.get_relationship_set_name(
            relationship=relationship,
        )

        owner_record = self.get_relationship_owner_record(
            relationship=relationship,
        )

        member_record = self.get_relationship_member_record(
            relationship=relationship,
        )

        if not owner_record or not member_record:
            continue

        owner_table = self.find_table_for_record(
            record_name=owner_record,
            table_lookup=table_lookup,
        )

        member_table = self.find_table_for_record(
            record_name=member_record,

```

```

        table_lookup=table_lookup,
    )

    if owner_table is None or member_table is None:
        continue

    owner_pk_columns = self.get_primary_key_columns(
        table=owner_table,
    )

    if not owner_pk_columns:
        owner_pk_columns = self.ensure_record_primary_key(
            table=owner_table,
            record_name=owner_record,
        )

    for owner_pk_column in owner_pk_columns:
        fk_column = self.ensure_fk_column_reusing_parent_name(
            child_table=member_table,
            parent_pk_column=owner_pk_column,
        )

        self.add_foreign_key_if_missing(
            child_table=member_table,
            child_column=fk_column.name,
            parent_table=owner_table,
            parent_column=owner_pk_column.name,
            set_name=set_name,
        )

def ensure_fk_column_reusing_parent_name(
    self,
    child_table: DB2Table,
    parent_pk_column: DB2Column,
) -> DB2Column:
    column_name = getattr(parent_pk_column, "name", "") or ""

    existing_column = self.find_column(
        table=child_table,
        column_name=column_name,
    )

    if existing_column is not None:
        existing_column.nullable = True
        existing_column.generated = True
        existing_column.primary_key = False
        existing_column.source_kind = "SET_FK"

        if not getattr(existing_column, "datatype", None):
            existing_column.datatype = getattr(parent_pk_column, "datatype", "") or ""

        return existing_column

    new_column = DB2Column(
        name=column_name,
        datatype=getattr(parent_pk_column, "datatype", "") or self.TECHNICAL_KEY_DATATYPE,
        nullable=True,
        primary_key=False,
        generated=True,
        source_kind="SET_FK",
    )

    child_table.columns.append(new_column)

    return new_column

def add_foreign_key_if_missing(
    self,
    child_table: DB2Table,
    child_column: str,
    parent_table: DB2Table,
    parent_column: str,
    set_name: str = "",
) -> None:
    if self.foreign_key_exists(
        child_table=child_table,

```

```

        child_column=child_column,
        parent_table=parent_table,
        parent_column=parent_column,
    ):
        return

    child_table.foreign_keys.append(
        DB2ForeignKey(
            column_name=child_column,
            reference_table=parent_table.name,
            reference_column=parent_column,
            set_name=self.normalize_name(set_name),
        )
    )

def foreign_key_exists(
    self,
    child_table: DB2Table,
    child_column: str,
    parent_table: DB2Table,
    parent_column: str,
) -> bool:
    normalized_child_column = self.normalize_name(child_column)
    normalized_parent_table = self.normalize_name(parent_table.name)
    normalized_parent_column = self.normalize_name(parent_column)

    for foreign_key in getattr(child_table, "foreign_keys", []) or []:
        existing_child_column = self.normalize_name(
            getattr(foreign_key, "column_name", "") or ""
        )
        existing_parent_table = self.normalize_name(
            getattr(foreign_key, "reference_table", "") or ""
        )
        existing_parent_column = self.normalize_name(
            getattr(foreign_key, "reference_column", "") or ""
        )

        if (
            existing_child_column == normalized_child_column
            and existing_parent_table == normalized_parent_table
            and existing_parent_column == normalized_parent_column
        ):
            return True

    return False

def ensure_record_primary_key(
    self,
    table: DB2Table,
    record_name: str,
) -> list[DB2Column]:
    primary_key_columns: list[DB2Column] = []

    for primary_key_name in list(getattr(table, "primary_keys", []) or []):
        column = self.find_column(
            table=table,
            column_name=primary_key_name,
        )

        if column is None:
            continue

        column.nullable = False
        column.primary_key = True

        if not getattr(column, "source_kind", ""):
            column.source_kind = "COBOL"

        primary_key_columns.append(column)

    if primary_key_columns:
        table.primary_keys = [
            column.name
            for column in primary_key_columns
        ]
        table.primary_key = table.primary_keys[0]

```

```

        return primary_key_columns

    generated_pk_name = self.generated_primary_key_name(
        record_name=record_name,
    )

    existing_generated = self.find_column(
        table=table,
        column_name=generated_pk_name,
    )

    if existing_generated is not None:
        existing_generated.nullable = False
        existing_generated.primary_key = True
        existing_generated.generated = True
        existing_generated.source_kind = "GENERATED PK"

        table.primary_keys = [existing_generated.name]
        table.primary_key = existing_generated.name

        return [existing_generated]

    generated_column = DB2Column(
        name=generated_pk_name,
        datatype=self.TECHNICAL_KEY_DATATYPE,
        nullable=False,
        primary_key=True,
        generated=True,
        source_kind="GENERATED PK",
    )

    table.columns.insert(0, generated_column)
    table.primary_keys = [generated_column.name]
    table.primary_key = generated_column.name

    return [generated_column]

def generated_primary_key_name(
    self,
    record_name: str,
) -> str:
    record_code = self.record_code(record_name)

    if record_code:
        return f"ID_RECORD_{record_code}"

    table_name = self.normalize_table_name(record_name)

    if table_name:
        return f"ID_RECORD_{table_name}"

    return "ID_RECORD"

def get_primary_key_columns(
    self,
    table: DB2Table,
) -> list[DB2Column]:
    primary_key_names = list(getattr(table, "primary_keys", []) or [])

    if not primary_key_names and getattr(table, "primary_key", None):
        primary_key_names = [table.primary_key]

    if not primary_key_names:
        primary_key_names = [
            getattr(column, "name", "") or ""
            for column in getattr(table, "columns", []) or []
            if getattr(column, "primary_key", False)
        ]

    result: list[DB2Column] = []

    for primary_key_name in primary_key_names:
        column = self.find_column(
            table=table,
            column_name=primary_key_name,

```

```

        )

        if column is not None:
            result.append(column)

    return result

def find_column(
    self,
    table: DB2Table,
    column_name: str,
) -> DB2Column | None:
    normalized_column_name = self.normalize_name(column_name)

    if not normalized_column_name:
        return None

    for column in getattr(table, "columns", []) or []:
        current_name = self.normalize_name(
            getattr(column, "name", "") or ""
        )

        if current_name == normalized_column_name:
            return column

    return None

def build_table_lookup(
    self,
    model: DB2Model,
) -> dict[str, DB2Table]:
    lookup: dict[str, DB2Table] = {}

    for table in getattr(model, "tables", []) or []:
        table_name = getattr(table, "name", "") or ""
        normalized_table_name = self.normalize_name(table_name)

        if normalized_table_name:
            lookup[normalized_table_name] = table

        suffix_removed = self.remove_record_suffix(
            normalized_table_name,
        )

        if suffix_removed:
            lookup[suffix_removed] = table

    return lookup

def find_table_for_record(
    self,
    record_name: str,
    table_lookup: dict[str, DB2Table],
) -> DB2Table | None:
    normalized_record_name = self.normalize_table_name(record_name)

    if normalized_record_name in table_lookup:
        return table_lookup[normalized_record_name]

    suffix_removed = self.remove_record_suffix(
        normalized_record_name,
    )

    if suffix_removed in table_lookup:
        return table_lookup[suffix_removed]

    for table_key, table in table_lookup.items():
        if self.remove_record_suffix(table_key) == suffix_removed:
            return table

    return None

def normalize_table_name(
    self,
    value: str | None,
) -> str:

```

```

        return self.to_db2_name(value or "")

def normalize_column_name(
    self,
    value: str | None,
    record_name: str | None,
    field=None,
) -> str:
    raw_value = str(value or "").strip()

    if not raw_value:
        return ""

    if field is not None and self.is_date_field(field=field):
        return self.date_column_name(
            field_name=raw_value,
            record_name=record_name,
        )

    return self.normal_business_column_name(
        field_name=raw_value,
        record_name=record_name,
    )

def normal_business_column_name(
    self,
    field_name: str,
    record_name: str | None,
) -> str:
    base = self.business_field_base_name(
        field_name=field_name,
    )

    record_code = self.record_code(record_name)

    if record_code:
        return f"{base}_479{record_code}"

    return base

def date_column_name(
    self,
    field_name: str,
    record_name: str | None,
) -> str:
    base = self.date_field_base_name(
        field_name=field_name,
    )

    record_code = self.record_code(record_name)

    if record_code:
        return f"{base}_479{record_code}"

    return base

def business_field_base_name(
    self,
    field_name: str,
) -> str:
    normalized = self.to_db2_name(field_name)

    normalized = self.remove_record_suffix(normalized)

    parts = [
        part
        for part in normalized.split("_")
        if part
    ]

    if not parts:
        return normalized

    parts = self.remove_generic_trailing_tokens(
        parts=parts,
    )

```



```

        parts = [
            self.PROJECT_ABBREVIATIONS.get(part, part)
            for part in parts
        ]

        if len(parts) == 1:
            return parts[0]

        first = parts[0]
        rest = "".join(parts[1:])

        if rest:
            return f"{first}_{rest}"

        return first

def date_field_base_name(
    self,
    field_name: str,
) -> str:
    normalized = self.to_db2_name(field_name)

    normalized = self.remove_record_suffix(normalized)

    parts = [
        part
        for part in normalized.split("_")
        if part
    ]

    if not parts:
        return normalized

    if parts[0] == "DA":
        main_parts = parts[1:]
    else:
        main_parts = parts

    if not main_parts:
        return "DA"

    if main_parts[0] in {"UB", "UE"}:
        return f"DA_{main_parts[0]}DATE"

    main_parts = self.apply_date_project_abbreviations(
        parts=main_parts,
    )

    main_parts = self.remove_generic_trailing_tokens(
        parts=main_parts,
    )

    if not main_parts:
        return "DA"

    if len(main_parts) == 1:
        main = main_parts[0]
    else:
        main = "".join(main_parts)

    return f"DA_{main}"

def apply_date_project_abbreviations(
    self,
    parts: list[str],
) -> list[str]:
    output: list[str] = []

    for part in parts:
        if part == "EVPRCP":
            output.append("ERCP")
            continue

        if part == "GDIFAR":
            output.append("GD")

```

```

        continue

    if part in {"GDIFC", "GDIFR"}:
        continue

    output.append(
        self.PROJECT_ABBREVIATIONS.get(part, part)
    )

    return output

def remove_generic_trailing_tokens(
    self,
    parts: list[str],
) -> list[str]:
    output = list(parts or [])

    while output and output[-1] in self.GENERIC_TRAILING_TOKENS:
        output.pop()

    return output

def record_code(
    self,
    record_name: str | None,
) -> str:
    normalized = self.to_db2_name(record_name or "")
    compact = re.sub(r"^[A-Z0-9]", "", normalized)

    if not compact:
        return ""

    if len(compact) <= 4:
        return compact

    return compact[-4:]

def is_date_field(
    self,
    field,
) -> bool:
    datatype = str(
        getattr(field, "datatype", None)
        or getattr(field, "data_type", None)
        or getattr(field, "type", None)
        or "",
    ).strip().upper()

    basetype = str(
        getattr(field, "basetype", None)
        or getattr(field, "base_type", None)
        or "",
    ).strip().upper()

    if datatype == "DATE" or basetype == "DATE":
        return True

    field_name = getattr(field, "name", "") or ""

    if self.is_date_like_name(field_name):
        picture = str(
            getattr(field, "picture", None)
            or getattr(field, "pic", None)
            or getattr(field, "pic_clause", None)
            or "",
        ).strip()

        if self.is_yyyymmdd_picture(picture):
            return True

    return False

def is_yyyymmdd_picture(
    self,
    picture: str,
) -> bool:

```

```

    if not picture:
        return False

    clean = DB2DatatypeMapper.clean_picture(
        picture=picture,
    )
    core = DB2DatatypeMapper.picture_core(
        picture=clean,
    )

    return core in {
        "9(8)",
        "S9(8)",
        "99999999",
        "S99999999",
    }

def is_date_like_name(
    self,
    value: str,
) -> bool:
    parts = self.name_parts(value)

    if not parts:
        return False

    if parts[0] == "DA":
        return True

    if any(part in self.DATE_TOKENS for part in parts):
        return True

    compact = "".join(parts)

    return any(token in compact for token in self.DATE_TOKENS)

def is_date_part_name(
    self,
    value: str,
) -> bool:
    parts = self.name_parts(value)

    for part in parts:
        if part in self.YEAR_PARTS:
            return True

        if part in self.MONTH_PARTS:
            return True

        if part in self.DAY_PARTS:
            return True

    return False

def get_record_primary_keys(
    self,
    record,
) -> list[str]:
    primary_keys: list[str] = []

    explicit_primary_keys = getattr(record, "primary_keys", None)

    if explicit_primary_keys:
        if isinstance(explicit_primary_keys, list):
            primary_keys.extend(explicit_primary_keys)
        else:
            primary_keys.append(explicit_primary_keys)

    primary_key = getattr(record, "primary_key", None)

    if primary_key:
        primary_keys.append(primary_key)

    cleaned: list[str] = []

    for primary_key_value in primary_keys:

```

```

        normalized = self.normalize_name(
            primary_key_value,
        )

        if not normalized:
            continue

        if normalized in cleaned:
            continue

        cleaned.append(normalized)

    return cleaned

def get_relationship_set_name(
    self,
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("set_name")
            or relationship.get("set")
            or relationship.get("name")
            or "",
        )

    return str(
        getattr(relationship, "set_name", None)
        or getattr(relationship, "set", None)
        or getattr(relationship, "name", None)
        or "",
    )

def get_relationship_owner_record(
    self,
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("owner_record")
            or relationship.get("parent_record")
            or relationship.get("owner")
            or relationship.get("parent")
            or "",
        )

    return str(
        getattr(relationship, "owner_record", None)
        or getattr(relationship, "parent_record", None)
        or getattr(relationship, "owner", None)
        or getattr(relationship, "parent", None)
        or "",
    )

def get_relationship_member_record(
    self,
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("member_record")
            or relationship.get("child_record")
            or relationship.get("member")
            or relationship.get("child")
            or "",
        )

    return str(
        getattr(relationship, "member_record", None)
        or getattr(relationship, "child_record", None)
        or getattr(relationship, "member", None)
        or getattr(relationship, "child", None)
        or "",
    )

def normalize_name(

```

```

        self,
        value: str | None,
    ) -> str:
        return self.to_db2_name(value or "")

    def to_db2_name(
        self,
        value: str | None,
    ) -> str:
        text = str(value or "").strip().upper()

        text = text.replace("\u00a0", " ")
        text = text.replace("\t", " ")

        text = re.sub(r"^[A-Z0-9]+", "_", text)
        text = re.sub(r"_+", "_", text)
        text = text.strip("_")

        return text

    def name_parts(
        self,
        value: str | None,
    ) -> list[str]:
        normalized = self.to_db2_name(value or "")

        return [
            part
            for part in normalized.split("_")
            if part
        ]

    def remove_record_suffix(
        self,
        name: str | None,
    ) -> str:
        value = str(name or "").strip().upper()

        value = re.sub(r"[_\-\s]+[0-9]{4}$", "", value)
        value = re.sub(r"[0-9]{4}$", "", value)
        value = re.sub(r"[_\-\s]+$", "", value)

        return value

```

```

=====
FILE: src\idms_modernizer\services\db2_model_builder.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.canonical_models import CanonicalSchema
from idms_modernizer.domain.db2_models import DB2Model
from idms_modernizer.services.db2_mapping_service import DB2MappingService

```

```

class DB2ModelBuilder:

    def __init__(self):
        self.mapping_service = DB2MappingService()

    def build(
        self,
        schema: CanonicalSchema
    ) -> DB2Model:

        return self.mapping_service.build_db2_model(
            schema
        )

```

```

=====
FILE: src\idms_modernizer\services\db2_model_service.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.db2_models import (

```

```

    DB2Column,
    DB2Model,
    DB2Table
)

```

```

class DB2ModelService:

```

```

    def build(
        self,
        metadata
    ) -> DB2Model:

        tables = []

        for record in metadata.records:

            columns = []

            for field in record.fields:

                columns.append(
                    DB2Column(
                        name=field.name,
                        datatype="VARCHAR(255)"
                    )
                )

            tables.append(
                DB2Table(
                    name=record.name,
                    columns=columns,
                    foreign_keys=[]
                )
            )

        return DB2Model(
            tables=tables
        )

```

```

=====
FILE: src\idms_modernizer\services\db2_name_normalizer.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.services.name_normalizer import NameNormalizer

```

```

class DB2NameNormalizer:

```

```

    REMOVE_RECORD_SUFFIX = False

```

```

    @staticmethod
    def normalize_record_name(
        name: str | None
    ) -> str:

        return NameNormalizer.normalize(name)

```

```

    @staticmethod
    def normalize_column_name(
        name: str | None
    ) -> str:

        normalized = NameNormalizer.normalize(name)

        if not DB2NameNormalizer.REMOVE_RECORD_SUFFIX:
            return normalized

        parts = normalized.split("_")

        if (
            len(parts) > 1
            and parts[-1].isdigit()
            and len(parts[-1]) == 4

```

```

):
    return "_".join(parts[:-1])

return normalized

```

```

=====
FILE: src\idms_modernizer\services\document_segmenter.py
ENCODING: utf-8
=====

```

```

import re

from idms_modernizer.domain.section_models import (
    RecordSection
)

class DocumentSegmenter:

    RECORD_PATTERN = re.compile(
        r"RECORD\s+NAME.*?([A-Z0-9\-]+)",
        re.IGNORECASE
    )

    def segment(
        self,
        document
    ) -> list[RecordSection]:

        sections = []

        current_section = None

        for page in document.pages:

            for line in page.lines:

                text = line.text.strip()

                record_match = (
                    self.RECORD_PATTERN.search(
                        text
                    )
                )

                if record_match:

                    if current_section:

                        sections.append(
                            current_section
                        )

                        current_section = (
                            RecordSection(
                                record_name=record_match.group(1),
                                lines=[]
                            )
                        )

                    elif current_section:

                        current_section.lines.append(
                            text
                        )

                if current_section:

                    sections.append(
                        current_section
                    )

            return sections

```

```
=====
FILE: src\idms_modernizer\services\er_diagram_generator.py
ENCODING: utf-8
=====
```

```
import os
from pathlib import Path
from graphviz import Digraph

from idms_modernizer.domain.schema_models import (
    SchemaMetadata
)

def generate_er_diagram(
    metadata: SchemaMetadata,
    output_path: str
) -> str:
    """Generate a simple ER diagram PNG from the canonical metadata.

    Nodes are records and edges are relationships (labeled with set name
    and cardinality if present).
    Returns the path to the generated PNG file.
    """

    out = Path(output_path)
    out.parent.mkdir(parents=True, exist_ok=True)
    os.environ["PATH"] += (
        os.pathsep +
        r"C:\Program Files\Graphviz\bin"
    )

    dot = Digraph("er", format="png")
    dot.attr(rankdir="LR")
    dot.attr("node", shape="box")

    # Add record nodes
    for record in metadata.records:
        # simple label: record name
        dot.node(record.name, label=record.name)

    # Add relationships
    for rel in metadata.relationships:
        owner = rel.owner_record
        member = rel.member_record
        label = rel.set_name
        if getattr(rel, "cardinality", None):
            label = f"{label}\n{rel.cardinality}"

        dot.edge(owner, member, label=label)

    # Render to the requested path (without extension)
    filename_no_ext = str(out.with_suffix(""))

    try:
        dot.render(
            filename=filename_no_ext,
            cleanup=True
        )
    except Exception as ex:
        raise RuntimeError(
            "Graphviz not installed. "
            "Install Graphviz and add "
            "'C:\\Program Files\\Graphviz\\bin' "
            "to PATH."
        ) from ex

    # Graphviz writes the .png next to filename_no_ext
    png_path = filename_no_ext + ".png"

    return png_path
```



```
=====
FILE: src\idms_modernizer\services\excel_sheet_mapping_service.py
ENCODING: utf-8
=====
```

```
import re

from idms_modernizer.domain.db2_models import (
    DB2Model,
    DB2Table,
    DB2Column,
)
from idms_modernizer.domain.schema_models import SchemaMetadata
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.services.db2_datatype_mapper import DB2DatatypeMapper

print("LOADED ExcelSheetMappingService VERSION FULL-LATEST-CALC-PIC-DATE-FK-NAMING-2026-07-29")

class ExcelSheetMappingService:
    """
    Builds Excel Sheet Mapping rows.

    Rules:
    - Always add level 01 record row if missing.
    - Include all outer/group rows from mapping_fields.
    - Group rows do not create DB2 physical field names.
    - Outer CALC group does not show CALC.
    - Inner groups under CALC also do not show CALC or PK.
    - Only elementary non-date descendants of CALC show CALC and PK.
    - Date groups and date parts inside CALC show neither CALC nor PK.
    - FILLER rows are COBOL-only.
    - Outer date group shows DB2 record and DATE datatype.
    - Date parts remain visible but DB2 mapping stays blank.
    - Sheet Mapping DB2 datatype is calculated directly from IDMS PIC Clause.
    - Field end position = STRT + LGTH - 1.
    - SET/FK rows reuse referenced/master DB2 column names.
    - DB2 field names use project naming rules and _479<record-code>.
    """

    COLUMNS = [
        "Cobol Record IDMS",
        "Cobol Zone",
        "IDMS Key",
        "IDMS PIC Clause",
        "Length of Field Bytes",
        "Field end position",
        "DB2 Key",
        "New DB2 Record",
        "New DB2 Field name",
        "New DB2 Data Type",
        "Hopex Expression TypeRemark",
        "Relation",
        "Reference Field Name (CopyBook) ",
        "Reference Field PIC Clause",
        "Cross Application DB2 Field Name",
        "Cross Appln DB2 Data Type",
        "Basetype",
    ]

    YEAR_PARTS = {
        "YEAR",
        "YR",
        "Y",
        "YY",
        "YYYY",
        "DY",
    }

    MONTH_PARTS = {
        "MONTH",
        "MON",
        "MO",
        "M",
        "MM",
    }
```

```

        "DM",
    }

    DAY_PARTS = {
        "DAY",
        "D",
        "DD",
    }

    DATE_TOKENS = {
        "DATE",
        "DT",
        "DTE",
        "DA",
        "YYMMDD",
        "YYYYMMDD",
        "YMD",
    }

    DATE_CHILD_TOKENS = YEAR_PARTS | MONTH_PARTS | DAY_PARTS

    PROJECT_ABBREVIATIONS = {
        "FORM": "FM",
        "EVPRCP": "ERCP",
    }

    GENERIC_TRAILING_TOKENS = {
        "SIC",
        "FC",
        "GDIFC",
        "GDIFR",
        "GDIFAR",
        "GDIF",
        "GDI",
        "REC",
        "RECORD",
    }

    DATE_TRAILING_TOKENS = {
        "SIC",
        "FC",
        "GDIFC",
        "GDIFR",
        "GDIF",
        "GDI",
    }

    def build(
        self,
        metadata: SchemaMetadata,
        db2_model: DB2Model,
    ) -> list[dict[str, str]]:
        print("USING ExcelSheetMappingService.build VERSION FULL-LATEST-CALC-PIC-DATE-FK-NAMING-2026-07-29")

        rows: list[dict[str, str]] = []

        table_lookup = self.build_table_lookup(
            db2_model=db2_model,
        )

        relationship_lookup = self.build_relationship_lookup(
            metadata=metadata,
        )

        for record in getattr(metadata, "records", []) or []:
            mapping_fields = (
                getattr(record, "mapping_fields", None)
                or getattr(record, "fields", None)
                or []
            )

            if not self.has_level_01_row(
                record=record,
                mapping_fields=mapping_fields,
            ):
                rows.append(

```

```

        self.build_record_level_01_row(
            record=record,
        )
    )

    table = self.find_table_for_record(
        record_name=getattr(record, "name", "") or "",
        table_lookup=table_lookup,
    )

    rows.extend(
        self.build_record_mapping_rows(
            record=record,
            table=table,
            relationship_lookup=relationship_lookup,
        )
    )

    rows.extend(
        self.build_generated_pk_rows(
            record=record,
            table=table,
        )
    )

    rows.extend(
        self.build_audit_rows(
            record=record,
            table=table,
        )
    )

    rows.extend(
        self.build_set_fk_rows_for_record(
            metadata=metadata,
            record=record,
            table=table,
            table_lookup=table_lookup,
        )
    )

    rows = self.force_pic_based_datatypes(
        rows=rows,
    )

    return rows

def build_record_mapping_rows(
    self,
    record,
    table: DB2Table | None,
    relationship_lookup: dict[str, str],
) -> list[dict[str, str]]:
    rows: list[dict[str, str]] = []

    mapping_fields = (
        getattr(record, "mapping_fields", None)
        or getattr(record, "fields", None)
        or []
    )

    column_lookup = (
        self.build_column_lookup(table=table)
        if table is not None
        else {}
    )

    date_scope = self.build_date_scope(
        fields=mapping_fields,
    )

    date_group_info = self.collect_date_group_info(
        fields=mapping_fields,
    )

    calc_scope = self.build_calc_scope(

```

```

        record=record,
        fields=mapping_fields,
    )

    for field in mapping_fields:
        if self.is_filler_field(field=field):
            rows.append(
                self.build_filler_row(
                    record=record,
                    field=field,
                )
            )
            continue

        is_group = self.is_group_field(
            field=field,
        )

        is_outer_date = self.is_outer_date_group(
            field=field,
            date_group_info=date_group_info,
        )

        is_date_child = self.is_date_child(
            field=field,
            date_scope=date_scope,
        )

        is_direct_date = self.is_direct_date_field(
            field=field,
        )

        is_date = (
            is_outer_date
            or is_date_child
            or is_direct_date
        )

        calc_status = self.calc_status_for_field(
            field=field,
            calc_scope=calc_scope,
        )

        db2_column = self.find_column_for_field(
            record=record,
            field=field,
            column_lookup=column_lookup,
        )

        db2_key = self.db2_key_label(
            table=table,
            column=db2_column,
        )

        if calc_status == "ROOT":
            db2_key = ""

        if is_group:
            db2_key = ""

        if calc_status == "DESCENDANT" and not is_date and not is_group:
            db2_key = self.add_pk_to_db2_key(
                db2_key=db2_key,
            )

        if is_date:
            db2_key = ""

        row = self.empty_row()

        row["Cobol Record IDMS"] = getattr(record, "name", "") or ""
        row["Cobol Zone"] = self.format_cobol_zone(field=field)
        row["IDMS Key"] = self.idms_key_label(
            calc_status=calc_status,
            is_date=is_date,
            is_group=is_group,

```

```

        db2_key=db2_key,
    )
    row["IDMS PIC Clause"] = self.get_field_picture(field=field)
    row["Length of Field Bytes"] = self.to_string(
        self.get_field_length(field=field)
    )
    row["Field end position"] = self.to_string(
        self.get_field_end_position(field=field)
    )
    row["DB2 Key"] = db2_key

    if is_outer_date:
        row["New DB2 Record"] = self.db2_record_name(
            record=record,
            table=table,
        )
        row["New DB2 Field name"] = self.convert_date_field_to_db2_field_name(
            record=record,
            field=field,
        )
        row["New DB2 Data Type"] = "DATE"
        row["DB2 Key"] = ""

    elif is_date_child:
        row["New DB2 Record"] = ""
        row["New DB2 Field name"] = ""
        row["New DB2 Data Type"] = ""
        row["DB2 Key"] = ""

    elif is_direct_date:
        row["New DB2 Record"] = self.db2_record_name(
            record=record,
            table=table,
        )
        row["New DB2 Field name"] = self.convert_date_field_to_db2_field_name(
            record=record,
            field=field,
        )
        row["New DB2 Data Type"] = "DATE"
        row["DB2 Key"] = ""

    elif calc_status == "ROOT":
        row["New DB2 Record"] = ""
        row["New DB2 Field name"] = ""
        row["New DB2 Data Type"] = ""
        row["DB2 Key"] = ""

    elif is_group:
        row["New DB2 Record"] = ""
        row["New DB2 Field name"] = ""
        row["New DB2 Data Type"] = ""
        row["DB2 Key"] = ""

    elif self.should_emit_db2_field(field=field):
        row["New DB2 Record"] = self.db2_record_name(
            record=record,
            table=table,
        )

        if db2_column is not None and getattr(db2_column, "name", None):
            row["New DB2 Field name"] = getattr(db2_column, "name", "") or ""
        else:
            row["New DB2 Field name"] = self.convert_cobol_zone_to_db2_field_name(
                record=record,
                field=field,
            )

        row["New DB2 Data Type"] = self.infer_db2_datatype_from_field(
            field=field,
        )

    row["Hopex Expression TypeRemark"] = self.hopex_remark(
        field=field,
    )
    row["Relation"] = relationship_lookup.get(
        NameNormalizer.normalize(getattr(record, "name", "") or ""),

```

```

        "",
    )
    row["Basetype"] = self.get_field_basetype(
        field=field,
    )

    rows.append(row)

return rows

def build_calc_scope(
    self,
    record,
    fields,
) -> dict[str, str]:
    scope: dict[str, str] = {}

    primary_key = getattr(record, "primary_key", None)

    if not primary_key:
        return scope

    normalized_primary_key = NameNormalizer.normalize(primary_key)
    suffix_removed_primary_key = self.remove_record_suffix(
        normalized_primary_key,
    )

    mapping_fields = list(fields or [])

    key_field = None
    key_index = None

    for index, field in enumerate(mapping_fields):
        field_name = NameNormalizer.normalize(
            getattr(field, "name", "") or ""
        )

        if not field_name:
            continue

        if field_name == normalized_primary_key:
            key_field = field
            key_index = index
            break

        if self.remove_record_suffix(field_name) == suffix_removed_primary_key:
            key_field = field
            key_index = index
            break

    if key_field is None or key_index is None:
        if normalized_primary_key:
            scope[normalized_primary_key] = "DESCENDANT"
        return scope

    key_name = NameNormalizer.normalize(
        getattr(key_field, "name", "") or ""
    )

    if not key_name:
        return scope

    key_is_group = self.is_group_field(
        field=key_field,
    )

    key_level = getattr(key_field, "level", None)

    if not key_is_group:
        scope[key_name] = "DESCENDANT"
        return scope

    scope[key_name] = "ROOT"

    if key_level is None:
        return scope

```

```

try:
    key_level_int = int(key_level)
except Exception:
    return scope

for child_field in mapping_fields[key_index + 1:]:
    child_name = NameNormalizer.normalize(
        getattr(child_field, "name", "") or ""
    )
    child_level = getattr(child_field, "level", None)

    if child_level is None:
        continue

    try:
        child_level_int = int(child_level)
    except Exception:
        continue

    if child_level_int <= key_level_int:
        break

    if child_name:
        scope[child_name] = "DESCENDANT"

return scope

def calc_status_for_field(
    self,
    field,
    calc_scope: dict[str, str],
) -> str:
    field_name = NameNormalizer.normalize(
        getattr(field, "name", "") or ""
    )

    if not field_name:
        return ""

    if field_name in calc_scope:
        return calc_scope[field_name]

    suffix_removed = self.remove_record_suffix(field_name)

    for key, value in calc_scope.items():
        if self.remove_record_suffix(key) == suffix_removed:
            return value

    return ""

def idms_key_label(
    self,
    calc_status: str,
    is_date: bool,
    is_group: bool,
    db2_key: str,
) -> str:
    if is_date:
        return ""

    if is_group:
        return ""

    labels: list[str] = []

    if calc_status == "DESCENDANT":
        labels.append("CALC")

    db2_key_parts = [
        part.strip().upper()
        for part in str(db2_key or "").split("/")
        if part.strip()
    ]

    if "FK" in db2_key_parts:

```

```

        labels.append("SET")

    output: list[str] = []

    for label in labels:
        if label not in output:
            output.append(label)

    return "; ".join(output)

def add_pk_to_db2_key(
    self,
    db2_key: str,
) -> str:
    parts = [
        part.strip().upper()
        for part in str(db2_key or "").split("/")
        if part.strip()
    ]

    if "PK" not in parts:
        parts.insert(0, "PK")

    ordered_parts: list[str] = []

    if "PK" in parts:
        ordered_parts.append("PK")

    if "FK" in parts:
        ordered_parts.append("FK")

    return "/".join(ordered_parts)

def force_pic_based_datatypes(
    self,
    rows: list[dict[str, str]],
) -> list[dict[str, str]]:
    corrected_rows: list[dict[str, str]] = []

    for row in rows or []:
        pic_clause = str(row.get("IDMS PIC Clause", "") or "").strip()
        db2_record = str(row.get("New DB2 Record", "") or "").strip()
        db2_field = str(row.get("New DB2 Field name", "") or "").strip()
        current_type = str(row.get("New DB2 Data Type", "") or "").strip().upper()

        if current_type == "DATE":
            corrected_rows.append(row)
            continue

        if pic_clause and db2_record and db2_field:
            row["New DB2 Data Type"] = DB2DatatypeMapper.map_picture(
                picture=pic_clause,
                fallback_length=None,
                fallback_scale=None,
            )

        corrected_rows.append(row)

    return corrected_rows

def build_table_lookup(
    self,
    db2_model: DB2Model,
) -> dict[str, DB2Table]:
    lookup: dict[str, DB2Table] = {}

    if db2_model is None:
        return lookup

    for table in getattr(db2_model, "tables", []) or []:
        table_name = getattr(table, "name", "") or ""

        for key in self.name_lookup_keys(table_name):
            lookup[key] = table

    return lookup

```



```

def build_column_lookup(
    self,
    table: DB2Table,
) -> dict[str, DB2Column]:
    lookup: dict[str, DB2Column] = {}

    for column in getattr(table, "columns", []) or []:
        column_name = getattr(column, "name", "") or ""

        for key in self.name_lookup_keys(column_name):
            lookup[key] = column

    return lookup

def build_relationship_lookup(
    self,
    metadata: SchemaMetadata,
) -> dict[str, str]:
    lookup: dict[str, str] = {}

    for relationship in getattr(metadata, "relationships", []) or []:
        owner_record = self.get_relationship_owner_record(relationship=relationship)
        member_record = self.get_relationship_member_record(relationship=relationship)
        set_name = self.get_relationship_set_name(relationship=relationship)

        for record_name in [owner_record, member_record]:
            normalized_record = NameNormalizer.normalize(record_name)

            if not normalized_record:
                continue

            existing = lookup.get(normalized_record, "")

            if existing:
                if set_name and set_name not in existing:
                    lookup[normalized_record] = f"{existing}; {set_name}"
            else:
                lookup[normalized_record] = set_name

    return lookup

def build_date_scope(
    self,
    fields,
) -> dict[int, bool]:
    scope: dict[int, bool] = {}
    active_date_levels: list[int] = []

    for field in fields or []:
        level = self.safe_int(getattr(field, "level", None), default=0)

        active_date_levels = [
            current_level
            for current_level in active_date_levels
            if current_level < level
        ]

        if active_date_levels:
            scope[id(field)] = True

        if self.is_potential_outer_date(field=field):
            active_date_levels.append(level)

    return scope

def collect_date_group_info(
    self,
    fields,
) -> dict[str, dict]:
    info: dict[str, dict] = {}

    for index, field in enumerate(fields or []):
        if not self.is_group_field(field=field):
            continue

```

```

        if not self.is_potential_outer_date(field=field):
            continue

        if not self.group_has_ymd_children(
            field=field,
            fields=fields,
            start_index=index,
        ):
            continue

        field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")

        if field_name:
            info[field_name] = {"field": field}

    return info

def group_has_ymd_children(
    self,
    field,
    fields,
    start_index: int,
) -> bool:
    parent_level = self.safe_int(getattr(field, "level", None), default=0)

    found_year = False
    found_month = False
    found_day = False

    for child in list(fields or [])[start_index + 1:]:
        child_level = self.safe_int(getattr(child, "level", None), default=0)

        if child_level <= parent_level:
            break

        child_name = getattr(child, "name", "") or ""
        parts = self.name_parts(child_name)

        if any(part in self.YEAR_PARTS for part in parts):
            found_year = True

        if any(part in self.MONTH_PARTS for part in parts):
            found_month = True

        if any(part in self.DAY_PARTS for part in parts):
            found_day = True

    return found_year and found_month and found_day

def is_outer_date_group(
    self,
    field,
    date_group_info: dict[str, dict],
) -> bool:
    if not self.is_group_field(field=field):
        return False

    field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")

    return field_name in date_group_info

def is_date_child(
    self,
    field,
    date_scope: dict[int, bool],
) -> bool:
    if id(field) not in date_scope:
        return False

    return self.is_date_part_name(field_name=getattr(field, "name", "") or "")

def is_direct_date_field(
    self,
    field,
) -> bool:
    if self.is_group_field(field=field):

```

```

        return False

    datatype = str(
        getattr(field, "datatype", None)
        or getattr(field, "data_type", None)
        or getattr(field, "type", None)
        or "",
    ).strip().upper()

    basetype = str(
        getattr(field, "basetype", None)
        or getattr(field, "base_type", None)
        or "",
    ).strip().upper()

    if datatype == "DATE" or basetype == "DATE":
        return not self.is_date_part_name(field_name=getattr(field, "name", "") or "")

    pic_clause = self.get_field_picture(field=field)

    if self.is_date_like_name(getattr(field, "name", "") or "") and self.is_yyyymmdd_picture(pic_clause):
        return True

    return False

def is_potential_outer_date(
    self,
    field,
) -> bool:
    field_name = getattr(field, "name", "") or ""

    if self.is_date_like_name(field_name):
        return True

    datatype = str(
        getattr(field, "datatype", None)
        or getattr(field, "data_type", None)
        or getattr(field, "type", None)
        or "",
    ).strip().upper()

    basetype = str(
        getattr(field, "basetype", None)
        or getattr(field, "base_type", None)
        or "",
    ).strip().upper()

    return datatype == "DATE" or basetype == "DATE"

def is_date_part_name(
    self,
    field_name: str,
) -> bool:
    parts = self.name_parts(field_name or "")

    return any(part in self.DATE_CHILD_TOKENS for part in parts)

def is_yyyymmdd_picture(
    self,
    pic_clause: str,
) -> bool:
    clean = DB2DatatypeMapper.clean_picture(pic_clause)
    core = DB2DatatypeMapper.picture_core(clean)

    return core in {"9(8)", "S9(8)", "99999999", "S99999999"}

def is_date_like_name(
    self,
    value: str,
) -> bool:
    parts = self.name_parts(value or "")

    if any(part in self.DATE_TOKENS for part in parts):
        return True

    if parts and parts[0] == "DA":

```

```

        return True

compact = "".join(parts)

return any(token in compact for token in self.DATE_TOKENS)

def find_table_for_record(
    self,
    record_name: str,
    table_lookup: dict[str, DB2Table],
) -> DB2Table | None:
    for key in self.name_lookup_keys(record_name):
        if key in table_lookup:
            return table_lookup[key]

    return None

def find_column_for_field(
    self,
    record,
    field,
    column_lookup: dict[str, DB2Column],
) -> DB2Column | None:
    field_name = getattr(field, "name", "") or ""

    if not field_name:
        return None

    possible_names = [
        field_name,
        self.remove_record_suffix(field_name),
        self.cobol_field_base_name(field_name=field_name),
        self.date_field_base_name(field_name=field_name),
        self.convert_cobol_zone_to_db2_field_name(record=record, field=field),
        self.convert_date_field_to_db2_field_name(record=record, field=field),
    ]

    for name in possible_names:
        column = self.find_matching_column(
            normalized_column_name=name,
            column_lookup=column_lookup,
        )

        if column is not None:
            return column

    return None

def find_matching_column(
    self,
    normalized_column_name: str,
    column_lookup: dict[str, DB2Column],
) -> DB2Column | None:
    for key in self.name_lookup_keys(normalized_column_name):
        if key in column_lookup:
            return column_lookup[key]

    suffix_removed = self.remove_record_suffix(normalized_column_name)

    for key in self.name_lookup_keys(suffix_removed):
        if key in column_lookup:
            return column_lookup[key]

    target_suffix_removed = self.remove_record_suffix(
        self.to_db2_name(normalized_column_name)
    )

    for current_name, column in column_lookup.items():
        if self.remove_record_suffix(current_name) == target_suffix_removed:
            return column

    return None

def build_filler_row(
    self,
    record,

```

```

        field,
    ) -> dict[str, str]:
        row = self.empty_row()

        row["Cobol Record IDMS"] = getattr(record, "name", "") or ""
        row["Cobol Zone"] = self.format_cobol_zone(field=field)
        row["IDMS PIC Clause"] = self.get_field_picture(field=field)
        row["Length of Field Bytes"] = self.to_string(self.get_field_length(field=field))
        row["Field end position"] = self.to_string(self.get_field_end_position(field=field))

        return row

def build_generated_pk_rows(
    self,
    record,
    table: DB2Table | None,
) -> list[dict[str, str]]:
    if table is None:
        return []

    rows: list[dict[str, str]] = []

    for column in getattr(table, "columns", []) or []:
        if getattr(column, "source_kind", "") != "GENERATED PK":
            continue

        row = self.empty_row()
        row["Cobol Record IDMS"] = getattr(record, "name", "") or ""
        row["DB2 Key"] = "PK"
        row["New DB2 Record"] = getattr(table, "name", "") or ""
        row["New DB2 Field name"] = getattr(column, "name", "") or ""
        row["New DB2 Data Type"] = self.get_db2_datatype(db2_column=column)
        rows.append(row)

    return rows

def build_audit_rows(
    self,
    record,
    table: DB2Table | None,
) -> list[dict[str, str]]:
    if table is None:
        return []

    rows: list[dict[str, str]] = []
    record_name = getattr(record, "name", "") or ""
    normalized_record_name = self.to_db2_name(record_name)

    audit_fields = [
        (f"TS_CREATE_{normalized_record_name}", "TIMESTAMP"),
        (f"TS_UPDATE_{normalized_record_name}", "TIMESTAMP"),
        (f"ID_USERID_{normalized_record_name}", "CHAR(8)"),
    ]

    for field_name, datatype in audit_fields:
        row = self.empty_row()
        row["Cobol Record IDMS"] = record_name
        row["New DB2 Record"] = getattr(table, "name", "") or ""
        row["New DB2 Field name"] = field_name
        row["New DB2 Data Type"] = datatype
        rows.append(row)

    return rows

def build_set_fk_rows_for_record(
    self,
    metadata: SchemaMetadata,
    record,
    table: DB2Table | None,
    table_lookup: dict[str, DB2Table],
) -> list[dict[str, str]]:
    if table is None:
        return []

    rows: list[dict[str, str]] = []
    emitted: set[tuple[str, str, str]] = set()

```

```

rows.extend(
    self.build_set_fk_rows_from_db2_foreign_keys(
        record=record,
        table=table,
        emitted=emitted,
    )
)

rows.extend(
    self.build_set_fk_rows_from_metadata_relationships(
        metadata=metadata,
        record=record,
        member_table=table,
        table_lookup=table_lookup,
        emitted=emitted,
    )
)

return rows

def build_set_fk_rows_from_db2_foreign_keys(
    self,
    record,
    table: DB2Table,
    emitted: set[tuple[str, str, str, str]],
) -> list[dict[str, str]]:
    rows: list[dict[str, str]] = []
    cobol_record_name = getattr(record, "name", "") or ""

    column_lookup = self.build_column_lookup(table=table)

    for foreign_key in getattr(table, "foreign_keys", []) or []:
        fk_column_name = self.get_foreign_key_column_name(foreign_key=foreign_key)

        if not fk_column_name:
            continue

        fk_column = self.find_matching_column(
            normalized_column_name=fk_column_name,
            column_lookup=column_lookup,
        )

        if fk_column is None:
            continue

        fk_datatype = self.get_db2_datatype(db2_column=fk_column)
        set_name = getattr(foreign_key, "set_name", "") or ""

        key = (
            NameNormalizer.normalize(cobol_record_name),
            NameNormalizer.normalize(getattr(table, "name", "") or ""),
            NameNormalizer.normalize(getattr(fk_column, "name", "") or ""),
            fk_datatype,
        )

        if key in emitted:
            continue

        emitted.add(key)

        row = self.empty_row()
        row["Cobol Record IDMS"] = cobol_record_name
        row["IDMS Key"] = "SET"
        row["DB2 Key"] = "FK"
        row["New DB2 Record"] = getattr(table, "name", "") or ""
        row["New DB2 Field name"] = getattr(fk_column, "name", "") or ""
        row["New DB2 Data Type"] = fk_datatype
        row["Relation"] = set_name
        rows.append(row)

    return rows

def build_set_fk_rows_from_metadata_relationships(
    self,
    metadata: SchemaMetadata,

```

```

record,
member_table: DB2Table,
table_lookup: dict[str, DB2Table],
emitted: set[tuple[str, str, str, str]],
) -> list[dict[str, str]]:
    rows: list[dict[str, str]] = []

    cobol_record_name = getattr(record, "name", "") or ""
    normalized_current_record = NameNormalizer.normalize(cobol_record_name)
    normalized_current_table = NameNormalizer.normalize(getattr(member_table, "name", "") or "")

    for relationship in getattr(metadata, "relationships", []) or []:
        set_name = self.get_relationship_set_name(relationship=relationship)
        owner_record_name = self.get_relationship_owner_record(relationship=relationship)
        member_record_name = self.get_relationship_member_record(relationship=relationship)

        if not set_name:
            continue

        normalized_member_record = NameNormalizer.normalize(member_record_name)

        if normalized_member_record not in {normalized_current_record, normalized_current_table}:
            continue

        owner_table = self.find_table_for_record(
            record_name=owner_record_name,
            table_lookup=table_lookup,
        )

        owner_pk_column = self.find_primary_key_column(table=owner_table)

        if owner_pk_column is None:
            continue

        fk_column_name = getattr(owner_pk_column, "name", "") or ""
        fk_datatype = self.get_db2_datatype(db2_column=owner_pk_column)

        key = (
            NameNormalizer.normalize(cobol_record_name),
            NameNormalizer.normalize(getattr(member_table, "name", "") or ""),
            NameNormalizer.normalize(fk_column_name),
            fk_datatype,
        )

        if key in emitted:
            continue

        emitted.add(key)

        row = self.empty_row()
        row["Cobol Record IDMS"] = cobol_record_name
        row["IDMS Key"] = "SET"
        row["DB2 Key"] = "FK"
        row["New DB2 Record"] = getattr(member_table, "name", "") or ""
        row["New DB2 Field name"] = fk_column_name
        row["New DB2 Data Type"] = fk_datatype
        row["Relation"] = set_name
        rows.append(row)

    return rows

def find_primary_key_column(
    self,
    table: DB2Table | None,
) -> DB2Column | None:
    if table is None:
        return None

    primary_key_names = list(getattr(table, "primary_keys", []) or [])

    if not primary_key_names and getattr(table, "primary_key", None):
        primary_key_names = [table.primary_key]

    normalized_primary_keys = {
        NameNormalizer.normalize(primary_key)
        for primary_key in primary_key_names
    }

```

```

        if primary_key
    }

    for column in getattr(table, "columns", []) or []:
        column_name = NameNormalizer.normalize(getattr(column, "name", "") or "")

        if column_name in normalized_primary_keys:
            return column

        if getattr(column, "primary_key", False):
            return column

    columns = list(getattr(table, "columns", []) or [])

    return columns[0] if columns else None

def get_foreign_key_column_name(
    self,
    foreign_key,
) -> str:
    if isinstance(foreign_key, dict):
        return str(
            foreign_key.get("column_name")
            or foreign_key.get("column")
            or foreign_key.get("fk_column")
            or foreign_key.get("foreign_key_column")
            or "",
        )

    return str(
        getattr(foreign_key, "column_name", None)
        or getattr(foreign_key, "column", None)
        or getattr(foreign_key, "fk_column", None)
        or getattr(foreign_key, "foreign_key_column", None)
        or "",
    )

def get_relationship_set_name(
    self,
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("set_name")
            or relationship.get("set")
            or relationship.get("name")
            or "",
        )

    return str(
        getattr(relationship, "set_name", None)
        or getattr(relationship, "set", None)
        or getattr(relationship, "name", None)
        or "",
    )

def get_relationship_owner_record(
    self,
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("owner_record")
            or relationship.get("parent_record")
            or relationship.get("owner")
            or relationship.get("parent")
            or "",
        )

    return str(
        getattr(relationship, "owner_record", None)
        or getattr(relationship, "parent_record", None)
        or getattr(relationship, "owner", None)
        or getattr(relationship, "parent", None)
        or "",
    )

```



```

def get_relationship_member_record(
    self,
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("member_record")
            or relationship.get("child_record")
            or relationship.get("member")
            or relationship.get("child")
            or "",
        )

    return str(
        getattr(relationship, "member_record", None)
        or getattr(relationship, "child_record", None)
        or getattr(relationship, "member", None)
        or getattr(relationship, "child", None)
        or "",
    )

def get_db2_datatype(
    self,
    db2_column: DB2Column | None,
) -> str:
    if db2_column is None:
        return ""

    return str(
        getattr(db2_column, "datatype", None)
        or getattr(db2_column, "data_type", None)
        or getattr(db2_column, "type", None)
        or "",
    ).strip().upper()

def db2_key_label(
    self,
    table: DB2Table | None,
    column: DB2Column | None,
) -> str:
    if table is None or column is None:
        return ""

    labels: list[str] = []

    if self.is_primary_key_column(table=table, column=column):
        labels.append("PK")

    if self.is_foreign_key_column(table=table, column=column):
        labels.append("FK")

    return "/".join(labels)

def is_primary_key_column(
    self,
    table: DB2Table,
    column: DB2Column,
) -> bool:
    if getattr(column, "primary_key", False):
        return True

    column_name = NameNormalizer.normalize(getattr(column, "name", "") or "")
    primary_keys = list(getattr(table, "primary_keys", []) or [])

    if not primary_keys and getattr(table, "primary_key", None):
        primary_keys = [table.primary_key]

    normalized_primary_keys = {
        NameNormalizer.normalize(primary_key)
        for primary_key in primary_keys
        if primary_key
    }

    return column_name in normalized_primary_keys

```

```

def is_foreign_key_column(
    self,
    table: DB2Table,
    column: DB2Column,
) -> bool:
    column_name = NameNormalizer.normalize(getattr(column, "name", "") or "")

    for foreign_key in getattr(table, "foreign_keys", []) or []:
        fk_column_name = NameNormalizer.normalize(self.get_foreign_key_column_name(foreign_key))

        if fk_column_name == column_name:
            return True

    source_kind = str(getattr(column, "source_kind", "") or "").upper()

    return source_kind in {"SET_FK", "FK", "FOREIGN KEY"}

def should_emit_db2_field(
    self,
    field,
) -> bool:
    if self.is_filler_field(field=field):
        return False

    if self.is_group_field(field=field):
        return False

    if self.is_date_part_name(field_name=getattr(field, "name", "") or ""):
        return False

    picture = self.get_field_picture(field=field)

    if picture:
        return True

    datatype = str(getattr(field, "datatype", "") or "").upper()

    return datatype in {"CHAR", "VARCHAR", "DECIMAL", "NUMERIC", "DATE"}

def is_filler_field(
    self,
    field,
) -> bool:
    return NameNormalizer.normalize(getattr(field, "name", "") or "") == "FILLER"

def is_group_field(
    self,
    field,
) -> bool:
    return bool(getattr(field, "has_child", False) or getattr(field, "is_group", False))

def db2_record_name(
    self,
    record,
    table: DB2Table | None,
) -> str:
    if table is not None and getattr(table, "name", None):
        return getattr(table, "name", "") or ""

    return self.to_db2_name(getattr(record, "name", "") or "")

def convert_cobol_zone_to_db2_field_name(
    self,
    record,
    field,
) -> str:
    field_name = getattr(field, "name", "") or ""
    base = self.cobol_field_base_name(field_name=field_name)
    record_code = self.record_code(record_name=getattr(record, "name", "") or "")

    if record_code:
        return f"{base}_479{record_code}"

    return base

def convert_date_field_to_db2_field_name(

```

```

        self,
        record,
        field,
    ) -> str:
        field_name = getattr(field, "name", "") or ""
        base = self.date_field_base_name(field_name=field_name)
        record_code = self.record_code(record_name=getattr(record, "name", "") or "")

        if record_code:
            return f"{base}_479{record_code}"

        return base

def cobol_field_base_name(
    self,
    field_name: str,
) -> str:
    normalized = self.to_db2_name(field_name)
    normalized = self.remove_record_suffix(normalized)

    parts = [
        part
        for part in normalized.split("_")
        if part
    ]

    if not parts:
        return normalized

    parts = self.remove_generic_trailing_tokens(parts=parts)

    parts = [
        self.PROJECT_ABBREVIATIONS.get(part, part)
        for part in parts
    ]

    if len(parts) == 1:
        return parts[0]

    first = parts[0]
    rest = "".join(parts[1:])

    if rest:
        return f"{first}_{rest}"

    return first

def date_field_base_name(
    self,
    field_name: str,
) -> str:
    normalized = self.to_db2_name(field_name)
    normalized = self.remove_record_suffix(normalized)

    parts = [
        part
        for part in normalized.split("_")
        if part
    ]

    if not parts:
        return normalized

    if parts[0] == "DA":
        main_parts = parts[1:]
    else:
        main_parts = parts

    if not main_parts:
        return "DA"

    if main_parts[0] in {"UB", "UE"}:
        return f"DA_{main_parts[0]}DATE"

    main_parts = self.apply_date_project_abbreviations(
        parts=main_parts,

```

```

    )

    main_parts = self.remove_generic_trailing_tokens(
        parts=main_parts,
    )

    if not main_parts:
        return "DA"

    if len(main_parts) == 1:
        main = main_parts[0]
    else:
        main = "".join(main_parts)

    return f"DA_{main}"

def apply_date_project_abbreviations(
    self,
    parts: list[str],
) -> list[str]:
    output: list[str] = []

    for part in parts:
        if part == "EVPRCP":
            output.append("ERCP")
            continue

        if part == "GDIFAR":
            output.append("GD")
            continue

        if part in {"GDIFC", "GDIFR"}:
            continue

        output.append(
            self.PROJECT_ABBREVIATIONS.get(part, part)
        )

    return output

def remove_generic_trailing_tokens(
    self,
    parts: list[str],
) -> list[str]:
    output = list(parts or [])

    while output and output[-1] in self.GENERIC_TRAILING_TOKENS:
        output.pop()

    return output

def record_code(
    self,
    record_name: str,
) -> str:
    normalized = self.to_db2_name(record_name)
    compact = re.sub(r"^[A-Z0-9]", "", normalized)

    if len(compact) <= 4:
        return compact

    return compact[-4:]

def get_field_picture(
    self,
    field,
) -> str:
    if self.is_group_field(field=field):
        return ""

    text = str(
        getattr(field, "picture", None)
        or getattr(field, "pic", None)
        or getattr(field, "pic_clause", None)
        or "",
    ).strip()

```

```

    if not text:
        return ""

    if text.upper().startswith("PIC "):
        return text

    if text.upper().startswith("PICTURE "):
        return "PIC " + text[8:].strip()

    return f"PIC {text}"

def get_field_length(
    self,
    field,
):
    return (
        getattr(field, "length", None)
        or getattr(field, "storage_length", None)
        or getattr(field, "physical_length", None)
        or getattr(field, "byte_length", None)
        or getattr(field, "bytes", None)
    )

def get_field_end_position(
    self,
    field,
):
    start = (
        getattr(field, "start_position", None)
        or getattr(field, "start", None)
    )

    length = self.get_field_length(field=field)

    try:
        if start is not None and length not in ("", None):
            return int(start) + int(length) - 1
    except Exception:
        pass

    end_position = (
        getattr(field, "end_position", None)
        or getattr(field, "field_end_position", None)
        or getattr(field, "end", None)
    )

    if end_position is not None:
        return end_position

    return ""

def infer_db2_datatype_from_field(
    self,
    field,
) -> str:
    pic_clause = self.get_field_picture(field=field)

    if pic_clause:
        return DB2DatatypeMapper.map_picture(
            picture=pic_clause,
            fallback_length=None,
            fallback_scale=getattr(field, "scale", None),
        )

    datatype = str(getattr(field, "datatype", "") or "").upper()

    if datatype == "DATE":
        return "DATE"

    if datatype in {"TIMESTAMP", "DATETIME"}:
        return "TIMESTAMP"

    length = self.get_field_length(field=field) or 1

    if datatype in {"CHAR", "VARCHAR", "DISPLAY", "TEXT"}:

```

```

        return f"CHAR({int(length)})"

    if datatype in {"DECIMAL", "NUMERIC"}:
        scale = getattr(field, "scale", None)

        if scale is not None and int(scale) > 0:
            return f"DECIMAL({int(length)},{int(scale)})"

        return f"DECIMAL({int(length)})"

    return ""

def get_field_basetype(
    self,
    field,
) -> str:
    return str(
        getattr(field, "basetype", None)
        or getattr(field, "base_type", None)
        or getattr(field, "datatype", None)
        or "",
    )

def hopex_remark(
    self,
    field,
) -> str:
    if getattr(field, "occurs", False):
        occurs_min = getattr(field, "occurs_min", None)
        occurs_max = getattr(field, "occurs_max", None)

        if occurs_min is not None and occurs_max is not None:
            if occurs_min == occurs_max:
                return f"OCCURS {occurs_max}"

            return f"OCCURS {occurs_min} TO {occurs_max}"

    return ""

def has_level_01_row(
    self,
    record,
    mapping_fields,
) -> bool:
    record_name = NameNormalizer.normalize(getattr(record, "name", "") or "")

    for field in mapping_fields or []:
        level = getattr(field, "level", None)

        try:
            if int(level) != 1:
                continue
        except Exception:
            continue

        field_name = NameNormalizer.normalize(getattr(field, "name", "") or "")

        if field_name == record_name:
            return True

        if self.remove_record_suffix(field_name) == self.remove_record_suffix(record_name):
            return True

    return False

def build_record_level_01_row(
    self,
    record,
) -> dict[str, str]:
    row = self.empty_row()

    record_name = getattr(record, "name", "") or ""

    row["Cobol Record IDMS"] = record_name
    row["Cobol Zone"] = self.format_cobol_level_and_name(level=1, name=record_name)

```

```

        return row

def name_lookup_keys(
    self,
    value: str,
) -> list[str]:
    keys: list[str] = []

    candidates = [
        value,
        NameNormalizer.normalize(value or ""),
        self.to_db2_name(value or ""),
        self.remove_record_suffix(value or ""),
        self.remove_record_suffix(NameNormalizer.normalize(value or "")),
        self.remove_record_suffix(self.to_db2_name(value or "")),
    ]

    for candidate in candidates:
        candidate_text = str(candidate or "").strip()

        if not candidate_text:
            continue

        normalized_name = NameNormalizer.normalize(candidate_text)
        db2_name = self.to_db2_name(candidate_text)

        for key in [candidate_text, normalized_name, db2_name]:
            key = str(key or "").strip()

            if key and key not in keys:
                keys.append(key)

    return keys

def format_cobol_zone(
    self,
    field,
) -> str:
    level = getattr(field, "level", None)
    name = getattr(field, "name", "") or ""

    return self.format_cobol_level_and_name(level=level, name=name)

def format_cobol_level_and_name(
    self,
    level,
    name: str,
) -> str:
    level_text = self.to_string(level)

    if level_text:
        try:
            level_text = f"{int(level_text):02d}"
        except Exception:
            pass

    if level_text and name:
        return f"{level_text} {name}"

    if name:
        return str(name)

    return level_text

def remove_record_suffix(
    self,
    name: str,
) -> str:
    value = str(name or "").strip().upper()
    value = re.sub(r"[_\-\s]{4}$", "", value)
    value = re.sub(r"[0-9]{4}$", "", value)
    value = re.sub(r"[_\-\s]+$", "", value)

    return value

def to_db2_name(

```

```

        self,
        value: str,
    ) -> str:
        normalized = str(value or "").upper().strip()
        normalized = re.sub(r"^[A-Z0-9]+", "_", normalized)
        normalized = re.sub(r"_+", "_", normalized)
        normalized = normalized.strip("_")

        return normalized

    def name_parts(
        self,
        value: str,
    ) -> list[str]:
        normalized = self.to_db2_name(value or "")

        return [part for part in normalized.split("_") if part]

    def empty_row(
        self,
    ) -> dict[str, str]:
        return {column: "" for column in self.COLUMNS}

    def to_string(
        self,
        value,
    ) -> str:
        if value is None:
            return ""

        return str(value)

    def safe_int(
        self,
        value,
        default: int = 0,
    ) -> int:
        try:
            if value is None:
                return default

            text = str(value).strip()

            if not text:
                return default

            return int(text)
        except Exception:
            return default

```

```

=====
FILE: src\idms_modernizer\services\metadata_builder.py
ENCODING: utf-8
=====

```

```

import re

from idms_modernizer.domain.document_models import (
    DocumentModel
)

from idms_modernizer.domain.schema_models import (
    DataField,
    Record,
    SchemaMetadata,
)

class MetadataBuilder:

    RECORD_PATTERN = re.compile(
        r"RECORD\s+NAME.*?([A-Z][A-Z0-9\-\-]+)",
        re.IGNORECASE
    )

```



```

FIELD_PATTERN = re.compile(
    r"^\d{2}\s+([A-Z][A-Z0-9-]+)",
    re.IGNORECASE
)

def build(
    self,
    document: DocumentModel
) -> SchemaMetadata:

    metadata = SchemaMetadata()

    current_record = None

    seen_records = set()

    for page in document.pages:

        for line in page.lines:

            text = line.text.strip()

            if not text:
                continue

            #
            # RECORD DETECTION
            #

            record_match = (
                self.RECORD_PATTERN.search(
                    text
                )
            )

            if record_match:

                record_name = (
                    record_match.group(1)
                    .strip()
                    .upper()
                )

                if (
                    record_name
                    not in seen_records
                ):

                    current_record = Record(
                        name=record_name
                    )

                    metadata.records.append(
                        current_record
                    )

                    seen_records.add(
                        record_name
                    )

                continue

            #
            # FIELD DETECTION
            #

            if current_record:

                field_match = (
                    self.FIELD_PATTERN.match(
                        text
                    )
                )

                if field_match:

```

```

        field_name = (
            field_match.group(1)
            .strip()
            .upper()
        )

        current_record.fields.append(
            DataField(
                name=field_name
            )
        )

#
# DEBUG OUTPUT
#

print(
    "\n===== RECORDS ====="
)

for record in metadata.records:

    print(
        f"{record.name} -> "
        f"{len(record.fields)} fields"
    )

print(
    "=====\n"
)

return metadata

```

```

=====
FILE: src\idms_modernizer\services\metadata_service.py
ENCODING: utf-8
=====

```

```
import re
```

```

from idms_modernizer.parsers.text_extractor import TextExtractor
from idms_modernizer.services.document_segmenter import DocumentSegmenter
from idms_modernizer.extractors.owner_member_extractor import OwnerMemberExtractor
from idms_modernizer.extractors.field_extractor import FieldExtractor
from idms_modernizer.extractors.cobol_zone_extractor import CobolZoneExtractor
from idms_modernizer.services.schema_picture_enricher import SchemaPictureEnricher
from idms_modernizer.services.relationship_resolver import RelationshipResolver
from idms_modernizer.domain.schema_models import SchemaMetadata, Record
from idms_modernizer.extractors.primary_key_extractor import PrimaryKeyExtractor

```

```
print("LOADED MetadataService VERSION GENERIC-DEBUG-NO-FIELD-HARDCODE-2026-07-30")
```

```
class MetadataService:
```

```
    """
```

```
    Builds SchemaMetadata from the schema PDF.
```

```
    Critical rule:
```

- record.fields is used for physical DB2 / DDL / canonical schema.
- record.mapping_fields is used for Excel Sheet Mapping.
- BOTH must be enriched with:
 - PIC clause
 - datatype
 - length
 - scale
 - start_position
 - end_position
 - basetype

```
    Generic behavior:
```

- No hardcoded record names.
- No hardcoded field names.
- No hardcoded business/application field names.
- Debug logging is pattern-based, not field-name based.

Notes:

- Date identification should remain token/pattern based in downstream services.
- Date tokens such as YEAR, MONTH, DAY, YY, MM, DD, DA, DATE are generic schema concepts, not application-specific field hardcoding.

DEBUG = True

```
FIELD_LINE_PATTERN = re.compile(
    r"^\s*(0[1-9]|[1-4][0-9]|88)\s+([A-Z][A-Z0-9-]*|FILLER)\b",
    re.IGNORECASE,
)
```

```
PIC_OR_USAGE_PATTERN = re.compile(
    r"\b(PIC|PICTURE|DISPLAY|COMP-3|COMP)\b",
    re.IGNORECASE,
)
```

```
def __init__(self):
    self.extractor = TextExtractor()
    self.segmenter = DocumentSegmenter()
    self.field_extractor = FieldExtractor()
    self.picture_enricher = SchemaPictureEnricher()
    self.membership_extractor = OwnerMemberExtractor()
    self.zone_extractor = CobolZoneExtractor()
    self.relationship_resolver = RelationshipResolver()
    self.primary_key_extractor = PrimaryKeyExtractor()
```

```
def build_metadata(
    self,
    pdf_path: str,
) -> SchemaMetadata:
    print("USING MetadataService.build_metadata VERSION GENERIC-DEBUG-NO-FIELD-HARDCODE-2026-07-30")
```

```
    document = self.extractor.extract_document(
        pdf_path,
    )
```

```
    sections = self.segmenter.segment(
        document,
    )
```

```
    if self.DEBUG:
        print(
            "METADATA_DOCUMENT_DEBUG",
            f"sections_count={len(sections or [])}",
        )
```

```
    metadata = SchemaMetadata()
```

```
    for section in sections:
        record = Record(
            name=section.record_name,
        )

        record.cobol_zone = self.zone_extractor.extract(
            section.lines,
        )
```

```
        physical_fields = self.field_extractor.extract(
            section.lines,
        )
```

```
        mapping_fields = self.field_extractor.extract_all(
            section.lines,
            include_filler=True,
        )
```

```
    if self.DEBUG:
        print("=" * 100)
        print(f"METADATA DEBUG RECORD: {record.name}")
        print("=" * 100)
        print(f"RAW SECTION LINE COUNT: {len(section.lines or [])}")
        print(f"PHYSICAL FIELD COUNT BEFORE ENRICH: {len(physical_fields or [])}")
        print(f"MAPPING FIELD COUNT BEFORE ENRICH: {len(mapping_fields or [])}")
```

```

        self.print_generic_raw_schema_lines(
            record_name=record.name,
            lines=section.lines,
        )

    record.fields = self.picture_enricher.enrich(
        fields=physical_fields,
        lines=section.lines,
        debug_label=f"{record.name} PHYSICAL",
    )

    record.mapping_fields = self.picture_enricher.enrich(
        fields=mapping_fields,
        lines=section.lines,
        debug_label=f"{record.name} MAPPING",
    )

    record.set_memberships = self.membership_extractor.extract(
        section.record_name,
        section.lines,
    )

    record.primary_key = self.primary_key_extractor.extract(
        section.lines,
    )

    if self.DEBUG:
        print("=" * 100)
        print(f"FIELDS EXTRACTED FOR {record.name} - PHYSICAL")
        print("=" * 100)

        for field in record.fields or []:
            self.print_field_debug(
                label="PHYSICAL_FIELD_DEBUG",
                field=field,
            )

        print("=" * 100)
        print(f"FIELDS EXTRACTED FOR {record.name} - MAPPING")
        print("=" * 100)

        for field in record.mapping_fields or []:
            self.print_field_debug(
                label="MAPPING_FIELD_DEBUG",
                field=field,
            )

        print(
            f"MetadataService: {record.name} "
            f"PK={record.primary_key} "
            f"ZONE={record.cobol_zone} "
            f"PHYSICAL_FIELDS={len(record.fields or [])} "
            f"MAPPING_FIELDS={len(record.mapping_fields or [])}"
        )

    metadata.records.append(record)

metadata.relationships = self.relationship_resolver.resolve(
    metadata,
)

if self.DEBUG:
    print(
        f"Relationships Found: {len(metadata.relationships)}"
    )

return metadata

def print_generic_raw_schema_lines(
    self,
    record_name: str,
    lines,
) -> None:
    """
    Generic raw-line debug.

```

This replaces hardcoded target_tokens.

Prints:

- COBOL/IDMS data item lines such as 02 FIELD-NAME, 04 FIELD-NAME, etc.
- Lines containing PIC / PICTURE / DISPLAY / COMP / COMP-3.

This is generic and works for any schema listing.

"""

```
print("=" * 100)
print(f"RAW GENERIC SCHEMA LINES FOR {record_name}")
print("=" * 100)

for index, line in enumerate(lines or []):
    value = getattr(line, "text", None)

    if value is None:
        value = str(line)

    text = str(value or "").strip()

    if not text:
        continue

    is_field_line = self.FIELD_LINE_PATTERN.search(text) is not None
    is_pic_or_usage_line = self.PIC_OR_USAGE_PATTERN.search(text) is not None

    if is_field_line or is_pic_or_usage_line:
        print(
            "METADATA_RAW_SCHEMA_LINE",
            f"record={record_name}",
            f"index={index}",
            f"line={repr(text)}",
        )

def print_field_debug(
    self,
    label: str,
    field,
) -> None:
    print(
        label,
        "name=", getattr(field, "name", None),
        "level=", getattr(field, "level", None),
        "datatype=", getattr(field, "datatype", None),
        "length=", getattr(field, "length", None),
        "scale=", getattr(field, "scale", None),
        "picture=", getattr(field, "picture", None),
        "start=", getattr(field, "start_position", None),
        "end=", getattr(field, "end_position", None),
        "basetype=", getattr(field, "basetype", None),
        "group=", getattr(field, "is_group", None),
        "has_child=", getattr(field, "has_child", None),
    )
```

```
=====
FILE: src\idms_modernizer\services\name_normalizer.py
ENCODING: utf-8
=====
```

```
import re
```

```
class NameNormalizer:
```

```
    @staticmethod
    def normalize(
        name: str | None
    ) -> str:

        if not name:
            return ""

        normalized = name.strip().upper()
```

```

normalized = normalized.replace("-", "_")
normalized = re.sub(
    r"^[^A-Z0-9_]",
    "_",
    normalized
)
normalized = re.sub(
    r"_{1,}",
    "_",
    normalized
)

return normalized.strip("_")

```

```

=====
FILE: src\idms_modernizer\services\phase2_conversion_bridge.py
ENCODING: utf-8
=====

```

```

class Phase2ConversionBridge:
    """
    Bridge from Phase 1 outputs to the Phase 2 converter.

    Import is intentionally lazy so Phase 1 UI can still load even if
    Phase 2 package has a missing or broken module.
    """

    def convert(
        self,
        cobol_text: str,
        canonical_json: str,
        phase2_metadata_json: str,
        ddl_text: str,
        idms_schema_text: str | None = None,
        relationship_overrides_json: str | None = None,
        target_program: str | None = None,
        use_ddl: bool = True,
        use_idms_schema: bool = False,
        use_overrides: bool = False
    ) -> tuple[str, list[str]]:

        if not cobol_text or not cobol_text.strip():
            raise ValueError(
                "COBOL code is required for Phase 2 conversion."
            )

        try:
            from idms_db2_converter.services.conversion_service import (
                ConversionService
            )

        except Exception as exc:
            raise RuntimeError(
                "Phase 2 converter package could not be loaded. "
                "Check src/idms_db2_converter files. "
                f"Details: {exc}"
            ) from exc

        if not canonical_json or not canonical_json.strip():
            canonical_json = '{"records": [], "sets": [], "relationships": []}'

        if not phase2_metadata_json:
            phase2_metadata_json = ""

        if not ddl_text:
            ddl_text = ""

        if not idms_schema_text:
            idms_schema_text = ""

        if not relationship_overrides_json:
            relationship_overrides_json = ""

        service = ConversionService()

```

```

converted, validation_messages = service.convert_retrieval(
    cobol_text=cobol_text,
    canonical_json=canonical_json,
    phase2_metadata_json=phase2_metadata_json,
    ddl_text=ddl_text if use_ddl else None,
    idms_schema_text=(
        idms_schema_text
        if use_idms_schema
        else None
    ),
    relationship_overrides_json=(
        relationship_overrides_json
        if use_overrides and relationship_overrides_json.strip()
        else None
    ),
    target_program=(
        target_program.strip()
        if target_program and target_program.strip()
        else None
    )
)

return converted, validation_messages

```

```

=====
FILE: src\idms_modernizer\services\phase2_metadata_generator.py
ENCODING: utf-8
=====

```

```

import json
import re

from datetime import datetime
from typing import Any

from idms_modernizer.domain.canonical_models import CanonicalSchema
from idms_modernizer.domain.db2_models import DB2Model, DB2Table, DB2Column
from idms_modernizer.domain.schema_models import SchemaMetadata

```

```

class Phase2MetadataGenerator:
    VERSION = "1.0"

    DATE_PARTS = {
        "YEAR": {
            "substring_start": 3,
            "substring_length": 2,
        },
        "MONTH": {
            "substring_start": 6,
            "substring_length": 2,
        },
        "DAY": {
            "substring_start": 9,
            "substring_length": 2,
        },
    }

    YEAR_PARTS = {
        "YEAR",
        "YR",
        "Y",
        "YY",
        "YYYY",
        "DY",
    }

    MONTH_PARTS = {
        "MONTH",
        "MON",
        "MO",
        "M",
        "MM",
        "DM",
    }

```

```

DAY_PARTS = {
    "DAY",
    "D",
    "DD",
}

def generate_json(
    self,
    canonical_schema: CanonicalSchema,
    db2_model: DB2Model,
    metadata: SchemaMetadata,
) -> str:
    payload = self.generate_payload(
        canonical_schema=canonical_schema,
        db2_model=db2_model,
        metadata=metadata,
    )

    return json.dumps(
        payload,
        indent=2,
    )

def generate_payload(
    self,
    canonical_schema: CanonicalSchema,
    db2_model: DB2Model,
    metadata: SchemaMetadata,
) -> dict[str, Any]:
    db2_table_lookup = self.build_db2_table_lookup(
        db2_model=db2_model,
    )

    canonical_record_lookup = self.build_canonical_record_lookup(
        canonical_schema=canonical_schema,
    )

    records_payload = self.build_records_payload(
        db2_model=db2_model,
    )

    record_table_map = self.build_record_table_map(
        metadata=metadata,
        db2_table_lookup=db2_table_lookup,
    )

    field_map = self.build_field_map(
        metadata=metadata,
        db2_table_lookup=db2_table_lookup,
        record_table_map=record_table_map,
    )

    calc_key_map = self.build_calc_key_map(
        canonical_schema=canonical_schema,
        metadata=metadata,
        record_table_map=record_table_map,
        db2_table_lookup=db2_table_lookup,
    )

    relationships_payload = self.build_relationships_payload(
        canonical_schema=canonical_schema,
        db2_table_lookup=db2_table_lookup,
    )

    set_ordering_map = self.build_set_ordering_map(
        relationships=relationships_payload,
    )

    nullable_fk_map = self.build_nullable_fk_map(
        db2_model=db2_model,
    )

    table_key_map = self.build_table_key_map(
        db2_model=db2_model,
    )

```



```

date_part_map = self.build_date_part_map(
    metadata=metadata,
    db2_table_lookup=db2_table_lookup,
    record_table_map=record_table_map,
)

navigation_intent = self.build_navigation_intent(
    relationships=relationships_payload,
)

output_semantics = self.build_output_semantics()

validation_messages = self.build_validation_messages(
    canonical_schema=canonical_schema,
    db2_model=db2_model,
    metadata=metadata,
    field_map=field_map,
)

return {
    "version": self.VERSION,
    "generated_at": datetime.utcnow().isoformat() + "Z",
    "records": records_payload,
    "record_table_map": record_table_map,
    "field_map": field_map,
    "calc_key_map": calc_key_map,
    "relationships": relationships_payload,
    "set_ordering_map": set_ordering_map,
    "nullable_fk_map": nullable_fk_map,
    "table_key_map": table_key_map,
    "date_part_map": date_part_map,
    "navigation_intent": navigation_intent,
    "output_semantics": output_semantics,
    "validation_messages": validation_messages,
    "canonical_record_lookup_count": len(canonical_record_lookup),
}

def build_records_payload(
    self,
    db2_model: DB2Model,
) -> list[dict[str, Any]]:
    records_payload: list[dict[str, Any]] = []

    for table in getattr(db2_model, "tables", []) or []:
        fields_payload: list[dict[str, Any]] = []

        primary_keys = list(getattr(table, "primary_keys", []) or [])

        if not primary_keys and getattr(table, "primary_key", None):
            primary_keys = [table.primary_key]

        normalized_primary_keys = {
            self.to_db2_name(primary_key)
            for primary_key in primary_keys
            if primary_key
        }

        for column in getattr(table, "columns", []) or []:
            datatype, length, scale = self.parse_db2_datatype(
                getattr(column, "datatype", ""),
            )

            column_name = getattr(column, "name", "") or ""
            normalized_column_name = self.to_db2_name(column_name)

            fields_payload.append(
                {
                    "name": column_name,
                    "column": column_name,
                    "datatype": datatype,
                    "length": length,
                    "scale": scale,
                    "nullable": getattr(column, "nullable", True),
                    "primary_key": (
                        getattr(column, "primary_key", False)

```

```

        or normalized_column_name in normalized_primary_keys
    ),
    "generated": getattr(column, "generated", False),
    "source_kind": getattr(column, "source_kind", ""),
}
)

records_payload.append(
    {
        "name": getattr(table, "name", "") or "",
        "table": getattr(table, "name", "") or "",
        "primary_key": primary_keys[0] if primary_keys else None,
        "primary_keys": primary_keys,
        "fields": fields_payload,
    }
)

return records_payload

def build_table_key_map(
    self,
    db2_model: DB2Model,
) -> dict[str, Any]:
    table_key_map: dict[str, Any] = {}

    for table in getattr(db2_model, "tables", []) or []:
        table_name = self.to_db2_name(
            getattr(table, "name", "") or ""
        )

        if not table_name:
            continue

        primary_keys = list(getattr(table, "primary_keys", []) or [])

        if not primary_keys and getattr(table, "primary_key", None):
            primary_keys = [table.primary_key]

        foreign_keys = []

        for foreign_key in getattr(table, "foreign_keys", []) or []:
            foreign_keys.append(
                {
                    "column_name": getattr(foreign_key, "column_name", "") or "",
                    "reference_table": getattr(foreign_key, "reference_table", "") or "",
                    "reference_column": getattr(foreign_key, "reference_column", "") or "",
                    "set_name": getattr(foreign_key, "set_name", "") or "",
                }
            )

        table_key_map[table_name] = {
            "primary_keys": primary_keys,
            "foreign_keys": foreign_keys,
        }

    return table_key_map

def build_db2_table_lookup(
    self,
    db2_model: DB2Model,
) -> dict[str, DB2Table]:
    lookup: dict[str, DB2Table] = {}

    for table in getattr(db2_model, "tables", []) or []:
        table_name = self.to_db2_name(
            getattr(table, "name", "") or ""
        )

        if not table_name:
            continue

        for key in self.name_lookup_keys(table_name):
            lookup[key] = table

    return lookup

```

```

def build_canonical_record_lookup(
    self,
    canonical_schema: CanonicalSchema,
) -> dict[str, Any]:
    lookup: dict[str, Any] = {}

    for record in getattr(canonical_schema, "records", []) or []:
        record_name = self.to_db2_name(
            getattr(record, "name", "") or "",
        )

        if not record_name:
            continue

        for key in self.name_lookup_keys(record_name):
            lookup[key] = record

    return lookup

def build_record_table_map(
    self,
    metadata: SchemaMetadata,
    db2_table_lookup: dict[str, DB2Table],
) -> dict[str, str]:
    record_table_map: dict[str, str] = {}

    for record in getattr(metadata, "records", []) or []:
        record_name = self.to_db2_name(
            getattr(record, "name", "") or "",
        )

        if not record_name:
            continue

        table = self.find_table_for_record(
            record_name=record_name,
            db2_table_lookup=db2_table_lookup,
        )

        record_table_map[record_name] = (
            getattr(table, "name", "") if table is not None else record_name
        )

    return record_table_map

def build_field_map(
    self,
    metadata: SchemaMetadata,
    db2_table_lookup: dict[str, DB2Table],
    record_table_map: dict[str, str],
) -> dict[str, dict[str, Any]]:
    field_map: dict[str, dict[str, Any]] = {}

    for record in getattr(metadata, "records", []) or []:
        idms_record_name = self.to_db2_name(
            getattr(record, "name", "") or "",
        )

        if not idms_record_name:
            continue

        table_name = record_table_map.get(
            idms_record_name,
            idms_record_name,
        )

        table = self.find_table_for_record(
            record_name=table_name,
            db2_table_lookup=db2_table_lookup,
        )

        if table is None:
            continue

        column_lookup = self.build_column_lookup(
            table=table,

```

```

    )

    source_fields = (
        getattr(record, "mapping_fields", None)
        or getattr(record, "fields", None)
        or []
    )

    for field in source_fields:
        raw_legacy_field_name = str(
            getattr(field, "name", "") or ""
        ).strip()

        legacy_field_name = self.to_db2_name(
            raw_legacy_field_name,
        )

        if not legacy_field_name:
            continue

        column = self.find_column_for_legacy_field(
            legacy_field_name=legacy_field_name,
            column_lookup=column_lookup,
        )

        if column is None:
            continue

        column_name = getattr(column, "name", "") or ""

        host = self.host_variable_name(
            record=idms_record_name,
            column=column_name,
        )

        payload = {
            "record": idms_record_name,
            "table": getattr(table, "name", "") or "",
            "column": column_name,
            "host": host,
            "legacy_field": legacy_field_name,
        }

        for key in self.name_lookup_keys(legacy_field_name):
            field_map[key] = payload

        for key in self.name_lookup_keys(raw_legacy_field_name):
            field_map[key] = payload

        for key in self.name_lookup_keys(column_name):
            field_map.setdefault(
                key,
                payload,
            )

    return field_map

def build_column_lookup(
    self,
    table: DB2Table,
) -> dict[str, DB2Column]:
    lookup: dict[str, DB2Column] = {}

    for column in getattr(table, "columns", []) or []:
        column_name = self.to_db2_name(
            getattr(column, "name", "") or "",
        )

        if not column_name:
            continue

        for key in self.name_lookup_keys(column_name):
            lookup[key] = column

    return lookup

```

```

def build_calc_key_map(
    self,
    canonical_schema: CanonicalSchema,
    metadata: SchemaMetadata,
    record_table_map: dict[str, str],
    db2_table_lookup: dict[str, DB2Table],
) -> dict[str, dict[str, Any]]:
    calc_key_map: dict[str, dict[str, Any]] = {}

    for record in getattr(metadata, "records", []) or []:
        record_name = self.to_db2_name(
            getattr(record, "name", "") or "",
        )

        if not record_name:
            continue

        primary_keys = self.extract_primary_keys_from_record(record)

        if not primary_keys:
            continue

        table_name = record_table_map.get(
            record_name,
            record_name,
        )

        table = self.find_table_for_record(
            record_name=table_name,
            db2_table_lookup=db2_table_lookup,
        )

        physical_primary_keys = []

        if table is not None:
            column_lookup = self.build_column_lookup(table)

            for primary_key in primary_keys:
                column = self.find_column_for_legacy_field(
                    legacy_field_name=primary_key,
                    column_lookup=column_lookup,
                )

                if column is not None:
                    physical_primary_keys.append(
                        getattr(column, "name", "") or primary_key,
                    )
                else:
                    physical_primary_keys.append(primary_key)
            else:
                physical_primary_keys = primary_keys

        cleaned_primary_keys = self.unique_values(
            [
                self.to_db2_name(primary_key)
                for primary_key in physical_primary_keys
                if primary_key
            ]
        )

        if not cleaned_primary_keys:
            continue

        primary_key = cleaned_primary_keys[0]

        calc_key_map[record_name] = {
            "record": record_name,
            "table": getattr(table, "name", table_name) if table else table_name,
            "key": primary_key,
            "primary_key": primary_key,
            "primary_keys": cleaned_primary_keys,
            "column": primary_key,
            "host": self.host_variable_name(
                record=record_name,
                column=primary_key,
            ),
        },

```

```

    }

    for record in getattr(canonical_schema, "records", []) or []:
        record_name = self.to_db2_name(
            getattr(record, "name", "") or "",
        )

        if not record_name or record_name in calc_key_map:
            continue

        primary_keys = self.extract_primary_keys_from_record(record)

        if not primary_keys:
            continue

        primary_key = primary_keys[0]

        calc_key_map[record_name] = {
            "record": record_name,
            "table": record_name,
            "key": primary_key,
            "primary_key": primary_key,
            "primary_keys": primary_keys,
            "column": primary_key,
            "host": self.host_variable_name(
                record=record_name,
                column=primary_key,
            ),
        },
    }

    return calc_key_map

def build_relationships_payload(
    self,
    canonical_schema: CanonicalSchema,
    db2_table_lookup: dict[str, DB2Table],
) -> list[dict[str, Any]]:
    relationships_payload: list[dict[str, Any]] = []

    for relationship in getattr(canonical_schema, "relationships", []) or []:
        parent_record = self.to_db2_name(
            getattr(relationship, "parent_record", None)
            or getattr(relationship, "owner_record", None)
            or "",
        )

        child_record = self.to_db2_name(
            getattr(relationship, "child_record", None)
            or getattr(relationship, "member_record", None)
            or "",
        )

        set_name = self.to_db2_name(
            getattr(relationship, "set_name", None)
            or getattr(relationship, "name", None)
            or "",
        )

        parent_table = self.find_table_for_record(
            record_name=parent_record,
            db2_table_lookup=db2_table_lookup,
        )

        child_table = self.find_table_for_record(
            record_name=child_record,
            db2_table_lookup=db2_table_lookup,
        )

        parent_keys = []

        if parent_table is not None:
            parent_keys = list(getattr(parent_table, "primary_keys", []) or [])

            if not parent_keys and getattr(parent_table, "primary_key", None):
                parent_keys = [parent_table.primary_key]

```

```

child_fks = []
order_by = []

if child_table is not None and parent_table is not None:
    for foreign_key in getattr(child_table, "foreign_keys", []) or []:
        fk_set_name = self.to_db2_name(
            getattr(foreign_key, "set_name", "") or "",
        )

        reference_table = self.to_db2_name(
            getattr(foreign_key, "reference_table", "") or "",
        )

        parent_table_name = self.to_db2_name(
            getattr(parent_table, "name", "") or "",
        )

        if set_name and fk_set_name and fk_set_name != set_name:
            continue

        if reference_table and reference_table != parent_table_name:
            continue

        child_fk = getattr(foreign_key, "column_name", "") or ""

        if child_fk:
            child_fks.append(child_fk)
            order_by.append(child_fk)

relationships_payload.append(
    {
        "set_name": set_name,
        "parent_record": parent_record,
        "child_record": child_record,
        "owner_record": parent_record,
        "member_record": child_record,
        "parent_table": getattr(parent_table, "name", "") if parent_table else "",
        "child_table": getattr(child_table, "name", "") if child_table else "",
        "parent_key": parent_keys[0] if parent_keys else None,
        "parent_keys": parent_keys,
        "child_fk": child_fks[0] if child_fks else None,
        "child_fks": child_fks,
        "cardinality": getattr(relationship, "cardinality", "1:N") or "1:N",
        "order_by": order_by,
    }
)

return relationships_payload

def build_set_ordering_map(
    self,
    relationships: list[dict[str, Any]],
) -> dict[str, dict[str, Any]]:
    set_ordering_map: dict[str, dict[str, Any]] = {}

    for relationship in relationships:
        set_name = relationship.get("set_name") or ""

        if not set_name:
            continue

        set_ordering_map[set_name] = {
            "parent_record": relationship.get("parent_record") or "",
            "child_record": relationship.get("child_record") or "",
            "parent_key": relationship.get("parent_key"),
            "parent_keys": relationship.get("parent_keys") or [],
            "child_fk": relationship.get("child_fk"),
            "child_fks": relationship.get("child_fks") or [],
            "order_by": relationship.get("order_by") or [],
        }

    return set_ordering_map

def build_nullable_fk_map(
    self,
    db2_model: DB2Model,

```

```

) -> dict[str, Any]:
    nullable_fk_map: dict[str, Any] = {}

    for table in getattr(db2_model, "tables", []) or []:
        table_name = getattr(table, "name", "") or ""

        column_lookup = {
            getattr(column, "name", ""): column
            for column in getattr(table, "columns", []) or []
        }

        for foreign_key in getattr(table, "foreign_keys", []) or []:
            fk_column_name = getattr(foreign_key, "column_name", "") or ""
            fk_column = column_lookup.get(fk_column_name)

            nullable = (
                getattr(fk_column, "nullable", True)
                if fk_column is not None
                else True
            )

            payload = {
                "nullable": nullable,
                "set_name": getattr(foreign_key, "set_name", "") or "",
                "reference_table": getattr(foreign_key, "reference_table", "") or "",
                "reference_column": getattr(foreign_key, "reference_column", "") or "",
                "table": table_name,
                "column": fk_column_name,
                "null_indicator": self.null_indicator_name(
                    record=table_name,
                    column=fk_column_name,
                ),
            }

            key = f"{table_name}.{fk_column_name}"
            nullable_fk_map[key] = payload

            set_name = self.to_db2_name(
                getattr(foreign_key, "set_name", "") or "",
            )

            if set_name:
                nullable_fk_map[set_name] = payload

    return nullable_fk_map

def build_date_part_map(
    self,
    metadata: SchemaMetadata,
    db2_table_lookup: dict[str, DB2Table],
    record_table_map: dict[str, str],
) -> dict[str, dict[str, Any]]:
    date_part_map: dict[str, dict[str, Any]] = {}

    for record in getattr(metadata, "records", []) or []:
        idms_record_name = self.to_db2_name(
            getattr(record, "name", "") or "",
        )

        if not idms_record_name:
            continue

        table_name = record_table_map.get(
            idms_record_name,
            idms_record_name,
        )

        table = self.find_table_for_record(
            record_name=table_name,
            db2_table_lookup=db2_table_lookup,
        )

        if table is None:
            continue

        date_columns = {

```



```

        key: column
        for key, column in self.build_column_lookup(table).items()
        if str(getattr(column, "datatype", "") or "").upper() == "DATE"
    }

    if not date_columns:
        continue

    source_fields = (
        getattr(record, "mapping_fields", None)
        or getattr(record, "fields", None)
        or []
    )

    for field in source_fields:
        raw_legacy_field_name = str(
            getattr(field, "name", "") or ""
        ).strip()

        legacy_field_name = self.to_db2_name(
            raw_legacy_field_name,
        )

        if not legacy_field_name:
            continue

        parsed_candidates = self.parse_date_part_candidates(
            field_name=legacy_field_name,
        )

        if not parsed_candidates:
            continue

        for parsed in parsed_candidates:
            part = parsed["part"]
            candidate_keys = parsed["candidate_keys"]

            date_column = self.find_first_existing_column(
                candidates=candidate_keys,
                columns=date_columns,
            )

            if date_column is None:
                continue

            date_column_name = getattr(date_column, "name", "") or ""
            host = self.host_variable_name(
                record=idms_record_name,
                column=date_column_name,
            )

            payload = {
                "record": idms_record_name,
                "table": getattr(table, "name", "") or "",
                "column": date_column_name,
                "host": host,
                "substring_start": self.DATE_PARTS[part]["substring_start"],
                "substring_length": self.DATE_PARTS[part]["substring_length"],
                "part": part,
                "date_part": part,
                "legacy_field": legacy_field_name,
            }

            for key in self.name_lookup_keys(legacy_field_name):
                date_part_map[key] = payload

            for key in self.name_lookup_keys(raw_legacy_field_name):
                date_part_map[key] = payload

        break

    return date_part_map

def parse_date_part(
    self,
    field_name: str,

```

```

) -> dict[str, Any] | None:
    candidates = self.parse_date_part_candidates(
        field_name=field_name,
    )

    if not candidates:
        return None

    return candidates[0]

def parse_date_part_candidates(
    self,
    field_name: str,
) -> list[dict[str, Any]]:
    normalized = self.to_db2_name(field_name or "")

    if not normalized:
        return []

    tokens = [
        token
        for token in re.split(r"[\s_]+", normalized)
        if token
    ]

    candidates = []

    for index, token in enumerate(tokens):
        part = self.date_part_type(
            token=token,
            tokens=tokens,
        )

        if part is None:
            continue

        base_tokens = tokens[:index] + tokens[index + 1 :]
        date_tokens = tokens.copy()
        date_tokens[index] = "DATE"

        base_name = "_".join(base_tokens)
        date_name = "_".join(date_tokens)
        compact_base = self.compact_name(base_name)

        candidate_keys = self.unique_values(
            [
                date_name,
                base_name,
                compact_base + "_DATE",
                "DA_" + compact_base + "DATE",
                "DA_" + date_name,
                self.remove_record_suffix(date_name),
                self.remove_record_suffix(base_name),
                self.remove_record_suffix(compact_base + "_DATE"),
                self.remove_record_suffix("DA_" + compact_base + "DATE"),
            ]
        )

        candidates.append(
            {
                "date_key": date_name,
                "part": part,
                "tokens": date_tokens,
                "candidate_keys": candidate_keys,
            }
        )

    return candidates

def date_part_type(
    self,
    token: str,
    tokens: list[str],
) -> str | None:
    token = str(token or "").upper()

```

```

has_dy_dm_dd = (
    "DY" in tokens
    and "DM" in tokens
    and "DD" in tokens
)

if token in {"YEAR", "YR", "Y", "YY", "YYYY"}:
    return "YEAR"

if token in {"MONTH", "MON", "MO", "M", "MM", "DM"}:
    return "MONTH"

if token in {"DAY", "D", "DD"}:
    return "DAY"

if token == "DY":
    if has_dy_dm_dd:
        return "YEAR"

    return "DAY"

return None

def build_navigation_intent(
    self,
    relationships: list[dict[str, Any]],
) -> dict[str, Any]:
    navigation_intent: dict[str, Any] = {}

    for relationship in relationships:
        set_name = relationship.get("set_name") or ""

        if not set_name:
            continue

        navigation_intent[set_name] = {
            "access_pattern": "SET_NAVIGATION",
            "parent_record": relationship.get("parent_record"),
            "child_record": relationship.get("child_record"),
            "parent_key": relationship.get("parent_key"),
            "parent_keys": relationship.get("parent_keys") or [],
            "child_fk": relationship.get("child_fk"),
            "child_fks": relationship.get("child_fks") or [],
            "order_by": relationship.get("order_by") or [],
        }

    return navigation_intent

def build_output_semantics(
    self,
) -> dict[str, Any]:
    return {
        "generated_by": "idms-db2-modernizer-phase1",
        "usage": "IDMS retrieval COBOL to DB2 embedded SQL conversion",
        "physical_types_source": "DB2_DDL",
        "logical_mapping_source": "PHASE1_CANONICAL_METADATA",
        "db2_key_values": ["PK", "FK", "PK/FK"],
        "generated_primary_key_rule": "ID_RECORD_<record_name>",
        "generated_primary_key_datatype": "CHAR(20)",
        "foreign_key_rule": "Use SET relationship and owner primary key columns only.",
        "character_rule": "CHAR by default; VARCHAR only when length equals 100.",
        "numeric_rule": "Numeric COBOL PIC maps to DECIMAL.",
        "date_rule": "Date part recognition remains generic for YEAR, YR, Y, YY, YYYY, DY, MONTH, MON, MO, M, MM, D
M, DAY, D, DD.",
    }

def build_validation_messages(
    self,
    canonical_schema: CanonicalSchema,
    db2_model: DB2Model,
    metadata: SchemaMetadata,
    field_map: dict[str, dict[str, Any]],
) -> list[str]:
    messages: list[str] = []

    if not getattr(canonical_schema, "records", []) or []:

```

```

        messages.append("No canonical records found.")

    if not getattr(db2_model, "tables", []) or []:
        messages.append("No DB2 tables found.")

    if not getattr(metadata, "records", []) or []:
        messages.append("No IDMS metadata records found.")

    if not field_map:
        messages.append("No field map entries generated.")

    for table in getattr(db2_model, "tables", []) or []:
        primary_keys = list(getattr(table, "primary_keys", []) or [])

        if not primary_keys and getattr(table, "primary_key", None):
            primary_keys = [table.primary_key]

        if not primary_keys:
            messages.append(f"Table {table.name} has no primary key.")
            continue

        column_names = {
            self.to_db2_name(getattr(column, "name", "") or "")
            for column in getattr(table, "columns", []) or []
        }

        for primary_key in primary_keys:
            if self.to_db2_name(primary_key) not in column_names:
                messages.append(
                    f"Table {table.name} primary key {primary_key} is not present as a DB2 column."
                )

    return messages

def parse_db2_datatype(
    self,
    datatype: str,
) -> tuple[str, int | None, int | None]:
    value = (datatype or "").strip().upper()

    decimal_match = re.match(
        r"^(DECIMAL|NUMERIC)$($(\d+)(?:, \s*(\d+))?$)$",
        value,
    )

    if decimal_match:
        return (
            "DECIMAL",
            int(decimal_match.group(2)),
            int(decimal_match.group(3) or 0),
        )

    char_match = re.match(
        r"^(VARCHAR|CHAR|CHARACTER)$($(\d+)$)$",
        value,
    )

    if char_match:
        datatype_name = char_match.group(1)

        if datatype_name == "CHARACTER":
            datatype_name = "CHAR"

        return (
            datatype_name,
            int(char_match.group(2)),
            None,
        )

    if value in {"INTEGER", "INT"}:
        return "INTEGER", 9, 0

    if value == "BIGINT":
        return "BIGINT", 18, 0

    if value == "SMALLINT":

```

```

        return "INTEGER", 9, 0

    if value == "DATE":
        return "DATE", None, None

    if value == "TIMESTAMP":
        return "TIMESTAMP", None, None

    return value or "CHAR", None, None

def host_variable_name(
    self,
    record: str,
    column: str,
) -> str:
    record_name = self.to_cobol_name(record or "")
    column_name = self.to_cobol_name(column or "")

    return f"HV-{{record_name}}-{{column_name}}"

def null_indicator_name(
    self,
    record: str,
    column: str,
) -> str:
    record_name = self.to_cobol_name(record or "")
    column_name = self.to_cobol_name(column or "")

    return f"NI-{{record_name}}-{{column_name}}"

def find_table_for_record(
    self,
    record_name: str,
    db2_table_lookup: dict[str, DB2Table],
) -> DB2Table | None:
    for key in self.name_lookup_keys(record_name):
        if key in db2_table_lookup:
            return db2_table_lookup[key]

    return None

def find_column_for_legacy_field(
    self,
    legacy_field_name: str,
    column_lookup: dict[str, DB2Column],
) -> DB2Column | None:
    for key in self.name_lookup_keys(legacy_field_name):
        if key in column_lookup:
            return column_lookup[key]

    legacy_base = self.remove_record_suffix(legacy_field_name)
    legacy_compact = self.compact_name(legacy_base)

    for column_key, column in column_lookup.items():
        column_base = self.remove_record_suffix(column_key)
        column_generated_removed = self.remove_generated_suffix(column_key)
        column_compact = self.compact_name(column_generated_removed or column_base)

        if legacy_base == column_base:
            return column

        if legacy_base == column_generated_removed:
            return column

        if legacy_compact and legacy_compact == column_compact:
            return column

        if legacy_compact and column_compact.endswith(legacy_compact):
            return column

        if legacy_compact and legacy_compact in column_compact:
            return column

    return None

def find_first_existing_column(

```

```

self,
candidates: list[str],
columns: dict[str, DB2Column],
) -> DB2Column | None:
    for candidate in candidates:
        for key in self.name_lookup_keys(candidate):
            if key in columns:
                return columns[key]

    candidate_base = self.remove_record_suffix(candidate)
    candidate_generated_removed = self.remove_generated_suffix(candidate)
    candidate_compact = self.compact_name(candidate_generated_removed or candidate_base)

    for column_key, column in columns.items():
        column_base = self.remove_record_suffix(column_key)
        column_generated_removed = self.remove_generated_suffix(column_key)
        column_compact = self.compact_name(column_generated_removed or column_base)

        if candidate_base == column_base:
            return column

        if candidate_generated_removed == column_generated_removed:
            return column

        if candidate_compact and candidate_compact == column_compact:
            return column

        if candidate_compact and candidate_compact in column_compact:
            return column

        if candidate_compact and column_compact.endswith(candidate_compact):
            return column

    return None

def extract_primary_keys_from_record(
    self,
    record,
) -> list[str]:
    primary_keys = []

    explicit_primary_keys = getattr(record, "primary_keys", None)

    if explicit_primary_keys:
        if isinstance(explicit_primary_keys, list):
            primary_keys.extend(explicit_primary_keys)
        else:
            primary_keys.append(explicit_primary_keys)

    primary_key = getattr(record, "primary_key", None)

    if primary_key:
        primary_keys.append(primary_key)

    return self.unique_values(
        [
            self.to_db2_name(primary_key)
            for primary_key in primary_keys
            if primary_key
        ]
    )

def name_lookup_keys(
    self,
    value: str | None,
) -> list[str]:
    original = str(value or "").strip().upper()

    if not original:
        return []

    db2_name = self.to_db2_name(original)
    cobol_name = self.to_cobol_name(original)
    no_record_suffix = self.remove_record_suffix(db2_name)
    no_generated_suffix = self.remove_generated_suffix(db2_name)
    compact = self.compact_name(no_generated_suffix or no_record_suffix or db2_name)

```

```

        return self.unique_values(
            [
                original,
                db2_name,
                cobol_name,
                no_record_suffix,
                no_generated_suffix,
                compact,
            ]
        )

    def to_db2_name(
        self,
        value: str | None,
    ) -> str:
        text = str(value or "").strip().upper()

        if not text:
            return ""

        text = text.replace("\u00a0", " ")
        text = text.replace("\t", " ")
        text = re.sub(r"^[A-Z0-9]+", "_", text)
        text = re.sub(r"_" +, "_", text)
        text = text.strip("_")

        return text

    def to_cobol_name(
        self,
        value: str | None,
    ) -> str:
        text = self.to_db2_name(value)

        text = text.replace("_", "-")
        text = re.sub(r"-+", "-", text)

        return text.strip("-")

    def remove_record_suffix(
        self,
        value: str | None,
    ) -> str:
        text = self.to_db2_name(value)

        if not text:
            return ""

        text = re.sub(r"[_\-\s]+[0-9]{4}$", "", text)
        text = re.sub(r"[0-9]{4}$", "", text)
        text = re.sub(r"[_\-\s]+$", "", text)

        return text

    def remove_generated_suffix(
        self,
        value: str | None,
    ) -> str:
        text = self.to_db2_name(value)

        if not text:
            return ""

        text = re.sub(r"_479[A-Z0-9]+$", "", text)
        text = re.sub(r"479[A-Z0-9]+$", "", text)
        text = re.sub(r"[_\-\s]+$", "", text)

        return text

    def compact_name(
        self,
        value: str | None,
    ) -> str:
        return re.sub(
            r"^[A-Z0-9]",

```

```

        "",
        str(value or "").upper(),
    )

def unique_values(
    self,
    values: list[str],
) -> list[str]:
    result: list[str] = []
    seen: set[str] = set()

    for value in values:
        normalized = str(value or "").strip().upper()

        if not normalized:
            continue

        if normalized in seen:
            continue

        seen.add(normalized)
        result.append(normalized)

    return result

```

```

=====
FILE: src\idms_modernizer\services\relationship_resolver.py
ENCODING: utf-8
=====

```

```

from collections import defaultdict

from idms_modernizer.domain.relationship_models import (
    Relationship,
)

class RelationshipResolver:
    """
    Resolves IDMS SET owner/member relationships from record set memberships.

    Generic behavior only:
    - No hardcoded SET names.
    - No hardcoded record names.
    - OWNER and MEMBER are read from schema listing memberships.
    """

    def resolve(
        self,
        metadata,
    ) -> list[Relationship]:
        owner_map: dict[str, list[str]] = defaultdict(list)
        member_map: dict[str, list[str]] = defaultdict(list)

        for record in getattr(metadata, "records", []) or []:
            record_name = getattr(record, "name", "") or ""

            if not record_name:
                continue

            for membership in getattr(record, "set_memberships", []) or []:
                set_name = (
                    getattr(membership, "set_name", None)
                    or getattr(membership, "name", None)
                    or ""
                )

                role = (
                    getattr(membership, "role", None)
                    or getattr(membership, "relation_type", None)
                    or getattr(membership, "type", None)
                    or ""
                )

                set_name = str(set_name or "").strip().upper()

```



```

role = str(role or "").strip().upper()

if not set_name:
    continue

if not role:
    continue

if set_name == "CALC":
    continue

if role == "OWNER":
    if record_name not in owner_map[set_name]:
        owner_map[set_name].append(record_name)

elif role == "MEMBER":
    if record_name not in member_map[set_name]:
        member_map[set_name].append(record_name)

relationships: list[Relationship] = []

all_sets = set(owner_map.keys()) | set(member_map.keys())

for set_name in sorted(all_sets):
    owners = owner_map.get(set_name, [])
    members = member_map.get(set_name, [])

    if not owners:
        continue

    if not members:
        continue

    for owner in owners:
        for member in members:
            if not owner or not member:
                continue

            if owner == member:
                continue

            relationships.append(
                Relationship(
                    set_name=set_name,
                    owner_record=owner,
                    member_record=member,
                    cardinality="1:N",
                )
            )

metadata.relationships = relationships

return relationships

```

```

=====
FILE: src\idms_modernizer\services\schema_picture_enricher.py
ENCODING: utf-8
=====

```

```
import re
```

```

from idms_modernizer.domain.schema_models import DataField
from idms_modernizer.services.name_normalizer import NameNormalizer

```

```
print("LOADED SchemaPictureEnricher VERSION GENERIC-PIC-PARENS-V-COMP3-FIX-2026-07-30")
```

```

class SchemaPictureEnricher:
    """

```

```
    Enriches DataField objects using schema listing lines.
```

```
    Critical rules:
```

- IDMS PIC Clause must come from schema PICTURE, not byte length.
- PIC S9(13)V9(2) must remain PIC S9(13)V9(2).
- PIC S9(13)V9(2) COMP-3 must remain PIC S9(13)V9(2) COMP-3.

- Do not convert PIC S9(13)V9(2) to PIC S9(8).
- Do not convert PIC 9(8) COMP-3 to PIC 9(5) COMP-3.
- Storage length remains byte length.
- Numeric logical precision comes from PIC.

Generic behavior:

- No hardcoded record names.
- No hardcoded field names.
- No hardcoded application-specific debug target list.
- Debug is controlled by DEBUG / DEBUG_VERBOSE only.

```
DEBUG = True
DEBUG_VERBOSE = False

FIELD_START_PATTERN = re.compile(
    r"^\s*(?P<level>0[1-9]|[1-4][0-9]|88)\s+"
    r"(?P<name>[A-Z][A-Z0-9-]*|FILLER)\b"
    r"(?P<rest>.*)$",
    re.IGNORECASE,
)

USAGE_PATTERN = re.compile(
    r"\b(?:USAGE\s+IS\s+)?(?:P<usage>DISPLAY|COMP-3|COMP)\b",
    re.IGNORECASE,
)

START_LENGTH_PATTERN = re.compile(
    r"\b(?:P<start>[0-9]+\s+(?P<length>[0-9]+\s*)$",
    re.IGNORECASE,
)

COMPACT_PIC_PATTERNS = [
    re.compile(r"S?9[()][0-9]+[()]V9[()][0-9]+[()]", re.IGNORECASE),
    re.compile(r"S?9[()][0-9]+[()]V[90]+", re.IGNORECASE),
    re.compile(r"S?[90]+V9[()][0-9]+[()]", re.IGNORECASE),
    re.compile(r"S?[90]+V[90]+", re.IGNORECASE),
    re.compile(r"S?9[()][0-9]+[()]", re.IGNORECASE),
    re.compile(r"S?[90]+", re.IGNORECASE),
    re.compile(r"X[()][0-9]+[()]", re.IGNORECASE),
    re.compile(r"X+", re.IGNORECASE),
]

DATE_NAME_TOKENS = {
    "DA",
    "DATE",
    "DT",
    "DTE",
    "YYMMDD",
    "YYYYMMDD",
    "YMD",
}

YEAR_PARTS = {
    "YEAR",
    "YR",
    "Y",
    "YY",
    "YYYY",
    "DY",
}

MONTH_PARTS = {
    "MONTH",
    "MON",
    "MO",
    "M",
    "MM",
    "DM",
}

DAY_PARTS = {
    "DAY",
    "D",
    "DD",
}
```

```

def enrich(
    self,
    fields: list[DataField],
    lines: list[str],
    debug_label: str = "",
) -> list[DataField]:
    if self.DEBUG:
        print(
            "USING SchemaPictureEnricher.enrich VERSION GENERIC-PIC-PARENS-V-COMP3-FIX-2026-07-30",
            f"debug_label={debug_label}",
        )

    normalized_lines = self.normalize_lines(
        lines=lines,
    )

    if self.DEBUG:
        print(
            "SCHEMA_ENRICH_DEBUG_LINES",
            f"debug_label={debug_label}",
            f"line_count={len(normalized_lines)}",
        )

    field_blocks = self.build_field_blocks(
        normalized_lines=normalized_lines,
        debug_label=debug_label,
    )

    enriched_fields: list[DataField] = []

    for field in fields or []:
        field_name = getattr(field, "name", "") or ""

        block = self.find_block_for_field(
            field_name=field_name,
            field_blocks=field_blocks,
        )

        if not block:
            if self.DEBUG:
                print(
                    "SCHEMA_ENRICH_NO_BLOCK",
                    f"debug_label={debug_label}",
                    f"field={repr(field_name)}",
                )

            enriched_fields.append(
                self.copy_field_with_defaults(
                    field=field,
                )
            )
            continue

        enriched_fields.append(
            self.enrich_field(
                field=field,
                block=block,
                debug_label=debug_label,
            )
        )

    enriched_fields = self.enrich_group_positions_from_children(
        fields=enriched_fields,
        debug_label=debug_label,
    )

    return enriched_fields

def normalize_lines(
    self,
    lines: list[str],
) -> list[str]:
    result: list[str] = []

    for line in lines or []:

```

```

        if line is None:
            continue

        value = getattr(line, "text", None)

        if value is None:
            value = str(line)

        value = str(value)
        value = value.replace("\t", " ")
        value = value.replace("\u00a0", " ")
        value = value.replace("PICTURE .", "PICTURE")
        value = value.replace("PIC .", "PIC")
        value = re.sub(r"<[^>+>", " ", value)
        value = re.sub(r"\s+", " ", value).strip()

        if value:
            result.append(value)

    return result

def build_field_blocks(
    self,
    normalized_lines: list[str],
    debug_label: str = "",
) -> dict[str, str]:
    starts: list[tuple[int, str]] = []

    for index, line in enumerate(normalized_lines or []):
        match = self.FIELD_START_PATTERN.match(line)

        if not match:
            continue

        field_name = match.group("name") or ""

        starts.append(
            (
                index,
                field_name.upper(),
            )
        )

    if self.DEBUG:
        print(
            "SCHEMA_BLOCK_STARTS_DEBUG",
            f"debug_label={debug_label}",
            f"starts_count={len(starts)}",
            f"starts={starts[:80]}",
        )

    blocks: dict[str, str] = {}

    for index, item in enumerate(starts):
        start_index, field_name = item

        if index + 1 < len(starts):
            end_index = starts[index + 1][0]
        else:
            end_index = len(normalized_lines)

        block = " ".join(
            normalized_lines[start_index:end_index],
        )

        normalized_field_name = NameNormalizer.normalize(
            field_name,
        )

        if normalized_field_name:
            blocks[normalized_field_name] = block

        suffix_removed = self.remove_record_suffix(
            value=normalized_field_name,
        )

```

```

        if suffix_removed:
            blocks.setdefault(
                suffix_removed,
                block,
            )

        if self.DEBUG_VERBOSE:
            print(
                "SCHEMA_BLOCK_DEBUG",
                f"debug_label={debug_label}",
                f"field={repr(normalized_field_name)}",
                f"block={repr(block)}",
            )

    return blocks

def find_block_for_field(
    self,
    field_name: str,
    field_blocks: dict[str, str],
) -> str:
    normalized_name = NameNormalizer.normalize(
        field_name or "",
    )

    if not normalized_name:
        return ""

    if normalized_name in field_blocks:
        return field_blocks[normalized_name]

    suffix_removed = self.remove_record_suffix(
        value=normalized_name,
    )

    if suffix_removed in field_blocks:
        return field_blocks[suffix_removed]

    for block_name, block in field_blocks.items():
        if self.remove_record_suffix(block_name) == suffix_removed:
            return block

    return ""

def enrich_field(
    self,
    field: DataField,
    block: str,
    debug_label: str = "",
) -> DataField:
    field_name = getattr(field, "name", "") or ""

    if not block:
        return self.copy_field_with_defaults(
            field=field,
        )

    parsed_line = self.parse_schema_block(
        block=block,
        field_name=field_name,
        debug_label=debug_label,
    )

    usage = parsed_line["usage"]
    start_position = parsed_line["start_position"]
    storage_length = parsed_line["storage_length"]
    picture_core = parsed_line["picture_core"]

    is_group = bool(
        getattr(field, "has_child", False)
        or getattr(field, "is_group", False)
    )

    if is_group:
        return self.make_field(
            source=field,

```

```

        datatype=getattr(field, "datatype", None),
        length=storage_length or getattr(field, "length", None),
        scale=getattr(field, "scale", None),
        picture=None,
        start_position=start_position or getattr(field, "start_position", None),
        end_position=self.calculate_end_position(
            start_position=start_position,
            storage_length=storage_length,
            fallback=getattr(field, "end_position", None),
        ),
        basetype=getattr(field, "basetype", None),
    )

    rendered_picture = self.render_cobol_pic_clause(
        picture=picture_core,
        usage=usage,
        storage_length=storage_length,
    )

    datatype, logical_length, scale = self.derive_type_from_picture(
        picture=rendered_picture,
        usage=usage,
        storage_length=storage_length,
        fallback_datatype=getattr(field, "datatype", None),
    )

    if self.is_date_field(
        field=field,
        picture=rendered_picture,
        storage_length=storage_length,
        datatype=datatype,
    ):
        datatype = "DATE"
        scale = None

    end_position = self.calculate_end_position(
        start_position=start_position,
        storage_length=storage_length,
        fallback=getattr(field, "end_position", None),
    )

    actual_length = storage_length

    if actual_length is None:
        actual_length = logical_length

    if self.DEBUG_VERBOSE:
        print(
            "SCHEMA_ENRICH_RESULT",
            f"debug_label={debug_label}",
            f"field={repr(field_name)}",
            f"block={repr(block)}",
            f"usage={repr(usage)}",
            f"picture_core={repr(picture_core)}",
            f"rendered_picture={repr(rendered_picture)}",
            f"datatype={repr(datatype)}",
            f"logical_length={logical_length}",
            f"storage_length={storage_length}",
            f"actual_length={actual_length}",
            f"scale={scale}",
            f"start={start_position}",
            f"end={end_position}",
        )

    return self.make_field(
        source=field,
        datatype=datatype,
        length=actual_length,
        scale=scale,
        picture=rendered_picture,
        start_position=start_position,
        end_position=end_position,
        basetype=self.basetype_for_datatype(
            datatype=datatype,
        ),
    )

```

```

def parse_schema_block(
    self,
    block: str,
    field_name: str,
    debug_label: str = "",
) -> dict:
    text = str(block or "").strip()

    usage = self.extract_usage(
        block=text,
    )

    start_position, storage_length = self.extract_start_and_length(
        block=text,
    )

    body_without_positions = self.remove_trailing_start_and_length(
        value=text,
    )

    rest = self.extract_rest_after_level_and_name(
        block=body_without_positions,
    )

    picture_area = self.remove_usage_words(
        value=rest,
    )

    picture_area = self.remove_schema_noise(
        value=picture_area,
    )

    picture_core = self.extract_picture_core(
        value=picture_area,
        field_name=field_name,
        debug_label=debug_label,
    )

    if self.DEBUG_VERBOSE:
        print(
            "SCHEMA_PARSE_DEBUG",
            f"debug_label={debug_label}",
            f"field={repr(field_name)}",
            f"block={repr(block)}",
            f"usage={repr(usage)}",
            f"start={start_position}",
            f"length={storage_length}",
            f"rest={repr(rest)}",
            f"picture_area={repr(picture_area)}",
            f"picture_core={repr(picture_core)}",
        )

    return {
        "usage": usage,
        "start_position": start_position,
        "storage_length": storage_length,
        "picture_core": picture_core,
    }

def extract_rest_after_level_and_name(
    self,
    block: str,
) -> str:
    match = self.FIELD_START_PATTERN.match(
        block or "",
    )

    if not match:
        return str(block or "")

    return str(match.group("rest") or "").strip()

def extract_usage(
    self,
    block: str,

```

```

) -> str:
    upper = str(block or "").upper()

    if "COMP-3" in upper:
        return "COMP-3"

    if re.search(r"\bCOMP\b", upper):
        return "COMP"

    usage_match = self.USAGE_PATTERN.search(
        block or "",
    )

    if usage_match:
        return usage_match.group("usage").upper()

    return "DISPLAY"

def extract_start_and_length(
    self,
    block: str,
) -> tuple[int | None, int | None]:
    match = self.START_LENGTH_PATTERN.search(
        block or "",
    )

    if not match:
        return None, None

    return int(match.group("start")), int(match.group("length"))

def remove_trailing_start_and_length(
    self,
    value: str,
) -> str:
    return re.sub(
        r"\b[0-9]+\s+[0-9]+\s*$",
        "",
        str(value or ""),
    ).strip()

def remove_usage_words(
    self,
    value: str,
) -> str:
    text = str(value or "")

    text = re.sub(r"\bUSAGE\s+IS\s+COMP-3\b", " ", text, flags=re.IGNORECASE)
    text = re.sub(r"\bUSAGE\s+IS\s+COMP\b", " ", text, flags=re.IGNORECASE)
    text = re.sub(r"\bUSAGE\s+IS\s+DISPLAY\b", " ", text, flags=re.IGNORECASE)
    text = re.sub(r"\bCOMP-3\b", " ", text, flags=re.IGNORECASE)
    text = re.sub(r"\bCOMP\b", " ", text, flags=re.IGNORECASE)
    text = re.sub(r"\bDISPLAY\b", " ", text, flags=re.IGNORECASE)

    return re.sub(r"\s+", " ", text).strip()

def remove_schema_noise(
    self,
    value: str,
) -> str:
    text = str(value or "")

    text = re.sub(r"\bOCCURS\b.*$", " ", text, flags=re.IGNORECASE)
    text = re.sub(r"\bREDEFINES\b.*$", " ", text, flags=re.IGNORECASE)
    text = text.replace(".", " ")
    text = text.replace("\u00a0", " ")
    text = text.replace("\t", " ")

    return re.sub(r"\s+", " ", text).strip()

def extract_picture_core(
    self,
    value: str,
    field_name: str = "",
    debug_label: str = "",
) -> str | None:

```



```

original = str(value or "").upper()

compact = re.sub(r"\s+", "", original)
compact = compact.replace("PICTURE", "")
compact = compact.replace("PIC", "")
compact = compact.replace(".", "")

candidates: list[tuple[int, int, str]] = []

for pattern in self.COMPACT_PIC_PATTERNS:
    for match in pattern.finditer(compact):
        raw = match.group(0)

        core = self.clean_picture_core(
            value=raw,
        )

        if not core:
            continue

        score = self.picture_score(
            core=core,
        )

        candidates.append(
            (
                score,
                match.start(),
                core,
            )
        )

if self.DEBUG_VERBOSE:
    print(
        "PIC_EXTRACT_DEBUG_INPUT",
        f"debug_label={debug_label}",
        f"field={repr(field_name)}",
        f"value={repr(value)}",
        f"compact={repr(compact)}",
        f"patterns=[[pattern.pattern for pattern in self.COMPACT_PIC_PATTERNS]]",
        f"candidates_before_sort={candidates}",
    )

if not candidates:
    return None

candidates.sort(
    key=lambda item: (item[0], item[1]),
)

selected = candidates[-1][2]

if self.DEBUG_VERBOSE:
    print(
        "PIC_EXTRACT_DEBUG_RESULT",
        f"debug_label={debug_label}",
        f"field={repr(field_name)}",
        f"candidates_after_sort={candidates}",
        f"selected={repr(selected)}",
    )

return selected

def picture_score(
    self,
    core: str,
) -> int:
    text = str(core or "").upper()

    score = 0

    if "V" in text:
        score += 1000

    if "(" in text and ")" in text:
        score += 100

```

```

        if text.startswith("S"):
            score += 10

        if text.startswith("X"):
            score += 5

        score += len(text)

    return score

def clean_picture_core(
    self,
    value: str | None,
) -> str | None:
    if value is None:
        return None

    text = str(value).upper()
    text = text.replace("PICTURE", "")
    text = text.replace("PIC", "")
    text = text.replace(".", "")
    text = text.replace("\u00a0", " ")
    text = text.replace("\t", " ")
    text = re.sub(r"\s+", "", text)

    return text.strip() or None

def render_cobol_pic_clause(
    self,
    picture: str | None,
    usage: str,
    storage_length: int | None,
) -> str | None:
    if not picture:
        return None

    normalized_picture = self.normalize_picture_preserving_v(
        picture=picture,
        storage_length=storage_length,
    )

    if not normalized_picture:
        return None

    usage_upper = str(usage or "DISPLAY").upper()

    if usage_upper in {"COMP", "COMP-3"}:
        if usage_upper not in normalized_picture.upper():
            normalized_picture = f"{normalized_picture} {usage_upper}"

    if normalized_picture.upper().startswith("PIC "):
        return normalized_picture

    return f"PIC {normalized_picture}"

def normalize_picture_preserving_v(
    self,
    picture: str | None,
    storage_length: int | None = None,
) -> str | None:
    if not picture:
        return None

    core = self.picture_core(
        picture,
    )

    if not core:
        return None

    if "V" in core:
        before_v, after_v = core.split("V", 1)

        before_v_normalized = self.normalize_9_part(
            part=before_v,

```

```

        storage_length=None,
    )

    after_v_normalized = self.normalize_9_part(
        part=after_v,
        storage_length=None,
    )

    return f"{before_v_normalized}V{after_v_normalized}"

if core.startswith("X"):
    return self.normalize_x_part(
        part=core,
        storage_length=storage_length,
    )

return self.normalize_9_part(
    part=core,
    storage_length=storage_length,
)

def normalize_x_part(
    self,
    part: str,
    storage_length: int | None = None,
) -> str:
    text = str(part or "").upper()

    if text.startswith("X(") and text.endswith(")"):
        return text

    if text and set(text) == {"X"}:
        if len(text) == 1 and storage_length and int(storage_length) > 1:
            return f"X({int(storage_length)})"

        if len(text) == 1:
            return "X"

        return f"X({len(text)})"

    return text

def normalize_9_part(
    self,
    part: str,
    storage_length: int | None = None,
) -> str:
    text = str(part or "").upper()
    signed = False

    if text.startswith("S"):
        signed = True
        text = text[1:]

    if text.startswith("9(") and text.endswith(")"):
        return f"{'S' if signed else ''}{text}"

    count = self.count_9_digits(
        value=text,
    )

    if count <= 0:
        return f"{'S' if signed else ''}{text}"

    if count == 1 and storage_length and int(storage_length) > 1:
        return f"{'S' if signed else ''}9({int(storage_length)})"

    if count == 1:
        return f"{'S' if signed else ''}9"

    return f"{'S' if signed else ''}9({count})"

def derive_type_from_picture(
    self,
    picture: str | None,
    usage: str,

```

```

        storage_length: int | None,
        fallback_datatype: str | None = None,
    ) -> tuple[str | None, int | None, int | None]:
        if not picture:
            return fallback_datatype, storage_length, None

        core = self.picture_core(
            picture=picture,
        )

        if not core:
            return fallback_datatype, storage_length, None

        if self.picture_is_character(core):
            length = self.character_length(
                picture=core,
            )

            return "CHAR", length, None

        if self.picture_is_decimal(core):
            precision, scale = self.decimal_precision_scale(
                picture=core,
            )

            return "DECIMAL", precision, scale

        if self.picture_is_numeric(core):
            precision = self.numeric_precision(
                picture=core,
            )

            return "DECIMAL", precision, 0

        return fallback_datatype, storage_length, None

def picture_core(
    self,
    picture: str | None,
) -> str:
    text = str(picture or "").upper()
    text = text.replace("PICTURE", "")
    text = text.replace("PIC", "")
    text = text.replace("COMP-3", "")
    text = re.sub(r"\bCOMP\b", "", text)
    text = text.replace("DISPLAY", "")
    text = text.replace("USAGE", "")
    text = text.replace("IS", "")
    text = text.replace(".", "")
    text = re.sub(r"\s+", "", text)

    return text.strip()

def picture_is_character(
    self,
    picture: str,
) -> bool:
    text = str(picture or "").upper()

    if text.startswith("X(") and text.endswith(")"):
        return self.extract_parenthesized_int(text) is not None

    return bool(text) and set(text) == {"X"}

def character_length(
    self,
    picture: str,
) -> int:
    text = str(picture or "").upper()

    if text.startswith("X(") and text.endswith(")"):
        parsed = self.extract_parenthesized_int(text)

        if parsed is not None:
            return parsed

```

```

        return text.count("X")

def picture_is_numeric(
    self,
    picture: str,
) -> bool:
    text = str(picture or "").upper()

    if text.startswith("S"):
        text = text[1:]

    return "9" in text and "V" not in text

def picture_is_decimal(
    self,
    picture: str,
) -> bool:
    text = str(picture or "").upper()

    if text.startswith("S"):
        text = text[1:]

    return "V" in text and "9" in text

def decimal_precision_scale(
    self,
    picture: str,
) -> tuple[int, int]:
    text = str(picture or "").upper()

    if text.startswith("S"):
        text = text[1:]

    before_v, after_v = text.split("V", 1)

    integer_digits = self.count_9_digits(
        value=before_v,
    )

    decimal_digits = self.count_9_digits(
        value=after_v,
    )

    return integer_digits + decimal_digits, decimal_digits

def numeric_precision(
    self,
    picture: str,
) -> int:
    text = str(picture or "").upper()

    if text.startswith("S"):
        text = text[1:]

    return self.count_9_digits(
        value=text,
    )

def count_9_digits(
    self,
    value: str,
) -> int:
    text = str(value or "").upper()

    if text.startswith("S"):
        text = text[1:]

    total = 0
    index = 0

    while index < len(text):
        if text[index] not in {"9", "0"}:
            index += 1
            continue

        if index + 1 < len(text) and text[index + 1] == "(":

```

```

        close_index = text.find(")", index + 2)

        if close_index != -1:
            number_text = text[index + 2:close_index]

            if number_text.isdigit():
                total += int(number_text)
                index = close_index + 1
                continue

        total += 1
        index += 1

    return total

def extract_parenthesized_int(
    self,
    value: str,
) -> int | None:
    text = str(value or "")
    open_index = text.find("(")

    if open_index == -1:
        return None

    close_index = text.find(")", open_index + 1)

    if close_index == -1:
        return None

    number_text = text[open_index + 1:close_index].strip()

    if not number_text.isdigit():
        return None

    return int(number_text)

def calculate_end_position(
    self,
    start_position: int | None,
    storage_length: int | None,
    fallback=None,
) -> int | None:
    try:
        if start_position is not None and storage_length is not None:
            return int(start_position) + int(storage_length) - 1
    except Exception:
        pass

    try:
        if fallback is not None:
            return int(fallback)
    except Exception:
        pass

    return None

def is_date_field(
    self,
    field: DataField,
    picture: str | None,
    storage_length: int | None,
    datatype: str | None,
) -> bool:
    datatype_upper = str(datatype or "").upper()

    if datatype_upper == "DATE":
        return True

    field_name = NameNormalizer.normalize(
        getattr(field, "name", "") or "",
    )

    parts = [
        part
        for part in field_name.replace("-", " ").replace("_", " ").split()

```

```

        if part
    ]

    if "DATE" in parts or "DA" in parts:
        if storage_length == 8:
            return True

    core = self.picture_core(
        picture=picture,
    )

    if core in {"9(8)", "S9(8)", "99999999", "S99999999"}:
        if field_name.startswith("DA ") or " DATE" in field_name:
            return True

    return False

def basetype_for_datatype(
    self,
    datatype: str | None,
) -> str | None:
    datatype_upper = str(datatype or "").upper()

    if datatype_upper == "DATE":
        return "DATE"

    if datatype_upper in {"DECIMAL", "NUMERIC", "INTEGER"}:
        return "NUMERIC"

    if datatype_upper in {"CHAR", "VARCHAR"}:
        return "CHAR"

    return datatype

def enrich_group_positions_from_children(
    self,
    fields: list[DataField],
    debug_label: str = "",
) -> list[DataField]:
    output = list(fields or [])

    for index, field in enumerate(output):
        is_group = bool(
            getattr(field, "has_child", False)
            or getattr(field, "is_group", False)
        )

        if not is_group:
            continue

        level = getattr(field, "level", None)

        try:
            parent_level = int(level)
        except Exception:
            continue

        child_starts: list[int] = []
        child_ends: list[int] = []

        for child in output[index + 1:]:
            child_level = getattr(child, "level", None)

            try:
                current_level = int(child_level)
            except Exception:
                continue

            if current_level <= parent_level:
                break

            start = getattr(child, "start_position", None)
            end = getattr(child, "end_position", None)

            if start is not None:
                try:

```

```

        child_starts.append(int(start))
    except Exception:
        pass

    if end is not None:
        try:
            child_ends.append(int(end))
        except Exception:
            pass

    if not child_starts and not child_ends:
        continue

    group_start = min(child_starts) if child_starts else getattr(field, "start_position", None)
    group_end = max(child_ends) if child_ends else getattr(field, "end_position", None)

    group_length = getattr(field, "length", None)

    try:
        if group_start is not None and group_end is not None:
            group_length = int(group_end) - int(group_start) + 1
    except Exception:
        pass

    output[index] = self.make_field(
        source=field,
        datatype=getattr(field, "datatype", None),
        length=group_length,
        scale=getattr(field, "scale", None),
        picture=None,
        start_position=group_start,
        end_position=group_end,
        basetype=getattr(field, "basetype", None),
    )

    return output

def copy_field_with_defaults(
    self,
    field: DataField,
) -> DataField:
    return self.make_field(
        source=field,
        datatype=getattr(field, "datatype", None),
        length=getattr(field, "length", None),
        scale=getattr(field, "scale", None),
        picture=getattr(field, "picture", None),
        start_position=getattr(field, "start_position", None),
        end_position=getattr(field, "end_position", None),
        basetype=getattr(field, "basetype", None),
    )

def make_field(
    self,
    source: DataField,
    datatype: str | None,
    length: int | None,
    scale: int | None,
    picture: str | None,
    start_position: int | None,
    end_position: int | None,
    basetype: str | None,
) -> DataField:
    data = source.model_dump()

    data.update(
        {
            "datatype": datatype,
            "length": length,
            "scale": scale,
            "picture": picture,
            "start_position": start_position,
            "end_position": end_position,
            "basetype": basetype,
        }
    )

```



```

        return DataField(**data)

def remove_record_suffix(
    self,
    value: str,
) -> str:
    text = str(value or "").strip().upper()
    text = re.sub(r"[\s_-]+[0-9]{4}$", "", text)
    text = re.sub(r"[0-9]{4}$", "", text)
    text = re.sub(r"[\s_-]+$", "", text)

    return text

```

```

=====
FILE: src\idms_modernizer\services\validation_service.py
ENCODING: utf-8
=====

```

```

class ValidationService:

    def validate_metadata(
        self,
        metadata
    ) -> list[str]:

        errors = []

        record_names = {
            record.name
            for record in metadata.records
        }

        for rel in metadata.relationships:

            if rel.owner_record not in record_names:

                errors.append(
                    f"Missing Owner Record: {rel.owner_record}"
                )

            if rel.member_record not in record_names:

                errors.append(
                    f"Missing Member Record: {rel.member_record}"
                )

        return errors

```

```

=====
FILE: src\idms_modernizer\tests\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_modernizer\tests\test_date_field_consolidator.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.services.date_field_consolidator import (
    DateFieldConsolidator
)

```

```

from idms_modernizer.domain.schema_models import (
    DataField
)

```

```

def test_consolidate_year_month_day():
    fields = [
        DataField(name="START_YEAR"),

```

```

        DataField(name="START_MONTH"),
        DataField(name="START_DAY")
    ]

    out = DateFieldConsolidator.consolidate(fields)

    assert len(out) == 1
    assert out[0].name == "START_DATE"
    assert out[0].datatype == "DATE"

def test_consolidate_variants():
    fields = [
        DataField(name="FINISH_YR"),
        DataField(name="FINISH_MO"),
        DataField(name="FINISH_DY")
    ]

    out = DateFieldConsolidator.consolidate(fields)

    assert len(out) == 1
    assert out[0].name == "FINISH_DATE"
    assert out[0].datatype == "DATE"

def test_preserve_existing_full_date():
    fields = [
        DataField(name="BIRTH_DATE", datatype="DATE"),
        DataField(name="BIRTH_YEAR"),
        DataField(name="BIRTH_MONTH")
    ]

    out = DateFieldConsolidator.consolidate(fields)

    # Should keep the existing BIRTH_DATE and not create duplicate
    names = [f.name for f in out]
    assert "BIRTH_DATE" in names
    assert all(n.startswith("BIRTH_") for n in names)

def test_keep_year_only():
    fields = [
        DataField(name="CLAIM_YEAR")
    ]

    out = DateFieldConsolidator.consolidate(fields)

    assert len(out) == 1
    assert out[0].name == "CLAIM_YEAR"

=====
FILE: src\idms_modernizer\tests\test_diagram_parser.py
ENCODING: utf-8
=====

from idms_modernizer.parsers.pdf_diagram_parser import (
    PdfDiagramParser
)

def test_diagram_parser():
    parser = (
        PdfDiagramParser()
    )

    relationships = (
        parser.parse(
            """
            DEPARTMENT
            EMPLOYEE
            """
        )
    )

```

```

    assert (
        len(relationships)
        == 1
    )

    assert (
        relationships[0].owner
        == "DEPARTMENT"
    )

=====
FILE: src\idms_modernizer\tests\test_field_extractor.py
ENCODING: utf-8
=====

import re
import sys
from dataclasses import dataclass, asdict
from typing import List, Optional, Set

# ----- PDF READER ----- #
def read_pdf_lines(pdf_path: str) -> List[str]:
    """
    Extract text lines from a PDF file using pdfminer.six.
    Returns a list of lines.
    """
    try:
        from pdfminer.high_level import extract_text
    except ImportError as e:
        raise RuntimeError(
            "pdfminer.six is required. Install it with:\n\n"
            "    pip install pdfminer.six\n"
        ) from e

    text = extract_text(pdf_path) or ""
    # Split into lines while preserving basic line structure
    lines = text.splitlines()
    # Trim trailing spaces for consistency
    return [ln.rstrip("\r\n") for ln in lines]

# ----- DATA TYPES ----- #
@dataclass(frozen=True)
class DataField:
    name: str
    datatype: str
    length: Optional[int]
    scale: Optional[int]
    picture: str

# ----- EXTRACTOR ----- #
class FieldExtractor:
    """
    Robust parser for IDMS-like listing lines.

    - Extracts PICTURE regardless of position and with/without PIC/PICTURE keyword
    - Recognizes DISPLAY, COMP, COMP-3 in any order relative to PICTURE
    - Skips FILLER items by default (configurable)
    - Removes duplicate names
    - Maps:
      * X -> CHAR(1)
      * X(n) -> VARCHAR(n)
      * DISPLAY + 9/S9[/ (n)] -> INTEGER(length = digits)
      * COMP/COMP-3 + 9/S9[/ (n)] -> DECIMAL(precision = digits)
      * S9(n)V9(m)/9(n)V9(m) -> DECIMAL(precision = n+m, scale = m)
    """

    def __init__(
        self,
        require_explicit_length: bool = False,
        include_filler: bool = False,
        auto_number_filler: bool = True,
    ):

```

```

self.require_explicit_length = require_explicit_length
self.include_filler = include_filler
self.auto_number_filler = auto_number_filler

# Start: <level> <name>
_R_START = re.compile(r"^\s*(\d{2})\s+([A-Z0-9-]+\s)", re.IGNORECASE)

# USAGE anywhere (allow "USAGE IS ...")
_R_USAGE = re.compile(r"\b(?:USAGE\s+IS\s+)?(DISPLAY|COMP-3|COMP)\b", re.IGNORECASE)

# PICTURE anywhere; optional PIC/PICTURE keyword; tolerate spaces; no trailing \b
_R_PIC_ANYWHERE = re.compile(
    r"""
    (?:\b(?:PIC|PICTURE)\s*)?
    (
        S\s*9\s*$\s*\d+\s*$\s*V\s*9\s*$\s*\d+\s*$ | # S9(n)V9(m) / 9(n)V9(m)
        S\s*9\s*$\s*\d+\s*$ | # S9(n)
        9\s*$\s*\d+\s*$ | # 9(n)
        S\s*9 | # S9
        9 | # 9
        X\s*$\s*\d+\s*$ | # X(n)
        X | # X
    )
    """,
    re.IGNORECASE | re.VERBOSE,
)

# Helpers (operate on normalized picture like X(30), 9(7), S9(7)V9(2))
_R_CHAR = re.compile(r"^\s*X(\d+)\s*$", re.IGNORECASE) # X(n)
_R_INT = re.compile(r"^\s*9(?:\s*(\d+)\s*)?\s*$", re.IGNORECASE) # 9 / 9(n) / S9 / S9(n)
_R_DEC = re.compile(r"^\s*9\s*(\d+)\s*$\s*V9\s*(\d+)\s*$", re.IGNORECASE) # S9(n)V9(m) / 9(n)V9(m)

def extract(self, lines: List[str]) -> List[DataField]:
    seen: Set[str] = set()
    out: List[DataField] = []
    filler_idx = 0

    for raw in lines:
        line = raw.strip()
        if not line:
            continue

        m_start = self._R_START.match(line)
        if not m_start:
            continue

        _, name = m_start.groups()
        name_up = name.upper()

        # Handle FILLER items
        if name_up.startswith("FILLER"):
            if not self.include_filler:
                continue
            if self.auto_number_filler:
                filler_idx += 1
                name_up = f"FILLER_{filler_idx}"

        if name_up in seen:
            continue

        # USAGE default to DISPLAY if absent
        m_usage = self._R_USAGE.search(line)
        usage = m_usage.group(1).upper() if m_usage else "DISPLAY"

        # PICTURE capture
        m_pic = self._R_PIC_ANYWHERE.search(line)
        if not m_pic:
            continue

        # Normalize: remove spaces inside the picture and uppercase
        pic = re.sub(r"\s+", "", m_pic.group(1).upper())

        # Optional filter: require explicit length (skip X, 9, S9)
        if self.require_explicit_length and pic in {"X", "9", "S9"}:
            continue

```

```

dtype: str = "VARCHAR"
length: Optional[int] = None
scale: Optional[int] = None

# Character
if pic == "X":
    dtype, length = "CHAR", 1
elif self._R_CHAR.fullmatch(pic):
    length = int(self._R_CHAR.fullmatch(pic).group(1))
    dtype = "VARCHAR"

# Decimal with V
elif self._R_DEC.fullmatch(pic):
    int_len, frac_len = map(int, self._R_DEC.fullmatch(pic).groups())
    length = int_len + frac_len
    scale = frac_len
    dtype = "DECIMAL"

# Integer-like 9/S9 with optional (n)
elif self._R_INT.fullmatch(pic):
    digits_str = self._R_INT.fullmatch(pic).group(1)
    digits = int(digits_str) if digits_str else 1
    length = digits
    dtype = "INTEGER" if usage == "DISPLAY" else "DECIMAL"
else:
    # Unrecognized picture; skip
    continue

seen.add(name_up)
out.append(
    DataField(
        name=name_up,
        datatype=dtype,
        length=length,
        scale=scale,
        picture=pic,
    )
)

```

```

return out

```

```

# ----- DEMO (PDF PATH) ----- #
if __name__ == "__main__":
    # Option A: Provide the PDF file path here (edit this string)
    pdf_path = r"C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master\src\idms_modernizer\data\IDMS_Schema_Listing.pdf"

    # Option B: Or pass the path as first CLI argument
    if len(sys.argv) > 1:
        pdf_path = sys.argv[1]

    # Read lines from PDF
    lines = read_pdf_lines(pdf_path)

    # Configure extractor as needed
    extractor = FieldExtractor(
        require_explicit_length=False, # set True to skip X / 9 / S9 without (n)
        include_filler=False,         # set True to include FILLER items
        auto_number_filler=True
    )

    fields = extractor.extract(lines)
    for f in fields:
        print(asdict(f))

```

```

=====
FILE: src\idms_modernizer\tests\test_normalization_mapping.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.schema_models import DataField
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.services.date_field_consolidator import DateFieldConsolidator
from idms_modernizer.services.db2_mapping_service import DB2MappingService

```

```

def test_name_normalizer_removes_suffix_and_normalizes():
    assert (
        NameNormalizer.normalize(
            "CLAIM-YEAR-0405"
        )
        == "CLAIM_YEAR"
    )
    assert (
        NameNormalizer.normalize(
            "EMP-ID 0415"
        )
        == "EMP_ID"
    )
    assert (
        NameNormalizer.normalize(
            " CLAIM--MONTH-0405 "
        )
        == "CLAIM_MONTH"
    )

def test_date_field_consolidator_consolidates_suffixed_date_fields():
    fields = [
        DataField(name="CLAIM-YEAR-0405"),
        DataField(name="CLAIM-MONTH-0405"),
        DataField(name="CLAIM-DAY-0405"),
        DataField(name="OTHER-FIELD-0405"),
    ]

    consolidated = DateFieldConsolidator.consolidate(
        fields
    )

    assert any(
        field.name == "CLAIM_DATE"
        and field.datatype == "DATE"
        for field in consolidated
    )
    assert any(
        field.name == "OTHER-FIELD-0405"
        and field.datatype is None
        for field in consolidated
    )
    assert len(consolidated) == 2

def test_db2_mapping_service_preserves_date_and_varchar_length():
    mapper = DB2MappingService()

    date_column = DataField(
        name="CLAIM_DATE",
        datatype="DATE"
    )
    assert mapper.map_datatype(date_column) == "DATE"

    timestamp_column = DataField(
        name="EVENT_TS",
        datatype="TIMESTAMP"
    )
    assert mapper.map_datatype(timestamp_column) == "TIMESTAMP"

    string_column = DataField(
        name="NAME",
        datatype="DISPLAY",
        length=40,
    )
    assert mapper.map_datatype(string_column) == "VARCHAR(40)"

```

```
=====
FILE: src\idms_modernizer\tests\test_schema_parser.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.parsers.pdf_schema_parser import (
    PdfSchemaParser
)
```

```
def test_schema_parser():
```

```
    sample_text = """
```

```
    RECORD NAME..... EMPLOYEE
```

```
    EMP-ID-0415
```

```
    EMP-FIRST-NAME-0415
```

```
    EMP-LAST-NAME-0415
```

```
    RECORD NAME..... DEPARTMENT
```

```
    DEPT-ID-0416
```

```
    DEPT-NAME-0416
```

```
    """
```

```
    parser = (
        PdfSchemaParser()
    )
```

```
    metadata = (
        parser.parse(
            sample_text
        )
    )
```

```
    assert (
        len(metadata.records)
        == 2
    )
```

```
    assert (
        metadata.records[0].name
        == "EMPLOYEE"
    )
```

```
    assert (
        len(
            metadata.records[0].fields
        )
        == 3
    )
```

```
=====
FILE: src\idms_modernizer\ui\__init__.py
ENCODING: utf-8
=====
```

```
[Empty file]
```

```
=====
FILE: src\idms_modernizer\ui\metadata_page.py
ENCODING: utf-8
=====
```

```
import streamlit as st
```

```
def render_metadata(
    metadata
```

```

):

st.subheader(
    "Detected Records"
)

for record in metadata.records:

    st.markdown(
        f"### {record.name}"
    )

    st.write(
        f"Fields: {len(record.fields)}"
    )

    field_names = [
        field.name
        for field in record.fields
    ]

    st.dataframe(
        field_names
    )

=====
FILE: src\idms_modernizer\ui\phase2_converter_panel.py
ENCODING: utf-8
=====

import json
from pathlib import Path
from typing import Any

import streamlit as st

from idms_modernizer.parsers.cobol_pdf_extractor import CobolPdfExtractor
from idms_modernizer.services.phase2_conversion_bridge import (
    Phase2ConversionBridge
)

try:
    from idms_db2_converter.exceptions import ConversionError
except Exception:
    class ConversionError(Exception):
        pass

class Phase2ConverterPanel:
    """
    Streamlit panel for running Phase 2 conversion inside Phase 1 app.

    Inputs:
    - COBOL text, COBOL PDF, or COBOL text file
    - Canonical JSON generated by Phase 1
    - Phase 2 metadata JSON generated by Phase 1
    - DB2 DDL generated by Phase 1
    - Optional IDMS schema text
    - Optional relationship overrides

    Important:
    - No hardcoded relationship overrides.
    - Relationship override JSON is dynamically generated from Phase 2 metadata
      when the user chooses to use overrides.
    """

    def render(
        self,
        canonical_json: str,
        phase2_metadata_json: str,
        ddl_text: str,
        idms_schema_text: str | None = None
    ) -> None:

        st.header(

```



```

        "Phase 2 - IDMS Retrieval COBOL to DB2 COBOL"
    )

    st.markdown(
        """
Use this section to convert an IDMS retrieval COBOL program into DB2 embedded SQL COBOL using the Phase 1 outputs gener
ated above.
        """
    )

    with st.expander(
        "Phase 1 Inputs Passed to Phase 2",
        expanded=False
    ):
        st.subheader(
            "Canonical JSON"
        )

        st.code(
            canonical_json or "",
            language="json"
        )

        st.subheader(
            "Phase 2 Metadata JSON"
        )

        st.code(
            phase2_metadata_json or "",
            language="json"
        )

        st.subheader(
            "DB2 DDL"
        )

        st.code(
            ddl_text or "",
            language="sql"
        )

    st.subheader(
        "COBOL Input"
    )

    cobol_text = self._render_cobol_input()

    st.subheader(
        "Conversion Settings"
    )

    target_program = st.text_input(
        "Target PROGRAM-ID",
        value="",
        placeholder="Example: DEPTDB2"
    )

    col1, col2, col3 = st.columns(3)

    with col1:
        use_ddl = st.checkbox(
            "Use DB2 DDL",
            value=True,
            help="Recommended. DDL should be the physical source of truth."
        )

    with col2:
        use_phase2_metadata = st.checkbox(
            "Use Phase 2 Metadata",
            value=True,
            help="Recommended. Uses Phase 1 generated mapping metadata."
        )

    with col3:
        use_idms_schema = st.checkbox(
            "Use IDMS Schema Text",

```

```

        value=False,
        help="Optional. Usually not required if canonical, DDL, and Phase 2 metadata are provided."
    )

    use_overrides = st.checkbox(
        "Use relationship key overrides",
        value=False,
        help="Optional. Dynamically generated from Phase 2 metadata when enabled."
    )

    relationship_overrides_json = ""

    if use_overrides:
        relationship_overrides_json = self._render_relationship_overrides(
            phase2_metadata_json=phase2_metadata_json
        )

    phase2_idms_schema_text = ""

    if use_idms_schema:
        phase2_idms_schema_text = st.text_area(
            "Paste IDMS schema/subschema listing text",
            height=300,
            value=idms_schema_text or ""
        )

    convert_clicked = st.button(
        "Convert to DB2 COBOL",
        type="primary"
    )

    if convert_clicked:
        self._run_conversion(
            cobol_text=cobol_text,
            canonical_json=canonical_json,
            phase2_metadata_json=(
                phase2_metadata_json
                if use_phase2_metadata
                else ""
            ),
            ddl_text=(
                ddl_text
                if use_ddl
                else ""
            ),
            idms_schema_text=(
                phase2_idms_schema_text
                if use_idms_schema
                else None
            ),
            relationship_overrides_json=(
                relationship_overrides_json
                if use_overrides
                else None
            ),
            target_program=target_program,
            use_ddl=use_ddl,
            use_idms_schema=use_idms_schema,
            use_overrides=use_overrides
        )

    def _render_cobol_input(
        self
    ) -> str:

        input_mode = st.radio(
            "Choose COBOL input method",
            [
                "Paste COBOL",
                "Upload COBOL PDF",
                "Upload COBOL Text File"
            ],
            horizontal=True
        )

        cobol_text = ""

```

```

if input_mode == "Paste COBOL":
    cobol_text = st.text_area(
        "Paste IDMS COBOL retrieval program",
        height=420,
        placeholder="Paste DEPTRPT or another IDMS retrieval program here..."
    )

elif input_mode == "Upload COBOL PDF":
    cobol_text = self._load_cobol_from_pdf()

elif input_mode == "Upload COBOL Text File":
    cobol_text = self._load_cobol_from_text_file()

return cobol_text

def _load_cobol_from_pdf(
    self
) -> str:

    cobol_text = ""

    cobol_pdf = st.file_uploader(
        "Upload COBOL PDF",
        type=[
            "pdf"
        ],
        key="phase2_cobol_pdf"
    )

    if cobol_pdf is None:
        return cobol_text

    temp_path = self._save_uploaded_file(
        uploaded_file=cobol_pdf,
        folder="data/temp"
    )

    extractor = CobolPdfExtractor()

    try:
        cobol_text = extractor.extract_text(
            temp_path
        )

        st.success(
            "COBOL PDF extracted successfully."
        )

        cobol_text = st.text_area(
            "Extracted COBOL text",
            value=cobol_text,
            height=420
        )

    except Exception as ex:
        st.error(
            f"Failed to extract COBOL PDF: {ex}"
        )

    return cobol_text

def _load_cobol_from_text_file(
    self
) -> str:

    cobol_text = ""

    cobol_file = st.file_uploader(
        "Upload COBOL source file",
        type=[
            "cbl",
            "cob",
            "txt"
        ],
        key="phase2_cobol_text"
    )

```

```

    )

    if cobol_file is None:
        return cobol_text

    try:
        cobol_text = cobol_file.read().decode(
            "utf-8",
            errors="ignore"
        )

        st.success(
            "COBOL text file loaded successfully."
        )

        cobol_text = st.text_area(
            "Loaded COBOL text",
            value=cobol_text,
            height=420
        )

    except Exception as ex:
        st.error(
            f"Failed to read COBOL file: {ex}"
        )

    return cobol_text

def _render_relationship_overrides(
    self,
    phase2_metadata_json: str
) -> str:

    st.markdown(
        """
Relationship overrides are generated from the current Phase 2 metadata.
You can edit them before conversion if needed.
"""
    )

    generated_overrides = self._build_overrides_from_phase2_metadata(
        phase2_metadata_json
    )

    if not generated_overrides:
        st.info(
            "No complete relationship overrides could be generated from Phase 2 metadata."
        )

        generated_overrides = {
            "relationships": []
        }

    return st.text_area(
        "Relationship key overrides JSON",
        height=300,
        value=json.dumps(
            generated_overrides,
            indent=2
        )
    )

def _build_overrides_from_phase2_metadata(
    self,
    phase2_metadata_json: str
) -> dict[str, Any]:

    if not phase2_metadata_json or not phase2_metadata_json.strip():
        return {
            "relationships": []
        }

    try:
        payload = json.loads(
            phase2_metadata_json
        )

```

```

except json.JSONDecodeError:
    return {
        "relationships": []
    }

relationships = payload.get(
    "relationships",
    []
)

override_relationships = []

for relationship in relationships:
    set_name = self._get_first_non_empty(
        relationship,
        [
            "set_name",
            "name",
            "relationship"
        ]
    )

    parent_record = self._get_first_non_empty(
        relationship,
        [
            "parent_record",
            "parent_table"
        ]
    )

    child_record = self._get_first_non_empty(
        relationship,
        [
            "child_record",
            "child_table"
        ]
    )

    parent_key = self._get_first_non_empty(
        relationship,
        [
            "parent_key",
            "parent_pk"
        ]
    )

    child_fk = self._get_first_non_empty(
        relationship,
        [
            "child_fk",
            "foreign_key"
        ]
    )

    order_by = relationship.get(
        "order_by",
        []
    )

    if not set_name:
        continue

    if not parent_record:
        continue

    if not child_record:
        continue

    item = {
        "set_name": self._to_idms_name(set_name),
        "parent_record": self._normalize_name(parent_record),
        "child_record": self._normalize_name(child_record)
    }

    if parent_key:

```

```

        item[
            "parent_key"
        ] = self._normalize_name(parent_key)

    if child_fk:
        item[
            "child_fk"
        ] = self._normalize_name(child_fk)

    if order_by:
        item[
            "order_by"
        ] = [
            self._normalize_name(value)
            for value in order_by
            if value
        ]

    override_relationships.append(
        item
    )

    return {
        "relationships": override_relationships
    }

def _run_conversion(
    self,
    cobol_text: str,
    canonical_json: str,
    phase2_metadata_json: str,
    ddl_text: str,
    idms_schema_text: str | None,
    relationship_overrides_json: str | None,
    target_program: str | None,
    use_ddl: bool,
    use_idms_schema: bool,
    use_overrides: bool
) -> None:

    if not cobol_text or not cobol_text.strip():
        st.error(
            "COBOL code is required."
        )
        st.stop()

    bridge = Phase2ConversionBridge()

    try:
        converted, validation_messages = bridge.convert(
            cobol_text=cobol_text,
            canonical_json=canonical_json,
            phase2_metadata_json=phase2_metadata_json,
            ddl_text=ddl_text,
            idms_schema_text=idms_schema_text,
            relationship_overrides_json=relationship_overrides_json,
            target_program=target_program,
            use_ddl=use_ddl,
            use_idms_schema=use_idms_schema,
            use_overrides=use_overrides
        )

        st.success(
            "Phase 2 conversion completed."
        )

    if validation_messages:
        st.warning(
            "Conversion completed with validation messages."
        )

        for message in validation_messages:
            st.write(
                f"- {message}"
            )

```

```

        else:
            st.info(
                "Validation passed."
            )

        st.subheader(
            "Final DB2 COBOL"
        )

        st.text_area(
            "Copy converted DB2 COBOL",
            value=converted,
            height=700
        )

    except ConversionError as exc:
        st.error(
            "Phase 2 conversion failed."
        )

        st.code(
            str(exc),
            language="text"
        )

    except Exception as exc:
        st.error(
            "Unexpected Phase 2 conversion error."
        )

        st.code(
            str(exc),
            language="text"
        )

def _save_uploaded_file(
    self,
    uploaded_file,
    folder: str
) -> str:

    target_folder = Path(
        folder
    )

    target_folder.mkdir(
        parents=True,
        exist_ok=True
    )

    target_path = (
        target_folder
        / uploaded_file.name
    )

    with open(
        target_path,
        "wb"
    ) as file:
        file.write(
            uploaded_file.getbuffer()
        )

    return str(
        target_path
    )

def _get_first_non_empty(
    self,
    source: dict[str, Any],
    keys: list[str]
) -> str | None:

    for key in keys:
        value = source.get(
            key

```

```

    )

    if value is None:
        continue

    value = str(
        value
    ).strip()

    if value:
        return value

    return None

def _normalize_name(
    self,
    value: str | None
) -> str:

    if value is None:
        return ""

    return str(
        value
    ).strip().upper().replace(
        "-",
        "_"
    )

def _to_idms_name(
    self,
    value: str | None
) -> str:

    if value is None:
        return ""

    return str(
        value
    ).strip().upper().replace(
        "-",
        "_"
    )

```

```

=====
FILE: src\idms_modernizer\ui\schema_listing_only_app.py
ENCODING: utf-8
=====

```

```

from pathlib import Path
from tempfile import NamedTemporaryFile

import streamlit as st

from idms_modernizer.core.config import settings
from idms_modernizer.core.utils import ensure_directory
from idms_modernizer.services.metadata_service import MetadataService
from idms_modernizer.services.canonical_schema_builder import (
    CanonicalSchemaBuilder,
)
from idms_modernizer.services.canonical_reconciler import (
    CanonicalReconciler,
)
from idms_modernizer.services.db2_model_builder import DB2ModelBuilder
from idms_modernizer.generators.ddl_generator import DDLGenerator
from idms_modernizer.services.excel_sheet_mapping_service import (
    ExcelSheetMappingService,
)

def initialize_directories() -> None:
    ensure_directory(
        settings.output_dir,
    )

```



```

def initialize_session_state() -> None:
    defaults = {
        "schema_only_generated": False,
        "schema_only_error": "",
        "schema_only_schema_path": "",
        "schema_only_metadata": None,
        "schema_only_canonical_schema": None,
        "schema_only_db2_model": None,
        "schema_only_ddl_text": "",
        "schema_only_excel_mapping_rows": [],
    }

    for key, value in defaults.items():
        if key not in st.session_state:
            st.session_state[key] = value

def reset_schema_only_outputs() -> None:
    st.session_state.schema_only_generated = False
    st.session_state.schema_only_error = ""
    st.session_state.schema_only_schema_path = ""
    st.session_state.schema_only_metadata = None
    st.session_state.schema_only_canonical_schema = None
    st.session_state.schema_only_db2_model = None
    st.session_state.schema_only_ddl_text = ""
    st.session_state.schema_only_excel_mapping_rows = []

def save_uploaded_schema_file(
    uploaded_file,
) -> str:
    suffix = Path(
        uploaded_file.name,
    ).suffix or ".pdf"

    with NamedTemporaryFile(
        delete=False,
        suffix=suffix,
    ) as temp_file:
        temp_file.write(
            uploaded_file.getbuffer(),
        )

    return temp_file.name

def generate_schema_outputs(
    schema_pdf,
) -> None:
    reset_schema_only_outputs()

    try:
        schema_path = save_uploaded_schema_file(
            uploaded_file=schema_pdf,
        )

        metadata_service = MetadataService()

        metadata = metadata_service.build_metadata(
            pdf_path=schema_path,
        )

        canonical_builder = CanonicalSchemaBuilder()

        canonical_schema = canonical_builder.build(
            metadata=metadata,
        )

        canonical_reconciler = CanonicalReconciler()

        canonical_schema = canonical_reconciler.reconcile(
            canonical_schema=canonical_schema,
            metadata=metadata,
        )

```

```

db2_builder = DB2ModelBuilder()

db2_model = db2_builder.build(
    canonical_schema,
)

ddl_generator = DDLGenerator()

ddl_text = ddl_generator.generate(
    db2_model,
)

excel_mapping_service = ExcelSheetMappingService()

excel_mapping_rows = excel_mapping_service.build(
    metadata=metadata,
    db2_model=db2_model,
)

st.session_state.schema_only_schema_path = schema_path
st.session_state.schema_only_metadata = metadata
st.session_state.schema_only_canonical_schema = canonical_schema
st.session_state.schema_only_db2_model = db2_model
st.session_state.schema_only_ddl_text = ddl_text
st.session_state.schema_only_excel_mapping_rows = excel_mapping_rows
st.session_state.schema_only_generated = True

except Exception as exc:
    st.session_state.schema_only_generated = False
    st.session_state.schema_only_error = str(
        exc,
    )
)

def build_mapping_row_count_by_table(
    rows: list[dict],
    include_total: bool = True,
) -> list[dict[str, int | str]]:
    """
    Builds table-level counts from the actual Excel Sheet Mapping rows.

    This count matches the detailed mapping table because it counts the
    generated mapping rows, not only DB2 physical columns.

    Output column order:
    - Column Field Count
    - Table Name
    """
    table_counts: dict[str, int] = {}

    for row in rows or []:
        table_name = (
            row.get("New DB2 Record")
            or row.get("Cobol Record IDMS")
            or ""
        )

        table_name = str(
            table_name,
        ).strip()

        if not table_name:
            continue

        table_counts[table_name] = table_counts.get(
            table_name,
            0,
        ) + 1

    output_rows: list[dict[str, int | str]] = []

    total_count = 0

    for table_name in sorted(table_counts):
        count = table_counts[table_name]

```

```

        output_rows.append(
            {
                "Column Field Count": count,
                "Table Name": table_name,
            }
        )

        total_count += count

    if include_total:
        output_rows.append(
            {
                "Column Field Count": total_count,
                "Table Name": "TOTAL",
            }
        )

    return output_rows

def build_db2_physical_column_count_rows(
    db2_model,
    include_total: bool = True,
) -> list[dict[str, int | str]]:
    """
    Builds table-level physical DB2 column counts.

    This is separate from the mapping row count because DATE child fields
    can create multiple mapping rows but only one DB2 physical DATE column.
    """
    table_rows: list[dict[str, int | str]] = []

    if db2_model is None:
        return table_rows

    total_count = 0

    for table in getattr(
        db2_model,
        "tables",
        [],
    ) or []:
        table_name = getattr(
            table,
            "name",
            "",
        )

        columns = getattr(
            table,
            "columns",
            [],
        ) or []

        count = len(
            columns,
        )

        table_rows.append(
            {
                "Column Field Count": count,
                "Table Name": table_name,
            }
        )

        total_count += count

    if include_total:
        table_rows.append(
            {
                "Column Field Count": total_count,
                "Table Name": "TOTAL",
            }
        )

    return table_rows

```

```

def render_main_tab() -> None:
    st.markdown("## Schema Listing Only")

    st.info(
        "Upload only the Schema Listing PDF to generate Metadata Overview, "
        "DB2 DDL, and Excel Sheet Mapping."
    )

    schema_pdf = st.file_uploader(
        "Schema Listing PDF",
        type=["pdf"],
        key="schema_only_schema_listing_pdf",
    )

    generate_clicked = st.button(
        "Generate Schema Outputs",
        type="primary",
        use_container_width=True,
    )

    if generate_clicked:
        if schema_pdf is None:
            st.error(
                "Please upload a Schema Listing PDF."
            )
            return

        with st.spinner(
            "Processing schema listing and generating outputs..."
        ):
            generate_schema_outputs(
                schema_pdf=schema_pdf,
            )

        if st.session_state.schema_only_generated:
            st.success(
                "Schema outputs generated successfully."
            )
        else:
            st.error(
                "Schema output generation failed."
            )

            if st.session_state.schema_only_error:
                st.error(
                    st.session_state.schema_only_error,
                )

    if st.session_state.schema_only_generated:
        st.markdown("### Output Status")

        metadata = st.session_state.schema_only_metadata
        db2_model = st.session_state.schema_only_db2_model
        mapping_rows = st.session_state.schema_only_excel_mapping_rows

        col1, col2, col3, col4 = st.columns(
            4,
        )

        with col1:
            st.metric(
                "Records",
                len(
                    metadata.records,
                )
                if metadata is not None
                else 0,
            )

        with col2:
            st.metric(
                "Relationships",
                len(
                    metadata.relationships,

```

```

        )
        if metadata is not None
        else 0,
    )

    with col3:
        st.metric(
            "DB2 Tables",
            len(
                db2_model.tables,
            )
            if db2_model is not None
            else 0,
        )

    with col4:
        st.metric(
            "Sheet Mapping Rows",
            len(
                mapping_rows,
            )
            if mapping_rows is not None
            else 0,
        )

def render_metadata_overview_tab() -> None:
    if not st.session_state.schema_only_generated:
        st.info(
            "Generate outputs from the Main tab first."
        )
        return

    metadata = st.session_state.schema_only_metadata
    db2_model = st.session_state.schema_only_db2_model
    mapping_rows = st.session_state.schema_only_excel_mapping_rows

    if metadata is None:
        st.info(
            "No metadata available."
        )
        return

    st.markdown("## Metadata Overview")

    st.markdown("### Records")

    record_rows = []

    for record in metadata.records:
        record_rows.append(
            {
                "Record": record.name,
                "Cobol Zone": record.cobol_zone or "",
                "Primary Key": record.primary_key or "",
                "Field Count": len(
                    record.fields,
                ),
                "Set Membership Count": len(
                    record.set_memberships,
                ),
            }
        )

    st.dataframe(
        record_rows,
        use_container_width=True,
        hide_index=True,
    )

    st.divider()

    st.markdown("### Table List and Mapping Row Counts")

    mapping_count_rows = build_mapping_row_count_by_table(
        rows=mapping_rows,

```

```

        include_total=True,
    )

    if mapping_count_rows:
        st.dataframe(
            mapping_count_rows,
            use_container_width=True,
            hide_index=True,
        )
    else:
        st.info(
            "No table mapping row counts available."
        )

    st.caption(
        "This count is based on the actual Excel Sheet Mapping rows, "
        "so the TOTAL matches the detailed mapping table."
    )

    st.divider()

    st.markdown("### DB2 Physical Column Counts")

    physical_column_count_rows = build_db2_physical_column_count_rows(
        db2_model=db2_model,
        include_total=True,
    )

    if physical_column_count_rows:
        st.dataframe(
            physical_column_count_rows,
            use_container_width=True,
            hide_index=True,
        )
    else:
        st.info(
            "No DB2 physical column counts available."
        )

    st.caption(
        "This count is based on DB2 physical columns. It can be lower than "
        "the mapping row count when COBOL YEAR, MONTH, and DAY fields map "
        "to one DB2 DATE column."
    )

    st.divider()

    st.markdown("### Records and Fields")

    record_names = [
        record.name
        for record in metadata.records
    ]

    if not record_names:
        st.info(
            "No records found."
        )
        return

    selected_record = st.selectbox(
        "Select a record",
        options=record_names,
    )

    selected_record_obj = None

    for record in metadata.records:
        if record.name == selected_record:
            selected_record_obj = record
            break

    if selected_record_obj is not None:
        st.write(
            f"Fields found: `{len(selected_record_obj.fields)}`"
        )

```

```

field_rows = []

for field in selected_record_obj.fields:
    field_rows.append(
        {
            "Field": field.name,
            "Level": field.level,
            "Datatype": field.datatype,
            "Length": field.length,
            "Scale": field.scale,
            "Picture": field.picture,
            "Start Position": field.start_position,
            "End Position": field.end_position,
            "Basetype": field.basetype,
        }
    )

st.dataframe(
    field_rows,
    use_container_width=True,
    hide_index=True,
)

with st.expander(
    "View all records and fields as JSON",
    expanded=False,
):
    st.json(
        [
            {
                "record": record.name,
                "cobol_zone": record.cobol_zone,
                "primary_key": record.primary_key,
                "fields": [
                    field.model_dump()
                    for field in record.fields
                ],
                "set_memberships": [
                    {
                        "set_name": getattr(
                            membership,
                            "set_name",
                            "",
                        ),
                        "relation_type": getattr(
                            membership,
                            "relation_type",
                            "",
                        ),
                        "owner_record": getattr(
                            membership,
                            "owner_record",
                            "",
                        ),
                        "member_record": getattr(
                            membership,
                            "member_record",
                            "",
                        ),
                    }
                    for membership in record.set_memberships
                ],
            }
            for record in metadata.records
        ]
    )

```

```

def render_ddl_tab() -> None:
    if not st.session_state.schema_only_generated:
        st.info(
            "Generate outputs from the Main tab first."
        )
    return

```

```

st.markdown("## DB2 DDL")

ddl_text = st.session_state.schema_only_ddl_text

if not ddl_text:
    st.info(
        "No DDL generated."
    )
    return

st.code(
    ddl_text,
    language="sql",
)

def render_excel_sheet_mapping_tab() -> None:
    if not st.session_state.schema_only_generated:
        st.info(
            "Generate outputs from the Main tab first."
        )
        return

    st.markdown("## Excel Sheet Mapping")

    rows = st.session_state.schema_only_excel_mapping_rows

    if not rows:
        st.info(
            "No mapping rows generated."
        )
        return

    st.markdown("### Sheet Map Abstract")

    abstract_rows = build_mapping_row_count_by_table(
        rows=rows,
        include_total=True,
    )

    if abstract_rows:
        st.dataframe(
            abstract_rows,
            use_container_width=True,
            hide_index=True,
        )
    else:
        st.info(
            "No table abstract rows available."
        )

    st.caption(
        "The TOTAL row matches the number of detailed mapping rows below."
    )

    st.markdown("### Sheet Mapping Details")

    st.dataframe(
        rows,
        use_container_width=True,
        hide_index=True,
    )

def main() -> None:
    st.set_page_config(
        page_title="Schema Listing Mapping",
        layout="wide",
    )

    initialize_directories()
    initialize_session_state()

    st.title(
        "Schema Listing Mapping"
    )

```



```

st.caption(
    "Upload Schema Listing PDF only and generate DB2 DDL plus "
    "Excel-style mapping."
)

tabs = st.tabs(
    [
        "Main",
        "Metadata Overview",
        "DB2 DDL",
        "Excel Sheet Mapping",
    ]
)

with tabs[0]:
    render_main_tab()

with tabs[1]:
    render_metadata_overview_tab()

with tabs[2]:
    render_ddl_tab()

with tabs[3]:
    render_excel_sheet_mapping_tab()

if __name__ == "__main__":
    main()

=====
FILE: src\idms_modernizer\ui\upload_page.py
ENCODING: utf-8
=====

from pathlib import Path

import streamlit as st

from idms_modernizer.core.config import (
    settings
)

def render_upload_page():

    st.subheader(
        "Upload IDMS Schema PDF"
    )

    uploaded_file = (
        st.file_uploader(
            "Schema PDF",
            type=["pdf"]
        )
    )

    if uploaded_file:

        save_path = (
            Path(
                settings.input_dir
            )
            / uploaded_file.name
        )

        with open(
            save_path,
            "wb"
        ) as file:

            file.write(
                uploaded_file.read()
            )

```

```

        st.success(
            f"Saved: {save_path}"
        )

        return str(save_path)

    return None

```

```

=====
FILE: src\tools\audit_hardcoding.py
ENCODING: utf-8
=====

```

```

import re
from pathlib import Path

```

```

SEARCH_TERMS = [
    "0400",
    "0405",
    "0410",
    "0415",
    "0420",
    "0425",
    "0430",
    "0435",
    "0440",
    "0445",
    "0450",
    "0455",
    "0460",
    "DEPARTMENT",
    "EMPLOYEE",
    "EMPOSITION",
    "JOB",
    "OFFICE",
    "COVERAGE",
    "EXPERTISE",
    "NON-HOSP",
    "SKILL",
    "STRUCTURE",
    "DEPT-EMPLOYEE",
    "EMP-EMPOSITION",
    "JOB-EMPOSITION",
    "OFFICE-EMPLOYEE",
    "NEW-DEPT-STORE-DEPARTMENT",
    "OLD-DEPT-ERASE-DEPARTMENT",
    "OLD-LASTNAME-MODIFY-EMPLOYEE",
]

```

```

SKIP_DIRS = {
    ".git",
    ".venv",
    "__pycache__",
    "data",
    "output",
    "temp",
    "logs",
    "tests",
}

```

```

SKIP_FILES = {
    "audit_hardcoding.py",
    "installed.txt",
}

```

```

INCLUDE_SUFFIXES = {
    ".py",
}

```

```

FALSE_POSITIVE_SUBSTRINGS = {
    "STRUCTUREDBLOCKS",
    "CLASS STRUCTUREDBLOCKS",
    "FROM IDMS_DB2_CONVERTER.TRANSFORMERS.STRUCTURED_BLOCKS",
    "IDMS_DB2_CONVERTER.TRANSFORMERS.STRUCTURED_BLOCKS",
}

```

```

def should_skip(
    path: Path,
) -> bool:
    if path.name in SKIP_FILES:
        return True

    parts = set(
        part.lower()
        for part in path.parts
    )

    if parts.intersection(SKIP_DIRS):
        return True

    if path.suffix.lower() not in INCLUDE_SUFFIXES:
        return True

    return False

```

```

def strip_docstrings(
    text: str,
) -> str:
    text = re.sub(
        r'(""".*?""")',
        "",
        text,
        flags=re.DOTALL,
    )

    text = re.sub(
        r'('''.*?'')',
        "",
        text,
        flags=re.DOTALL,
    )

    return text

```

```

def is_likely_comment_line(
    line: str,
) -> bool:
    stripped = line.strip()

    if not stripped:
        return True

    return stripped.startswith(
        (
            "#",
            "*",
            "- ",
        )
    )

```

```

def contains_false_positive(
    line: str,
) -> bool:
    upper_line = line.upper()

    for value in FALSE_POSITIVE_SUBSTRINGS:
        if value in upper_line:
            return True

    return False

```

```

def term_pattern(
    term: str,
) -> re.Pattern:
    escaped = re.escape(
        term,
    )

    if re.fullmatch(
        r"\d+",
        term,
    ):
        return re.compile(
            rf"(?<!\d){escaped}(?!\\d)",
            re.IGNORECASE,
        )

    return re.compile(
        rf"(?<![A-Z0-9_-]){escaped}(?![A-Z0-9_-])",
        re.IGNORECASE,
    )

def main() -> None:
    root = Path.cwd()

    findings: list[str] = []

    compiled_terms = [
        (
            term,
            term_pattern(term),
        )
        for term in SEARCH_TERMS
    ]

    for path in root.rglob("*"):
        if not path.is_file():
            continue

        if should_skip(path):
            continue

        try:
            text = path.read_text(
                encoding="utf-8",
                errors="ignore",
            )
        except Exception:
            continue

        text_without_docstrings = strip_docstrings(
            text,
        )

        for line_number, line in enumerate(
            text_without_docstrings.splitlines(),
            start=1,
        ):
            if is_likely_comment_line(
                line,
            ):
                continue

            if contains_false_positive(
                line,
            ):
                continue

            for term, pattern in compiled_terms:
                if pattern.search(line):
                    findings.append(
                        f"{path}:{line_number}: {line.strip()}"
                    )

    if not findings:

```

```

        print(
            "No hardcoded demo terms found in scanned Python logic."
        )
        return

    print(
        "Potential hardcoded demo terms found in Python logic:"
    )

    print()

    for item in findings:
        print(
            item,
        )

if __name__ == "__main__":
    main()

```

```

=====
FILE: test
ENCODING: utf-8
=====

```

```

digraph G { A -> B }

```

```

=====
EXPORT SUMMARY
=====
Project folder: idms-db2-modernizer-master
Text/code files processed: 104

```